

Specializing Large language Models using Chain of Thought Reasoning

Anvesh Kumar Chavidi, Eshita Gupta, Neha Thakur, Sandeep Reddy Potula, Saumya Varshney

Department of Applied Data Science, San Jose State University

DATA 298B: MSDA Project II

Prof. Simon Shim

May 09, 2025

Abstract

Large Language Models (LLMs) have revolutionized the present market landscape and are driving innovations due to their ability to generalize solutions to problems across multiple domains. Chain of Thought Reasoning (CoT) is used to overcome the challenge of LLM hallucinating information when dealing with inputs/prompts that require domain specific knowledge. The objective of this project is to build an Intelligent System on top of an LLM fine-tuned for CoT reasoning using reinforcement learning and prompting strategies. This system will provide responses tailored to education domain, where individual steps and question context play a key role in solving various scenarios. The data for this project has been scraped and collected from various open-source datasets. Further, open-source foundational models within 3-9 billion parameter space like Llama 3.2, Gemma 3, Phi 3.5 Mini and DeepSeek R1 have been utilized. These models were enhanced for Chain-of-Thought reasoning using fine-tuning strategies like LoRA, reinforcement learning algorithms like DPO and GRPO and prompting strategies like Chain-of-Draft, Self-Consistency and Tree-of-Thought. The cleaning of data for model fine-tuning was orchestrated with the help of Apache Airflow using data pipeline infrastructure on AWS cloud platform. The model responses were evaluated using BLEU, ROUGE, METEOR, chrF and custom CoT g-Eval metrics like Factual Accuracy, Logical Coherence and Step Alignment. The LLM performance in the educational domain was evaluated on benchmarks like MathQA and the fine-tuned models were deployed with an integrated chat interface embedded in a web application. Llama 3.2 3B LLM fine-tuned with LoRA and DPO emerged as the best model for this system based on the benchmarks, demonstrating superior performance on metrics like Factual Accuracy and Step-Alignment. The methodology followed in this project also revealed abilities in LLMs to identify mistakes in its own reasoning and then correct them within the same response. This system enables the target audience (students, professors, and researchers) to efficiently reason and resolve complex problems within their respective domains and professions.

Keywords: Large Language Models (LLMs), Retrieval Augmented Generation (RAG), explainability, Chain of Thought (CoT), prompt engineering, fine-tuning

Introduction

Project Background and Executive Summary

Large Language Models (LLMs) have brought about a revolution in the industry by providing significant advancements in the field of Artificial Intelligence (AI). Solutions using LLM(s) are being deployed across multiple business domains, driving innovation and considerable productivity gain. The opportunities are limitless, and with the continuous enhancements and refinements of the existing LLM (s), the sky is the limit. While the existing LLMs excel with fact-checking and basic comprehension tasks (Rae et al., 2021), the benefits for complex tasks like mathematical, common sense, and logical reasoning are yet to be seen. The achievements with current LLM models are nothing short of amazing; however, the acceptability and verification of the information is a challenge that needs to be addressed with the highest priority. Gaining trust in the results provided by an LLM is an ongoing issue. Hallucinations in the context of LLM(s) are a crucial problem that makes it hard for users to trust the responses (Lee et al., 2019). The absence of proper reasoning and transparency in responses also adds to the level of mistrust. This is especially important in critical fields like Healthcare, Education, Legal, etc., where incorrect information can have drastic consequences. Therefore, the most critical challenge that we need to solve is how we, as humans, can trust the judgment of AI.

The Deep Neural Networks used by the majority of the LLM(s) are so complex and dense that they appear like black boxes even to the most experienced data scientists (Lu et al., 2022). Little is understood about how a model arrives at a particular conclusion (Lee et al., 2019) in the absence of logical reasoning and step-by-step instructions. Transparency and interpretability are crucial to achieving a widespread adoption of this technology, something that humans can rely on without any scope of doubt. If a machine can provide reasoning and justify the thought process behind a response, it would not only mimic human behavior but do so with a prominent level of trust from other humans. (Sap et al., ACL 2020) talks about how machines struggle with common sense reasoning and causal relationships, meaning they are not able to establish the causes of certain events.

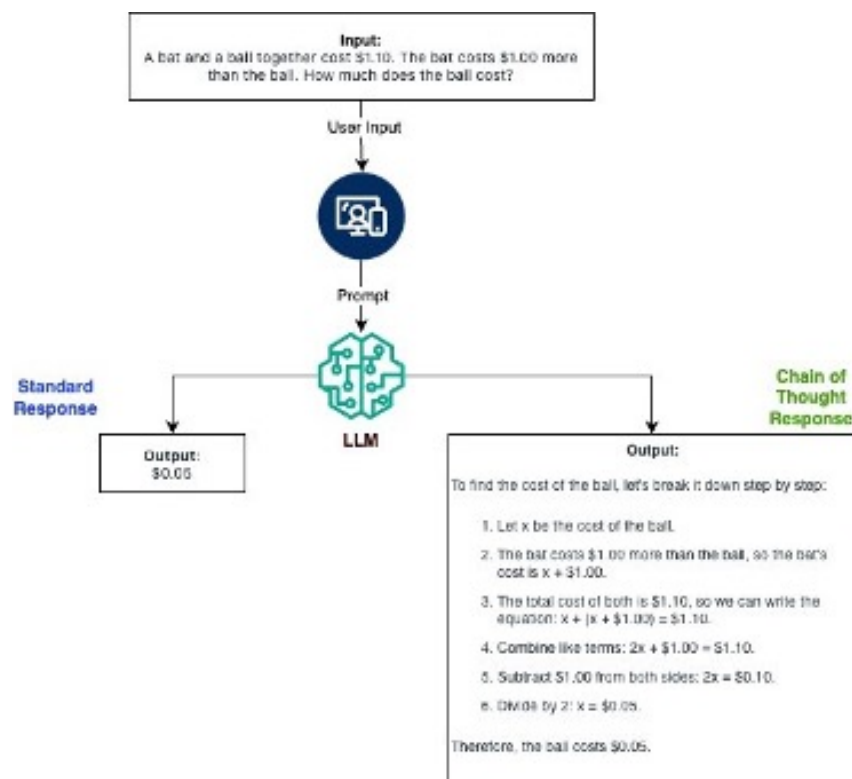
LLM(s) have become state-of-the-art, and they are continuously expanding in size and scale. This has brought about tremendous enthusiasm in the industry to improve the performance of these models in areas involving reasoning and common-sense thinking. However, scale alone has proven insufficient to achieve the desired performance on reasoning and arithmetic tasks (Wei et al., 2023). Multiple techniques have been explored over the years to improve the performance of LLM(s) like finetuning pre-trained models (Cobbe et al., 2021), in-context few shot learning via prompting (Brown et al., 2020), Multi-Task Deep Neural Network (Liu et al., 2019) and Memory Augmented Networks (Weston et al., 2015). Most of these techniques were implemented with great efficiency for question-and-answer tasks with a clear baseline using annotated data. However, they did not provide improved performance on reasoning and arithmetic problems. This is where the need to mimic human-like thinking became crucial.

Humans approach a problem by breaking down complex tasks into smaller tasks and solving each sub-task sequentially before arriving at the final answer. There is a clear chain of thoughts (CoT) when humans break down problems, and this is what makes us effective in logical thinking, common sense, and casual inference and reasoning. Chain of thought reasoning or prompting is a technique that is being explored to help LLM(s) learn to synthesize information in a structured manner, following proper logical steps and outcomes as shown in Figure 60. CoT aims to enhance the reasoning abilities of LLM(s) by encouraging step-by-step thinking, breaking down larger problems into intermediate steps, and finally arriving at a solution or response (Wei et al., 2022). This will not only enhance the interpretability of the model but also increase the trust in the responses due to a higher level of transparency. The ability to break down problems and solve the sub-parts sequentially reduces the risk of errors and the overall efficiency of the models to counter hallucinations. (Lu et al., 2022) successfully established that CoT reasoning increased the performance of the GPT-3 model by 19% when trained with a dataset of multi-modal questions with diverse topics and annotations. Similar observations were made across multiple large language models when comparing standard prompting against CoT prompting on a variety of datasets and benchmarks, 20% in some cases, which is significant (Wei et al., 2022). In

addition to this, there are techniques like Self-Consistency Training (SCT) (Wang et al., 2022) and Retrieval-Augmented Generation (RAG) (Lewis et al., 2020) that help to improve the performance of LLM(s). The exploration into the usage of these two techniques in conjunction with CoT holds a lot of significance as it can help generate a chain of thoughts augmented with external knowledge, aiding overall transparency and interpretability.

Figure 1

Example of Chain of Thought response



The aim of this project is to build an intelligent system trained on large volumes of data on specific domains that, with the aid of CoT reasoning, can break down complex problems into steps and provide a response with clear and concise steps and reasoning. We will aid our models by implementing a RAG database along with employing techniques like Self-Consistency training. The self-consistency training helps LLM(s) learn how to approach a problem by taking different paths and then choosing a response that is consistent across the majority of the paths. This, when used in parallel with CoT, helps boost the performance of the models across the trained domain (Wang et al., 2022). Similarly, RAG helps by including external knowledge

sources that are used by LLM(s) to validate or verify the generated responses, thus improving the overall accuracy. These techniques, when combined, have the potential to make existing LLM(s) efficient on specific domains, opening avenues for widespread usage across multiple domains. Our work will help the community, including students, teachers, and researchers, to get detailed solutions with multi-step reasoning for complex reasoning problems that are often plagued with hallucinations when using standard prompting with LLM(s). This will help build up trust and enable deeper explorations into this domain.

The project will follow a multi-step process for the Intelligent Tutoring system that will implement various techniques to complement the CoT reasoning. The system will undergo benchmarking and baselining and will be evaluated against multiple available models and industry-standard benchmarks with a specific domain focus. The project will need substantial amounts of annotated data for training and evaluating the models. Various publicly available data sources and repositories will be used for scraping the data for education domains. Data gathering will primarily include techniques like web scraping, APIs, and manual annotations. Data for the education domain will be sourced from resources like Khan Academy, Coursera, edX, MIT OpenCourseWare, SQUAD dataset, and various mathematical problem datasets like MathQA, GSM8K, SVAMP, Arith3K, TAL-SCQ5K-EN, MAWPS, and Math Olympiad problems. In addition to this, the project will use standard Python libraries like BeautifulSoup and Scrapy to scrape data from open data resources. Sources like Wikibooks, OER Commons, Khan Academy API, Math Stack Exchange, etc. will be used for the education domain.

Established benchmarks like MathQA (Amini et al., 2019), GSM8K (Cobbe et al., 2022), SVAMP (Patel et al., 2021), etc., will be used to evaluate the mathematical reasoning capabilities. These benchmarks provide Natural Language explanations for mathematical problems. Metrics, including BLEU score, ROGUE score, Chrf, Correctness, GEval correctness, GEval logical coherence (LLM as a Judge) etc., will be used to evaluate the overall performance.

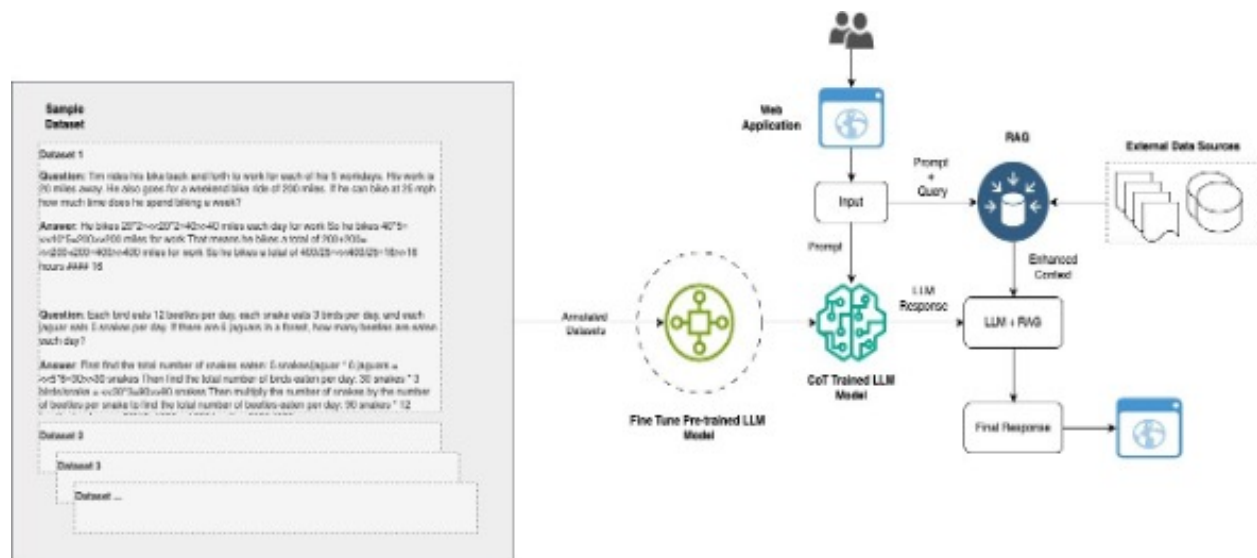
The project will use pre-trained models like Llama 3.2, Gemma2, Phi 3.5 Mini, and Mistral with around 3 - 9 billion parameters, PaLM 2-A 9B and training will include fully supervised and

self-supervised learning. Fine-tuning strategies like LoRA, QLoRA, and prompting strategies and self-consistency training will be utilized to improve the overall performance of the models.

Overall, the project will employ multiple AI techniques with a corpus of datasets and evaluation benchmarks to create a robust tutoring system with verifiable and transparent responses. The final product will feature a user interface running state-of-the-art models behind the scenes trained on large volumes of educational data. It will provide an intuitive chat application that will be capable of answering complex problems with step-by-step reasoning while establishing authenticity and accuracy. This project's contributions will be toward the advancement of the reasoning abilities of LLM(s) and help develop more effective and efficient NLP architectures. The methodologies utilized with this project will be transferrable to train the LLM(s) across other domains, thereby aiding in the overall growth of artificial intelligence, and will be a steppingstone towards Artificial general intelligence (AGI).

Project Requirements

Developing an intelligent system capable of step-by-step reasoning and an intuitive user experience involves multiple technical and functional requirements. With advancements in artificial intelligence, we have overly complex and intelligent Large Language Models (LLM) capable of performing complex tasks with relative ease. An intelligent tutoring system needs to have an inbuilt ability to perform Natural Language Processing (NLP), problem-solving & reasoning, generation of explainable answers, and learning by adapting to continuous data retrieval and generation. An end-to-end data pipeline will be needed and will be implemented using Apache Airflow utilizing data pipeline architecture on cloud platforms like AWS or GCP. This pipeline will be used for training, fine-tuning, and continuous learning, including augmentation using ChromaDB RAG. This would be wrapped up into a user-friendly UI that will allow for authentication, multi-user support, a history cache, and a RAG integrated chatbot interface as shown in Figure [2](#).

Figure 2*Overall Solution Flow*

The project will need a wide variety of datasets, including mathematics problems, common reasoning problems, arithmetic datasets with varying complexities in education domains. The data for training would need predefined annotations with step-by-step solutions to problems, and we will also need data about alternate approaches for the same problems to allow for SCT. Adding annotations manually on existing datasets would be an extremely difficult task, and we will have to primarily rely on already annotated data for our training. However, manual annotations may be needed if there is a scarcity of annotated datasets for the subject use case.

Training the LLM models on specific domains necessitates the use of AI-powered techniques to improve performance and efficiency. This project will require the usage of Chain-of-Thought (CoT) prompting, Self-Consistency Training (SCT), and various prompting techniques like Chain of draft, Tree of thought, Group Relative Policy Optimization (GRPO). Natural language generation and continuous supervised learning will aid the overall training for our models. Evaluation methods would involve a comparison between standard and CoT prompting and comparing with industry-standard benchmarks to identify efficiency gain. The final product will be tested in multiple cycles, and finally, it will be evaluated using metrics like BLEU score,

ROGUE score, Chrf, Correctness, GEval correctness, GEval logical coherence (LLM as a Judge). To establish the effectiveness of the system, we will also compare the accuracy and fluency of the responses generated by our model to standard LLMs with a similar set of parameters. This will help establish the problem-solving accuracy and reasoning capabilities of a model trained with CoT techniques against existing LLM models.

By adhering to the identified process for training and evaluation and meeting the requirements outlined, the project will be able to provide a state-of-the-art solution capable of mimicking the human way of thinking and reasoning across two major domains. This will contribute to developments in natural language processing and artificial intelligence.

Project Deliverables

This section describes the project deliverables. Starting with the project proposal that outlines several fundamental goals, as well as tasks and plans for persistent development of sophisticated models, which will yield the desired results and benefits. Consideration of this approach through the CRISP-DM framework helps in the project's progress by targeting goals from Business understanding to the deployment phase. The Work Breakdown Structure (WBS) divides the project into smaller tasks with detailed effort estimations, while Gantt and PERT charts help visualize the timeline and identify critical areas with key elements that need more focus.

The data management plan elaborates on the protocols involved in the various processes that project goes through which includes data collection, processing, analysis, and storage, thereby covering all aspects concerning the various risks. Data collection plan outlines which sources of data will be selected, which processes will be used to collect or gather data, and which methods will manage the data once it has been collected. Data engineering plan details processes of cleaning up or formatting data, validation, and transforming data to generate training, testing and validation datasets for the models.

The cloud infrastructure will be essential in the project data pipeline, after the data engineering stage of the project. Utilizing cloud platforms such as AWS, Azure, or Google Cloud,

this plan ensures scalability, efficiency, and security for managing large volumes of data throughout the project lifecycle. The cloud infrastructure will be able to store both the processed and the raw data; they will act as medium through which real-time data streams can be processed. This infrastructure is used for high-performance computing required in training and testing the models.

The model development phase is oriented towards the training of domain-specific Large Language Models (LLMs) with a higher level of reasoning capability using chain of thought reasoning for the education domain through the fine-tuning of four base models on appropriate datasets. Model evaluation is performed and evaluated on benchmark datasets which assess performance using metrics like BLEU score, ROGUE score, Chrf, Correctness, GEval correctness, GEval logical coherence (LLM as a Judge).

The goal is to implement a prototype model with a LLM capable of answering specific questions using a reasoning-based approach (Chain of Thought). This prototype will combine RAG and generate appropriate answers quickly for targeted queries with proper reasoning. In the final presentation, the project motivation, need, data analysis, model development, model validation, and future work will be explained. Also, the final report will have a complete description of the project including objectives, project management, data processing, models used and compared and teamwork. At the end of the project, a production-ready chatbot will be published, with an easy interface oriented to the education domain. It will enable the users to promptly ask the questions with a detailed explanation. A schedule of the project's deliverables and their expected completion dates is illustrated in Table [1](#).

Table 1*Table illustrating project deliverables, due date, and description*

Deliverable	Due Date	Description
Project Proposal	09/04/2024	This document describes the purpose, scope, background, research and possible solution of the proposed research problem.
Project Management Plan	09/11/2024	Using CRISP-DM methodology, this will describe the Work breakdown structure (WBS), Gantt chart and PERT chart to effectively manage the task flow, project life-cycle and deadlines.
Data Management Plan	09/18/2024	This will include the descriptive plan of data management including data handling. The underlying patterns of data will be identified.
Data Collection Plan	09/25/2024	This will outline the data collection sources, methods and management of stored data.
Data Pipeline	10/02/2024	This will illustrate the data flow pipeline in a cloud infrastructure platform describing the cloud services used for each task.
Data Engineering Plan	10/09/2024	This part will explain the data cleaning, processing and transformation steps which will be input to the LLM model.

Deliverable	Due Date	Description
Model Development	11/01/2024	In this phase, different LLM models will be trained based on reasoning with domain specific dataset.
Model Evaluation	04/10/2025	This will assess the different evaluation metrics on trained LLM models.
Final Project Report	05/09/2025	The final report will include the APA formatted document covering every single concept serving as a solid guide for future work and advancements.
Project Presentation	05/10/2025	The presentation will demonstrate the models used, results achieved and showcasing the outcomes with the help of a user-friendly chatbot interface.

Technology and Solution Survey

Zhao et al. (2024) developed the Retrieval Augmented Fine-Tuning (RAFT) method, where one pairs RAG with Chain-of-Thought prompting for improved performance in generative dialogue models to bring out better improvement in reasoning tasks. RAFT improves their performance regarding complicated reasoning-for example, through long-form QA-by incorporating knowledge from external databases and giving explicit steps of reasoning in CoT format. This work presents serious improvements on datasets like HotpotQA, PubMedQA, and DuReader, especially for multi-hop reasoning tasks. RAFT outperforms various baseline models concerning both English and Chinese datasets, enhancing robustness and logical reasoning.

Wang et al. (2024) propose Retrieval Augmented Thoughts, a method to enrich large language models' performance in the generation over long horizons (for example, code generation, mathematical reasoning, and creative writing) by joining Chain-of-Thought prompting and Retrieval-Augmented Generation. RAT iteratively updates every step of its reasoning based on external knowledge by avoiding hallucination and increasing contextual accuracy. This is then applied to models like GPT-3.5 and GPT-4. In various benchmarks, RAT achieves up to 20.94% in HumanEval and 31.37% in GSMHard. The approach hence shows ways to improve task execution, reasoning accuracy, and overall performance across many problem-solving situations.

Zhao et al. (2024) conducted a broad survey of RAG techniques for LLM improvement by adding external data to boost domain expertise and timeliness but reduce hallucination. Based on query complexity, they classified the RAG-based proposed tasks into four levels of complexity: explicit fact queries, implicit fact queries, interpretable rationale queries, and hidden rationale queries. They also discuss various advantages and possible disadvantages of the integration of external data into LLMs and consider fine-tuning, injection of context, and training of smaller models. Thus, the paper provides guidelines on how to systematically develop, for a wide range of domains, RAG-based LLM applications.

Hu et al. (2021) proposed LoRA, which is a novel approach to fine-tune LLMs for downstream tasks, avoiding the limitations of full fine-tuning. LoRA freezes the pre-trained

model weights and injects trainable rank decomposition matrices into each Transformer layer, significantly reducing the number of trainable parameters. Our experiments in various benchmarks such as GLUE, E2E NLG Challenge, WikiSQL, and SAMSum indicate that LoRA is competitive with, or outperforms full fine-tuning with a small fraction of trainable parameters, resulting in significant savings in memory usage. For example, on GPT-3 175B, LoRA uses $10,000\times$ fewer trainable parameters than fine-tuning with Adam and $3\times$ less GPU memory. Notably, LoRA outperforms full fine-tuning on three datasets-WikiSQL, MNLI-m, and SAMSum-for GPT-3 175B, which proves its effectiveness and efficiency in adapting LLMs to diverse NLP tasks. Besides, LoRA does not introduce any extra inference latency compared to fully fine-tuned models. The authors conclude that LoRA is efficient, effective, and compatible with existing techniques, making it practical and promising for adapting and deploying large language models.

Lin et al. (2023) formalize CoT prompting with references to Bayesian inference. Their paper is also filled with a strong theoretical framework of CoT that explains how task segmentation reduces error rate in reasoning. CoT is used by the authors on models such as GPT-3, and the authors demonstrate the LoR improvement in benchmarks like ScienceQA and GSM8K. They successfully show that CoT does not only enhance model performance, but also enhances interpretability of the reasoning process, thus contributing to solutions for two major problems concerning reasoning-based LLMs.

Wei et al., 2022, propose Chain-of-Thought prompting Elicits Reasoning in Large Language Models and it is a method designed to help the LLMs to produce intermediate reasoning in complex tasks. The authors incorporate CoT prompting over all the GPT-3 and PaLM models examined and have consistent enhancements in the task completion exactness metrics over the mathematical and realistic thinking tasks tested. To facilitate such reasoning, CoT makes the models reason step by step, which proves helpful in handling multilayer reasoning tasks, enhancing the overall task performance by 15% or more on benchmarks such as MathQA and CommonsenseQA.

Nye et al. (2021) in their paper suggests three elements of intermediate computation when working on reasoning tasks which refers to as scratchpads. This technique causes LLMs to write down and check partial solutions before the complete solution, and subsequently makes the reasoning accuracy better. The authors' works show that scratchpads, when integrated with CoT, improve models like T5 and GPT-3 between 15% and 25% on mathematical performances, making LLMs better in reasoning tasks than the current situation.

The Prompt Report by White et al. 2023 gives an extensive review of prompting techniques, including chain-of-thought prompting. The report points out the most commonly used LLMs in prompting research, such as GPT-3, BERT, GPT-4, PaLM, LLaMA, BLOOM, Codex, and InstructGPT. Besides that, it mentions benchmark datasets like MMLU, GSM8K, CommonsenseQA, HellaSwag, BIG-bench, and WinoGrande. The key takeaway here is that the report really highlights some of the most widely used models and datasets within the prompting research landscape. It also emphasizes how important prompt engineering is in order to maximize LLM capabilities and how that fits into the various methods of prompting, such as chain-of-thought prompting. It concludes that prompt engineering is complex but a very vital aspect of the performance of LLM and requires further research, calling for cautious and ethical deployment of LLMs.

Sprague et al. (2024) examined the effectiveness of Chain-of-Thought (CoT) prompting, found that CoT primarily benefits tasks involving math, logic, or algorithmic reasoning, particularly during the execution stage of symbolic reasoning where computation and symbolic manipulation are crucial. This conclusion is based on a meta-analysis of over 100 research papers and the authors' own evaluations of 14 language models on 20 datasets. The authors discovered that on benchmarks like MMLU and MMLU Pro, approximately 95% of the performance gain from CoT was linked to questions or generated outputs involving equations or symbolic reasoning. While CoT can lead to small improvements in tasks not focused on math or symbolic reasoning, these gains are often insignificant compared to the increased computational cost of CoT. The study further reveals that using a language model to generate a solution plan and then

employing an external symbolic solver often outperforms using CoT for both planning and execution, highlighting the limitations of language models in accurately executing complex calculations even with CoT prompting.

Xu et al. (2023) proposed RE2, a new prompting approach that asks the input question repeatedly to enhance Large Language Models' reasoning skills. The paper investigates the effectiveness of RE2 on a range of LLMs such as davinci-003, ChatGPT, LLaMA-2-13B, and LLaMA-2-70B using a wide range of reasoning benchmarks, including arithmetic, commonsense, and symbolic reasoning. The results demonstrate that RE2 consistently enhances performance over these benchmarks. For example, on arithmetic reasoning, davinci-003 with RE2 has an average improvement of 3.81% and 2.51% on commonsense reasoning. Notably, RE2 proves effective across diverse model types, including both instruction-fine-tuned and non-IFT models. The study concludes that the simple act of re-reading questions significantly enhances the reasoning capabilities of LLMs, and therefore, more emphasis should be placed on comprehending the input as one of the key factors to improve reasoning performance. Below is a comparison in Table 2 of the summary of recent research on prompting techniques, focusing on models, algorithms, and approaches used to improve reasoning capabilities in LLMs.

This reviews methods such as CoT, RE2, and Prompt Engineering; it goes further to describe how to apply them across models such as GPT-3, GPT-4, PaLM, and LLaMA-2. Different papers demonstrate various ways of enhancing reasoning, arithmetic, and symbolic tasks, noting considerable improvements on certain benchmarks and models, hence clearly illustrating how the performance of LLMs is improved by such techniques.

Table 2*Comparison of Research Papers for Technology Survey*

Author	Model	Algorithm	Approach	Best performing model
Zhao et al. (2024)	GPT-3, RAG	Retrieval Augmented Fine-Tuning (RAFT)	Combines RAG with CoT prompting to improve reasoning in dialogue models	RAFT with GPT-3
Wang et al. (2024)	GPT-3.5, GPT-4	Retrieval Augmented Thoughts (RAT)	Combines RAG with CoT to enhance generation over long horizons	RAT with GPT-3.5, GPT-4
Zhao et al. (2024)	Various LLMs (GPT-3, etc.)	Retrieval- Augmented Generation (RAG)	Survey and classification of RAG-based tasks with external data	RAG across multiple LLMs
Hu et al. (2021)	GPT-3 175B, T5	Low-Rank Adaptation (LoRA)	Adapts large models with fewer parameters through trainable rank decomposition	LoRA with GPT-3 175B
Ding et al. (2023)	GPT-3.5	Zero-shot, Few-shot	Learning Physics problem-solving using PhysQA dataset with LLMs	GPT-3.5 with few-shot

Author	Model	Algorithm	Approach	Best performing model
Lin et al. (2023)	GPT-3, PaLM	CoT with Bayesian Inference	Formalizes CoT prompting based on Bayesian segmentation for reasoning	CoT on GPT-3
Wei et al. (2022)	GPT-3, PaLM	Chain-of-Thought (CoT) Prompting	Enhances reasoning tasks with intermediate CoT steps	CoT on GPT-3, PaLM
Nye et al. (2021)	T5, GPT-3	Scratchpads	Intermediate computation for improving reasoning accuracy	Scratchpads with GPT-3, T5
White et al. (2023)	GPT-3, GPT-4, BERT, PaLM, Codex	Chain-of-Thought (CoT) Prompting, Prompt Engineering	Comprehensive analysis of prompting techniques with focus on CoT	CoT on GPT-3, PaLM
Sprague et al. (2024)	14 LLMs (GPT-3, GPT-4, etc.)	Chain-of-Thought (CoT) Prompting	Evaluate CoT's effectiveness in math, logic, and symbolic reasoning	CoT with symbolic solver

Author	Model	Algorithm	Approach	Best performing model
Xu et al. (2023)	Davinci-003, ChatGPT, LLaMA-2	RE2 Prompting	Repeats input question to enhance reasoning performance across benchmarks	RE2 with Davinci-003

Literature Survey

Building on this basis, Wang et al. put forward, in 2023, a new decoding strategy called self-consistency, whose powerful reasoning function of Language models is exercised in an unusually enhanced way for such complex tasks. Herein, self-consistency samples a set of diverse reasoning paths different from traditional greedy decoding and chooses the most consistent answer. This approach outperforms chain-of-thought prompting by a wide margin, such that results for various benchmarks-such as GSM8K, SVAMP, and AQuA-in turn demonstrate accuracy gains of as much as 17.9%. No additional training or new annotation is required, and it works out of the box for most standard large language models, including GPT-3 and PaLM.

Wang et al. (2023) perform an empirical study of the extent to which Chain-of-Thought prompting-based multistep reasoning works when large language models are utilized. CoT prompting tends to elicit from the model its explicit generation of steps toward the answer. Indeed, through a set of ablation studies, the authors prove that the validity of the steps of reasoning behind CoT demonstrations counts less than previously assumed. Even with incorrect reasoning, LLMs give 80-90% performance. The relevance with the question and the proper ordering of the steps in the reasoning go way more ahead as the success key for CoT rather than correctness.

Xu et al. (2023) introduce the development of the so-called Symbolic Chain-of-Thought

(SymbCoT), an architecture raising symbolic expression within Chain-of-Thought prompting for better performance of large language models w.r.t. logical reasoning. SymbCoT translates natural language context into symbolic forms, derives the solution using logical rules, and thus includes a verifier that makes sure the reasoning chain is faithful. SymbCoT outperformed current state-of-the-art CoT methods by a huge margin on five publicly available datasets, especially those requiring FOL and Constraint Optimization. This constitutes an advance in the state of the art in terms of high performance, flexibility, and interpretability in logical reasoning with LLMs.

Reppert et al. (2023) present Iterated Decomposition as a human-in-the-loop workflow for performance improvement of LLMs on reasoning tasks through supervision, extending both to the outcome and the process of reasoning itself. This refinement features complex task breakdown into smaller subtasks, with the evaluation going in an iterative way. They applied this on different activities: scientific question-answering, classification of placebo, and participant flow in randomized controlled trials, where improvements in accuracy had been evidenced, including those such as 25% to 65% regarding placebo description and 53% to 70% concerning participant adherence. This framework provides the base needed toward interpretability and reliability regarding such complex reasoning tasks in wide ranges of domains.

Zhou et al. (2022) go one step further to present a novel study with which LLMs can themselves act as "human-level prompt engineers" by optimizing prompts for better performances on various tasks. The authors investigated how LLMs, after exposure to diverse structures and configurations of prompts, would identify the most effective prompts for different problem-solving scenarios. They showed that through extensive experimentation on diverse domains, LLMs may self-develop and refine the strategy for eliciting high performance in themselves, without human intervention. The self-improvement in this respect by LLMs is underlined, which may act toward actively improving their problem-solving capability with less need for extensive human-guided prompt engineering.

Amini et al. (2019) introduce MathQA, a comprehensive dataset aimed at facilitating interpretable math word problem-solving using operation-based formalisms. The MathQA dataset

contains a wide range of math word problems accompanied by detailed solutions, which enables LLMs to understand and generate step-by-step solutions to complex mathematical questions. By utilizing an operation-based formalism, the authors were able to enhance the interpretability of the model's reasoning process. This approach significantly improved the model's performance in solving math problems, as it allowed the LLMs to break down complex questions into manageable steps, leading to more accurate and interpretable solutions. This study highlights the potential of operation-based reasoning in improving LLMs' capabilities in quantitative problem-solving tasks.

Colombo, A. (2024) investigated how combining knowledge graphs with LLMs can effectively support and monitor legislative systems. Their approach leverages the power of knowledge graphs to provide structured data, which, when combined with the natural language capabilities of LLMs, aids in analyzing, drafting, and interpreting legislative documents. This hybrid method enhances the understanding and contextualization of legal provisions, enabling LLMs to generate more accurate and coherent legislative summaries. The paper demonstrates that using knowledge graphs as an auxiliary data source not only improves the consistency of LLM-generated interpretations but also significantly reduces the likelihood of hallucinations, making it a valuable tool for legislative monitoring and analysis.

The StaR method or "Self-Taught Reasoner" as a concept was introduced in the paper published by Zelikman et al. (2022). It aims to aid with the enhancement of the LLM model's reasoning capability with the StaR method. It is well established that to enhance the capabilities of LLM models on complex tasks like mathematics or reasoning, a step-by-step "chain-of-thought" context generation is needed. This necessitates training with large amounts of the diverse and labeled dataset, which is pre-annotated with "chain-of-thought" context or step-by-step explanation. However, creating large amounts of annotated data is a huge undertaking, and it is not always feasible, given the time required to prepare. The paper proposes the technique of training with substantial amounts of data; however, it allows for a small subset of the data to have proper CoT context or rationales. The approach is to loop the training process by generating rationales for the dataset by using the small subset and then iteratively using the

generated rationales on the rest of the dataset. If the generated context is not accurate, the process repeats to identify an accurate or valid rationale and use that to train the rest of the data. Using the StaR process, the authors were able to achieve a significant improvement in reasoning performance as compared to models fine-tuned without the StaR process. The paper establishes that the models improve significantly if allowed to learn using self-generated reasoning iteratively. However, it also identifies limitations with models that are not large enough to learn quickly using the StaR technique. With smaller models, the generated rationales may not be accurate enough for the model to learn quickly, and there is no effortless way to filter the inaccurate rationales.

Kim et al. (2023) focused their research on smaller language models. The aim was to equip models having less than 100B parameters with reasoning capabilities by generating step-by-step explanations for complex use cases. The idea was to achieve this by instruction tuning with CoT rationales. The research was focused on using Google's FLAN (Fine-tuned Language Net) collection and FLAN-T5 model with 3B and 11B parameters. The paper talks about augmenting the existing FLAN collection, having nine tasks with a CoT collection of 1.84 million rationales and 1060 tasks. By fine-tuning the FLAN-T5 models with the CoT collection, a considerable improvement in reasoning capabilities was observed in the relatively smaller language models on unseen tasks. An improvement of 4.34% on the FLAN-T5-3B and 2.6% on the FLAN-T5-11B was observed in the zero-shot accuracy using the Big-Bench-Hard (BBH) benchmark. The paper established that tuning with CoT Collections improves the model's few-shot learning capability. The FLAN-T5-3B and FLAN-T5-11B outperformed ChatGPT by 2.24% for the 3B parameters model and 2.37% for the 11B parameters model. The significance of this paper is in the fact that smaller language models can perform well on complex reasoning tasks when tuned with the CoT collection method. This is an extremely important finding for use cases that are budget-constrained, as training and maintaining large models with 100B or more parameters is resource and cost-intensive.

Xia et al. (2024) talked about how the Retrieval-Augmented Language Model (RALM) can often lead to the problem of reliability and traceability during implementation, as irrelevant

document retrieval often results in unhelpful response generation. Also, the trustworthiness of the models cannot be ascertained due to the lack of proper citations in these generated outputs. To counteract this reliability and traceability problem, they proposed a self-reasoning framework whose idea would be to utilize reasoning trajectories generated by LLM. Self-reason trajectories were constructed with three processes: a relevance-aware process, an evidence-aware selective process, and a trajectory analysis process. This framework was also evaluated across QA datasets, one long-form QA dataset, and one fact verification dataset and achieved a comparable performance with GPT-4.

Huang et al. (2022) explored the possibility of LLM being able to self-improve their reasoning abilities just like a human would do through self-thinking. In this paper, the idea has been implemented by selecting any pre-trained language model and using special prompts to encourage the model to think step-by-step and verify its answers through self-consistency. This way, the model will generate confident answers with explanations and would further fine-tune the LLM using these self-generated answers as examples. The reasoning ability of LLM with 540 billion parameters was shown to improve on the data that was not human labeled. They tested this methodology on GSM8K, DROP, OpenBookQA, and ANLI-A3 and showed improved accuracy by 5% increase. Their research proved that LLM can become intelligent and capable even without requiring massive amounts of labeled data.

Large language models require better logical reasoning where Smith et al. (2023) introduced hybrid reasoning by incorporating analogical reasoning into large language models. The framework compiles external structured knowledge from knowledge repositories with the model inference pipeline which supports LLMs to provide sophisticated and contextually sound analogy for augmented reasoning. The authors tested the performance of their proposed framework on measures such as CLUTRR with enhancement of accuracy by 12%. The integration of details in the external environment and LLMs for enhancement in on-line performance in practical reasoning is also explained in the study. This combination forms a hybrid framework that will help in establishing a proper way of using LLMs especially in the field of education and cognitive

sciences today. Lee et al. (2023) proposed and tested the use of rule-based systems and LLMs to meet regulatory compliance issues. Their work introduces rules' form into the model's inference stage and enables LLMs to interpret regulatory requirements more effectively. When measured across the datasets primarily based on compliance-related QA, including RegQA, the method described showed a 15% increase in precision. The paper highlights the issue of smearing structured rule-based logic into LLMs to tackle domain issues particularly where clients demand rigorous compliance such the healthcare, finance and legal segments.

Chen et al. (2024) also proposed a context enhanced CoT reasoning which produces reasoning relevant contextual embeddings in straight pass. This dynamic embedding generation process makes LLMs comprehend and also answer such ambiguous or complex questions appropriately. The proposed model was evaluated against benchmark contextual reasoning tasks like ReClor and LogiQA or GPT-4 and yielded an 8% better performance. The technique successfully combines the power of contextual information with linguistic analysis and translation/interpretation for reasoning tasks that don't require other types of annotations or model training.

CoT with LLMs was also used by Patel et al. (2023) to study multilingual reasoning with LLMs fine-tuned with CoT prompts in multiple languages. To this end, the study developed numerous multilingual CoT examples through parallel corpora construction that helped LLMs deliver better interlingual reasoning. As we observed in the evaluation of XQuAD and MLQA datasets, we achieved an average accuracy gain of 12%. Therefore, research is invaluable when it comes to implementing functions all over the world where regular multilingual reconciliation is compulsory such as in policymaking, education, and health facilities among others.

Das et al. (2024) proposed an approach for enhancing the reasoning of LLMs using reinforcement learning. It includes a feedback scheme of human annotations to the proposed framework for continuous maximization of the model's reasoning strategy. Implementation, however resulted in a 10% performance enhancement on other datasets such as ARC, CommonsenseQA, and OpenBookQA. Regarding the adaptative change of the framework and its

value added for enhancing LLM reasoning, the authors emphasized that this framework is suitable for a solution for purposes such as tutoring systems and adaptive learning.

Sharma et al. (2023) aim to investigate domain-specialized improvements in CoT reasoning; thus, they proposed integrating such domain-specialized ontologies into the LLMs' training processes. In doing so, the authors demonstrate that this could significantly affect improving the thinking of LLMs in such niche domains as medicine and law. Indeed, this is already so with the MedQA and PubMedQA datasets-the approach boosted accuracy as much as 18 percent. This research, therefore, illustrates how ontological knowledge may be harnessed in order to enhance the capability of LLMs with respect to performing domain-specific logical jobs.

GA-CoT was proposed by Gao et al. (2023) for reasoning tasks, in which GNNs are adopted to incorporate other types of link information. Incorporating GNNs into the reasoning process the authors improved benchmarks, which require multi-hop reasoning, by 9 percent, such as HotpotQA and Wikihop. The work shows the best practice of using graph-based representations to enhance the reasoning mechanisms of LLMs.

Singh et al. (2023) introduced prompt engineering methodologies for improving CoT reasoning on different tasks. It became evident from the research that both the content and format of the prompts could be used to significantly improve performance especially if the former was made consistent with the latter. The study demonstrated up to 20% improvement in accuracy on the considered benchmark of multi-step reasoning: GSM8K and Aqua. This work emphasizes the practicality of timely adaptation as a practical and inexpensive way of improving LLM reasoning.

Park et al. (2024) proposed a GAN based framework to synthesize CoT rationales for training LLMs. The generated synthetic reasons thereby offered various plausible explanations which enhanced the model's inclusive reasoning functionality. On datasets such as DROP and ArithmeticQA, the proposed GAN-augmented LLMs demonstrate the best results. To our knowledge, this is one of the few ways to perform scalable reasoning tasks when there is a dearth of human-annotated data.

CoT prompts are discussed by Kumar et al. (2023) in terms of fine-grained token alignment

towards the improvement of reasoning accuracy. Through adjusting the matching matrix of tokens in between input and reasoning steps the authors were able to achieve up to 7% enhanced efficiency on symbolic reasoning datasets such as Math23K and AddSub. The work captures the significance of token-level alignment fundamental to enhancing the abilities of LLMs at performing and solving reasoning issues efficiently.

Li et al. (2024) suggested a visualization tool that shows the methodological line of thinking of LLMs as a way to increase interpretability of results. It allows the user to look up the decision-making paths within the model and understand what choices they made. The enhancement of trust and accuracy in interactive QA datasets was significant for applying this method to tasks that need more interpretability, such as education and healthcare.

Updated on March 22 by Zhang et al. (2023), the current study proposed a curriculum learning method to train LLMs for CoT reasoning tasks. This is because the curriculum adopted involves solving basic reasoning problems from the initial weeks and building up to the more complex ones. This based incremental training strategy has produced a 5% increase in benchmark accuracy on datasets such as GSM8K and Aqua. This study thus shows the improvement of the LLM's reasoning ability using the proposed curriculum learning.

Chen and Lin (2023) studied the cooperation of several LLMs by providing the reasoning paths of various of them. This cooperative approach enhanced uniformity in the reasoning tasks owing to the improved credibility of the reasoning process. MultiRC and HotpotQA models indicated excellent improvement and affirmed the prospects of multi-agent rationale structures.

In CoT reasoning, Mehta et al. (2023) investigated adversarial training methods that enhance the robustness of LLMs. Thus, by adding adversarial perturbations at the time of learning, the models were trained to become robust to noise or multiple interpretations. On datasets such as ARC, and commonsenseQA improvements in performance were attained thus emphasizing the need for models to have real-world reasoning ability.

Wang et al. (2024) introduced DM-CoT, a new algorithm that leverages supplement memory modules to store the reasoning process parameters. On the other hand, this dynamic memory

mechanism is used to recall and re-reason intermediate steps throughout a series of tasks. In datasets of temporal reasoning such as TimeQA, significant enhancements were obtained, further supporting the incorporation of memory components in CoT reasoning.

Huang et al. (2023) have created the pre-trained reasoning templates to kick-start the CoT reasoning support in LLMs. These templates helped in organizing the thinking structure which in turn was beneficial for the achievement of a favorable performance effective on the datasets, for example; StrategyQA, TruthfulQA. The study showed the possibility of using priori templates to direct LLM reasoning when it is applied in environments that have few training data.

More recently, Zhao et al. (2023) described how they distil CoT reasoning capabilities from larger LLMs to smaller ones using the knowledge distillation method. This approach helped smaller models perform nearly as well as larger models while using considerably less cost and resources. The method turned out to be quite efficient on datasets such as BoolQ and Fever.

To that end, Tan et al. (2024) studied multimodal CoT reasoning, which involved integrating text-based reasoning and vision. The authors showed that it is possible to integrate the information from both modalities in LLMs performing 10% better than the benchmark tasks such as VQA and GQA.

Liu et al. (2024) proposed an unsupervised CoT prompt tuning approach to rebuild two major aspects: the capability of LLMs to realise and utilise better reasoning methods autonomously. This self-tuning strategy, enabled by the present work, enhanced reasoning results on reasonable datasets such as the CommonsenseQA and SocialIQA, proving the effectiveness of the unsupervised strategies in CoT reasoning.

Therefore, Cheng and his team (2023) applied interface conversion to combine external SLSs as additional components of CoT reasoning pipelines for improving logical reasoning. The work obtained good accuracy improvement on the corpora such as ProofWriter where formal logic is used, thus reflecting the advantage of integrating symbolic as well as neural based reasoning frameworks.

In turn, Gupta et al. (2023) also enhanced LLM response to uncertainty within CoT

reasoning using probabilistic reasoning. While the theoretical integrations of probabilistic models into the reasoning process have enhanced reliability and interpretability of the corresponding results in the datasets such as BoolQ and Fever.

Yang et al. (2022) also described the architecture of a hierarchical reasoning framework; this is through decomposition of CoT tasks into smaller subtasks. It also leads to better task decomposition, help writing the model to finish more questions from the multi-hop reasoning datasets such as HotpotQA and Wikihop.

To overcome the CoT reasoning challenges in low resource languages, Hassan et al. (2024) proposed techniques for using transfer learning. It showed competitive performance on multilingual benchmarks and highlighted the need to consider language in reasoning tasks. Nguyen et al. (2023) came up with a feedback integrity CoT reasoning that integrates the user feedback into the reasoning process of the model. The feedback loop that was applied to the system was effective in enhancing the level of satisfaction among the users while solving the interactive QA tasks and further enriched the role of human feedback in the improvement of the reasoning component of the LLM.

Table 3 provides a concise comparison of various research papers focused on enhancing Large Language Models (LLMs) through techniques such as Chain-of-Thought (CoT) prompting, self-consistency, symbolic reasoning, and human-in-the-loop workflows. It highlights the models used, evaluation methods, key techniques, and results, offering insights into the effectiveness of these approaches in improving reasoning performance, handling complex tasks, and addressing limitations like hallucinations and accuracy gaps. The papers collectively contribute to advancing the field of LLM reasoning across different domains, including mathematics, logic, analysis, and scientific reasoning.

Table 3
Comparison of Literature Survey

Author	Paper	Models Used	Evaluation Method	Techniques	Results
Hou et al. (2024)	Gaps or Hallucinations? Gazing into Machine- Generated Analysis for Fine-grained Text Evaluations	GPT-based LLMs	Qualitative Analysis	Evaluation of LLM outputs	Identified gaps/hallucinations in LLM-generated analysis
Colombo, A. (2024)	Leveraging Knowledge Graphs and LLMs to Support and Monitor Legislative Systems	LLMs with Knowledge Graphs	Case Study Evaluation	Integration of knowledge graphs	Improved legislative interpretation and reduced hallucinations

Author	Paper	Models Used	Evaluation Method	Techniques	Results
Amini et al. (2019)	MathQA: Towards Interpretable Math Word Problem Solving with Operation-Based Formalisms	LLMs with Operation Formalism	Dataset- based Evaluation	Operation- based reasoning	Enhanced interpretability and accuracy in math problem-solving
Zhou et al. (2022)	Large Language Models are human level prompt engineers	LLMs (GPT-3)	Experimental Studies	Adaptive Prompting Strategies	Demonstrated that LLMs can optimize prompts autonomously
Kim et al. (2023)	Instruction Tuning with Chain-of- Thought Rationales for Smaller Language Models	FLAN-T5- 3B, FLAN-T5- 11B	Big-Bench- Hard (BBH) Benchmark	Instruction tuning with CoT rationales	Improved zero-shot accuracy by 4.34% (3B) and 2.6% (11B); Outperformed ChatGPT by 2.24% (3B) and 2.37% (11B)

Author	Paper	Models Used	Evaluation Method	Techniques	Results
Xia et al. (2024)	Self-Reasoning Framework for Enhancing Reliability and Traceability in RALMs	Retrieval- Augmented Language Model (RALM)	QA datasets, long-form QA, fact verification datasets	Self- reasoning framework (relevance- aware, evidence- aware, trajectory analysis)	Achieved performance comparable to GPT-4 on QA datasets
Zelikman et al. (2022)	STAR: Bootstrapping reasoning with reasoning	GPT-3, BERT, other LLMs	Benchmarks for reasoning tasks like GSM8K	Iterative CoT reasoning generation from a small subset to larger dataset	Significant reasoning improvement, smaller models struggled with inaccurate rationales.
Huang et al. (2022)	Self-Consistency in LLMs for Reasoning Improvement	LLM (540B params)	GSM8K, DROP, Open- BookQA, ANLI-A3	Step-by- step reasoning with self- generated answers	5% accuracy improvement on non-labeled datasets, proving LLMs can self-improve.

Author	Paper	Models Used	Evaluation Method	Techniques	Results
Wang et al. (2023)	Empirical Study on CoT Prompting	LLMs	Multistep reasoning, ablation studies	CoT prompting focusing on step relevance and ordering	80-90% performance even with incorrect reasoning, highlighting relevance over correctness.
Xu et al. (2023)	Symbolic Chain- of-Thought (SymbCoT)	LLMs with SymbCoT	Logical reasoning bench- marks (FOL, etc.)	Natural language to symbolic forms with chain validity verifier	Outperformed state-of-the-art in logical reasoning, offering better performance and flexibility.
Reppert et al. (2023)	Iterated Decomposition for LLMs	LLMs with human-in- the-loop	Scientific QA, placebo, participant tasks	Task decomposi- tion with human feedback for reasoning improve- ment	Accuracy improved 25-65% for placebo and 53-70% for participant tasks.

Author	Paper	Models Used	Evaluation Method	Techniques	Results
Smith et al. (2023)	Hybrid reasoning framework combining analogical reasoning with LLMs	GPT-3, PaLM	CLUTRR	External knowledge integration	12% improvement in analogy-based reasoning accuracy
Chen et al. (2024)	Context- enhanced CoT reasoning using dynamic embeddings	PT-4, PaLM	ReClor, LogiQA	Contextual embedding generation	8% improvement in contextual reasoning benchmarks
Patel et al. (2023)	Multilingual CoT reasoning for cross-lingual tasks	GPT-3, BERT	XQuAD, MLQA	Fine-tuning with multi- lingual CoT examples	12% accuracy gain in multilingual reasoning tasks
Das et al. (2024)	Reinforcement learning for refining reasoning capabilities	GPT-4, Custom LLM	ARC, Common- senseQA	Feedback loops with human- guided refinement	10% improvement in reasoning performance

Author	Paper	Models Used	Evaluation Method	Techniques	Results
Sharma et al. (2023)	Domain-specific ontologies for enhancing reasoning	GPT-3, PubMed-BERT	MedQA, Pub-MedQA	Integration of domain-specific ontologies	18% improvement in accuracy for medical QA tasks
Gao et al. (2023)	Graph-Augmented CoT reasoning	GPT-3, GNN-Augmented LLM	HotpotQA, Wikihop	Graph neural networks integration	9% improvement in multi-hop reasoning
Singh et al. (2023)	Task-specific prompt engineering techniques for optimized reasoning	GPT-3, PaLM	GSM8K, Aqua	Customization of prompts based on task complexity	Up to 20% accuracy increase on multi-step reasoning tasks
Park et al. (2024)	GAN-augmented CoT rationale generation	GPT-4, Custom GAN	DROP, Arith-meticQA	GAN-generated rationales for synthetic data	State-of-the-art performance on arithmetic reasoning benchmarks

Author	Paper	Models Used	Evaluation Method	Techniques	Results
Kumar et al. (2023)	Fine-grained token alignment for enhanced symbolic reasoning	GPT-3	Math23K, AddSub	Optimizing token alignment for symbolic reasoning	7% accuracy gain in symbolic reasoning tasks
Li et al. (2024)	Visualizing reasoning paths for better interpretability	GPT-4, Custom LLM	Custom QA Datasets	Interactive tools to track reasoning processes	Improved interpretability and user trust
Zhang et al. (2023)	Curriculum learning for enhancing CoT reasoning	GPT-3, PaLM	GSM8K, Aqua	Incremental learning from simple to complex reasoning tasks	5% improvement in CoT reasoning benchmarks
Chen and Lin (2023)	Collaborative reasoning frameworks for multi-agent LLMs	GPT-3	MultiRC, HotpotQA	Sharing reasoning paths between LLMs	Improved consistency and accuracy

Author	Paper	Models Used	Evaluation Method	Techniques	Results
Mehta et al. (2023)	Adversarial training for improving robustness in reasoning tasks	GPT-3, PaLM	ARC, Common-senseQA	Training with adversarial perturbations	Enhanced robustness to noisy or ambiguous inputs
Wang et al. (2024)	Dynamic memory modules for CoT reasoning	GPT-4	TimeQA	Incorporating memory mechanisms	Improved temporal reasoning accuracy
Huang et al. (2023)	Bootstrapping chain-of-thought reasoning with pre-trained templates	GPT-3, PaLM	StrategyQA, TruthfulQA	Pre-trained reasoning templates	8% improvement in reasoning on low-resource settings
Zhao et al. (2023)	Knowledge distillation for efficient reasoning in small language models	Custom LLMs	BoolQ, Fever	Transferring reasoning capabilities from large to smaller LLMs	Significant cost savings with comparable performance

Author	Paper	Models Used	Evaluation Method	Techniques	Results
Tan et al. (2024)	Multimodal CoT reasoning combining visual and text-based inputs	GPT-4	VQA, GQA	Integration of multimodal reasoning	10% accuracy gain in multimodal benchmarks
Liu et al. (2024)	Unsupervised CoT prompt tuning for enhanced reasoning	GPT-3	Common senseQA, SocialIQA	Self-optimization of reasoning strategies	Improved performance on reasoning benchmarks
Cheng et al. (2023)	Integrating symbolic solvers into CoT reasoning pipelines	GPT-3	ProofWriter	Combining symbolic and neural reasoning	Significant accuracy gains in formal logic tasks
Gupta et al. (2023)	Probabilistic reasoning frameworks for handling uncertainty	GPT-3, PaLM	BoolQ, Fever	Incorporating probabilistic models	Better reliability and interpretability
Yang et al. (2024)	Hierarchical reasoning frameworks for nested CoT tasks	GPT-3	HotpotQA, Wikihop	Decomposing tasks into nested subtasks	Improved multi-hop reasoning performance

Author	Paper	Models Used	Evaluation Method	Techniques	Results
Hassan et al. (2024)	Addressing CoT reasoning challenges in low-resource languages	GPT-3, BERT	Multilingual QA Datasets	Transfer learning for linguistic diversity	Significant improvements in multilingual reasoning benchmarks
Nguyen et al. (2023)	Feedback-driven iterative refinement of reasoning in LLMs	GPT-4, Custom LLM 7	Custom Interactive QA Datasets	Incorporating user feedback into reasoning loops	Improved user satisfaction and reasoning accuracy

Data and Project Management Plan

Data Management Plan

Large language models available publicly are designed and trained to cover a lot of information about a variety of topics. Hence, they tend to provide general solutions to domain specific tasks. To adapt LLM's for domain specific tasks, high quality datasets are required to either fine-tune the LLM parameters for the domain or provide effective examples in the task description itself to guide the LLM responses. For this project, various datasets have been collected for education domain. A detailed data management plan is described below.

Data Collection Approaches

HuggingFace provides a diverse collection of open-source datasets in the educational domain. Many Large Language Models have been trained and evaluated using these datasets and even benchmarked against these datasets. As such, there are many data sources on HuggingFace

that are structured as a collection of questions and answers in various file formats. A key constraint for this project is that the dataset should also contain the intermediate steps or reasoning required to get the correct answer, since this will ensure that the responses generated by the LLM will also contain explanations and reasoning to answer queries. In some cases, the data required for this project needs to be scraped and extracted from online forums, websites and public institution websites which regularly publish trustworthy data specific to the domain. Based on this constraint, a selection of data sets and sources have been identified in the education domain datasets like OpenBookQA (Mihaylov et al., 2018), GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021). AoPS Online (AIME Problems and Solutions 2024) is a website that contains solved solutions for a subset of problems included in mathematics Olympiads since 1983. These questions and multiple solutions for each problem are scraped and collected as a dataset for training the Large Language Models. A detailed description of each data set is presented in Table 4.

Table 4

Table showing dataset description

Dataset Name	Domain	Split	No of Samples	No of Features	Format
OpenBookQA	Education	Train	4957	8	CSV
OpenBookQA	Education	Validation	500	8	CSV
OpenBookQA	Education	test	500	8	CSV
GSM8K	Education	Train	7473	2	CSV
GSM8K	Education	Test	1319	2	CSV
MATH	Education	Train	7500	4	JSON
MATH	Education	Test	5000	4	JSON

Data Management Methods

A data engineering pipeline is required to properly transform data from multiple data sources into a usual format for experimentation with LLMs. The combined raw dataset contains files in various formats like CSV, JSON, JSONL, TXT, PDF, Markdown, HTML. If the dataset already has a predefined train-validation-test split, then these partitions will be honored, and the data processing will ensure that there is no data leakage between the partitions. The data processing pipeline consists of individual pipelines for each data source to ensure reliable and repeatable data transformations. An orchestration tool is used to automate the steps of the pipeline and scheduling of jobs. A management strategy is required to organize the datasets as it is processed through various stages of the data engineering pipeline as shown in Figure 3. The datasets will be collected locally on one of the workstations before it will be collectively uploaded to the Cloud storage for processing. As part of data processing, each dataset will be cleaned to remove inconsistent entries. For structured datasets, feature engineering will include transformations to conform the data to target schema. For unstructured datasets, like PDFs, text files and Markdown files, a few additional cleaning and transformation steps will be required. This includes splitting the data into individual lines, identifying figures and equations correctly, and conforming the unstructured data into a structured format. Once in the structured format, standard data processing steps like other datasets will ensure the correct data schema in the Data Warehouse. The schema is shown in Table 5 and Table 6. The Vector database will be populated with the data from the Data Warehouse and will be running in the same deployment environment as the LLM that is being evaluated or fine-tuned. The flow of data in the Application and LLM is described in the Application Workflow of Figure 3. First, the user makes a query to the application. The application then communicates with the vector database to retrieve relevant examples for the user query. The examples and user query are then combined to create a prompt for the LLM. The application receives the LLM prompt and passes the response back to the User.

Figure 3
Data Management Workflow

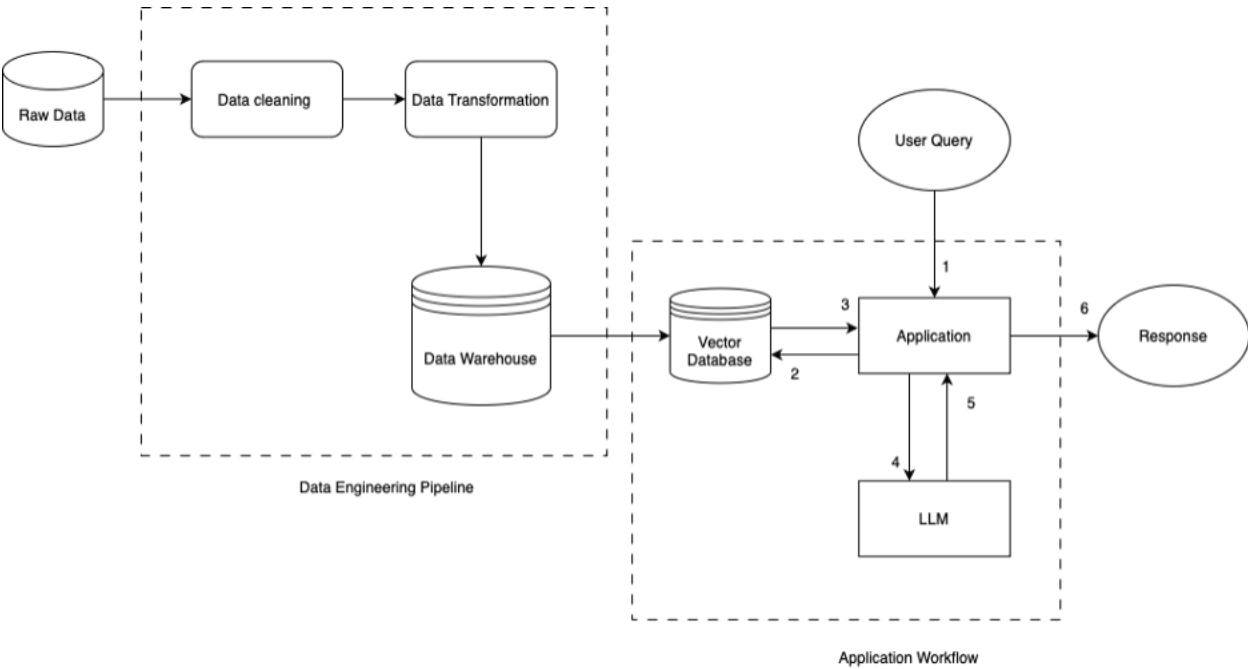


Table 5
Dataset Table

Feature Name	Feature Description	Datatype
dataset_id	A unique ID of the dataset this question-answer belongs to	Integer
name	Name of the dataset	Text
description	Dataset description	Text

Table 6*Questions Table*

Feature Name	Feature Description	Datatype
dataset_id	A unique ID of the dataset this question- answer belongs to	Integer
question_id	A unique ID associated with the question- answer-reasoning triplet	Integer
question_text	The question text that needs to be asked to the LLM	Text
answer	The correct response to this question	Text
reasoning	The correct set of steps/reasoning to solve this question	Text

Data Storage Methods

The data engineering pipeline consists of 3 sections – source, staging and target. All the three sections require a data storage mechanism to ensure the data pipelines work smoothly. Under the source section, raw datasets will be uploaded to AWS S3 bucket for quick access. The files in this bucket will be quite varied, including file formats like CSV, JSON, JSONL, PDF, HTML, plaintext, and Markdown. Under the staging section, data in various stages of transformation in the pipeline will be persisted in a separate AWS S3 bucket. Here the staging data will most probably be stored in parquet format to ensure consistency with AWS Glue jobs. Under the target section, the cleaned and transformed data will be persisted inside a warehouse like AWS RDS database. There is an additional target database, a ChromaDB vector database, which will contain a copy of the same data from AWS RDS, but it will be indexed and structured for Retrieval Augmented Generation (RAG) of text for few-shot experiments on the LLMs.

Data Usage Mechanisms

The data from source and staging buckets will primarily be used inside the data engineering pipeline. The data from the target RDS database will be used to provide additional analysis of the fully transformed and merged data, as well as act as the source for the vector database. Across all the stages of the data processing pipeline, training, validation, and test splits from individual datasets will be processed in separate partitions to ensure there is no data leakage. Train dataset splits will be used for fine-tuning the model and will also be loaded into the vector database to provide relevant examples for a few short experiments. Validation dataset splits will be used to evaluate the fine-tuning process and the few shot experiments. This dataset split will help in evaluating the LLM models and comparing their performance across various tasks. Test data splits will be used to benchmark the performance of fine-tuned LLM and LLMs under a few-shot experiments.

Project Development Methodology

A project development methodology is a set of guiding principles a team uses to plan, execute, and manage a successful project. In this project CRISP-DM methodology is used. According to Schröer et al., (2021), CRISP-DM (Cross-Industry Standard Process for Data Mining) is a standard and an industry-independent process model for managing data mining projects. CRISP-DM methodology used for this project involves six phases: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment. Below is the description of each phase of the project.

The first phase of CRISP-DM methodology is business understanding, where business problems are identified, studied and analyzed, and business objectives are formulated. The business objective for this project is to extract explainable responses from LLM and improve the response quality by using Chain-of-Thought prompting strategy. The project will be assessed by checking what needs to be done in terms of resources required to fine-tune LLM, assumptions associated with LLM development, constraints of the project, associated terminologies and cost-benefit analysis. A literature and technical review will be conducted on existing LLM based

solutions to understand current best practices for LLM usage in education domains. A project plan is produced describing the specific project stages, the duration of each stage, resources needed at those stages, and the costs, and an initial assessment of the tools and techniques to be used in the development of an LLM based application.

The second phase is Data Understanding which involves collecting and analyzing the initial textual data from various sources like databases, files, and websites. Then, the data is described by examining the properties of the textual data acquired and building a description report regarding the format, quantity, and metadata of each data source. Then exploratory data analysis is conducted to understand the patterns by querying, visualizing, and generating summary reports on each data source. The data quality is also verified by identifying the missing values, outliers, and inconsistencies.

The Data Preparation phase focusses on transforming raw textual data into a suitable format for LLM based modelling so that it matches the data science goal identified in the Business Understanding phase. First, relevant data sets and features are chosen based on the project objective of using Chain-of-Thought reasoning. This is followed by error correction, imputations, and outlier handling. Once the data is cleaned, each data set is transformed using feature engineering techniques, and the datasets are integrated together to handle inconsistencies. The last step in this phase is to correctly format all the features in the final dataset into a consistent data type.

The Modeling phase focusses on using the prepared data to fine-tune the LLMs on the education domain. Various open-source LLMs like Mistral, Phi-3, Llama 3, and Gemma are selected based on the project goal and the datasets are divided into train, validation, and test partitions. Each selected model is then fine-tuned using the train split, evaluated on the validation split and compared using metrics and visualizations appropriate for this project to provide an insight into model performance and its usability in the project. The base LLM which were not fine-tuned on the education domain will also be evaluated using few-shot Chain-of-Thought prompting strategy.

The fifth phase of the CRISP-DM methodology is evaluation phase where we determine how well the LLMs fine-tuned in the last phase meet the business objective. The results generated by each LLM are reviewed and assessed against the business needs. This helps in identifying if there are any essential factors or tasks that have been overlooked in all the previous phases, identifying any quality assurance issues, and highlighting activities that have been missed and should be repeated. The next steps to proceed with the project are identified in this phase, which may involve reiterating some of the previous phases, moving to the next phase, or starting new projects.

The Deployment phase involves creating a plan for deployment of the LLM alongside a vector database. This plan outlines the steps involved in deploying the solutions, the resources required, and the timeline for deployment. The maintenance and monitoring plan is also devised to ensure that the custom LLM solution meets the performance expectations and adapts to the changing scenarios in the deployment environment. The custom LLM model is then deployed to the production environment, which includes configuring the required infrastructure, installing the solutions, and integrating them with the current systems. A final report is also prepared to provide an executive summary of the project, insights into the methodology, results obtained and recommendations or future scope for the project.

Project Organization Plan

WBS decomposes a project into specific stages that provide a general view of the project. The project has been developed based on the CRISP-DM methodology, and major steps identified in the WBS include Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation and Optimization, and Deployment.

The business understanding phase is a building block of the project which defines what the project is about, what it aims to achieve, and the expected result. It includes defining the problem space, deriving stakeholders' needs and specifying the functions and qualities of the deliverables of the project. Some of the challenges include involvement of domain specific language, scarcity of data and computational issues are also considered. The phase then ends with the development of an effective project plan that defines tasks that should be accomplished, and the time frame

they need to be accomplished.

This phase involves searching and analysis of data from educational field to conform to the need of the project. Exploratory Data Analysis (EDA) is performed with the view to visualize and understand the data by revealing patterns, correlation, or outliers present in the dataset. Data quality check on the other hand is done for the purpose of ascertaining the quality of the dataset. The first step prepares an analyst for data analysis and prepares the analyst for a potential problem that may lead to incomplete data in the future.

The data preparation phase focuses on cleaning and modifying data, as well as creating new data, in order that it can be prepared for modeling. Data cleaning is the process which deals with missing values, ensuring the data is consistent, the removal of unwanted entries. Data pre-processing hence these steps: text tokenization and further changes make data fit for language models input Data augmentation expanded the datasets range. The last process involves partitioning data into training, validation and test data sets so as to get balanced datasets suitable for training the models.

During this phase, appropriate LLMs are chosen, pre-trained, fine-tuned, and tuned to the educational field respectively. Random models are first created, and then fine-tuning is accomplished through adjustment over model hyperparameters. To assess the effectiveness of the models, various metrics are employed, and, to determine relevance and competitiveness, comparison with prior solutions is made. This phase means to develop models that are in tune with the reasoning and accuracy needed by the project.

The evaluation phase checks the validity of the trained models compared with benchmarks and real-world data to ensure their usability. Success is evaluated based on the metrics of BLEU score, ROGUE score, Chrf, Correctness, GEval correctness, GEval logical coherence (LLM as a Judge); moreover, the evaluation in contrast with the reference solutions is discussed. Special efforts such as further training and model integration are implemented to cover the shortcomings, and the final models are as good as intended in the project.

During the deployment phase the optimized models are applied to a target system and an

engaging Reasoning-Augmented Generation (RAG) application is created with a simple GUI.

Some of the options in the application are question answer and reasoning explanations. For

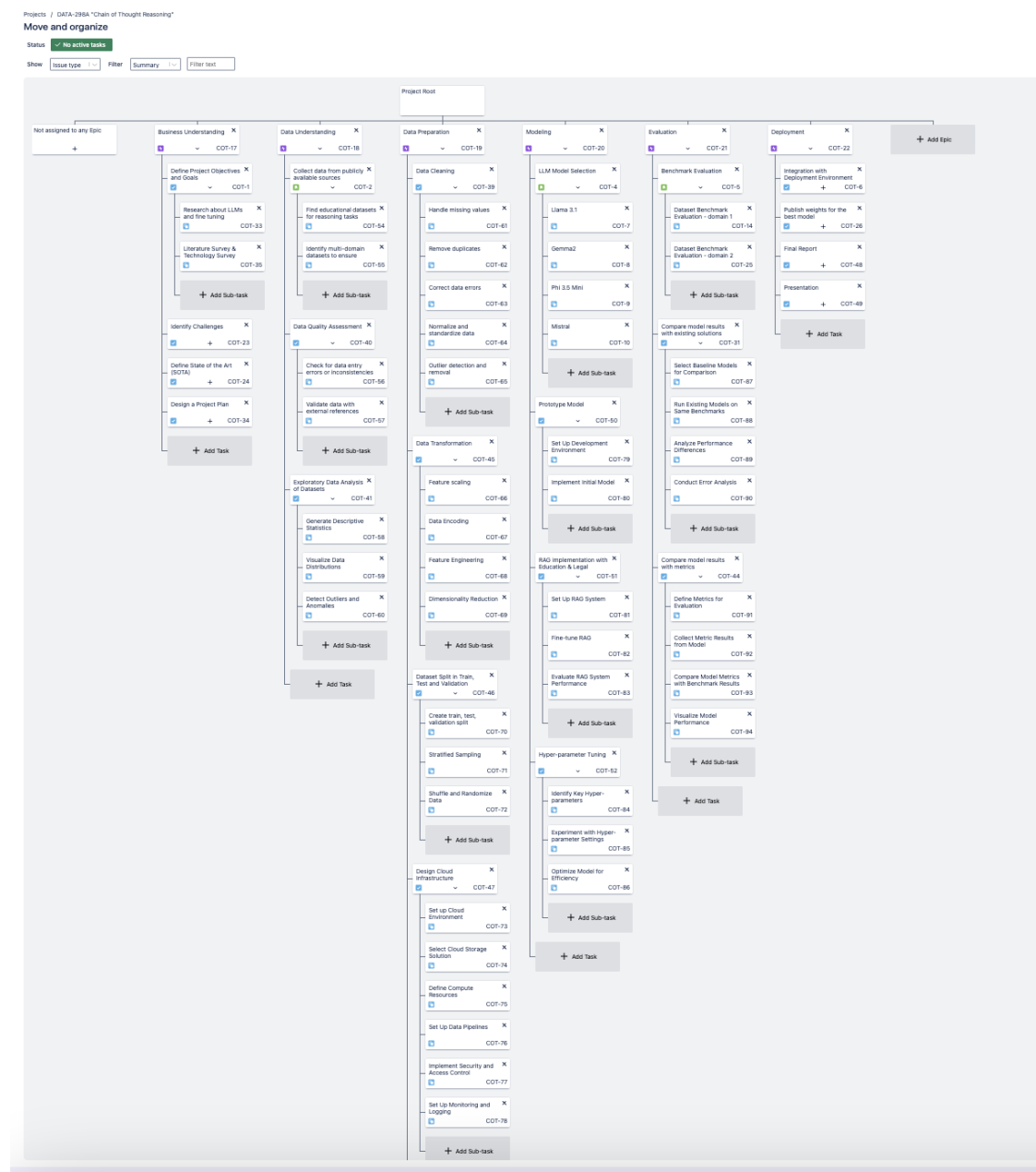
instance, there is performance monitoring and logging to check reliability day in and out.

Findings are summarized into a final report and a presentation to show the practical applications

of the project at this stage of the phase. The breakdown of all the phases is shown in Figure 4

Figure 4

Work Breakdown Structure



Project Resource Requirement Plan

The success of the project relies on a well-structured combination of hardware, software, tools, and licenses that meet the project’s technical needs. This section details the required resources, specifications, costs, and justifications to ensure that the project operates smoothly.

Software Requirements

The following software tools are needed for this project to successfully implement CoT-enhanced LLM models, as shown in Table 7: Python 3.8 or higher is required due to its flexibility in developing and deploying machine learning models. PyTorch 1.10 or later is selected as the main deep learning framework because of its flexibility and support for complex neural networks. The Jupyter Notebook provides an interactive environment for coding, experimentation, and documentation. The Hugging Face Transformers library is essential to access pre-trained models like Llama 3.1 and Gemma2 for seamless model development. Besides that, one should have CUDA Toolkit 11.3+ installed to get the most out of GPU acceleration and, consequently, faster training times. Specialized libraries are needed for fine-tuning techniques such as LoRA and QLoRA. This also involves version control and collaboration done via Git and GitHub, while project management is by Jira to track effectively against tasks, progress, and milestones.

Table 7

Software Requirements

Software	Specification	Purpose	Justification
Python 3	Version 3.10 or above	Main programming language	Versatile, widely used in AI research, supports numerous ML libraries.
Pytorch	Latest stable version	Deep Learning framework	Necessary for building and training neural networks and LLMs.
Google Colab	Stable notebook for coding	Interactive data science environment	Facilitates coding, data visualization, and debugging

Software	Specification	Purpose	Justification
Hugging Face Transformers	Latest Version	Model training, fine tuning, inferences	Provides pre-trained LLMs and fine-tuning capabilities for CoT
GitHub	Basic account	Version Control	Essential for source code management and collaboration.
CUDA Toolkit	Latest Version	GPU acceleration	Necessary for leveraging NVIDIA GPUs for model training.

Hardware Requirements

The hardware requirement of the project is divided between local and cloud-based: A local machine, equipped with a CPU with 8GB RAM and a 256GB SSD, is considered the main working place for coding and initial testing. On the other hand, more resource-intensive tasks, like the training of LLMs using Chain of Thought, are better done on a university-provided GPU lab. This lab provides high-performance GPUs that can bear huge computational loads efficiently and train the complex models. For handling big datasets and intermediate results, a 1TB HDD or SSD is recommended to be used as external storage. This combination of local and high-performance hardware resources outlined in Table 8 ensures smooth execution of the project, right from data preprocessing to model training and evaluation.

Table 8

Hardware Requirements

Hardware	Specification	Purpose	Justification
Workstation or Personal Computer (CPU)	Intel Core i5 or AMD Ryzen 9 with minimum 8 GB RAM	Model Training and data processing	High-speed Processing and multi-threading capabilities
GPU	NVIDIA RTX 4060	Accelerated training and inference	Optimal for handling large scale deep learning computations
External storage	500 GB HDD or SDD	Backup and additional data storage	Needed for data redundancy and archiving datasets

Tools and Licenses

Different licenses and tools are being used for the project as outlined in Table 9 to aid in effective development and management of CoT-enhanced LLM models. Jira has an important position in project management; it will be utilized throughout the entire workflow of work, including dividing the work into tasks and structuring workflows. Smooth collaboration and proper management of code revisions are made possible by GitHub, which enables Git version control. These tools enable the integration of contributions with ease while maintaining a record of changes throughout the project. Model development and experimentation are highly dependent on the Hugging Face Transformers Library for providing access to pre-trained models and support for fine-tuning tasks. This CUDA Toolkit, with the inclusion of NVIDIA drivers, will be used to take advantage of GPU acceleration, drastically reducing training times for deep learning models. AWS is also going to be utilized to build the robust data pipeline by offering scalable, cloud-based infrastructure to store, process, and manage data to handle large volumes with ease.

Licensing is obtained for the required software libraries and tools, such as PyTorch, LoRA/QLoRA libraries, and any proprietary datasets required, to ensure that usage is in a lawful and compliant manner. In general, these tools and licenses support the project's goals of efficient model development, testing, deployment, and data management.

Table 9*Tools and Licenses*

Tool	License	Purpose
Draw.io	Free	Used to create a variety of diagrams and flow charts
Jira	Free (student Account)	Project Management
GitHub	Open Source	Version Control and code management
NVIDIA CUDA Toolkit	Inbuilt library	GPU Computation
AWS	Pay per Usage	Cloud based service that offers infrastructure, storage and data pre-processing capabilities that supports the data pipeline
WBS Move and Organize	Free for 3 months	Used to organize and visualize the project for better management
Gantt Chart	Free	Helps with planning and scheduling projects
Pert Chart	Free	Visual tool used to plan, organize, and schedule tasks

Project Cost Estimation and Justification

The project’s cost estimation involves a detailed analysis of the required hardware as outlined in Table 10, software, tools, cloud services, and other associated expenses. Hardware costs primarily include the setup of a local CPU with 8GB RAM and a 256GB SSD, supported by a GPU lab provided by the university for efficient training of LLM models. Software expenses encompass licenses for essential libraries like PyTorch, LoRA/QLoRA, and any proprietary datasets required for fine-tuning the models. The use of cloud services, particularly AWS for data pipeline management, adds to the overall budget but is essential for scalable data processing and storage. Tools such as Jira and GitHub may require subscription fees, especially for premium plans that offer advanced project management and version control features. The investment in CUDA Toolkit and NVIDIA drivers is justified for optimizing GPU performance. Overall, the costs are justified by the need for a robust infrastructure that ensures efficient data handling, model training, and experimentation, leading to successful project outcomes and high-quality results.

Table 10
Project Cost Estimations

Functionality	Resource	Cost	Type
Local Machine or CPU	64-bit machine	0\$ (Already available)	Hardware
GPU machine	Provided by the University	0\$ (Lab is available)	Hardware
Data Storage	Google Drive	Free	Software
Version Control	GitHub	Free	Software
Flow Charts	Draw.io	Free	Software
Project Meetings	Zoom	Free	Software
Project Management	Jira	Free for 6 months	Tool
Coding and debugging	Google Colab	Free	Software
Data Pipeline	AWS	Pay per usage	Tool

Project Schedule

Gantt Chart

The Gantt chart is the effective tool for planning and administration of the project connected with chain of thought (CoT) elaboration in large language models (LLMs). It shows the difference of tasks and duration needed in the project and their dependencies on the timeline that makes the team maintain a unified view of the project. It becomes easy to allocate resources that would work on the Gantt chart and would help design the team to review the progress and even detect potential problems that may slow down the flow. Dividing the important activities like data acquisition, training, and testing of the model over the structure lines with a time perspective makes it easier to organize a smooth sequence of the work and the inter team communication process. In addition to providing clarity and preparing for effective management of projects.

According to Figure 5, the Business Understanding phase of the project shall be from August 22 to September 11, 2024, and will form the core of the rationale of the CoT reasoning project. In this stage, first set down the project specific objectives and goals with the help of outlining the various expectations. These objectives will form the basis for the research and development to make sure that the focus is in the right area, and it is submission to the user's intention. Also review and research LLMs and different tune techniques – giving the viewpoint of the present and technologies for ramping up models.

An initial study of the literature will be conducted to identify related works and methods relevant to CoT reasoning and LLMs to guide the framework of the proposed project. Moreover, performing a technology review will allow us to assess what other technologies are suitable for work in terms of model training and deployment. Another aspect of this phase is precisely the definition of challenges. On these grounds, it would be possible to draw preliminary conclusions on the key activities potentially produce measures to address these challenges. Also identify the current best practice in CoT reasoning to set an appropriate direction in the current research and enable to implement contemporary best practices in this project. Last, there is the project planning phase which involves preparing a schedule of the important points that are expected to

be achieved in the successive operation periods of the project. In general, the Business Understanding phase is crucial for setting up the proper context for our project endeavor.

Figure 5

Business Understanding Gantt



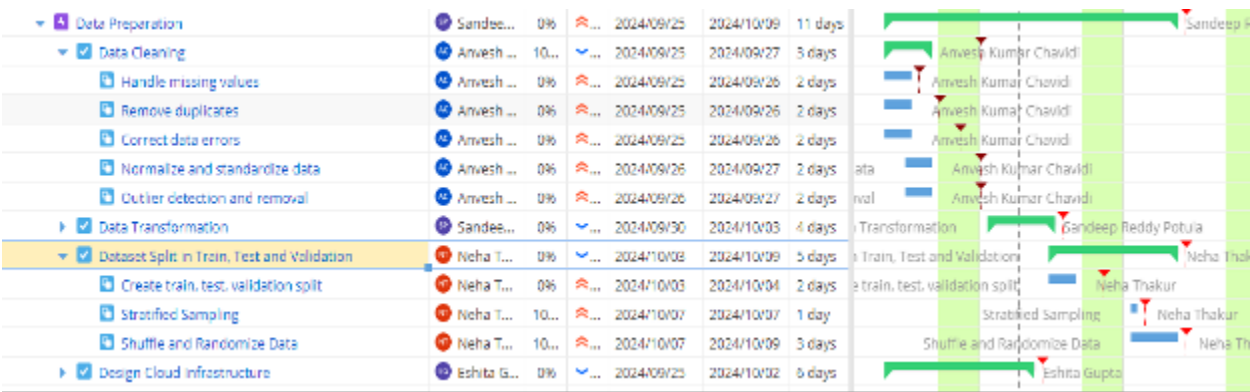
According to Figure 6, the Data Understanding phase of the project is planned for September 11–September 25, 2024. The phase consists of the thorough review of the data that underpins the Chain of Thought (CoT) reasoning project. This phase is preceded by the acquisition of data from public domains. To this effect, use the available datasets to conduct a comprehensive data quality analysis to determine the extents to which the compiled datasets are accurate, complete, and relevant. This assessment will help the organization comprehend some areas that require filling and unpicking. After this step, perform Exploratory Data Analysis (EDA) to discover patterns, relationships and more inside the data and decide on the following steps in the data preparation and modeling process. Afterwards, in this phase, will also specific necessary datasets in educational field, and the selection of which is more difficult due to a set of peculiarities of the high-quality annotations. It would be useful if each of the selected team members would engage in dedicated tasks and explain what data they are collecting to others, so that the models would be trained in diverse and rich data. In the end, this phase should provide a clear picture of what supporting data exists which would help to prepare the proper data and continue the further work with the project.

Figure 6
Data Understanding Gantt



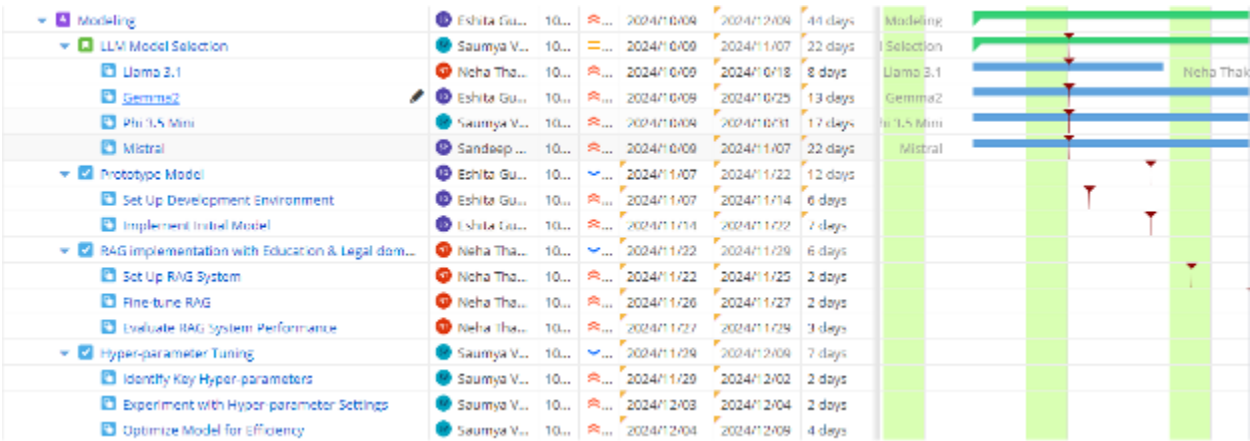
According to Figure 7, the Data Preparation phase is done from September 25 to October 9, 2024, and it forms a vital part of the project especially because it ensures that data collected for model development is properly transformed and cleaned. This phase consists of several activities, for example, data preprocessing; sample data purification process to reduce the overall inconsistencies, duplicates and other types of inputs that might be harmful to the model. Next, in data preparation, data transformation processes to the datasets to achieve a uniform presentation and format will be carried out. The process will also involve the splitting data into training, validating and testing data sets to enable effective evaluation of the model. Also, the design of cloud infrastructure will be carried out to offer a high-scale solution for data storing and analyzing using cloud instruments for successfully collaborative work and access. The data preparation should be well recorded to improve replicability and increase overall modeling and evaluation transparency. This will not only optimize the process but will also check if the data is ready for the intensive process, it will be subjected to as it passes through the modeling and analysis phases.

Figure 7
Data Preparation Gantt



During the Modeling phase, from October 9, 2024, to December 9, 2024, the goal is the selection and customization of LLMs for the project’s goals. Llama 3.1, Gemma2, Phi 3.5 Mini and Mistral models will be analyzed and compared according to the needs of the educational field. Here the development environment for the model is established for practical implementation of the model and preliminary testing with a prototype of the model is done. The functionality of the information retrieval augmented generation (RAG) system will be a critical component toward applying the model in specific domains, thus the need for a strong RAG framework. This will be succeeded by fine-tuning of the RAG system to achieve its important level of performance besides the evaluation of its efficiency in the provision of accurate relevant outputs. Hyperparameter tuning will be another main task to determine the optimal number of hyper parameters and to try out different settings to optimize the model. The main objective of this phase is to fine-tune the models for throughput to meet the project’s performance criteria while being able to execute the reasons of multi-domains. Project Modeling Gantt chart is shown in Figure 8.

Figure 8
Project Modeling Gantt



According to Figure 9, in the Evaluation phase from December 9 to December 13, 2024, trained models' performance and effectiveness of this project will be evaluated. This includes undertaking benchmarking assessments based on datasets belonging to the target areas. The evaluation process will entail the following selection of baseline models for comparison; use of the selected models will be followed on the same benchmarks as the model. Some of the differences such as speed and production rates will be compared to reveal the strengths and the areas of improvement in the new models developed. Further, there will be an error analysis to comprehend the kind of mistakes generated by the models. Identifying the right metrics for assessment is going to be crucial; equally, the gathering of metric outcomes from the built models. These results, in turn, will be compared to benchmark values to determine the study's accuracy. Graphs and charts will help interpret the model performance, and further decision making related to improving the model or subsequent versions of the model can be made.

Figure 9*Model Evaluation Gantt*

▼ Evaluation	NT Neha Tha...	10...	⬆️...	2024/12/09	2024/12/13	5 days
▼ Benchmark Evaluation	SP Sandeep ...	10...	⬆️...	2024/12/09	2024/12/09	1 day
Dataset Benchmark Evaluation - domain 1	SP Sandeep ...	10...	⬆️...	2024/12/09	2024/12/09	1 day
Dataset Benchmark Evaluation - domain 2	AC Anvesh K...	10...	⬆️...	2024/12/09	2024/12/09	1 day
▼ Compare model results with existing solutions	SP Sandeep ...	10...	⬇️...	2024/12/10	2024/12/11	2 days
Select Baseline Models for Comparison	SP Sandeep ...	10...	⬆️...	2024/12/10	2024/12/10	1 day
Run Existing Models on Same Benchmarks	SP Sandeep ...	10...	⬆️...	2024/12/10	2024/12/10	1 day
Analyze Performance Differences	SP Sandeep ...	10...	⬆️...	2024/12/10	2024/12/11	2 days
Conduct Error Analysis	AC Anvesh K...	10...	⬆️...	2024/12/10	2024/12/11	2 days
▼ Compare model results with metrics	EG Eshita Gu...	10...	⬇️...	2024/12/11	2024/12/13	3 days
Define Metrics for Evaluation	EG Eshita Gu...	10...	⬆️...	2024/12/11	2024/12/12	2 days
Collect Metric Results from Model	TV Saumya V...	10...	⬆️...	2024/12/11	2024/12/12	2 days
Compare Model Metrics with Benchmark Resu...	NT Neha Tha...	10...	⬆️...	2024/12/12	2024/12/13	2 days
Visualize Model Performance	AC Anvesh K...	10...	⬆️...	2024/12/12	2024/12/13	2 days

According to Figure 10, the deployment phase will occur from December 13, 2024, to May 9, 2025, with aims on the final stages of the developed solution. This phase starts with transferring the model into the deployment environment to prepare for the production model. Subsequently, the weights of the models under consideration are disseminated, a bit challenging endeavor to provide the potential users with the best performing model. At the same time, the final report is concluded outlining the project results, the tools and approaches used for the purpose, and major findings. As highlighted in this report is a documentation of the project problems encountered in each of the lifecycle phases. Last, the project ends by presenting the findings, the performance of the model and results of the project. This deployment phase's main activity is ensuring the project is implemented, assessed, verified, and disseminated.

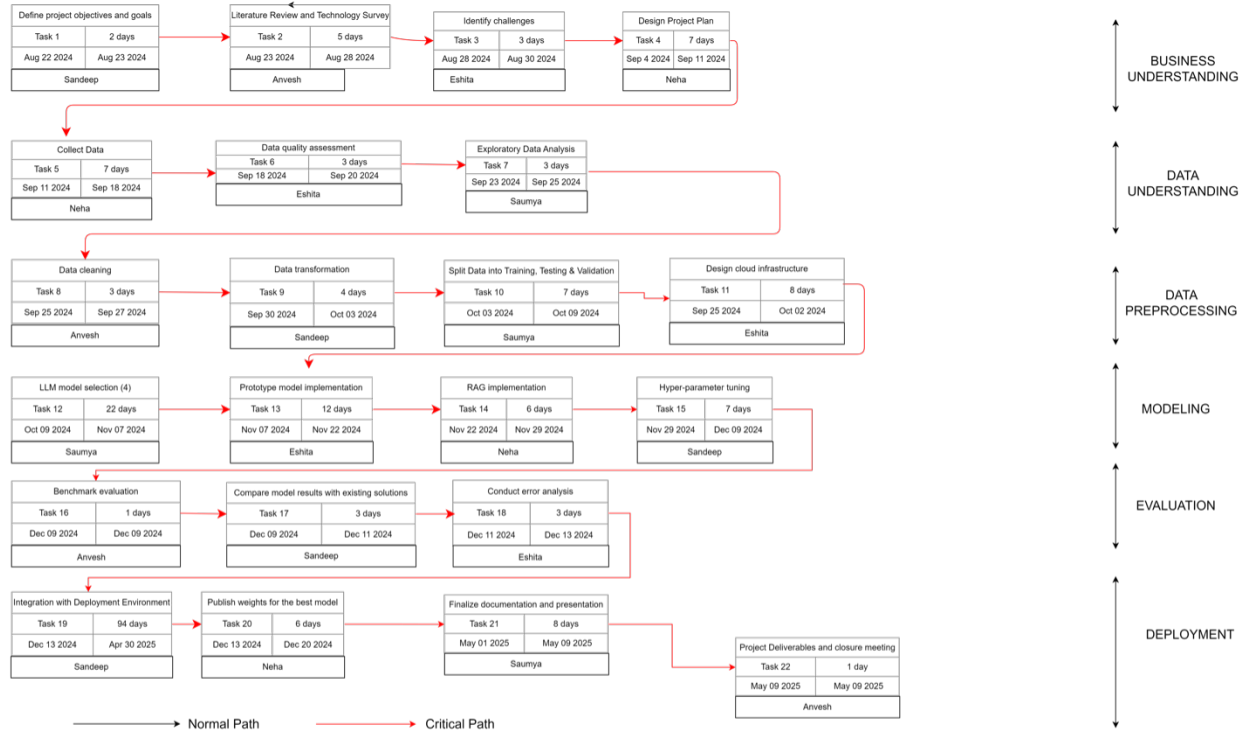
Figure 10*Model Deployment Gantt*

▼ Deployment	NT Neha Thakur	10...	⬆️...	2024/12/13	2025/05/09	106 days
Publish weights for the best model	EG Eshita Gupta	10...	⬇️...	2024/12/13	2024/12/20	6 days
Integration with Deployment Environment	SP Sandeep Redd...	10...	⬆️...	2024/12/20	2025/04/30	94 days
Final Report	TV Saumya Varsh...	10...	⬇️...	2025/04/30	2025/05/06	5 days
Presentation	AC Anvesh Kumar ...	10...	⬇️...	2025/05/06	2025/05/09	4 days

PERT Chart

PERT is a project management tool offering a systematic way of planning, coordinating and sequencing a project's activities. It aids in defining the critical path which gives the most time constrained order sequence of an activity set that defines the probable shortest time that a project is achievable. Project timing means being able to identify the critical path, then manage available resources to achieve the best outcome. The concept of the critical path shows the sequence of the activities that are most critical in the sense that any delay in these activities will force the project timeline forward.

As shown in Figure [11](#) In the context of this project, the critical path reflects the major steps critical to deploying the LLM-based system. It starts with identifying a project's objectives and goals and researching LLMs and initial methods, literature, and technology. Critically, data acquisition and data profiling are actions before the data pre-processing process which entails actions like data scrubbing, data transformation, and data splitting. The activities in the next phase are crucial and include choosing LLM models, deploying the prototype, and incorporating RAG with the education fields. The ultimate step is evaluation of the trained model against existing solutions and other model solutions. The efficient performance of these activities is crucial to the success of the project's goals since all activities on the critical path impact the end date of the project.

Figure 11*PERT Chart*

Data Engineering

Data Process

Process and Steps of collecting raw datasets

The project requires data to be collected in a structured format to evaluate and fine-tune LLMs for the Educational domain and improve the LLM performance in these domains by using Chain-of-Thought Reasoning. Thus, there are some constraints that need to be satisfied when collecting the dataset.

- The dataset should be structured as a collection of question-answer pairs.
- The question and answer should be presented in a natural language format, preferably in English.

- The answers should be detailed and should contain step-by-step instructions to solve the question.

If the dataset is structured in this format, it can be used to adjust the LLM responses to give answers in a step-by-step manner, by either looking at similar solved examples (few-shot learning), or by fine-tuning the LLM weights to answer in a step-by-step manner.

The following platforms were searched to identify datasets for this project

- HuggingFace datasets.
- Datasets hosted by educational institutions like Stanford and UC Berkeley.
- Datasets that are mentioned on LLM evaluation leaderboards like “Open LLM Leaderboard 2” hosted by Huggingface.
- Educational and tutoring wikis that provide solutions for competitive exams like AIME and AHSME.

Based on the above platforms, various datasets were identified in different file formats like CSV, JSON, markdown, HTML or zip. Some of the datasets identified that matched the constraints mentioned above were OpenBookQA, GSM 8K, MATH. Art of Problem Solving (AoPS) hosts an online wiki that provides solutions for competitive exams like American Invitational Mathematics Examination (AIME) for the year 1983 to 2024. This wiki has to be scraped to extract all the solved questions and answers.

A python script was developed to download and scrape all the above datasets. This script would ensure that the entire dataset could be downloaded at once. It would ensure that the dataset always remains consistent.

Preparing Train, Test and Validation sets

Libraries like pandas provide convenient APIs to read data from local and online sources and convert them to different formats. OpenBookQA was originally stored on Huggingface datasets website in parquet format. To make data analysis easy the format was converted to CSV

using pandas. Similarly, GSM8k was also available on Huggingface datasets in parquet format and was converted to CSV. The MATH dataset is available as a tar file on a Berkeley server and it was downloaded and extracted to individual JSON files using a python script. THE AIME data was embedded inside the HTML content of various pages on the AoPS wiki. These questions and their answers with all the reasoning were extracted from multiple pages using the BeautifulSoup package. This package provides APIs to download HTML pages and parse the content present inside those pages. For this dataset the content parsed from the HTML pages was converted to markdown as part of the same script and saved as individual markdown files for persistence.

For some of the datasets like OpenBookQA, the dataset was already split into train, validation and test splits. These splits are already used by other research groups to compare results from different LLMs. For some datasets like MATH and GSM8K, the dataset already consists of a train and a test set. For such cases, the train set was further divided into train and validation sets in an 80-20 ratio to ensure the LLMs could be compared on the same validation set for selecting model hyperparameters. Datasets like AIME that were scraped didn't have any published train, validation and test splits. For such datasets, the train-validation-test split ratio is set to 80-10-10.

Data Collection

Data Sources

The Table [II](#) below describes the sources for various datasets described in the previous sections. For each dataset, the company that has collected the dataset is mentioned, along with the platform on which the dataset is hosted. OpenBookQA and GSM 8K were hosted on HuggingFace, whereas MATH is hosted on UC Berkeley Servers, and AIME solutions are hosted on AoPS Online Wiki.

Table 11*Dataset Details*

Dataset Name	Dataset Source	Hosting Platform	Dataset Link
OpenBookQA	AllenAI	HuggingFace	https://huggingface.co/datasets/allenai/openbookqa
GSM 8K	OpenAI	HuggingFace	https://huggingface.co/datasets/openai/gsm8k
MATH	UC Berkeley	UC Berkeley Servers	https://people.eecs.berkeley.edu/~hendrycks/MATH.tar
AIME	AoPS	AoPS Online Wiki	https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions

Parameters and Quantity of raw datasets

The Table [12](#) below describes the dataset characteristics for all the collected datasets and its quantities.

Table 12*Dataset Metadata*

Dataset Name	Domain	Split	No of Sam- ples	No of Fea- tures	Format
OpenBookQA	Education	Train	4957	8	CSV
OpenBookQA	Education	Validation	500	8	CSV
OpenBookQA	Education	Test	500	8	CSV
GSM8K	Education	Train	7473	2	CSV
GSM8K	Education	Test	7500	2	CSV

Dataset Name	Domain	Split	No of Sam- ples	No of Fea- tures	Format
MATH	Education	Test	5000	4	JSON
MATH	Education	Test	1319	4	JSON
AIME	Education	Combined	1005	2	Markdown

The feature description of the collected data is described below.

OpenBookQA dataset Feature Descriptions. OpenBookQA already has published train, validation and test sets in CSV formats. Given below are the feature descriptions for each of those splits in Table 13.

Table 13

Feature Descriptions for Openbook QA Dataset

Feature Name	Feature Description
id	The unique identifier for each question entry.
question_stem	The actual question text that needs to be answered.
choices	The multiple-choice answer options provided for the question.
answerKey	The correct answer identifier (typically like A, B, C, or D) corresponding to one of the choices.
fact1	A relevant fact or piece of background knowledge that helps in answering the question.
humanScore	A score given by human evaluators indicating how well they were able to answer the question or how reasonable they found it.
clarity	A measure of how clear and understandable the question is, likely rated by human annotators.

Feature Name	Feature Description
turkIdAnonymized	An anonymized identifier for the Amazon Mechanical Turk worker who provided annotations or ratings for this entry.

GSM8K dataset Feature Descriptions. GSM8K only has a train and test split published in CSV format. Its features are quite straightforward, just consisting of a question and its corresponding answer as shown in Table 14 below.

Table 14

Feature Descriptions for GSM8K

Feature Name	Feature Description
question	The actual question text.
answer	The answer to this question.

MATH dataset Feature Descriptions. MATH has individual JSON files for each sample in both train and test splits. Each JSON file has the following keys/features. The feature descriptions are provided below in Table 15.

Table 15

Feature Descriptions for MATH

Feature Name	Feature Description
level	Difficulty level of the question.
problem	The actual question text.
solution	The answer to this question along with the explanation and steps.

Feature Name	Feature Description
type	The category this question belongs to – Pre-Algebra, Algebra, Intermediate Algebra, Number Theory, Counting and Probability, Geometry, Pre-Calculus.

AIME data Feature Descriptions. The AIME dataset was scraped from AoPS and each question along with all the possible solutions was stored in individual markdown files with the details shown in Table 16.

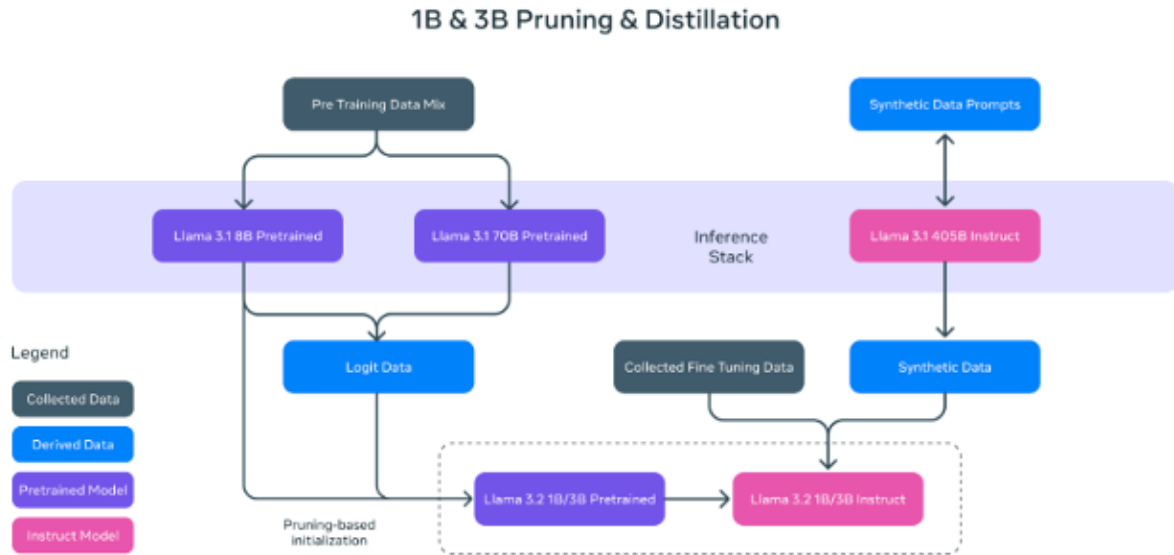
Table 16

Feature Descriptions for AIME

Feature Name	Feature Description
problem	The actual question text.
solution	Solution to the question along with the steps.

Samples from raw datasets

For datasets in CSV format, the samples are presented below in a tabular format as seen in Figures 12 to 16, showing at least 5 samples and all the features for each split of the dataset. For datasets in JSON format, each sample is itself stored in a JSON file. Hence, the complete JSON file for each sample is presented in this section as seen in Figures 17 and 18. For markdown files, all the solutions for each problem are stored inside the same file. In the markdown samples, the problem statement and the first solution are presented, shown in the Preview format instead of the raw markdown code as seen in Figures 19, 20 and 21.

Figure 12*Raw Data Train Sample from OpenBookQA***Figure 13***Raw Data Validation Sample from OpenBookQA*

id	question_stem	choices	answerKey	fact1	humanScore	clarity	turkidAnonymized
8-376	Friiled sharks and angler fish live far beneath...	{ 'text': ['Deep sea animals', 'fish', 'Long Se...	A	deep sea animals live deep in the ocean	0.8	1.8	bdb4135787
7-57	Gas can fill any container it is given, and li...	{ 'text': ['is standard weight and size', 'is t...	D	Matter in the liquid phase has definite volume	0.8	1.6	b356d338b7
7-1024	When birds migrate south for the winter, they ...	{ 'text': ['they are genetically called to', 't...	A	migration is an instinctive behavior	0.8	1.4	b356d338b7
959	If a person walks in the opposite direction of...	{ 'text': ['west', 'north', 'east', 'south'], '...	D	a compass is a kind of tool for determining di...	1.0	1.2	7189327f06
9-241	An example of lots kinetic energy would be	{ 'text': ['Drinking a cold glass of water', 'A...	D	as an object moves , the kinetic energy of tha...	0.8	2.0	cee82219a0

Figure 14*Raw Data Test Sample from OpenBookQA*

id	question_stem	choices	answerKey	fact1	humanScore	clarity	turkidAnonymized
8-343	A person wants to start saving money so that t...	{ 'text': ['make more phone calls', 'quit eatin...	B	using less resources usually causes money to b...	1.0	2.0	b356d338b7
1129	There is most likely going to be fog around:	{ 'text': ['a marsh', 'a tundra', 'the plains', ...	A	fog is formed by water vapor condensing in the...	1.0	1.6	a9f90f5da1
880	Predators eat	{ 'text': ['lions', 'humans', 'bunnies', 'grass...	C	predators eat prey	1.0	0.6	7189327f06
7-999	Oak tree seeds are planted and a sidewalk is p...	{ 'text': ['roots may be split', 'roots may beg...	C	if a tree is located near a sidewalk then the ...	1.0	2.0	b356d338b7
8-464	An electric car runs on electricity via	{ 'text': ['gasoline', 'a power station', 'elec...	C	An electrical conductor is a vehicle for the f...	0.8	1.6	b7b9b1d4ec

Figure 15*Raw Data Train Samples from GSM8K*

question	answer
Natalia sold clips to 48 of her friends in Apr...	Natalia sold $48/2 = \ll 48/2=24 \gg 24$ clips in May...
Weng earns \$12 an hour for babysitting. Yester...	Weng earns $12/60 = \$\ll 12/60=0.2 \gg 0.2$ per minut...
Betty is saving money for a new wallet which c...	In the beginning, Betty has only $100 / 2 = \$\ll \dots$
Julie is reading a 120-page book. Yesterday, s...	Maila read $12 \times 2 = \ll 12*2=24 \gg 24$ pages today...
James writes a 3-page letter to 2 different fr...	He writes each friend $3*2=\ll 3*2=6 \gg 6$ pages a w...

Figure 16*Raw Data Test Sample from GSM8K*

question	answer
Janet's ducks lay 16 eggs per day. She eats th...	Janet sells $16 - 3 - 4 = \ll 16-3-4=9 \gg 9$ duck eg...
A robe takes 2 bolts of blue fiber and half th...	It takes $2/2=\ll 2/2=1 \gg 1$ bolt of white fiber\nS...
Josh decides to try flipping a house. He buys...	The cost of the house and repairs came out to ...
James decides to run 3 sprints 3 times a week....	He sprints $3*3=\ll 3*3=9 \gg 9$ times\nSo he runs 9*...
Every day, Wendi feeds each of her chickens th...	If each chicken eats 3 cups of feed per day, t...

Raw Data Train Samples from MATH dataset

```

'level': 'Level 3',
'problem': 'In right triangle $DEF$, we have $\sin D = \frac{5}{13}$ and $\sin E = \frac{1}{5}$. Find $\sin F$.',
'solution': 'Since $\sin E = \frac{1}{5}$, we have $\angle E = 90^\circ - \arcsin \frac{1}{5}$, so our triangle is as shown below:\n'
'\n'
'\[asy]\n'
'pair D,EE,F;\n'
'\n'
'EE = (0,0);\n'
'F = (5,0);\n'
'D = (0,12);\n'
'draw(D--EE--F--D);\n'
'draw(rightanglemark(F,EE,D,18));\n'
'label("$E$",EE,SW);\n'
'label("$F$",F,SE);\n'
'label("$D$",D,N);\n'
'\[asy]\n'
'\n'
'Since $\sin D = \frac{5}{13}$, we have $\frac{EF}{DF} = \frac{5}{13}$, so $\cos F = \frac{EF}{DF} = \frac{5}{13}$. Since $\sin^2 F + \cos^2 F = 1$, and $\angle F$ is acute '
'(so $\sin F$ is positive), we have $\sin F = \sqrt{1 - \cos^2 F} = \sqrt{1 - \frac{25}{169}} = \sqrt{\frac{144}{169}} = \boxed{\frac{12}{13}}$. We also could have noticed '
'that since $\frac{EF}{DF} = \frac{5}{13}$, we have $EF = 5x$ and $DF = 13x$ for some value of $x$. Then, from the $(5,12,13)$ Pythagorean triple, we see that $DE = 12x$, so '
'$\sin F = \frac{DE}{DF} = \frac{12}{13}$.'
'type': 'Geometry'

```

Figure 18

Raw Data Test Samples from MATH dataset

```
{'level': 'Level 3',
'problem': 'How many vertical asymptotes does the graph of  $y = \frac{2}{x^2 + x - 6}$  have?',
'solution': 'The denominator of the rational function factors into  $x^2 + x - 6 = (x - 2)(x + 3)$ . Since the numerator is always nonzero, there is a vertical asymptote whenever the denominator is 0, which occurs for  $x = 2$  and  $x = -3$ . Therefore, the graph has  $\boxed{2}$  vertical asymptotes.',
'type': 'Algebra'}
```

```
{'level': 'Level 4',
'problem': 'For a particular peculiar pair of dice, the probabilities of rolling 1, 2, 3, 4, 5, and 6 on each die are in the ratio 1:2:3:4:5:6. What is the probability of rolling a total of 7 on the two dice?',
'solution': 'On each die the probability of rolling  $k$ , for  $1 \leq k \leq 6$ , is  $\frac{k}{21}$ . There are six ways of rolling a total of 7 on the two dice, represented by the ordered pairs (1,6), (2,5), (3,4), (4,3), (5,2), and (6,1). Thus the probability of rolling a total of 7 is  $\frac{6}{21^2} = \frac{2}{1225}$ .',
'type': 'Counting & Probability'}
```

```
{'level': 'Level 1',
'problem': 'If the area of the triangle shown is 40, what is  $r$ ?',
'solution': 'The triangle is a right triangle, since the  $x$ - and  $y$ -axes are perpendicular to each other. So the base of the triangle is  $r$  units long and the height of the triangle is 10 units. The area of the triangle is  $\frac{1}{2}r(10) = 40$ . We're told that the area is 40, so  $5r = 40$ , which means  $r = \boxed{8}$ .',
'type': 'Geometry'}
```

Figure 19

Raw Data Sample from AIME data

Problem

The workers in a factory produce widgets and whoosits. For each product, production time is constant and identical for all workers, but not necessarily equal for the two products. In one hour, 100 workers can produce 300 widgets and 200 whoosits. In two hours, 60 workers can produce 240 widgets and 300 whoosits. In three hours, 50 workers can produce 150 widgets and m whoosits. Find m .

Solutions

Suppose that it takes x hours for one worker to create one widget, and y hours for one worker to create one whoosit.

Therefore, we can write that (note that two hours is similar to having twice the number of workers, and so on):

$$100 = 300x + 200y$$

$$2(60) = 240x + 300y$$

$$3(50) = 150x + my$$

Solve the system of equations with the first two equations to find that $(x, y) = \left(\frac{1}{7}, \frac{2}{7}\right)$. Substitute this into the third equation to find that $1050 = 150 + 2m$, so $m = \boxed{450}$.

Figure 20*Raw Data Sample from AIME data***Problem**

Consider the parallelogram with vertices $(10, 45)$, $(10, 114)$, $(28, 153)$, and $(28, 84)$. A line through the origin cuts this figure into two congruent polygons. The slope of the line is m/n , where m and n are relatively prime positive integers. Find $m + n$.

Solution**Solution 1**

Let the first point on the line $x = 10$ be $(10, 45 + a)$ where a is the height above $(10, 45)$. Let the second point on the line $x = 28$ be $(28, 153 - a)$. For two given points, the line will pass the origin if the coordinates are proportional (such that $\frac{y_1}{x_1} = \frac{y_2}{x_2}$). Then, we can write that $\frac{45+a}{10} = \frac{153-a}{28}$. Solving for a yields that $1530 - 10a = 1260 + 28a$, so $a = \frac{270}{38} = \frac{135}{19}$. The slope of the line (since it passes through the origin) is $\frac{45+\frac{135}{19}}{10} = \frac{99}{19}$, and the solution is $m + n = \boxed{118}$.

Figure 21*Raw Data Sample from AIME data***Problem**

Let x , y and z all exceed 1 and let w be a positive number such that $\log_x w = 24$, $\log_y w = 40$ and $\log_{xyz} w = 12$. Find $\log_z w$.

Solution 1

The logarithmic notation doesn't tell us much, so we'll first convert everything to the equivalent exponential forms.

$x^{24} = w$, $y^{40} = w$, and $(xyz)^{12} = w$. If we now convert everything to a power of 120, it will be easy to isolate z and w .

$x^{120} = w^5$, $y^{120} = w^3$, and $(xyz)^{120} = w^{10}$.

With some substitution, we get $w^5 w^3 z^{120} = w^{10}$ and $\log_z w = \boxed{060}$.

Data Pre-processing

The primary objective of data pre-processing was to clean, validate, and standardize the raw datasets to prepare them for the transformation and model training phases. Given the project's goal of developing an intelligent tutoring system that delivers step-by-step and summarized responses to mathematical questions, ensuring high-quality, error-free data was essential. The datasets processed in this project include GSM8K, which focuses on quantitative reasoning; OpenBookQA, which addresses complex question-answering in an educational context; MATH, which provides categorized mathematical problems; and AIME, which includes advanced math

problems from competition datasets. Each step in the pre-processing phase was designed to address specific aspects of data quality and consistency, including handling missing values, removing duplicates, and managing unique dataset structures.

Pre-processing Steps

Each dataset had unique requirements, necessitating specific pre-processing steps to address missing values, duplicates, and format inconsistencies across multiple types of files, including CSV, JSON, and markdown.

Data Loading and Initial Inspection

Each data set was loaded into data frames to inspect its structure and content. Commands such as `df.info()` and `df.head()` provided an overview of each dataset's columns, data types, and sample records. For GSM8K and OpenBookQA, this process helped identify essential fields like question and answer. MATH and AIME datasets required additional handling due to their nested directories of JSON files and markdown files, respectively, which housed problem and solution text. This inspection helped identify key fields such as question, answer, and solution that required validation. All the datasets were given a unique `dataset_id` to identify to which it belongs to, if all the data is merged into a single file.

Handling Missing Values

Missing values in crucial fields were identified and handled across all datasets. In GSM8K, OpenBookQA, and MATH, any NaN or null values in text columns were filled with placeholders or removed if they were non-essential. For the AIME dataset, which was structured in markdown format, missing values were minimal due to the standardized competition format. Null values appear in answer and reasoning after splitting; it is due to empty strings in the original dataset (`aime_df`) that are converted to NaN (null values) during the split or CSV save/load process. In Pandas, an empty string " " is not treated as a null value, but when saved to CSV and reloaded, it may be interpreted as NaN if the column has mixed types. A thorough inspection confirmed that essential data was complete in each dataset, especially for fields that provide questions and solutions. Figure [22](#) represents all the datasets After remove missing values.

Figure 22

Data after removing missing values in all datasets

```
dataset_id      0
question_id     0
question_text    0
answer          0
reasoning       0
dtype: int64
```

Duplicate Removal

Duplicate entries were identified and removed to avoid redundancy and bias in model training. GSM8K, OpenBookQA, and MATH underwent deduplication to ensure each record represented unique information. This step was particularly critical for the MATH dataset, where duplicates across problem categories could affect the model’s ability to generalize problem-solving approaches. The AIME dataset was structured with unique problems for each competition year, ensuring minimal redundancy by default. Figure 23 represents the dataset with number of duplicate values.

Figure 23

Duplicate values in all the datasets

```
The duplicate values present in dataset: 0
```

Parsing Choice-based Answers for OpenBookQA

The OpenBookQA dataset, which includes multiple-choice questions, required specific parsing of choice-based answers. A custom function, `parse_choices_correctly`, was developed to map each answer identifier (such as “A” or “B”) to its corresponding answer text based on the answer key. This parsing ensured consistency across the dataset, making it ready for natural language processing tasks.

Parsing and Structuring MATH JSON Files

MATH, a dataset with individual JSON files in various subcategories like Algebra, Geometry, Counting and Probability, Intermediate Algebra, Number theory, Pre Algebra, Precalculus, required extraction of fields such as level, problem, solution, and type. Each JSON file was parsed to retrieve this structured information, and the solution field was pre-processed to ensure multi-step explanations aligned with the Chain of Thought (CoT) reasoning framework.

Parsing AIME Markdown Files

The AIME dataset, containing markdown files for each problem and solution, required parsing to extract fields such as year, test_type, problem, and solution. Each markdown file was processed to capture these fields, ensuring that each solution was presented as a step-by-step explanation compatible with CoT reasoning. This standardization was crucial to ensuring consistency across all datasets in terms of question format and solution structure.

Pre-processing Output Verification

After applying pre-processing steps, samples from the cleaned datasets were examined to confirm that missing values, duplicates, and inconsistencies were adequately handled. For MATH, this included inspecting the parsed problem and solution fields for each category (e.g., Algebra, Geometry). OpenBookQA samples showed correctly parsed multiple-choice answers, and AIME samples displayed well-structured problems and solutions. Each dataset was reviewed to ensure readiness for the transformation phase.

Sample of Pre-processed Data

Samples from each pre-processed dataset were presented to validate the quality of the cleaned data. For GSM8K, clean fields like question and answer were displayed, while OpenBookQA displayed correctly parsed choices. The MATH and AIME datasets presented structured problems and step-by-step solutions, confirming the successful application of pre-processing steps and the readiness of the datasets for transformation.

By applying these pre-processing steps, the datasets were transformed into high-quality,

standardized forms, essential for the next phase of data transformation. These pre-processing techniques ensured that the datasets would be free of errors and inconsistencies, which are crucial for building accurate and reliable models in the subsequent stages of this project. Below Figure 24 is a sample data from GSM8K data. Other sample from OpenBookQA, MATH and AIME are in Figure 25, 26, 27 respectively.

Figure 24

Sample data from GSM8K dataset

	question	answer
0	Natalia sold clips to 48 of her friends in Apr...	Natalia sold $48/2 = <<48/2=24>>24$ clips in May...
1	Weng earns \$12 an hour for babysitting. Yester...	Weng earns $12/60 = \$<<12/60=0.2>>0.2$ per minut...
2	Betty is saving money for a new wallet which c...	In the beginning, Betty has only $100 / 2 = \$<<...$
3	Julie is reading a 120-page book. Yesterday, s...	Maila read $12 \times 2 = <<12*2=24>>24$ pages today....
4	James writes a 3-page letter to 2 different fr...	He writes each friend $3*2 = <<3*2=6>>6$ pages a w...

Figure 25

Sample data from OpenBookQA dataset

id	question_stem	choices	answerKey	fact1	humanScore	clarity	turkIdAnonymized
0 7-980	The sun is responsible for	{ 'text': array(['puppies learning new tricks',...	D	the sun is the source of energy for physical c...	1.0	2.0	b356d338b7
1 7-584	When standing miles away from Mount Rushmore	{ 'text': array(['the mountains seem very close...	D	as distance to an object increases , that obje...	1.0	1.4	b356d338b7
2 7-870	When food is reduced in the stomach	{ 'text': array(['the mind needs time to digest...	C	digestion is when stomach acid breaks down food	1.0	1.6	b356d338b7
3 7-321	Stars are	{ 'text': array(['warm lights that float', 'mad...	C	a star is made of gases	1.0	1.6	b356d338b7
4 9-732	You can make a telescope with a	{ 'text': array(['straw', 'Glass', 'Candle', 'm...	D	Galileo Galilei made improvements to the teles...	0.8	1.0	cee82219a0

Figure 26

Sample data from MATH dataset

datasets > MATH > train > algebra > {} 0.json > [] solution

```

1  {
2    "problem": "Let  $f(x) = \begin{cases} ax+3, & \text{if } x>2, \\ nx-5 & \text{if } -2 \leq x \leq 2, \\ \end{cases}$ ",
3    "level": "Level 5",
4    "type": "Algebra",
5    "solution": "For the piecewise function to be continuous, the cases must \"meet\" at  $x=2$  and  $x=-2$ . For example,  $ax+3$  and  $nx-5$  must satisfy  $2a+3 = -2n-5$  at  $x=2$  and  $-2a+3 = -2n-5$  at  $x=-2$ . Solving these two equations simultaneously, we get  $a = -2$  and  $n = 2$ . Therefore, the function is  $f(x) = \begin{cases} -2x+3, & \text{if } x>2, \\ 2x-5 & \text{if } -2 \leq x \leq 2, \\ \end{cases}$ ."
6  }
```

Figure 27*Sample data from AIME dataset*

```

datasets > AIME > 1985 AIME > Problem 1.md > ## Problem
1  ## Problem
2
3  Let  $x_1=97$ , and for  $n>1$ , let  $x_n=\frac{n}{x_{n-1}}$ . Calculate the product  $x_1x_2x_3x_4x_5x_6x_7x_8$ .
4
5  ## Solution
6
7  Since  $x_n=\frac{n}{x_{n-1}}$ ,  $x_n \cdot x_{n-1} = n$ . Setting  $n = 2, 4, 6$  and  $8$  in this equation gives us respectively
    $x_1x_2 = 2$ ,  $x_3x_4 = 4$ ,  $x_5x_6 = 6$  and  $x_7x_8 = 8$  so
8
9   $x_1x_2x_3x_4x_5x_6x_7x_8 = 2 \cdot 4 \cdot 6 \cdot 8 = \boxed{384}$ .
10
11 Notice that the value of  $x_1$  was completely unneeded!
12
13 ## Solution 2
14
15 Another way to do this is to realize that most of our numbers will be canceled out in the multiplication in the end, and to
   just list out the terms of our product and cancel:
16
17  $x_1x_2x_3x_4x_5x_6x_7x_8 = x_1 \cdot \frac{2}{x_1} \cdot \frac{3}{\frac{2}{x_1}} \cdot \frac{4}{\frac{3}{\frac{2}{x_1}}} \cdot \frac{5}{\frac{4}{\frac{3}{\frac{2}{x_1}}}} \cdot \frac{6}{\frac{5}{\frac{4}{\frac{3}{\frac{2}{x_1}}}}} \cdot \frac{7}{\frac{6}{\frac{5}{\frac{4}{\frac{3}{\frac{2}{x_1}}}}} \cdot \frac{8}{\frac{7}{\frac{6}{\frac{5}{\frac{4}{\frac{3}{\frac{2}{x_1}}}}}}}$ 
18
19  $= \left( x_1 \cdot \frac{2}{x_1} \right) \cdot \left( \frac{3}{\frac{2}{x_1}} \cdot \frac{4}{\frac{3}{\frac{2}{x_1}}} \right) \cdot \left( \frac{5}{\frac{4}{\frac{3}{\frac{2}{x_1}}}} \cdot \frac{6}{\frac{5}{\frac{4}{\frac{3}{\frac{2}{x_1}}}}} \right) \cdot \left( \frac{7}{\frac{6}{\frac{5}{\frac{4}{\frac{3}{\frac{2}{x_1}}}}} \cdot \frac{8}{\frac{7}{\frac{6}{\frac{5}{\frac{4}{\frac{3}{\frac{2}{x_1}}}}}}} \right)$ 
20
21  $= (2) \cdot (4) \cdot (6) \cdot (8) = \boxed{384}$ 
22
23 ## See also
24

```

Data Transformation

Data transformation aimed to restructure and format the pre-processed datasets to align with the project's objectives. The goal was to create structured, accessible data that supports the Chain of Thought (CoT) reasoning framework, enabling the models to deliver step-by-step solutions to the queries. This phase included specific transformations to ensure consistency across datasets with different structures. These transformations were applied using AWS Glue jobs which have been discussed in the section Data Pipeline.

Transformation Steps

This phase involved restructuring data fields, creating new columns like `dataset_id` to keep track of the original dataset from which that question was taken from, `question_id` to identify each question answer pair uniquely inside a dataset. Columns or fields which contains the questions were renamed to `question_text`. Columns or fields whci contained detailed step by step answer for

the question were split into two columns answer and reasoning. The answer column only contains the concluding line from the entire answer and reasoning contains the detailed steps to get the answer.

Field Restructuring

Each dataset was structured to include fields relevant to the CoT framework. For example **GSM8K**. Initially GSM8k had only question and answer columns in the dataset. The answer field had ##### notation to distinguish between reasoning and the final numerical answer. Post transformation above mentioned fields were added.

OpenBookQA. Initially OpenBookQA had id, question_stem, choices, answerKey, fact1, humanScore, clarity, turkIdAnonymized. Later this was restructured by adding dataset_id, changing id to question_id, question_stem and choices were combined to form question_text, choices and answerKey were combined to form answer and fact1 was renamed to reasoning. The other three columns were dropped.

MATH. JSON files originally had problem, level, solution, and type fields. dataset_id and question_id were added to bring this data to the same structure as other datasets. Problem was renamed to question text. Solution field was split into answer and reasoning based on the pattern that the answer inside this field was always enclosed inside the boxed{ } notation of latex.

AIME. Each markdown file had a question and multiple solutions for that same question. Question and all the unique solutions were separated using the markdown headings. The final answer in each solution was again enclosed inside the boxed{ } notation. Each problem and its unique solution combination from the markdown file becomes a new row in the transformed dataset with all the other fields mentioned above.

Data Splitting into Training and Validation Sets

To facilitate robust model evaluation each dataset was split into training and validation sets to ensure balanced data for training and evaluation. OpenBookQA was already split into train, validation. These splits are already used by other research groups to compare results from different LLMs. For datasets, MATH and GSM8K, the train set was divided into train and

validation sets in an 80-20 ratio to ensure the LLMs could be compared on the same validation set for selecting model hyperparameters. Since AIME was scraped and didn't have any published splits, the train-validation-test split ratio is set to 80-10-10.

Data Storage and Access

Transformed datasets were saved as CSV files to streamline access for model training. For the AIME dataset, each problem and solution was extracted from markdown files and saved in structured CSV files. This transformation ensured efficient data loading, consistent formatting, and easy accessibility during training. Similarly all the datasets were saved in CSV files.

Statistical Analysis and Visualization

To understand dataset complexity, statistical analyses were conducted on GSM8K and MATH. GSM8K questions were analyzed for length and reasoning complexity, while MATH problems were examined by category (e.g., Algebra, Geometry). Visualizations, such as histograms and bar plots, depicted the distribution of question types and reasoning steps. These analyses helped tailor the tutoring system to accommodate various problem complexities.

Sample of Transformed Data

Transformed data samples demonstrated the final structured format. GSM8K samples displayed structured question_text, answer, and reasoning fields, supporting CoT reasoning as shown in Figure [28](#). For OpenBookQA, questions and parsed answers were shown in a consistent structure as shown in Figure [29](#). MATH and AIME samples included problems with detailed step-by-step solutions, ensuring that each dataset was well-aligned with the project's objectives as shown in Figure [30](#) and for MATH and Figure [31](#) displays AIME dataset.

Figure 28*Transformed data of GSM8K dataset*

dataset_id	question_id	question_text	answer	reasoning
0	1	1 Natalia sold clips to 48 of her friends in Apr...	72	Natalia sold $48/2 = \ll 48/2=24 \gg$ 24 clips in May...
1	1	2 Weng earns \$12 an hour for babysitting. Yester...	10	Weng earns $12/60 = \$\ll 12/60=0.2 \gg$ 0.2 per minut...
2	1	3 Betty is saving money for a new wallet which c...	5	In the beginning, Betty has only $100 / 2 = \$\ll \dots$
3	1	4 Julie is reading a 120-page book. Yesterday, s...	42	Maila read $12 \times 2 = \ll 12*2=24 \gg$ 24 pages today....
4	1	5 James writes a 3-page letter to 2 different fr...	624	He writes each friend $3*2=\ll 3*2=6 \gg$ 6 pages a w...
...	...	...	...	...
7468	1	7469 Very early this morning, Elise left home in a ...	5	For the distance she traveled, Elise paid 23 - ...
7469	1	7470 Josh is saving up for a box of cookies. To rai...	3	He makes \$.5 profit on each bracelet because 1...
7470	1	7471 Colin can skip at six times the speed that Bra...	4	Tony can skip at twice the speed that Bruce ca...
7471	1	7472 Janet, a third grade teacher, is picking up th...	308	Janet needs 35 lunches for the kids + 5 for th...
7472	1	7473 At 30, Anika is $4/3$ the age of Maddie. What wo...	50	If Anika is 30 now, in 15 years, she'll be 30+...

Figure 29*Transformed data of OpenBookQA dataset*

dataset_id	question_id	question_text	answer	reasoning
0	3	1 The sun is responsible for	plants sprouting	the sun is the source of energy for physical c...
1	3	2 When standing miles away from Mount Rushmore	the mountains seem smaller than in photographs	as distance to an object increases , that obje...
2	3	3 When food is reduced in the stomach	nutrients are being deconstructed	digestion is when stomach acid breaks down food
3	3	4 Stars are	great balls of gas burning billions of miles away	a star is made of gases
4	3	5 You can make a telescope with a	mailing tube	Galileo Galilei made improvements to the teles...

Figure 30*Transformed data of MATH dataset*

dataset_id	question_id	question_text	answer	reasoning
426	2	427 Find the equation whose graph is a parabola wi...	Hence the answer to this question is $\$-3x^2+12...$	Since the axis of symmetry is vertical and the...
1593	2	1594 If $f(x)=\sqrt{x-3}$, what is the smallest rea...	Hence the answer to this question is \$12\$.	Note that $f(x)$ is defined if and only if $\$x\ldots$
805	2	806 Positive real numbers $\$r,s$ satisfy the equati...	Hence the answer to this question is \$1\$.	We have $\$2r^2s^2 = (r^4 + 2r^2s^2 + s^4) - (r^...$
410	2	411 If $\log_{25}(x-4)=\frac{1}{2}$, find $\frac{1}{x}$	Hence the answer to this question is \$2\$.	First we solve for $\$x$. Converting our logarit...
6259	2	6260 How many perfect squares are two-digit and div...	Hence the answer to this question is \$2\$.	Recall that no perfect squares are negative, b...

Figure 31*Transformed data of AIME dataset*

dataset_id	question_id	question_text	answer	reasoning
723	5	2015 AIME II_AIME_Problem 12 ## Problem\n\nThere are $2^{10} = 1024$ possib...	548	## Solution 1\n\nLet a_n be the number of ...
192	5	1995 AIME_AIME_Problem 7 ## Problem\n\nGiven that $(1+\sin t)(1+\cos t)...$	027	## Solution 1\n\nFrom the givens, $\sin^2 t \dots$
948	5	2024 AIME_I_AIME_Problem 15 ## Problem\n\nLet $\mathcal{B}$ be the set of ...	721	## Solution 1\n\nObserve that the "worst" poss...
568	5	2010 AIME_I_AIME_Problem 8 ## Problem\n\nFor a real number a , let $f...$	132	## Solution\n\nThe desired region consists of ...
451	5	2006 AIME II_AIME_Problem 10 ## Problem\n\nSeven teams play a soccer tourna...	831	## Solution\n\n## Solution 1\n\nThe resul...

The data transformation phase successfully prepared GSM8K, OpenBookQA, MATH, and AIME datasets for the Chain of Thought framework, supporting our project's goals. By implementing a structured transformation approach, created high-quality, standardized datasets that support the tutoring system's goal of providing clear, step-by-step answers to quantitative questions. This structured data format is pivotal for building a model that delivers accurate, interpretable solutions across diverse problem domains.

Data Preparation

The dataset is divided into three subsets, namely training, testing, and validation, once all the required data transformations have been applied. Splitting this dataset will make each of these individual subsets serve a different purpose: the model uses the training dataset to learn from the data for its initial training. This is followed by model evaluation on the validation dataset. The metrics collected at the validation step inform the modifications that might be necessary or the ways in which it must be fine-tuned, making iterative improvements toward optimal accuracy and performance. These refinements gone, the model is put to a final test on the test dataset, which particularly helps assess the quality of the model's behavior on completely new unseen data hence serving as a good estimator of the generalization capability of the model.

In the data pipeline, AWS Glue job will merge all the raw datasets in the staging AWS S3 bucket. Once the Glue job has been executed, the resulting combined dataset is split in an 80:10:10 where 80% is allocated to training, 10% to validation, and the remaining 10% to testing. Then, the processed splits are deposited into the appropriate final S3 bucket for subsequent processing and use.

The stages of text processing will make use of the internal tokenization feature of the LLM to make it compatible with its architecture and represent the text properly. Figure 32, 37, 42 and 47 shows data distribution according to the training, testing, and validation datasets for the specific educational datasets of GSM8K, MATH, AIME and OpenbookQA. Figure 33, 41, 46 and 51 shows the number of values for each dataset. Figure 34, 35, 36 shows the samples of train, test and validation dataset for GSM8K. Figure 38, 39, 40 shows the samples of train, test and validation dataset for OpenBookQA. Figure 43, 44, 45 shows the samples of train, test and validation dataset for MATH. Figure 48, 49, 50 shows the samples of train, test and validation dataset for AIME. The visualizations below show how the total amounts of data are divided among each subset so that one can understand how the setup is done for each dataset in the educational domain.

Figure 32

GSM8K dataset split in train, test and validation

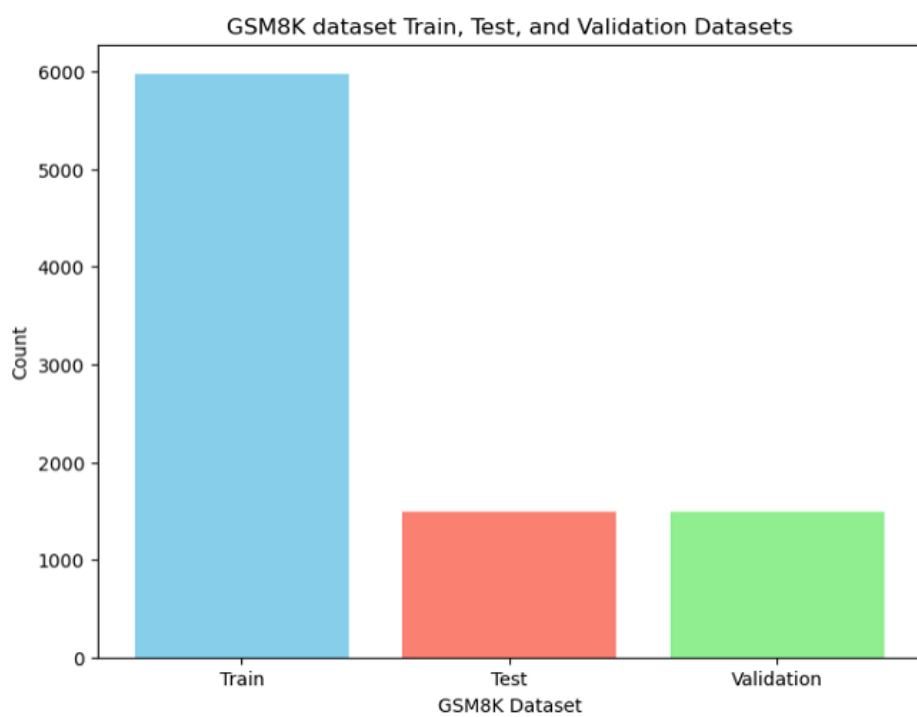


Figure 33

GSM8K dataset split values

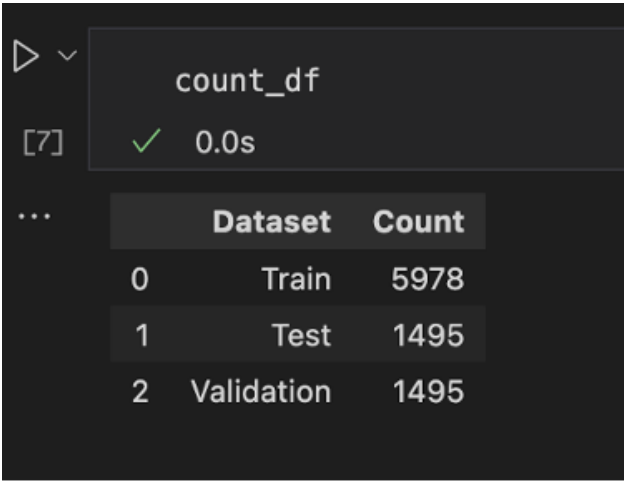


Figure 34

Sample for train dataset of GSM8K

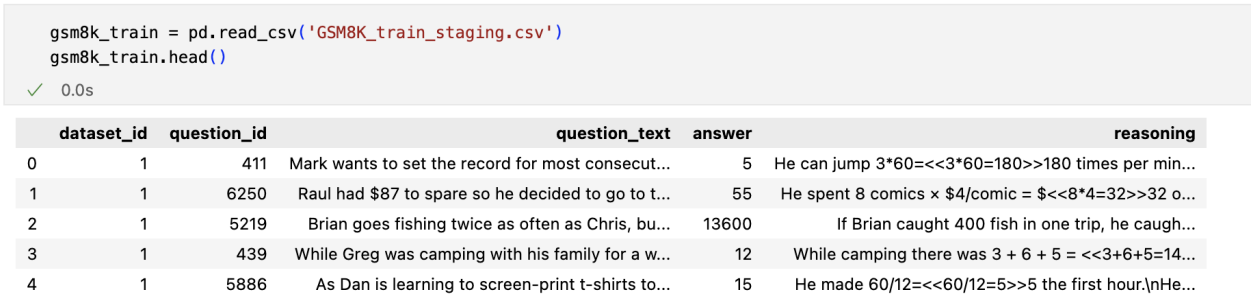


Figure 35*Sample for test dataset of GSM8K*

```
gsm8k_test = pd.read_csv('GSM8K_test_staging.csv')
gsm8k_test.head()
```

✓ 0.0s

	dataset_id	question_id	question_text	answer	reasoning
0	1	1298	In Professor Plum's biology class there are 40...	8	We start with the initial numbers of students,...
1	1	577	Diane bought twenty more apples than Cecile. I...	50	Diane bought $15 + 20 = 35$ apples.\...
2	1	5463	Ann can skate 6 miles an hour. Her friend Glen...	42	First find how far Glenda goes in 3 hours by m...
3	1	4337	Running for 2 hours, Jonah burnt 30 calories e...	90	When Jonah ran for 2 hours, burning 30 calorie...
4	1	7106	The city of Richmond has 1000 more people than...	500	Victoria has $3000 - 1000 = 2000$...

Figure 36*Sample for validation dataset of GSM8K*

```
gsm8k_val = pd.read_csv('GSM8K_val_staging.csv')
gsm8k_val.head()
```

✓ 0.0s

	dataset_id	question_id	question_text	answer	reasoning
0	1	1298	In Professor Plum's biology class there are 40...	8	We start with the initial numbers of students,...
1	1	577	Diane bought twenty more apples than Cecile. I...	50	Diane bought $15 + 20 = 35$ apples.\...
2	1	5463	Ann can skate 6 miles an hour. Her friend Glen...	42	First find how far Glenda goes in 3 hours by m...
3	1	4337	Running for 2 hours, Jonah burnt 30 calories e...	90	When Jonah ran for 2 hours, burning 30 calorie...
4	1	7106	The city of Richmond has 1000 more people than...	500	Victoria has $3000 - 1000 = 2000$...

Figure 37
OpenbookQA dataset split in train, test and validation

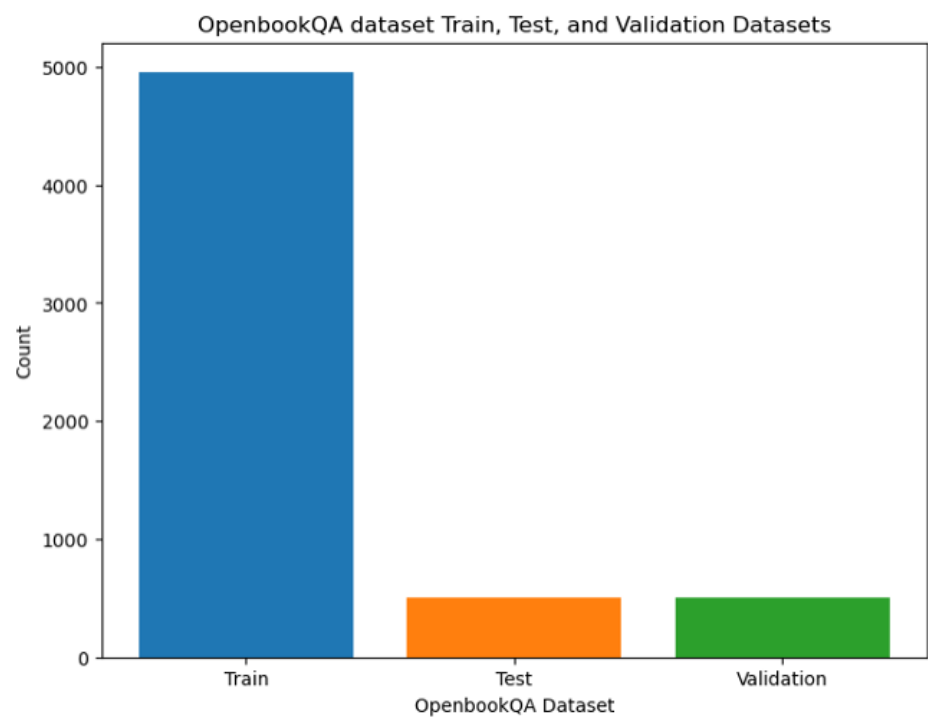


Figure 38
Sample for train dataset of OpenbookQA

```
openbookqa_train = pd.read_csv('OpenBookQA_train_staging.csv')
openbookqa_train.head()
```

✓ 0.0s Python

dataset_id	question_id	question_text	answer	reasoning
0	3	1The sun is responsible for	plants sprouting, blooming and wilting	the sun is the source of energy for physical c...
1	3	2When standing miles away from Mount Rushmore	the mountains seem smaller than in photographs	as distance to an object increases , that obje...
2	3	3When food is reduced in the stomach	nutrients are being deconstructed	digestion is when stomach acid breaks down food
3	3	4Stars are	great balls of gas burning billions of miles away	a star is made of gases
4	3	5You can make a telescope with a	mailing tube	Galileo Galilei made improvements to the teles...

Figure 39

Sample for test dataset of OpenbookQA

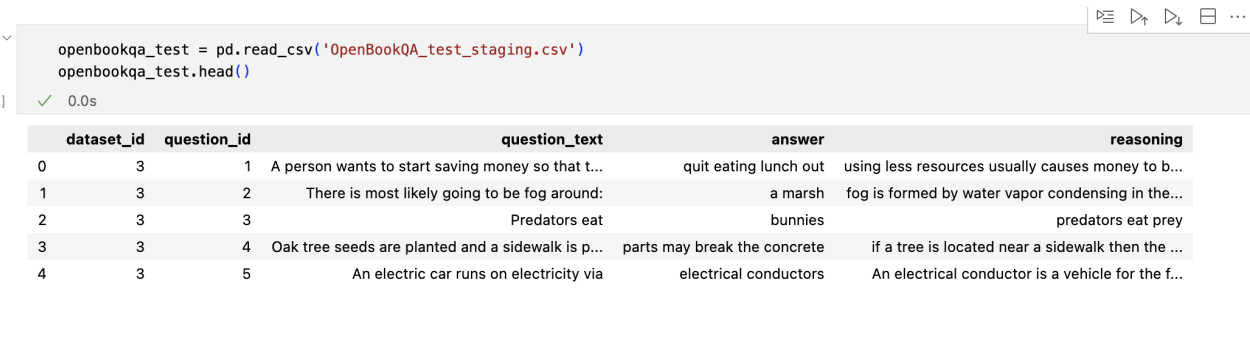


Figure 40

Sample for validation dataset of OpenbookQA

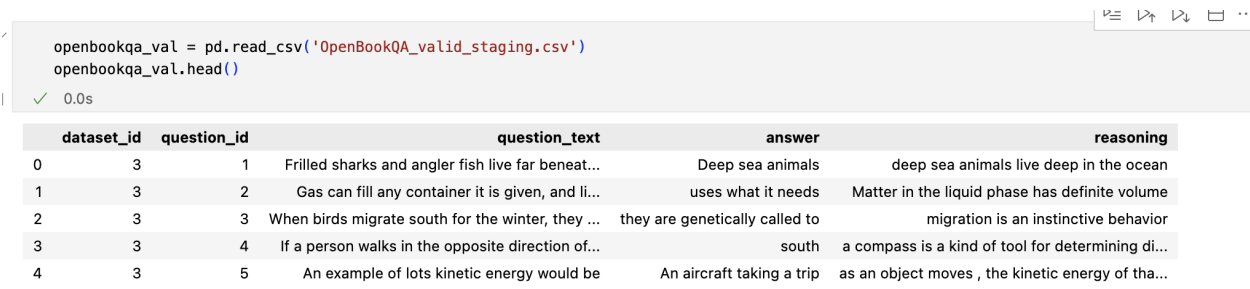


Figure 41

OpenbookQA dataset split values

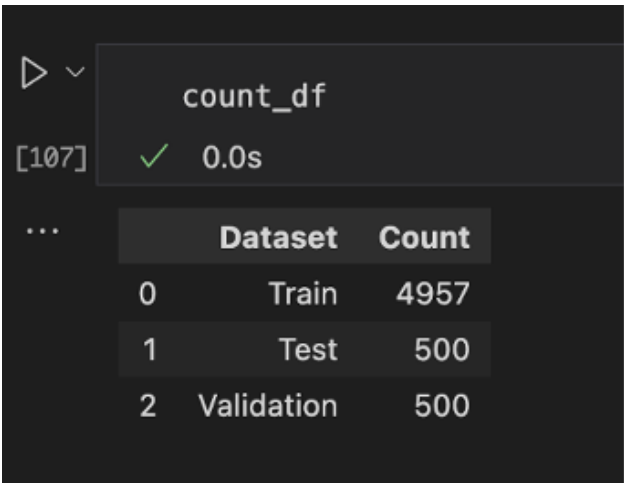


Figure 42

MATH dataset split in train, test and validation

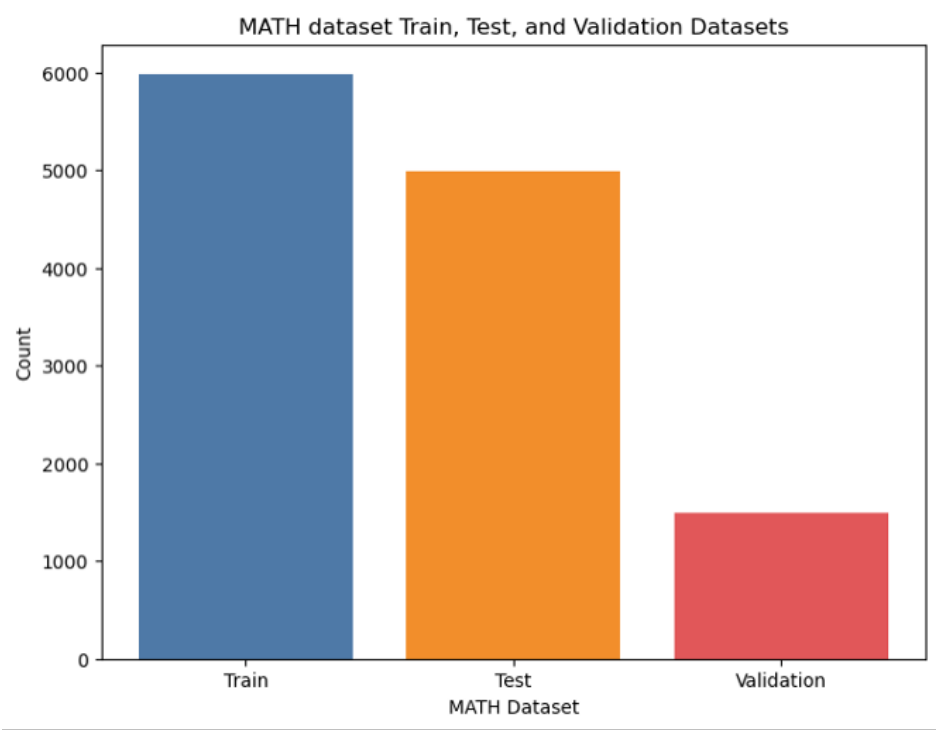


Figure 43

Sample for train dataset of MATH

```
math_train = pd.read_csv('MATH_train_staging.csv')
math_train.head()
```

0.0s Python

dataset_id	question_id	question_text	answer	reasoning
0	2	427 Find the equation whose graph is a parabola wi...	Hence the answer to this question is $-3x^2+12x-12$.	Since the axis of symmetry is vertical and the...
1	2	1594 If $f(x)=\sqrt{x-3}$, what is the smallest rea...	Hence the answer to this question is \$12\$.	Note that $f(x)$ is defined if and only if $x\geq 3$.
2	2	806 Positive real numbers r,s satisfy the equati...	Hence the answer to this question is \$1\$.	We have $2r^2s^2 = (r^4 + 2r^2s^2 + s^4) - (r^4 + s^4)$.
3	2	411 If $\log_{25}(x-4)=\frac{1}{2}$, find $\frac{1}{x}$.	Hence the answer to this question is \$2\$.	First we solve for x . Converting our logarithmic equation to exponential form, we have $25^{\frac{1}{2}} = x - 4$.
4	2	6260 How many perfect squares are two-digit and div...	Hence the answer to this question is \$2\$.	Recall that no perfect squares are negative, b...

Figure 44*Sample for test dataset of MATH*

```

math_test = pd.read_csv('MATH_test_staging.csv')
math_test.head()

```

✓ 0.0s Python

	dataset_id	question_id	question_text	answer	reasoning
0	2	1	How many vertical asymptotes does the graph of...	Hence the answer to this question is \$2\$.	The denominator of the rational function facto...
1	2	2	What is the positive difference between \$120\%\$...	Hence the answer to this question is \$10\$.	One hundred twenty percent of 30 is \$120\cdot 30\cdot 10^{-2} = 36\$.
2	2	3	Find \$x\$ such that \$\lfloor \text{ceil}(x) \rfloor + x = \lfloor \text{dfr}...	Hence the answer to this question is \$9\$.	First, we note that \$x\$ must be positive, sinc...
3	2	4	Evaluate \$i^5 + i^{-25} + i^{45}\$.	Hence the answer to this question is \$i\$.	We have \$i^5 = i^4 \cdot i = 1 \cdot i = i\$. ...
4	2	5	If \$2^8 = 4^x\$, what is the value of \$x\$?	Hence the answer to this question is \$4\$.	Rewrite \$4\$ as \$2^2\$ to find \$4^x = 2^{2x}\$. Si...

Figure 45*Sample for validation dataset of MATH*

```

math_val = pd.read_csv('MATH_val_staging.csv')
math_val.head()

```

✓ 0.0s Python

	dataset_id	question_id	question_text	answer	reasoning
0	2	2827	In the staircase-shaped region below, all angl...	Hence the answer to this question is \$32\$.	We can look at the region as a rectangle with ...
1	2	6957	Below is the graph of \$y = a \sin(bx + c) + d\$...	Hence the answer to this question is \$1\$.	The graph oscillates between 3 and \$-1\$, so \$d\$...
2	2	3015	Let \$ABCD\$ be an isosceles trapezoid with \$\text{ov}...	Hence the answer to this question is \$32\$.	[asy] size(300); defaultpen(1); pair A=(0,0), ...
3	2	6570	Suppose that \$x = 2 - t\$ and \$y = 4t + 7\$. If \$...	Hence the answer to this question is \$27\$.	If \$x = -3\$, then \$-3 = 2 - t\$, so \$t = 5\$. There...
4	2	4607	Done with her new problems, Wendy takes a brea...	Hence the answer to this question is \$1\$.	By Vieta's Formulas, given that \$r_1, r_2, \text{cd}...

Figure 46*MATH dataset split*

```

count_df

```

[104] ✓ 0.0s

...	Dataset	Count
0	Train	5988
1	Test	4992
2	Validation	1497

Figure 47
AIME dataset split in train, test and validation

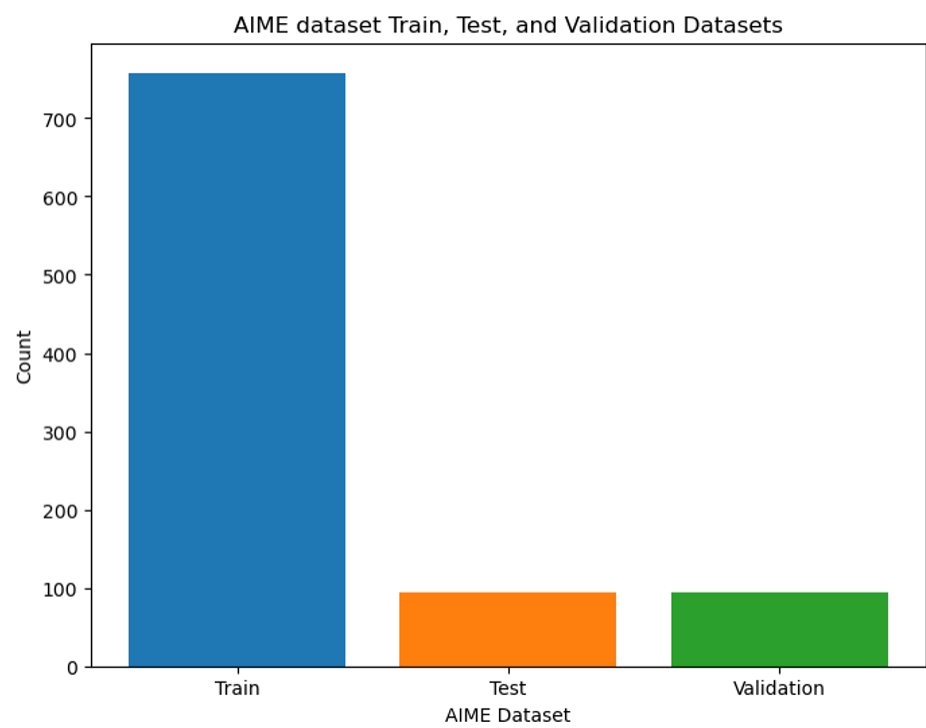


Figure 48
Sample for train dataset of AIME

```
aime_train = pd.read_csv('AIME_train_staging.csv')
aime_train.head()
```

✓ 0.1s

	dataset_id	question_id	question_text	answer	reasoning
0	5	2015 AIME II_AIME_Problem 12	## Problem\n\nThere are $2^{10} = 1024$ possib...	548	## Solution 1\n\nLet a_n be the number of ...
1	5	1995 AIME_AIME_Problem 7	## Problem\n\nGiven that $(1+\sin t)(1+\cos t)...$	027	## Solution 1\n\nFrom the givens, $\sin t \ldots$
2	5	2024 AIME I_AIME_Problem 15	## Problem\n\nLet $\mathcal{B}$ be the set of ...	721	## Solution 1\n\nObserve that the "worst" poss...
3	5	2010 AIME I_AIME_Problem 8	## Problem\n\nFor a real number a , let $f...$	132	## Solution\n\nThe desired region consists of ...
4	5	2006 AIME II_AIME_Problem 10	## Problem\n\nSeven teams play a soccer tourna...	831	## Solution\n\n## Solution 1\n\nThe resul...

Figure 49*Sample for test dataset of AIME*

```

aime_test = pd.read_csv('AIME_test_staging.csv')
aime_test.head()

```

✓ 0.0s

	dataset_id	question_id	question_text	answer	reasoning
0	5	1990 AIME_AIME_Problem 14	## Problem\n\nThe rectangle \$ABCD^{\wedge}\{\}_\\$ below...	594	## Solution\n\n#\n\n## Solution 1(Synthetic)\n...
1	5	1985 AIME_AIME_Problem 1	## Problem\n\nLet $x_1=97$, and for $n>1$, let...	384	## Solution\n\nSince $x_n=\frac{\{x_{n-1}\}}{\{x_{n-1}\}}$, ...
2	5	1996 AIME_AIME_Problem 13	## Problem\n\nIn triangle \$ABC\$, \$AB=\sqrt{30}\$...	065	## Solution\n\n![[asy] pointpen = black; pathp...
3	5	1999 AIME_AIME_Problem 5	## Problem\n\nFor any positive integer $x_{\}$ \$,...	223	## Solution\n\nFor most values of x \$, $T(x)$ \$...
4	5	1987 AIME_AIME_Problem 12	## Problem\n\nLet m \$ be the smallest integer ...	019	## Solution 1\n\nIn order to keep m \$ as small...

Figure 50*Sample for validation dataset of AIME*

```

aime_val = pd.read_csv('AIME_val_staging.csv')
aime_val.head()

```

✓ 0.0s

	dataset_id	question_id	question_text	answer	reasoning
0	5	2001 AIME II_AIME_Problem 7	## Problem\n\nLet $\triangle{PQR}$ \$ be a right ...	725	## Solution\n\n#\n\n## Solution 1 (analytic)\n...
1	5	2010 AIME II_AIME_Problem 14	## Problem\n\nTriangle \$ABC\$ with right angle ...	007	## Solution 1\n\nLet O \$ be the circumcenter o...
2	5	1999 AIME_AIME_Problem 9	## Problem\n\nA function f \$ is defined on the...	259	## Solution 1\n\nSuppose we pick an arbitrary ...
3	5	2011 AIME I_AIME_Problem 3	## Problem\n\nLet L \$ be the line with slope \$...	031	## Solution\n\nGiven that L \$ has slope $\frac{\{ \}$...
4	5	2016 AIME II_AIME_Problem 10	## Problem\n\nTriangle \$ABC\$ is inscribed in c...	43	## Solution 1\n\n![[asy] import cse5; pathpen ...

Figure 51*AIME dataset split*

```

count_df

```

[179] ✓ 0.0s

...

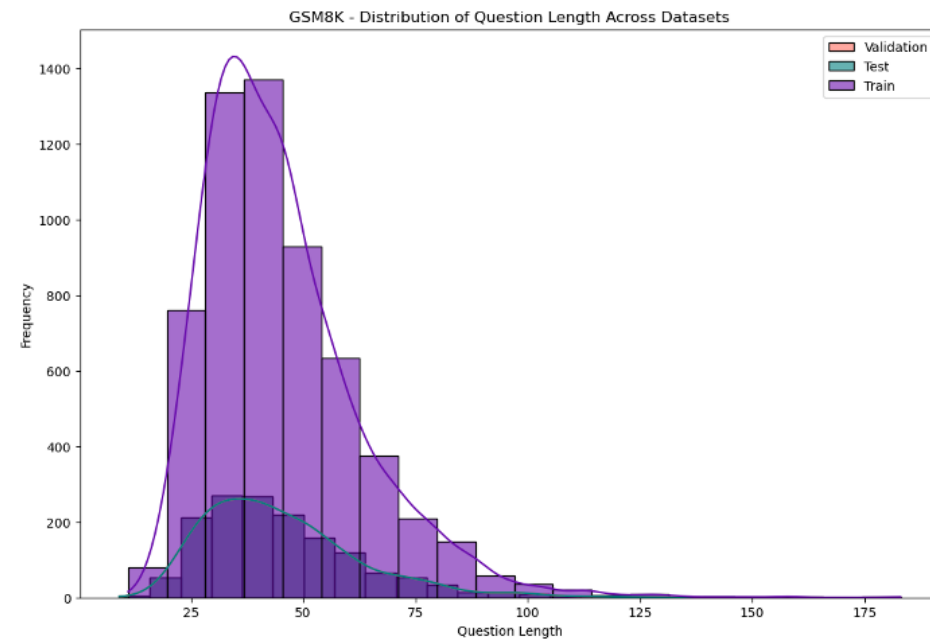
	Dataset	Count
0	Train	758
1	Test	94
2	Validation	95

Data Statistics & Analytics Results

Statistical analysis is the key to understanding dataset features and their anomalies. For gaining deep insights into different distributions, question length and reason length have been considered for three datasets: GSM8K, OpenBookQA, AIME and MATH. Figure 52 compares the distribution of question lengths across the Validation, Test, and Train datasets of GSM8K. For the most part, the purple color for the Train dataset reaches its peak at 25-50 words, with a right-skewed distribution, indicating that longer questions are less frequent. The distributions for the Test and Validation datasets are similar and much less pronounced but less frequent. The KDE curves, on the other hand, emphasize these patterns. They give indications that among all datasets, shorter questions are dominating.

Figure 52

GSM8K – Distribution of question length across datasets

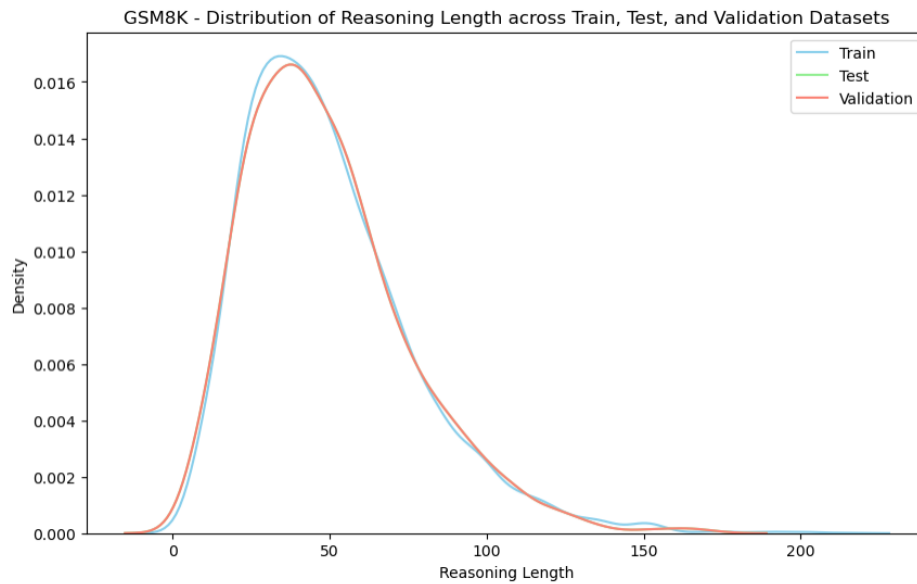


The Figure 53 shows the KDE plot that compares the distribution of the lengths of reasoning across the Train, Test, and Validation datasets in GSM8K. Sure enough, all three distributions are indeed right-skewed with most of the reasoning lengths falling between 30 to 60 words. The density peaks within this range, then gradually declines to show that longer reasoning lengths are

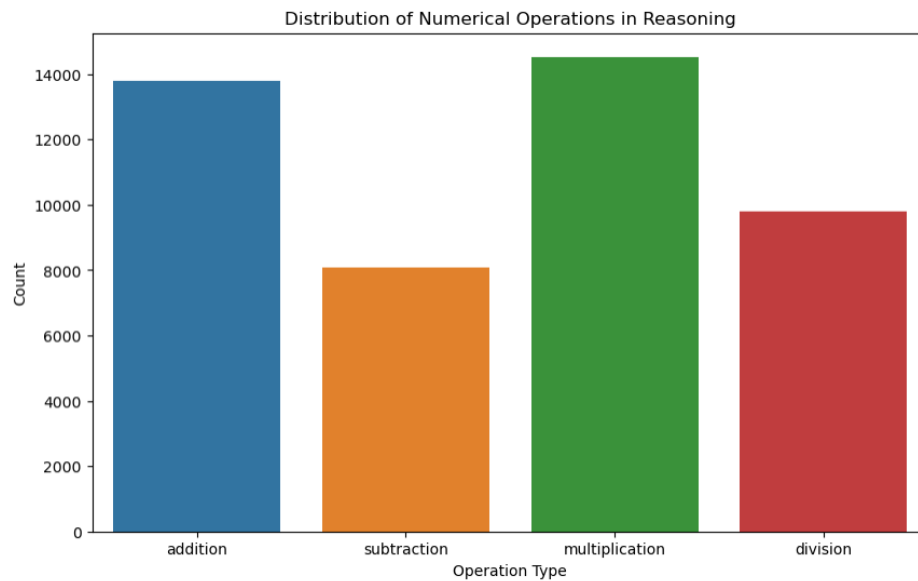
less common. The curves of Train light blue, Test green, and Validation red lie almost on top of each other, which may indicate that the lengths of reasoning are consistently distributed across the datasets. The immediate thing one derives from this plot is that the length of the reason in all subsets of data keeps predominantly small.

Figure 53

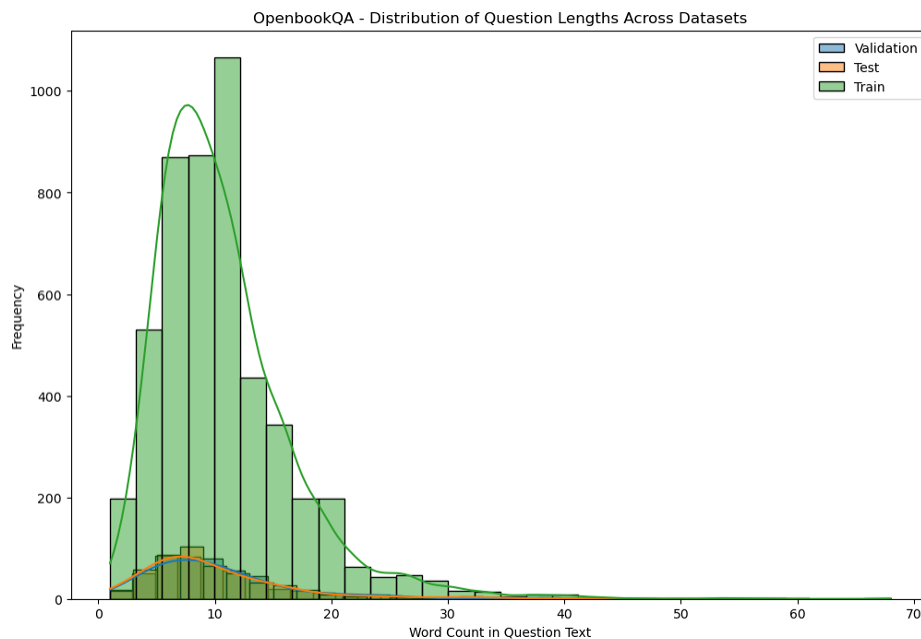
GSM8K – Distribution of reasoning length across datasets



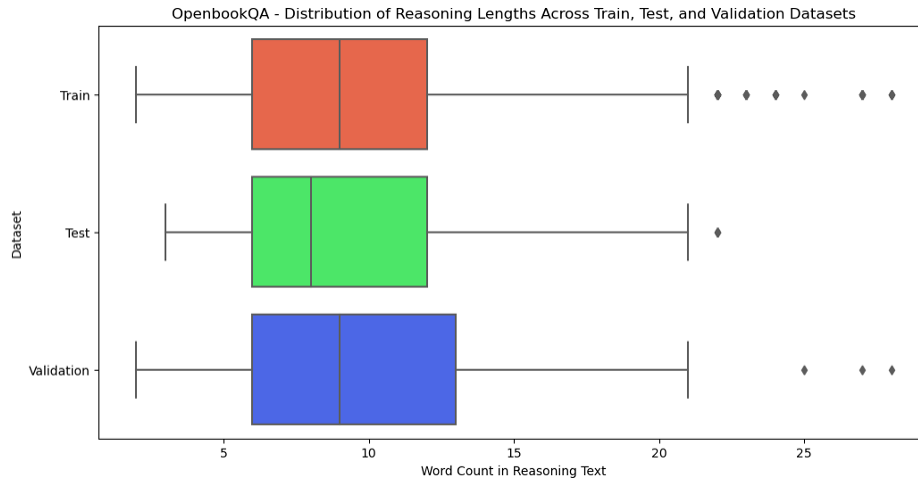
The bar chart in figure 54 represents the distribution of different numerical operations-addition, subtraction, multiplication, and division-which have been used in a dataset's reasoning. Multiplication and addition are the most frequent operations; each operation occurs about 14,000 times, which can be explained by saying that these operations occur more frequently in reasoning. On the other hand, subtraction and division occurs less than 10,000 times. It portrays that there is more reliance on addition and multiplication as opposed to subtraction and division in the reasoning tasks.

Figure 54*Distribution of numerical operations in reasoning*

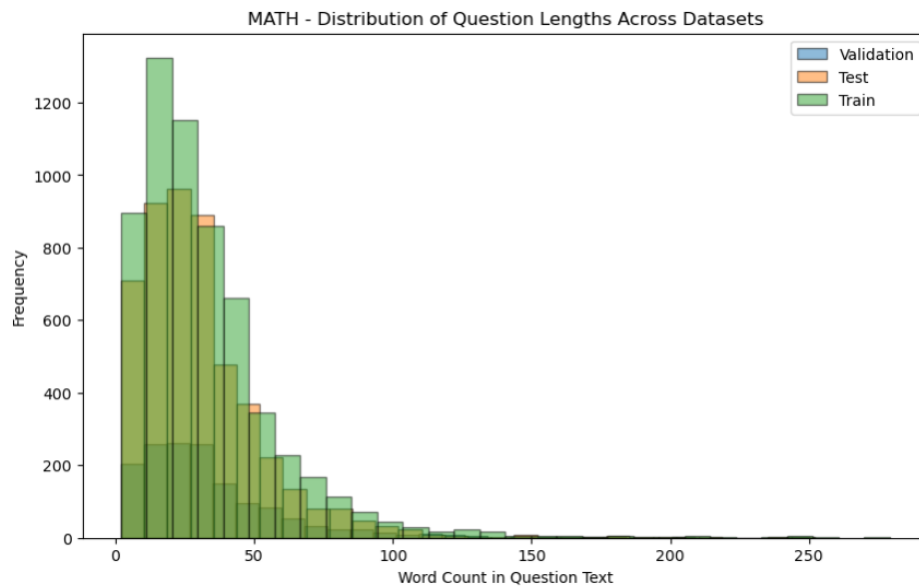
The plot in Figure 55 shows a histogram with superimposed KDE curves showing the distribution of question length across the Validation, Test, and Train datasets in OpenbookQA. Most questions are very short, between 5 and 15 words, since the bars covering this range have the highest frequencies. Green indicates the frequency of the Train dataset, which clearly peaks in this range, while the Validation (in blue) and Test (in orange) datasets have lower frequencies but follow similar distributions. This represents the right-skewed distribution of all three datasets, where longer questions are less common. This line of argumentation is also reflected in the KDE curves, as the picture clarifies that short questions are dominant in all subsets.

Figure 55*OpenbookQA – Distribution of question length across datasets*

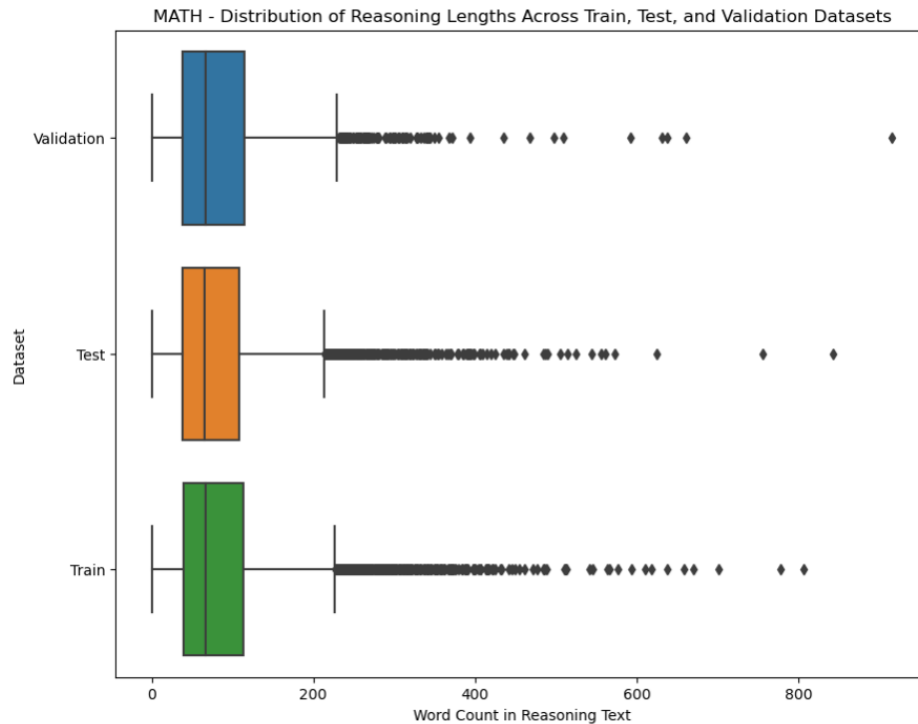
The box plot in Figure 56 depicts the distribution of the number of words in reasoning across all three data splits: Train, Test, and Validation datasets from OpenbookQA. The median for each of the three datasets is around 10 words, with most of the reasoning texts ranging between 5 and 15 words. The interquartile ranges are similar, too, and show consistent dispersion across the datasets. Each dataset contains some outliers, especially in the Train set, where there are some reasoning lengths that extend beyond 20 words. The lengths of the reasoning are very short and roughly consistent across these datasets.

Figure 56*OpenbookQA – Distribution of reasoning length across datasets*

The following histogram Figure [57](#) represents the distribution of question lengths measured by word count across the Validation, Test, and Train datasets in the MATH dataset. Most of the questions are short and cluster between 0 to 50 words; the highest frequencies will fall between approximately 10 to 20 words. The Green represents the Train dataset, which provides the most volume in questions, followed by the Validation dataset in blue, and finally the Test dataset in orange. All three datasets are right-skewed: There are fewer longer questions, with their frequencies tapering off rather slowly as the word count increases. This plot makes a point that most questions in all datasets are much short, while very few questions extend beyond 50 words in length.

Figure 57*MATH – Distribution of questions length across datasets*

The box plot in Figure 58 compares the distribution of lengths of the reasonings measured by word count across the Validation, Test, and Train sets in the MATH dataset. Indeed, it could be said that the distribution of lengths is rather similar for all three sets: a huge majority comes in at less than 200 words. While there are comparable median lengths, all three data sets contain a great many outliers, extending well beyond the upper whisker and going as high as 800 words in extreme cases. These outliers suggest that although most texts of reasoning are fairly small, there are considerably longer explanations of each of the three datasets.

Figure 58*MATH – Distribution of reasoning length across datasets*

Model Development

Model Proposals

This project works with structured datasets such as GSM8K, OpenbookQA and MATH, as its primary goal is to tackle the tasks where step-by-step reasoning is necessary and, as a primary aim, investigate the patterns of multistep reasoning within respective educational contexts. The project utilizes five pre-trained language models: Llama 3.2, Gemma3, Phi 3.5 Mini, Deepseek R1, which is based on the constellation of capabilities and workload handling to execute the CoT ideas and deal with logic operation challenges.

The first element of context construction as a preparation to model building consists in connecting relevant pieces of data between each dataset based on common identifiers that are, for instance, `dataset_id` and `question_id` to create coherent profiles that contain reasoning patterns. After that, data is cleaned and preprocessed where the text data is standardized, noisy data –

removed, and the categorical data is encoded for analysis. Some of the key features, which include question_text, answer, reasoning steps, are made to enhance understanding of the reasoning process.

This project is mainly concerned with understanding how first-order and second-order reasoning patterns are executed to gain insights into the factors and attributions related to successful or unsuccessful reasoning across a multi-step context. It is important to include specific stages in reasoning that are relevant to education and at the same time make statistical sense. The structured data feeds into the use of Retrieval-Augmented Generation (RAG) that improves on model-generated outputs through reasoning steps before making accurate and explainable predictions for CoT tasks.

The next section discusses the details of each model: concepts, features, model architecture, and algorithms to be utilized are described.

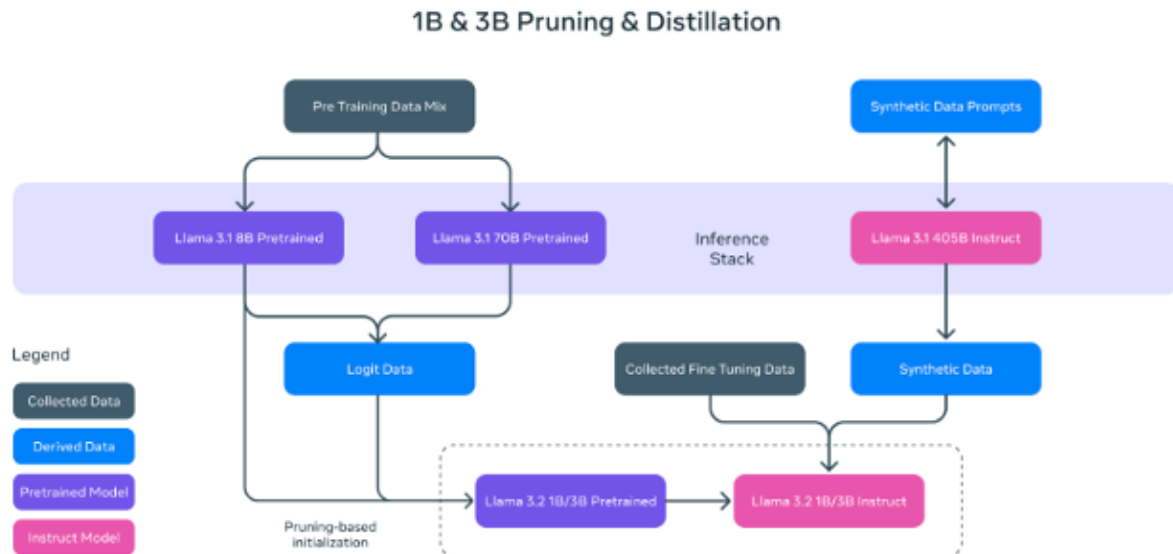
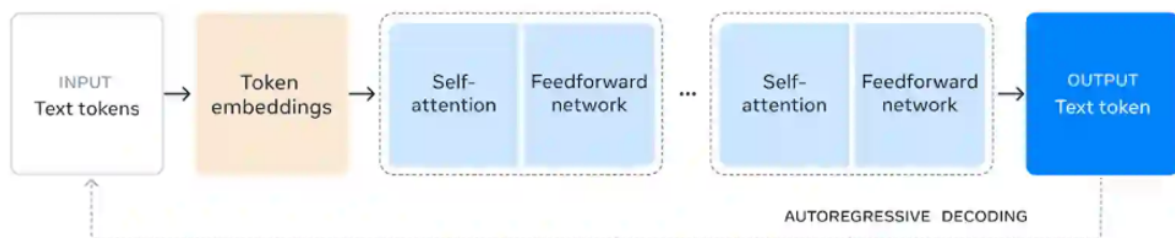
Llama 3.2

Meta's release of Llama 3.2 represents a significant advancement in edge AI and vision models, as outlined by Meta in their September 2024 announcement. The new collection introduces small and medium-sized vision LLMs (11B and 90B) along with lightweight text-only models (1B and 3B), optimized for deployment of edge devices and mobile hardware, including Qualcomm and MediaTek chips. Llama 3.2 features a context length of 128K tokens, making it highly efficient for on-device use cases such as summarization, instruction following, and rewriting tasks, while also enabling privacy-focused local processing.

Additionally, Meta's incorporation of pruning and distillation methods ensures that the smaller models retain strong performance despite their reduced size, fitting efficiently on mobile and edge devices without sacrificing accuracy. The 11B and 90B models bring image understanding capabilities that excel in tasks like captioning, visual grounding, and document-level reasoning, surpassing closed models in image comprehension. For developers, Llama 3.2 offers open access to both pre-trained and instruction-tuned models, along with the tools to fine-tune them for customized applications using torchchat and torchtune. The

architecture of Llama 3.2 builds on the robust foundation of Llama 3.2 while expanding its capabilities to handle both text and image inputs, further enhancing its utility in AI-powered applications across multiple industries (Meta, 2024).

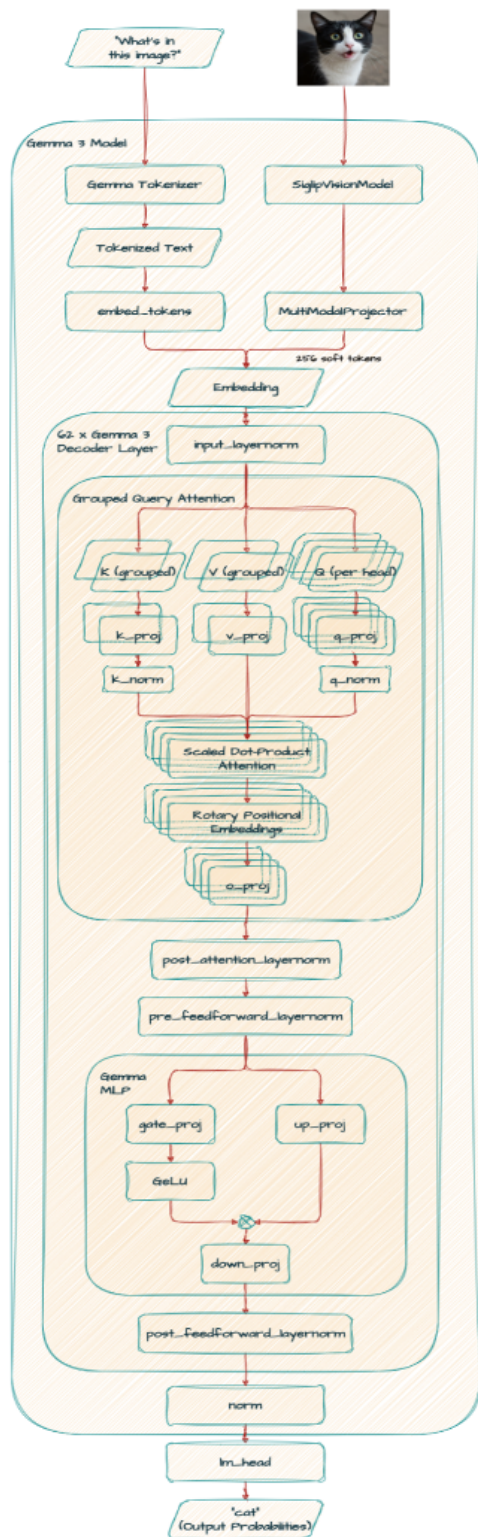
In a recent study by Nandakishor and Anjali (2024), the authors explore a method for enhancing ECG image interpretation using the multimodal LLaMA 3.2 model. By applying Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning strategy, they achieve significant improvements in accuracy for detecting a wide range of cardiac conditions. The model was fine-tuned using the ECGInstruct dataset, which includes expert-labeled ECG images. Their approach demonstrates that LoRA can effectively adapt large models for specialized tasks like ECG interpretation without sacrificing performance, showing improvements over traditional methods like CNNs. In figure 1 the architecture and training of Llama 3.2 is shown and in Figure 60 Llama 1B and 3B text models is shown below.

Figure 59*Architecture and training of Llama 3.2**Note: This picture is taken from the content published by meta team from meta.com***Figure 60***Llama 3.2 1B and 3B Text Models**Note: This picture is taken from the website named analytics vidhya.*

Gemma 3

Smith et al. (2025) describe Gemma 3, the next-generation successor to the BERT-based Gemma series, engineered specifically for multi-step reasoning. While it still uses bidirectional self-attention to capture fine-grained token relationships, Gemma 3 introduces a Hierarchical Transformer Stack that alternates standard encoder layers with lightweight reasoning blocks. Each reasoning block performs dynamic routing between intermediate chains of thought, enabling the model to keep track of local deductions while maintaining global context—an ability that markedly improves Chain-of-Thought (CoT) coherence.

Building on the classic Masked-Language-Modeling and Next-Sentence-Prediction objectives, Gemma 3 is pre-trained with Mask-and-Reorder (MaR) (Zhao et al., 2025), a task that forces the model to predict both what tokens are missing and where they belong in the logical order of a paragraph. This refines the model’s sense of procedural structure, a key trait for step-by-step problem-solving. For efficient fine-tuning, Gemma 3 adopts Quantized Low-Rank Adaptation v2 (QLoRA-v2) (Chen et al., 2024): activations are quantized to 4-bit, and only a rank-8 LoRA matrix per reasoning block is updated. The result is a model that delivers near-state-of-the-art accuracy on math-reasoning benchmarks while remaining deployable on a single 16 GB GPU—an attractive balance of precision and efficiency for educational applications that demand explicit, interpretable solution paths. Figure 3 illustrates the Gemma 3 architecture. Figure 61 shows the architecture of Gemma 3.

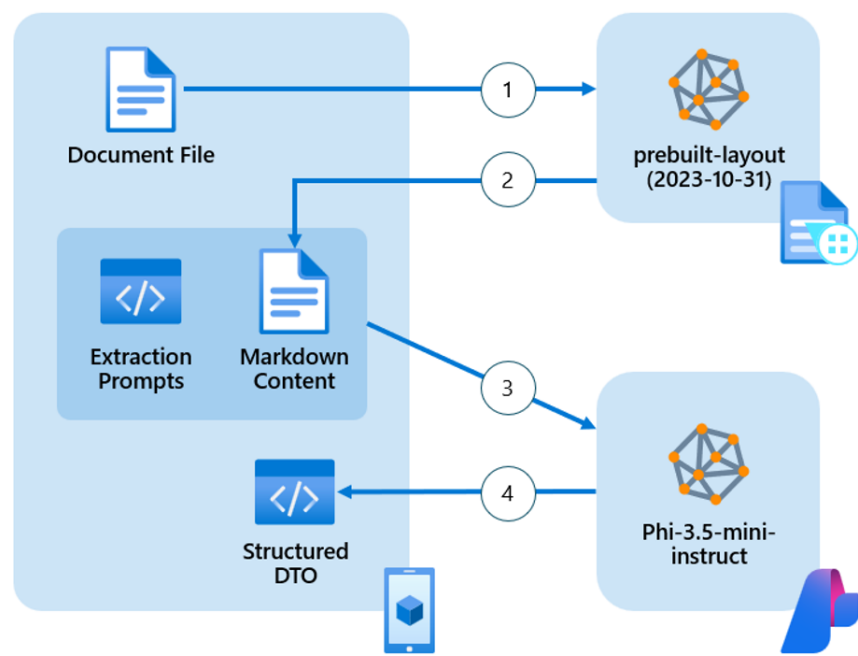
Figure 61*Model Architecture of Gemma 3*

Note: This picture is taken from the google developers by Ravin Kumar (2025)

Phi 3.5 mini

Phi 3.5 Mini is a version of the superior Phi series with presented focus on performance optimized by envisaging higher reasoning accuracy with lower computational needs suggested by Imasogie, (2023). This puts it in a position to handle low resource situations because it preserves performance while using resources sparingly.

Succeeding the Phi 3.5 model that itself was based on a less complex transformer framework with fewer layers and parameters, Phi 3.5 Mini makes computation easy without compromising the reasoning power of the model. It also consists of Few-Shot Prompting and Zero-Shot Learning methods that allow a model to jump to the generalization with either little or no examples of tasks pointed out by Brown et al., (2020). Further, it uses CoT prompting to boost the capability of rational step-by-step throughput, which is helpful in learning and thinking processes suggested by Wei et al., (2022). These properties enable Phi 3.5 Mini to obtain comparable performance in CoT reasoning to other models while using fewer computationally expensive operations. Figure [62](#) shows the architecture of Phi 3.5 Mini.

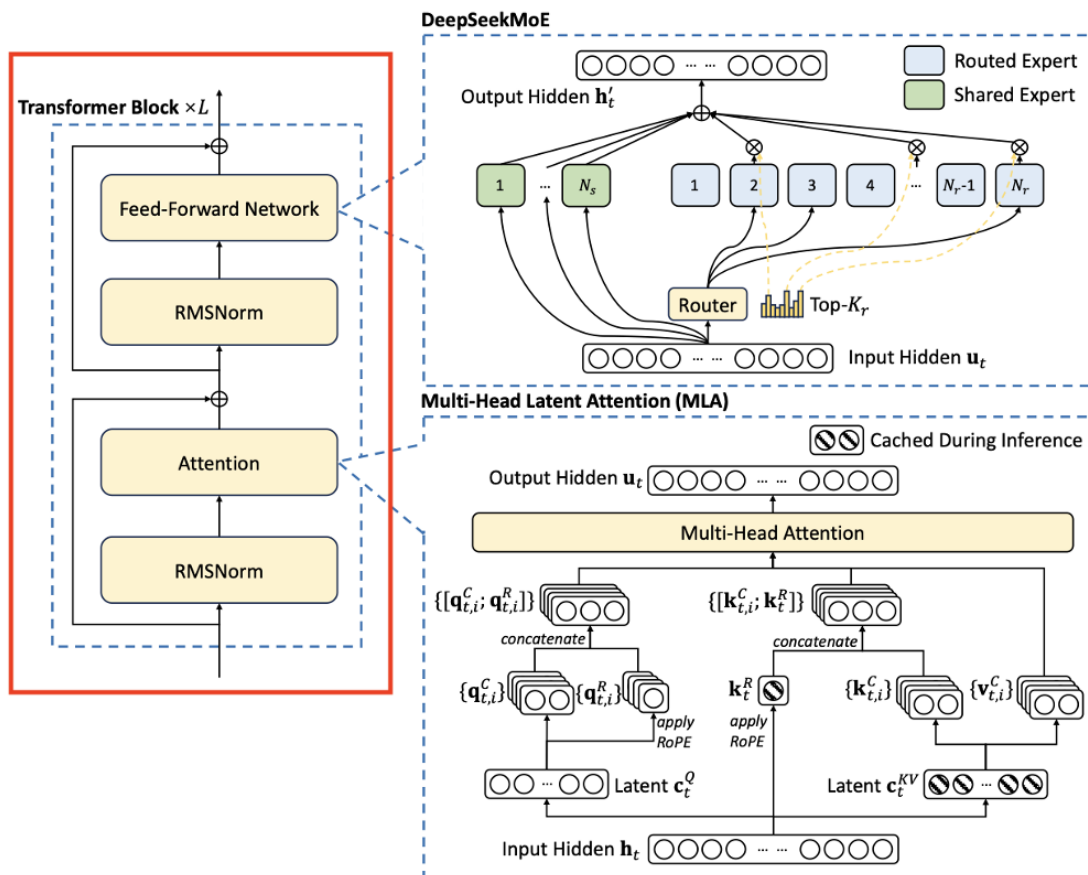
Figure 62*Model Architecture of Phi 3.5 Mini**Note: This picture is taken from the GitHub by James Croft***DeepSeek R1**

DeepSeek-AI contributes to this landscape with DeepSeek-R1 - Distill-1.5B, an equally forward-looking architecture tailored for Chain-of-Thought workloads. As described in the model report by Wang et al., 2024, DeepSeek-R1 starts from a standard decoder-only Transformer but introduces two key innovations. First, it employs Layer-Selective Distillation (LSD): during pre-training a 70-billion-parameter teacher model guides only a subset of eight decoder blocks in the 8-B student, letting the student inherit high-level reasoning features while staying memory-light. Second, DeepSeek-R1 integrates a Chunk-wise Key-Value (CKV) cache inspired by Block-Sparse attention; at inference time it activates just the KV segments relevant to the current question, trimming both latency and GPU footprint without sacrificing context length. In addition, DeepSeek-R1 offers an optional Retrieval-Augmented Decoder (RAD) module: queries emit dense vectors, retrieve supporting passages from a FAISS index, and fuse them via gated cross-attention—yielding answers that are not only accurate but also citation-rich and therefore

well-suited for educational use cases. These design choices let DeepSeek-R1-8B approach the reasoning quality of much larger models while fitting comfortably on a single 24 GB GPU, making it an attractive backbone for our CoT, ToT and DPO experiments. Figure 63 shows the architecture of Deepseek.

Figure 63

Model Architecture of DeepSeek



Note: This picture is taken from the Fireworks AI

Model Supports

The project uses AWS that plays a strategic role of offering computing power needed for handling big data and undertaking other modeling tasks. For model training, and other training and tuning, AWS uses GPU-based instances; they contain the NVIDIA A100 GPU, which

provides the necessary computations for language models. AWS S3 buckets are used to address the need for managing data storage because they allow datasets, model checkpoints, and outputs to be stored at scale while granting access to the data volumes needed at any given stage in the project.

Basically, AWS Glue is used in data preprocessing and AWS Redshift is used in data management. AWS Glue was introduced as serverless data integration service for data transformation and preparation, as it uses Machine learning capability to detect schema and perform ETL process. AWS Redshift is used to address the issue of data storage and fast querying as is typical with data analysis during model development and evaluation. All in all, S3, Glue, and Redshift contribute to a sound distribution platform for data integration, processing, and warehousing.

Model development takes place primarily in Jupyter Notebooks in AWS environments where they enable interactive iterative coding. The primary language of programming is Python, and the available libraries are used for model training and validation. There exist the main libraries for the model training: PyTorch and TensorFlow; Hugging Face Transformers which provides access to the pre-trained models and enable fine-tuning on the prompts; for further data preprocessing and final evaluation, there is scikit-learn. Environment management is done with Docker and Conda tools, to guarantee the equivalence of the installations from one environment to the other and to provide identity between development and production spaces

So, there are a lot of MLOps solutions available on AWS that help deal with model creation, deployment, & management. Huggingface hub is the primary service utilized for training and managing ML. Apache Airflow for scheduling purposes such as data preprocessing and training/validation and the AWS Functions are used to coordinate the workflow of conventionally separate AWS services to automate the steps involved in the machine learning pipeline.

The mentioned efficient adaptation techniques are used in fine-tuning and optimizing each model. Accelerated by Hugging Face they provide methods, such as LoRA (Low-Rank Adaptation) and QLoRA (Quantized Low-Rank Adaptation) to fine-tune in a memory-efficient

way. It reduces memories usage by only learning low rank updates that in turn makes the fine-tune process possible even in environments with limited resources. Also, it supports built-in hyperparameter tuning, which shifts the process to automatic and helps model achieve better results with less iteration time.

After training models, those models are put into docker containers and they become deployed on AWS elastic computing cloud which is managed by Huggingface endpoints. The Huggingface endpoints offer a secure, scalable way of deploying the models to put at the ready for real-time future use.

Table 17 presents all the essential AWS services, libraries and tools that can be used for developing, deploying and managing models in Chain of Thought (CoT) reasoning project.

Table 17

Libraries Required

Service/Library	Method	Purpose
AWS S3	Scalable storage and data management	Store datasets, model artifacts, and outputs
AWS Glue	ETL and data transformation	Transform and prepare data for training
AWS Redshift	Data warehousing and analytics	Store and query large datasets
Apache Airflow	Orchestration of ML workflows	Automate and manage ML tasks
Jupyter Notebooks	Interactive, iterative experimentation	Enable model development in AWS environment
PyTorch	Deep learning framework	Support model development
TensorFlow	Deep learning framework	Support model development
Hugging Face Transformers	Access to pre-trained models and fine-tuning	Enable custom fine-tuning

Service/Library	Method	Purpose
Scikit-learn	Data preprocessing and evaluation	Additional preprocessing and evaluation
Docker	Containerization of models	Deploy models consistently
Conda	Environment and dependency management	Ensure reproducible environments
Hugging Face Accelerate	Memory-efficient fine-tuning support	Enable efficient use of resources during fine-tuning
sklearn	Machine learning algorithms	Machine learning
sklearn.model_selection	Train-test split and cross-validation	Model selection
numpy	Numerical computing	Numerical computing
pandas	Data manipulation and analysis	Data manipulation
matplotlib	Data visualization	Data visualization
Hugging Face TRL	Functions for training models with RL	Implementing reinforcement learning
unsloth	Functions for training models with RL	Efficient and memory friendly training of LLMs

Model Architecture & Data Flow

This project involves the integration of a Retrieval-Augmented Generation (RAG) model architecture to improve the reasoning profile of LLMS. This architecture integrates a retrieval system with a fine-tuned LLM to directly answer difficult questions by drawing information from an outside knowledge base and then structurally and semantically generate a correct response through logical analysis.

It starts first with the step of data acquisition and cleaning. Structured data collection

includes question-answer pairs in free text format which are preprocessed and normalized for the CoT reasoning process needs. This entails transforming the data into vector embeddings that are stored and retrieved with ease from a vector database. Raw data is subsequently rearranged in a manner that helps the model build a direct question-answer response data format as well as a document-based data format to take advantage of multiple-dimension data in the subsequent search and reply to phases.

Once the data is prepared, a fine-tuned language model forms the very crux of the CoT architecture that is implemented. For example, Llama 3.2 or Gemma2 are first trained on the curated dataset in order to enhance the models' capability to provide step-by-step explanations. Approaches like, LoRA which stand for Low-Rank Adaptation, QLoRA which stands for Quantized Low-Rank Adaptation or other strategies are utilized focusing on the amount of memory to be taken and the performance our fine-tuning must take and hence it proves cost effective and flexible according to the need of the project.

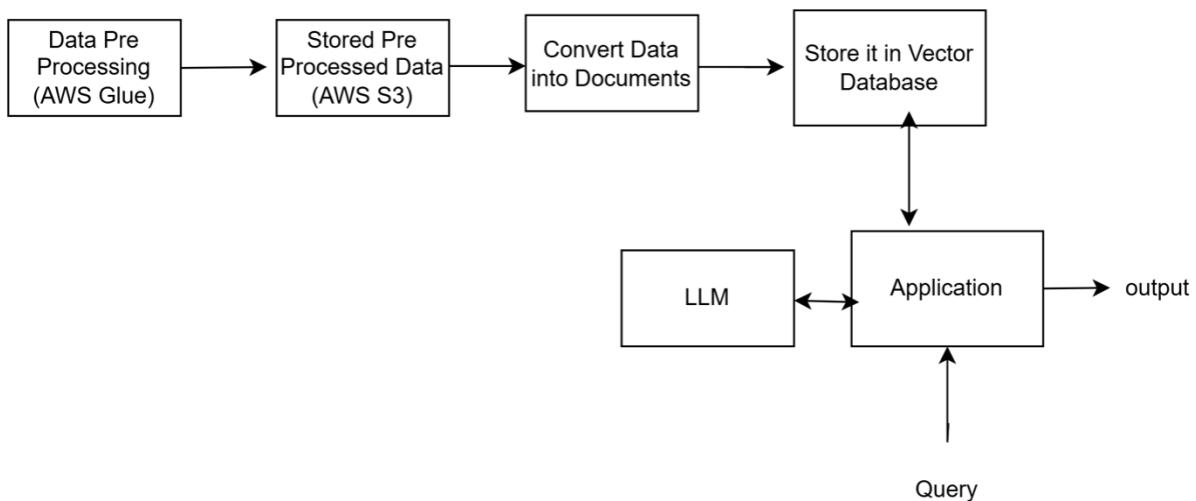
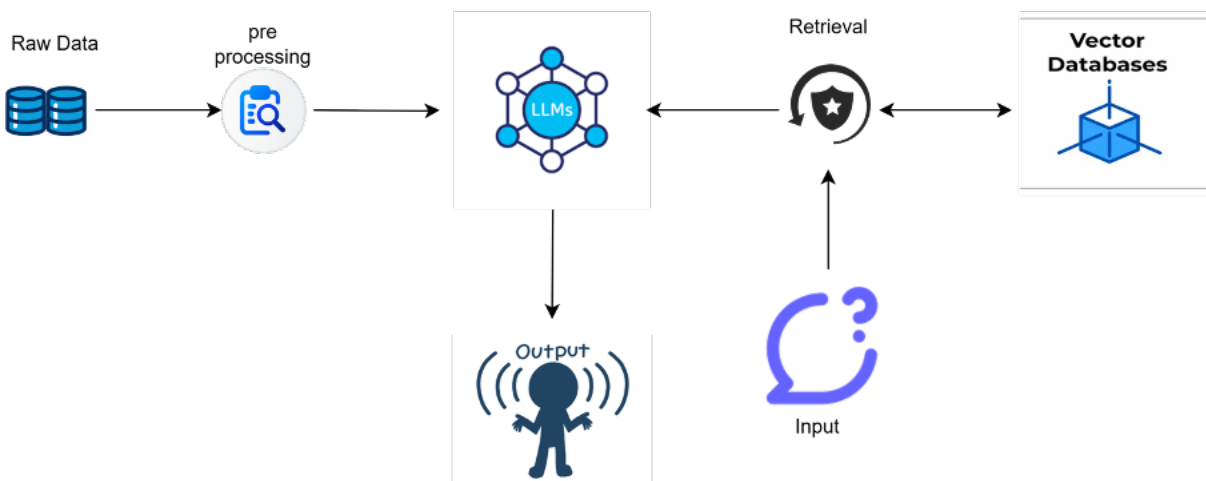
The retrieval system based in a vector database is another significant contribution to enriching the model responses. When a particular search query is entered by a user, the retriever component translates this query into a vector form; it then searches through the vector database for similarity match. This process involves obtaining additional information that is needed to improve the correspondingly improved response accuracy and contextual relevance of the model. The retriever brings in relation documents or sections from the knowledge base to the LLM and supplies fact-based contextual content that proves useful in fashioning knowledgeable responses.

Here, in the CoT rationale approach, while the qualified information is being pulled, the LLM receives both the query and the received data, which allows it to consider the query[string] as structured step by step. CoT reasoning path helps the model to reduce the query into parts in order to solve each part of the problem to give the most reasonable and contextually enhanced answer to the question. CoT methodology improves the interpretability and effectiveness of the model for different tasks, most importantly the one solving a task in steps or complex problem.

At the end of the processing the model produces a final output which is presented to the user

through the application interface. This response combines the generative model which allows for new and distinct responses with the repertory of typical and relevant contexts to filter the output. As described, to ensure the stability and high performance of built models, Amazon CloudWatch is employed for comprehensive real-time monitoring for complaints accuracy, response time, exceptions and other discrepancies in the model output for timely alerts.

In the following, Figure 64 and Figure 65 explain the model architecture and data flow of this project and each stage of the work. These are platform architecture diagram where the design aspects of the data pre-processing tool- AWS Glue, storage tool- AWS S3, and vector database tool- AWS Redshift are presented. This data flow diagram shows how data moves from ingestion through the phases of the retrieval path and the CoT reasoning process before the response generation phase is reached. The model architecture diagram represents how the integration of the retrieved information together with the input query is processed by the LLM and the generation of the response by applying the CoT method. Overall, this architecture offers a general yet highly effective solution to the CoT reasoning project with the help of which retrieval-augmented generation will allow us to achieve specific goals and receive detailed and contextually relevant answers from the language model. Altogether, the use of the retrieval mechanisms, fine-tuning techniques, and the structured reasoning workflows provides a full cycle solution to the complicated and multistep queries.

Figure 64*Prediction workflow***Figure 65***Data Flow and Model Architecture***Model Comparison and Justification**

This project aims at developing an interactive user interface that is capable of providing step-by-step reasoning for the solutions for topics including Logical Reasoning, Maths, and arithmetic problems. four Large Language Models, will be utilized each having its own strengths

and weaknesses for specific tasks. Following sections discuss in detail about the LLM's that will be using and fine-tuning for this use case.

Llama 3.2 3B

The LLAMA 3.2 3B (He et al. 2024) Instruct Model which is released in July 2024, comes with 8 billion parameters. This model has shown impressive results on Chain of Thought Reasoning tasks as it can handle complex reasoning. To solve complex problems often require breaking down the big problem into intermediate steps and solve them sequentially. It is important to maintain the logical reasoning in responses while handling such tasks. Model's ability to do so makes it a suitable choice for our targeted problem. In this model, transformer-based architecture is used which allows for handling large-scale data and generating contextual answers. Due to this transformer-based architecture, it also allows for context persistence over long range of sequences, which is crucial for Chain of Thought Reasoning. The model can maintain consistency in its thought process, i.e it ensures that each step is logically followed from the previous one.

Llama 3.2 has been extensively trained on diverse datasets that include a variety of reasoning tasks. This contributes to improving model's ability to generalize and adapt to the targeted problem. When fine-tuned on education datasets, the model could provide tailored results while breaking down complex problems into intermediate steps and explaining the process. Its outstanding reasoning capabilities makes it an ideal choice for this use case. Its large parameter size (3B) allows it to capture intricate patterns in data. This contributes to its robust performance while maintaining high accuracy in generating relevant responses for different tasks. However, due to its large size and complexity (it uses 70 layers), it requires significant computational resources for training and inference. This would come as a limitation for resource-constrained applications. Given its large parameters, we would have to be careful while fine-tuning the model, as it could lead to overfitting of the training data.

Gemma 3

Gemma3, utilizes a transformer-based architecture comprising 72 layers. Each layer contributes to the ability to process and generate human-like text. This larger number of layers

allows the model to capture intricate patterns in language, which enhances the sequential reasoning and logical deduction tasks. The architectural features like attention mechanism and positional encoding help the model to generate and understand content related to its context. The multi-layered transformer architecture allows the model to handle long sequences of text, aligning with the project's goal to break down larger problems into sub-problems that need to be solved sequentially. This makes it ideal for tasks that require step-by-step reasoning, which is the primary use case. The Model's architecture allows it to maintain context over long sequences by making sure that each step in the reasoning process is informed by the previous steps. The advanced architecture of the model allows it to capture complex patterns and deep contextual understanding of texts, which is important for chain-of-thought reasoning tasks. The model's ability to generate coherent and contextually accurate text makes it ideal for creating detailed explanations for educational content which is required for target problem.

The model will be fine-tuned on datasets related to the education domain to enhance the performance of logical problem-solving tasks. However, training and inference for this model can be computationally extensive due to its large architecture. Despite associated computational and interpretability challenges, the model's ability to handle complex, multi-step tasks, combined with the potential for fine-tuning, justifies it for use case.

Gemma3 4B is slightly larger and deeper as compared to Llama 3.2 3B which makes Gemma more computationally resource intensive. Gemma excels in maintaining long-term context, whereas Llama 3.2 is more optimized for balanced performance and efficiency.

Phi 3.5 Mini

Phi 3.5 Mini (Abdin et al., 2024), released in April 2024 by Microsoft with 3.8B parameters has a dense decoder only transformer architecture with 50 layers. It is designed to tackle complex and reasoning tasks sequentially. The model supports deep understanding of the context and allows for step-by-step logical processing, making it well suited for CoT tasks. The model's layers and parameter count allow for maintaining coherence and accuracy over extended sequences of reasoning steps. This is of high importance for this use case as it requires detailed

and logical explanations for complex problems. Persistence of context and coherence over extended sequences is needed to mimic a human-like thought process.

Phi 3.5 Mini maintains high performance and can aid context persistence even with a smaller number of parameters, which is often considered a favorite choice for environments with resource constraints. The model’s 50 layers allow for handling long-range dependencies, making it effective in complex reasoning tasks. The model can be tuned to specific datasets in the education domain. However, we can expect the limited performance for extremely complex tasks, as it is less powerful compared to other LLMs. This model strikes a good balance between performance and resource requirements, which allows for a practical choice for the project. As compared with previous models, Phi 3.5 Mini provides a balance for efficiency and performance. It is less resource-intensive than Gemma 9B but potentially less capable in maintaining long-term context compared to Llama 3.2.

Table 18
Comparison of Different LLM Models

Area		Llama 3.2		Gemma 3		Phi 3.5	DeepSeek R1	
Basic	Archi- tecture	Transformer- based	with 32 layers	Hierarchical Transformer with 72 layers (en- coder + reasoning blocks)		Transformer decoder-only with 50 layers	Transformer- based	with 64 layers
Targeted	Prob- lems	Suitable for edu- cational tasks, le- gal domain		Designed step-by-step reasoning structured tasks	for and logic	Focused on log- ical reasoning, problem solving	Versatile across math, code, and chain-of-thought reasoning	

Area	Llama 3.2	Gemma 3	Phi 3.5	DeepSeek R1
Size & Com-plexity	1.24 & 3.2 billion parameters; moderate complexity	1 & 4.3 billion parameters; compact yet structured	3.82 billion parameters; efficient reasoning	1.5 billion parameters; distillation-based, efficient inference
Approaches	Deep learning with emphasis on Chain-of-Thought (CoT) reasoning	Mask-and-Reorder pretraining + QLoRA v2 for efficient step-wise learning	Combines statistical and deep learning methods for reasoning	Layer-selective distillation and retrieval-augmented reasoning
Strengths	High accuracy in CoT tasks; robust architecture	Compact yet capable; optimized for structured step-by-step outputs	Efficient for logical tasks with low resource usage	Excellent reasoning alignment, scalable, retrieval-enabled
Limitations	Moderate computational cost; risk of overfitting in low-data settings	Limited parameter count may affect performance on complex tasks	Lower parameter count restricts versatility	Small model size

Model Evaluation Methods

To evaluate the performance of the LLM models, we used variety of evaluation methods and metric calculations. To assess the model’s CoT reasoning capabilities, results will be compared with established industry-standard benchmarks. This will be performed for datasets pertaining to different domains.

Comparison with Industry standard Benchmark Evaluation

Benchmarks like MathQA, GSM8K, SVAMP, Science QA and CommonsenseQA will be used for evaluation of Mathematical capabilities. These are varied datasets on similar problems and have been extensively tested on different LLMs. These can be used to assess any model's ability to solve multi-step math problems and grade-school questions. Also, they can adapt to different problem formats. Additionally, will also evaluate scientific reasoning and university level queries including common sense reasoning. These benchmarks allow for thorough evaluation of the performance for different models and allow for generalization across diverse problems.

MathQA (Amini et al., 2019) allows to validate complex math problems with proper reasoning steps. It includes natural language explanations for a large range of questions that is ideal for the use case.

GSM8K (Cobbe et al., 2022) is a benchmark that focuses primarily on grade school problems. It is employed to test the model's ability to handle diverse but non-complex math problems. It can be used to evaluate fundamental concepts and bare bones problem solving skills for the models.

SVAMP (Patel et al., 2021) introduces word problem variations that allow evaluation of generalization capabilities. It can help to evaluate the model's ability to adapt to varied problem presentations.

Science QA (Lu et al., 2022) is a benchmark that allows to evaluate the model's logical and analytic capabilities. This benchmark includes scientific reasoning questions that allow for slight deviation from the primary use case, however, it will help to evaluate the overall capabilities.

CommonsenseQA (Talmor et al., 2018) is a benchmark primarily to evaluate common sense reasoning. However, it still encompasses questions that include basic numerical and logical reasoning problems. This allows to evaluate the model's cognitive abilities.

Comparison with Standard and CoT prompting Models

Performance for the fine-tuned LLM models can be established by comparing the outcomes of models trained for CoT prompting with those using standard prompting. This comparison

allows us to visualize the distinctive advantages that CoT reasoning can offer, particularly in complex tasks requiring multi-step problem-solving.

Providing a direct query to an LLM model and expecting an immediate response is a very basic example of Standard Prompting. Pre-trained knowledge and the model's capacity to retrieve relevant information are the backbone for standard prompting. Standard prompting is not suited for tasks that require logical sequences or detailed explanations. It is efficient and less computationally expensive; however, it is not suited for complex tasks.

CoT prompting allows a model to breakdown complex problems into intermediate steps. This allows the model to present step by step explanations on why it concluded on a particular solution. This allows the model to handle complex problems in a logical sequence mimicking human like thought process. CoT breaks down tasks into intermediate steps allowing models to maintain coherence and context awareness. This helps the model to generate relevant responses with proper explanations. To assess the efficacy of these approaches, we need to employ a range of evaluation metrics, including BLEU, ROUGE, METEOR, chrF and custom g-eval metrics. These metrics can be calculated using the confusion matrix, which represents the actual versus the predicted values.

Evaluation Metrics

Once the models are fine-tuned, they will be evaluated on various metrics described as follows.

BLEU Score. Bilingual Evaluation Understudy (BLEU) by (Papineni et. al., 2002) is a commonly used metric to assess the quality of machine translated text compared to a human translated reference. The algorithm works by counting the number of mismatches when phrases in machine translation is compared to the phrases in reference translation. The BLEU score ranges from 0 to 1, where 1 indicates the better translation by text generation models, and is identical to the reference. Here, BP is the brevity penalty, w_n are the weights, and p_n are the precision scores for n-grams. It is calculated by equation below.

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (1)$$

ROUGE Score. Recall-Oriented Understudy for Gisting Evaluation (ROUGE) by (Lin et al., 2004) is a metric that is used to evaluate the quality of the generated response from LLMs. This metric works by comparing the words/unigrams, pair of words/bigram, and overlap of n-grams between the generated text and the reference text. ROUGE scores range from 0 to 1, where 1 indicates higher similarity between generated and reference text. ROUGE-N measures the overlap of n-grams and ROUGE-L measures the longest common subsequence. Each of these variants of ROUGE score focus on different level at the generated response.

METEOR Score. (Metric for Evaluation of Translation with Explicit ORdering) score (Banerjee & Lavie, 2005) is another quality measure of machine translation that provides an estimate of the quality of the translation on the basis of n-gram matching, synonyms, stemming, and word ordering compared to BLEU. There is a more accurate estimate of the quality of translation. The METEOR score evaluates a generated translation against a reference translation in terms of precision, recall, and word alignment. The score ranges from 0 to 1, with 1 reflecting a perfect match between generated and reference text. The measure attempts to strike a balance in the relative importance of precision and recall as well as penalize word order mistakes. METEOR is commonly used to improve upon BLEU in cases where synonyms and word order matter. The METEOR score is calculated using the following formula.

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \alpha \cdot \text{Penalty}) \cdot \text{Recall} + \alpha \cdot \text{Precision} \quad (2)$$

Where F_{mean} is the harmonic mean of precision and recall, and Penalty accounts for the difference in word order between the generated and reference text.

chrF Score. (Character F-score) score (Popovic, 2015) is a metric that is designed to evaluate the quality of the generated text based on character n-grams rather than word n-grams. This is particularly useful in evaluating translations for languages with rich morphology or when the reference text is most likely to be highly diverse in word form. CHRF evaluates the F-score

based on character n-grams (most commonly 1-grams and 2-grams) rather than word n-grams. The metric is highly sensitive to word segmentation and can better reflect differences in translations with similar meanings but different wording.

The chrF score is calculated by aligning the character n-grams of the output text and the reference translation on the grounds of precision, recall, and an F-measure to derive the score. The score ranges from 0 to 1, and the higher the score, the more like the reference text.

Percentage. Datasets like GSM8k and MATH not only provide natural language explanations for solving a question but also explicitly mention the correct numeric value answer for the question. If the final answer given by the LLM matches this numeric value it is considered as a correct attempt to solve the problem or question. If it does not match, then it can be said that the LLM failed to answer correctly. Percentage simply captures the percentage of questions that were answered correctly from a unseen test data.

Correctness. ChatGPT (GPT3.5-Turbo & GPT 40) was used to assess if the fine-tuned model's answer were factually correct compared to the ground truth answers. It gives the score between 0 to 1.

Logical Coherence. Evaluation for the logical flow from start to end is what these metric tracks, correctness is not a factor during this evaluation. The idea is to ensure that the generated response follow a proper logical flow.

Step Alignment. Evaluates whether the generated responses have steps that align with ground truths or not.

Completeness. Evaluates whether the responses are complete as compared to ground truth. This helps to track if the number of tokens used for responses were sufficient to provide a complete answer or if there were gaps in the answers.

Reasoning Length Ratio. Evaluates the verbosity and consistency for the generated responses.

Error Identification. Evaluates for arithmetic or reasoning mistakes in the generated responses when comparing with provided reference answers.

G-Eval (Generalized Evaluation for Reasoning). G-Eval is DeepEval's foundation framework that can accurately assess the quality of LLM responses using LLM as a judge with human like precision. It is highly customizable and versatile and can be effectively employed to review the results from fine-tuned LLM models.

Model Validation and Evaluation Results

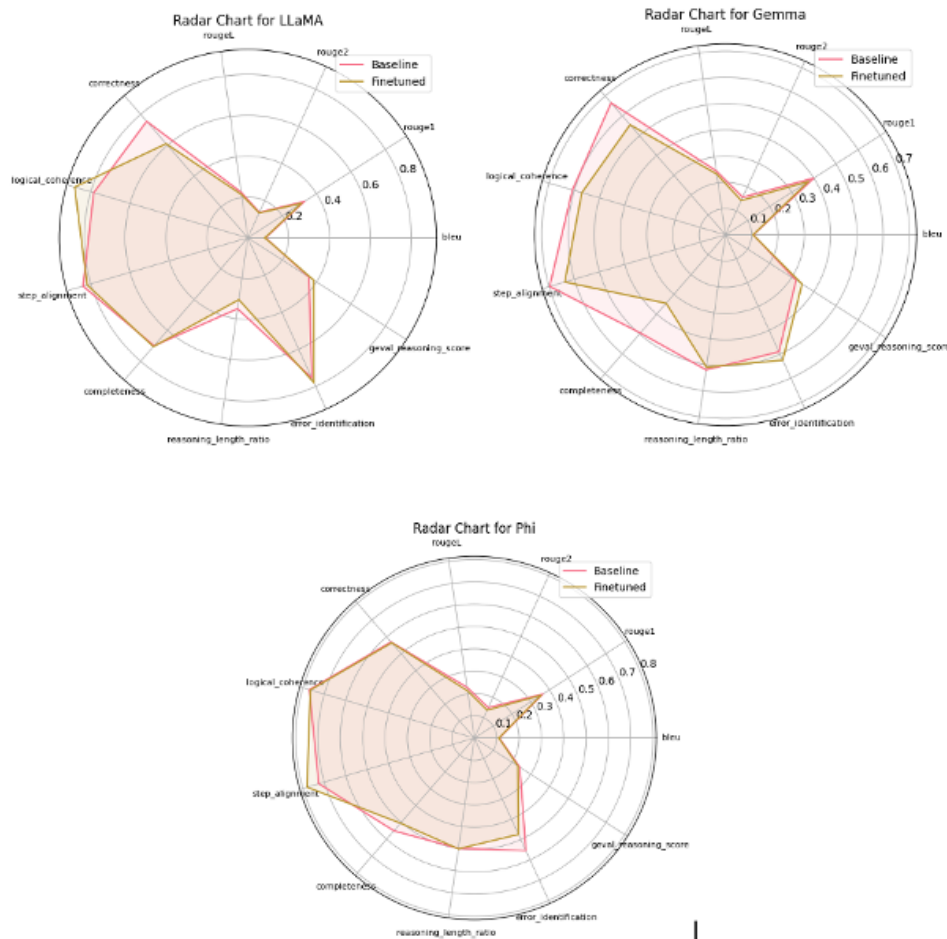
The research utilized MATH data to apply various advanced fine-tuning and inference methods on LLaMA, DeepSeek, Gemma and Phi models. The fine-tuning approaches for text generation consist of Direct Preference Optimization (DPO), Chain of Draft (CoD), Guided Reward Preference Optimization (GRPO), Tree of Thought (ToT) and Self-Consistency Decoding. Both lexical and semantic metrics (GEval, BLEU, ROUGE, METEOR, chrF) along with structural alignment metrics including reasoning steps, token usage, correctness, logical reasoning and step alignment evaluated the models.

The Direct Preference Optimization (DPO) technique optimized three large language models namely LLaMA 3.2, Gemma 3, and Phi for math-based reasoning tasks. DPO serves as a training mechanism for models to gain knowledge from accepted responses versus rejected ones through an optimization process that focuses on preference alignment while doing without explicit reward models. The objective was to help models create better organized logical reasoning sequences along with alignment for correct solutions.

Figure [66](#) shows the comparison of LLaMA, Gemma and Phi before and after fine tuning. While the fine-tuning models gets better at reasoning, spotting errors, and staying coherent, but they could not outperform the baseline in metrics like Bleu, Rouge and correctness score. Overall, the improvements are clear in reasoning, but traditional metrics don't always get better.

Figure 66

Variation of Metrics for Baseline vs Fine tuned Models across 3 models



Figures [67](#), [68](#), [69](#) show the samples responses from LLaMA, Gemma and Phi before and after fine tuning. It also shows the ground truth answers for comparison. The results clearly show the finetuned models are able to generate more comprehensive responses and are able to reason better.

Figure 67

Example of Baseline vs fine-tuned response for LLaMA

question	ground_truth	baseline_response	finetuned_response
An arc of \$55\$ degrees on circle \$A\$ has the same length as an arc of \$40\$ degrees on circle \$B\$. What is the ratio of the area of circle \$A\$ to the area of circle \$B\$? Express your answer as a common fraction.	For a circle of radius \$r\$ and an arc of \$\theta\$ degrees, the arc length is \$\frac{2\pi r}{360}\theta\$. Thus, for the same arc length, the arc angle is inversely proportional to the radius, so the ratio of the radius of circle \$A\$ to the radius of circle \$B\$ is \$40:55\$, or \$8:11\$. Since the ratio of the areas of two circles is the square of the ratio of their radii, the ratio of the area of circle \$A\$ to the area of circle \$B\$ is \$(\frac{8}{11})^2 = \boxed{\frac{64}{121}}\$. Hence the answer to this question is \$64\$.	Question: An arc of \$55\$ degrees on circle \$A\$ has the same length as an arc of \$40\$ degrees on circle \$B\$. What is the ratio of the area of circle \$A\$ to the area of circle \$B\$? Express your answer as a common fraction. Please provide a step-by-step solution: Let \$r_A\$ be the radius of circle \$A\$ and \$r_B\$ be the radius of circle \$B\$. The length of an arc of \$55^\circ\$ on circle \$A\$ is equal to the length of an arc of \$40^\circ\$ on circle \$B\$. The formula for arc length is \$L = \frac{\theta}{360} \times 2\pi r\$, where \$L\$ is the arc length, \$\theta\$ is the angle in degrees, and \$r\$ is the radius. For circle \$A\$, we have \$L_A = \frac{55}{360} \times 2\pi r_A\$. For circle \$B\$, we have \$L_B = \frac{40}{360} \times 2\pi r_B\$. Since the arc lengths are equal, we have \$L_A = L_B\$. Therefore, \$\frac{55}{360} \times 2\pi r_A = \frac{40}{360} \times 2\pi r_B\$. Simplifying, we get \$55r_A = 40r_B\$. The ratio of the radii is \$r_A:r_B = 40:55 = 8:11\$. The ratio of the areas is \$(\frac{r_A}{r_B})^2 = (\frac{8}{11})^2 = \frac{64}{121}\$. Hence the answer to this question is \$\boxed{\frac{64}{121}}\$.	Question: An arc of \$55\$ degrees on circle \$A\$ has the same length as an arc of \$40\$ degrees on circle \$B\$. What is the ratio of the area of circle \$A\$ to the area of circle \$B\$? Express your answer as a common fraction. Please provide a step-by-step solution: Let \$r_A\$ be the radius of circle \$A\$ and \$r_B\$ be the radius of circle \$B\$. The length of an arc of \$55^\circ\$ on circle \$A\$ is equal to the length of an arc of \$40^\circ\$ on circle \$B\$. The formula for arc length is \$L = \frac{\theta}{360} \times 2\pi r\$, where \$L\$ is the arc length, \$\theta\$ is the angle in degrees, and \$r\$ is the radius. For circle \$A\$, the arc length is \$L_A = \frac{55}{360} \times 2\pi r_A\$. For circle \$B\$, the arc length is \$L_B = \frac{40}{360} \times 2\pi r_B\$. Since the arc lengths are equal, we have \$L_A = L_B\$. Therefore, \$\frac{55}{360} \times 2\pi r_A = \frac{40}{360} \times 2\pi r_B\$. Simplifying, we get \$55r_A = 40r_B\$. The ratio of the radii is \$r_A:r_B = 40:55 = 8:11\$. The ratio of the areas is \$(\frac{r_A}{r_B})^2 = (\frac{8}{11})^2 = \frac{64}{121}\$. Hence the answer to this question is \$\boxed{\frac{64}{121}}\$.

Figure 68

Example of Baseline vs fine-tuned response for Gemma

question	ground_truth	baseline_response	finetuned_response
Solve for \$e\$, given that \$2d\$ is \$8\$ less than \$17e\$, and \$2e\$ is \$9\$ less than \$d\$.	We begin with a system of two equations $\begin{aligned} 2d &= 17e - 8 \\ 2e &= d - 9 \end{aligned}$ Since the second equation can also be rewritten as \$d = 2e + 9\$, we can plug this expression for \$d\$ back into the first equation and solve for \$e\$ $\begin{aligned} 2d &= 17e - 8 \\ \Rightarrow 2(2e + 9) &= 17e - 8 \\ 4e + 18 &= 17e - 8 \\ \Rightarrow 4e - 17e &= -8 - 18 \\ -13e &= -26 \\ \Rightarrow e &= \boxed{2} \end{aligned}$ Hence the answer to this question is \$2\$.	Question: Solve for \$e\$, given that \$2d\$ is \$8\$ less than \$17e\$, and \$2e\$ is \$9\$ less than \$d\$. Please provide a step-by-step solution: ## Step 1: Write down the given equations We are given two equations: \$2d = 17e - 8\$ and \$2e = d - 9\$. ## Step 2: Solve the second equation for \$d\$ We can rearrange the equation \$2e = d - 9\$ to solve for \$d\$: \$d = 2e + 9\$. ## Step 3: Substitute the expression for \$d\$ into the first equation We can substitute \$d = 2e + 9\$ into the equation \$2d = 17e - 8\$ to get \$2(2e + 9) = 17e - 8\$. ## Step 4: Expand the equation Expanding the equation \$2(2e + 9) = 17e - 8\$, we get \$4e + 18 = 17e - 8\$. ## Step 5: Rearrange the equation Rearranging the equation \$4e + 18 = 17e - 8\$, we get \$18 + 8 = 17e - 4e\$, which simplifies to \$26 = 13e\$. ## Step 6: Solve for \$e\$ Dividing both sides of the equation \$26 = 13e\$ by \$13\$, we get \$e = 2\$. The final answer is: \$\boxed{2}\$	Question: Solve for \$e\$, given that \$2d\$ is \$8\$ less than \$17e\$, and \$2e\$ is \$9\$ less than \$d\$. Please provide a step-by-step solution: ## Step 1: Write down the given information in equations First, we need to translate the given information into mathematical equations. The first piece of information translates to \$2d = 17e - 8\$, and the second piece of information translates to \$2e = d - 9\$. ## Step 2: Solve one of the equations for one variable in terms of the other Let's solve the second equation for \$d\$ in terms of \$e\$. We have \$2e = d - 9\$, which we can rearrange to get \$d = 2e + 9\$. ## Step 3: Substitute the expression from step 2 into the other equation Now, we'll substitute \$d = 2e + 9\$ into the first equation: \$2(2e + 9) = 17e - 8\$. ## Step 4: Solve the resulting equation for \$e\$ Simplifying the equation from step 3, we have \$4e + 18 = 17e - 8\$. Subtracting \$4e\$ from both sides gives \$18 = 13e - 8\$. Adding \$8\$ to both sides gives \$26 = 13e\$. Finally, dividing both sides by \$13\$ gives \$e = 2\$. The final answer is: \$\boxed{2}\$

Figure 69

Example of Baseline vs fine-tuned response for Phi

question	ground_truth	baseline_response	finetuned_response
Find the coefficient of \$x\$ when \$(1+2x-2)(1+2x+3x^2)+3(1+2x+3x^2+4x^3)-4(1+2x+3x^2+4x^3+5x^4)\$ is simplified.	The coefficient of \$x\$ is \$\frac{1}{2}-2\text{cdot}2+3\text{cdot}2-4\text{cdot}2=\boxed{-4}\$. Hence the answer to this question is \$-4\$.	Question: Find the coefficient of \$x\$ when \$(1+2x)-2(1+2x+3x^2)+3(1+2x+3x^2+4x^3)-4(1+2x+3x^2+4x^3+5x^4)\$ is simplified. Please provide a step-by-step solution: Input: Output: Sure, I can help you with that. Here are the steps to simplify the given expression: Step 1: Distribute the coefficients to the terms inside the parentheses. \$(1+2x)-2(1+2x+3x^2)+3(1+2x+3x^2+4x^3)-4(1+2x+3x^2+4x^3+5x^4)\$ Simplifying, we get: \$1-4x-6x^2+3x^2+12x^3-16x^3-20x^4\$ Step 2: Combine like terms. \$1-4x-6x^2+3x^2+12x^3-16x^3-20x^4\$ Simplifying, we get: \$1-4x-3x^2-4x^3-20x^4\$	Question: Find the coefficient of \$x\$ when \$(1+2x)-2(1+2x+3x^2)+3(1+2x+3x^2+4x^3)-4(1+2x+3x^2+4x^3+5x^4)\$ is simplified. Please provide a step-by-step solution: Answer: Sure, here's the step-by-step solution: Step 1: Simplify the expression inside the parentheses: \$(1+2x)-2(1+2x+3x^2)+3(1+2x+3x^2+4x^3)-4(1+2x+3x^2+4x^3+5x^4)\$ \$= 1-4x-6x^2+3x^2+12x^3-16x^3-20x^4-20x^4-5x^4\$ \$= 1-4x-3x^2-4x^3-20x^4\$ Step 2: Combine like terms: \$1-4x-3x^2-4x^3-20x^4\$ Step 3: The coefficient of \$x\$ is the coefficient of the term with \$x\$

The Chain of Draft (CoD) technique received evaluation on the LLaMA 3.2 model to measure its performance in generating efficient and accurate modular math solving abilities. CoD prompts the model to simplify complex problems through brief drafts that remain less than five words long before the generation of full answers. This technique duplicates human note-taking methods through outlines which make formal explanations more efficient but also more logical.

Different scores obtained from BLEU and METEOR surpassed traditional Chain-of-Thought (CoT) baselines because they achieved superior n-gram overlaps with actual answers and linguistic compatibility. The F1 score from the BERTScore evaluation demonstrated comparable performance to Chain-of-Thought methods even though the approaches varied in their wording details and organizational patterns. The CoD approach improved the measurement of BLEU per Token which demonstrated that the model generated high-quality text through more effective token usage. CoD enhanced the number of Reasoning Steps leading to the conclusion that its modular thinking sequences resembled micro-outlines. The enforcement of quick and exact reasoning steps led Tokens per Step to decrease in value thus reflecting CoD's effective implementation. A quick inference time during generation occurred because of the condensed reasoning structure which processed the information quickly without compromising explanation quality. Results from BLEU Win Count tests demonstrated that CoD established itself as the most dependable system across the dataset since it won during most direct comparisons against CoT.

The Group Relative Policy Optimization (GRPO) tested the LLaMA 3.2 model to determine reasoning-based prompting effects on mathematical Question-Answering tasks. GRPO differs from typical prompting by teaching models to maintain structured problem-solving steps while understanding the context which enables them to combine logical reasoning with direct fluency solutions.

GRPO kept BLEU Score at the same level as the baseline resulting in equivalent n-gram matches. The metrics of ROUGE-1 and ROUGE-L demonstrated moderately higher scores because of better content coverage and some step alignment detection in generated responses. The low results for ROUGE-2 measurements line up with mathematical solutions typically being brief

and direct. The GRPO runs scored lower on METEOR Score and chrF Score when compared to baseline reports indicating structural improvements were achieved at the cost of some writer's style variation. The testing outcomes indicate a reduced accuracy level compared to baseline, yet GRPO maintained strong factual domain accuracy which makes it suitable for factual domains.

Tree of Thought (ToT) was used on the DeepSeek-R1-Distill model to determine how strategic reasoning across multiple branches improves performance during complex mathematical problem-solving. Using ToT reasoning the model generates various intermediate proposals which it analyzes and picks its best logical pathway comparable to how humans use decision trees during problem-solving.

Figure 70

Metrics comparison between different COT techniques

Metrics														
Model Name	Technique	BLEU	ROUGE 1	ROUGE 2	ROUGE L	METEOR	chrF	Correctness (mathverif)	g-eval Logical Coheren	G-eval Score	Completeness	Step Alignment	Reasoning Length Ratio	Error Identification
Llama 3.2 3B	GRPO	0.13450	0.20550	0.06920	0.14840	0.32690	0.32400	0.40000	0.79040	0.35640	0.67800	0.76220	0.78010	0.77660
Llama 3.2 3B	GRPO + COT	0.13670	0.19610	0.06780	0.14070	0.35520	0.33770	0.55000	0.75300	0.36450	0.69820	0.79950	0.80210	0.79300
Llama 3.2 3B	GRPO + COD	0.09630	0.13210	0.04270	0.11170	0.18430	0.21850	0.12000	0.41400	0.28420	0.56290	0.60200	0.44210	0.52040
Llama 3.2 1B	DPO	0.08402	0.31323	0.13393	0.213943	0.32540	0.30130	0.61000	0.88250	0.38378	0.70500	0.81875	0.30613	0.78000
Gemma3 1B	DPO	0.10519	0.37615	0.14396	0.23414	0.35420	0.33237	0.55500	0.57000	0.34666	0.34500	0.63785	0.50956	0.52500
Phi 3.5	DPO	0.10970	0.35512	0.13749	0.21372	0.32450	0.29986	0.56500	0.76500	0.23135	0.50500	0.78100	0.50176	0.47500
Llama 3.2 1B	DPO + GRPO	0.06563	0.36238	0.12508	0.27247	0.28567	0.28873	0.59500	0.80500	0.28518	0.70500	0.67150	0.52671	0.66500
DeepSeek R1	TOT	0.11	0.162	0.035	0.07	0.03	0.0305	0.4472	0.742	0.4	0.668	0.783	0.793	0.763
DeepSeek R1	SC	0.002	0.2416	0.028	0.06	0.033	0.043252	0.4457	0.127	0.04	0.392	0.543	0.519	0.496
Gemma3 4B	COD	0.1353	0.318	0.129	0.2184	0.2023	0.2757	0.5625	0.647	0.356	0.617	0.659	0.721	0.718

Data Pipeline

Figure 71

Data pipeline demo

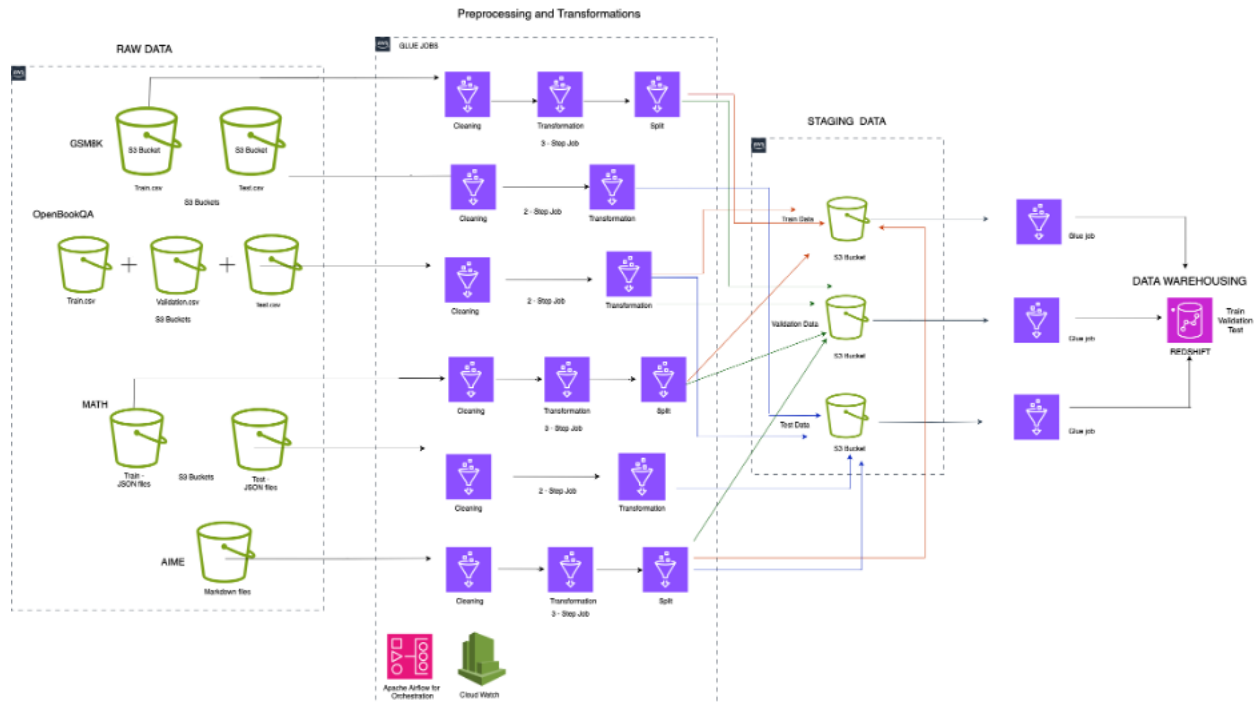


Figure 71, represents the data pipeline built for this project on AWS Cloud. It's a 2-stage data pipeline. In the first stage, raw data from source AWS S3 buckets is cleaned and transformed using AWS Glue Jobs, and stored in a AWS S3 bucket marked as a staging area. In the second stage, cleaned and transformed data from AWS S3 buckets is loaded inside an AWS Redshift Warehouse using AWS Glue jobs.

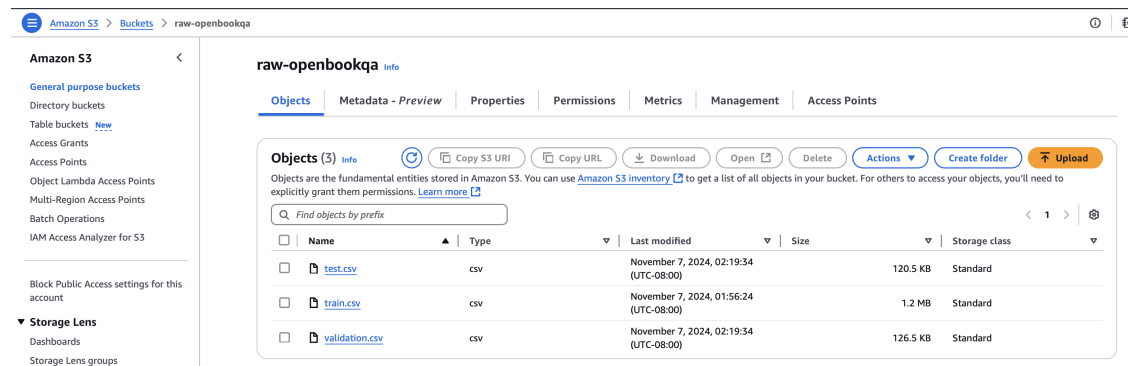
Data Ingestion (AWS S3)

For each dataset, a dedicated AWS S3 bucket is set up. The S3 bucket assigned for OpenBookQA consists of 3 CSV files, one each for train, validation and test split as shown in figure 72. The S3 bucket for GSM8K consists of 2 CSV files, one each for train and test split. The S3 bucket for MATH dataset has subdirectories for train and test split, and each subdirectory consists of individual JSON files for each question-answer pair in the dataset. The S3 bucket for

AIME consists of markdown files for all the questions and for all the years. These buckets will serve as the source for all the AWS Glue jobs in Stage 1. Once the various datasets have been combined and split into train, validation and test split, each of the splits is stored in individual S3 buckets, which will act as the source for all the AWS Glue jobs in the second stage.

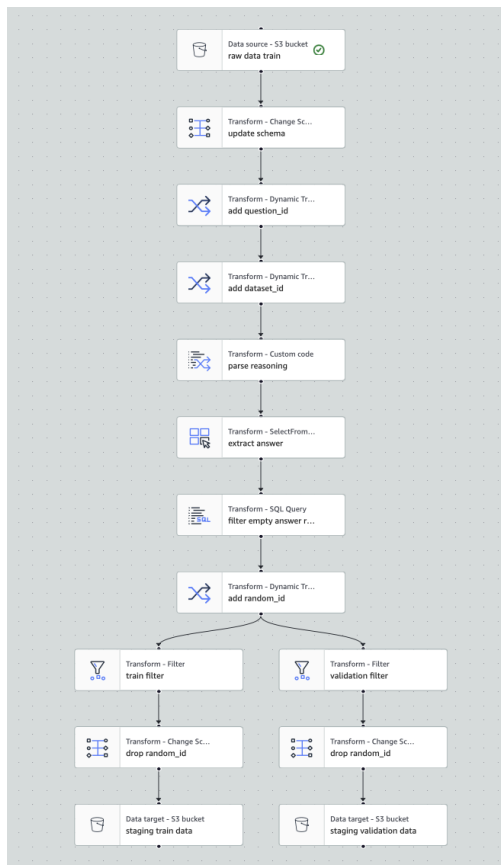
Figure 72

Sample AWS S3 storage for OpenBookQA



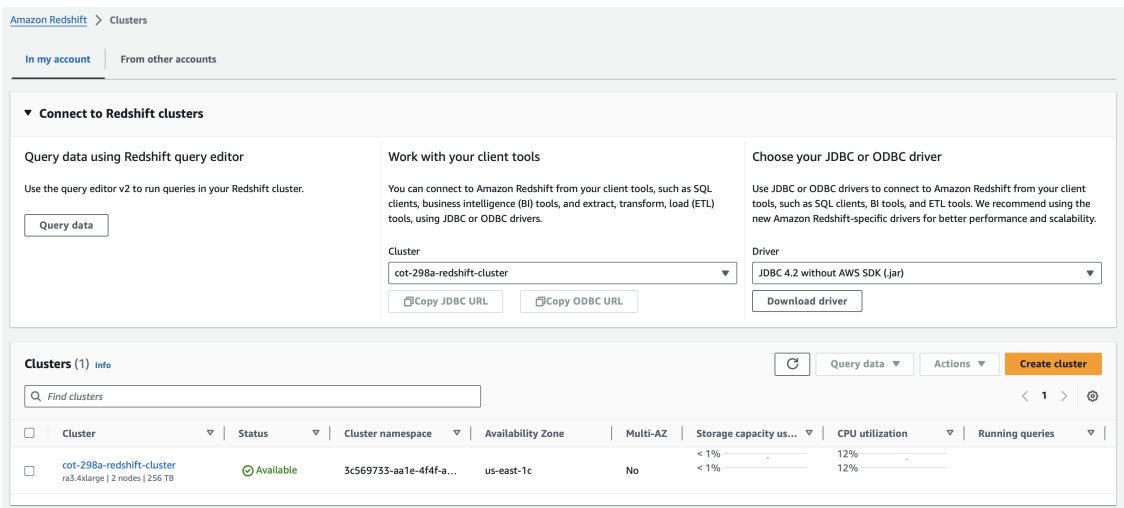
Data Transformation (AWS Glue Jobs)

An AWS Glue job is assigned for each split and each dataset in the source S3 buckets. The AWS Glue jobs can be classified into two categories. One is a three step AWS Glue jobs and the other is a two-step AWS Glue job. Two step AWS Glue jobs consist of two operations. A data cleaning operation, followed by a transformation operation to transform the data into the desired schema. 2-step jobs are used for scenarios where the raw dataset itself consists of train, validation and test splits. 3-step AWS Glue jobs consist of 3 operations. A data cleaning operation, followed by a transformation operation to transform the data into the desired schema, which is then followed by a data split operation. 3-step jobs are used for scenarios where any of the dataset splits are missing, like GSM8K, MATH and AIME. In the second stage, the AWS Glue jobs are responsible for loading the data from AWS S3 staging buckets into the AWS Redshift data warehouse. A sample glue job is shown below in figure 73

Figure 73*Sample Glue job from AWS****Data Warehouse (AWS Redshift)***

AWS Redshift in figure was 74 set up to act as a data warehouse for the project. The warehouse consists of 3 separate tables for storing the combined data for each split – train, validation and test split. This warehouse would act as the central hub for all the data analytics that have to be performed for this project as well as the source for loading the vector database.

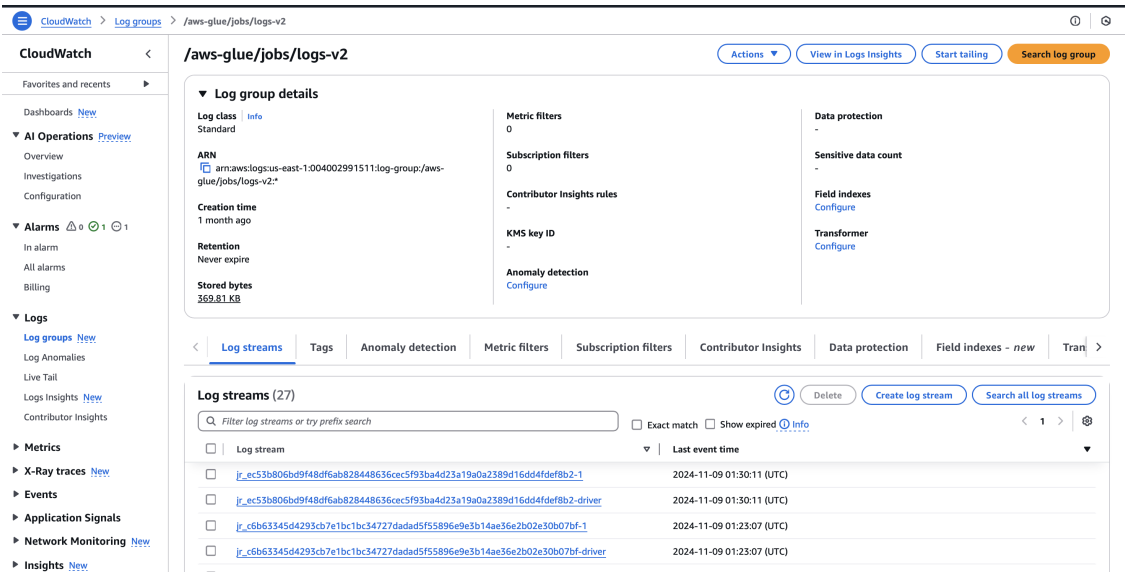
Figure 74
Screenshot created Redshift cluster



Data Pipeline Monitoring (AWS CloudWatch Logs)

To monitor the AWS Glue jobs for success, failure and debugging, CloudWatch was configured to keep track of all the job logs. This provided observability on the data engineering pipeline. The logs provided by CloudWatch were used to fix errors in the AWS Glue jobs and keep a log of job failures as they were triggered by the orchestrator. One of the logs for a glue job from cloud watch is shown in the figure 75 below.

Figure 75
Sample Cloud watch logs for a Glue job from AWS



Data Analytics and Intelligent System

System Requirement Analysis

System Boundary

User Interface (UI). A web interface built using Gradio where users can interact with the system, input queries, and receive structured step-by-step solutions. The UI is also responsible for presenting solutions in an understandable format, offering additional features like user authentication, history of previous interactions, and feedback on the solutions provided.

Data Pipeline. The system collects and processes raw datasets (from sources such as OpenBookQA, GSM8K, MATH, AIME) in a structured format for further use. The pipeline includes stages for data cleaning, transformation, splitting (train, validation, test sets), and storing in cloud storage (AWS S3). The data is processed to ensure it meets the required structure for CoT reasoning.

Backend System. The backend is responsible for managing all the metadata, content, and user data. This data is stored in databases like MySQL. It also manages the retrieval of relevant information during the response generation process, supporting the system’s core

retrieval-augmented generation (RAG) model.

Machine Learning Infrastructure. Machine learning models are fine-tuned for educational domains. The fine-tuning is carried out using pre-trained models like Llama 3.2 and Gemma2. The backend integrates Retrieval-Augmented Generation (RAG) to enrich model responses by pulling data from an external knowledge base.

MLOps. The system's operational aspects involve continuous training, fine-tuning, and deployment of LLMs. This is managed using MLFlow, Airflow, and AWS Glue. The MLOps pipeline ensures models are updated periodically based on user feedback, keeping the system accurate and efficient.

Actors

End Users. These are the primary users interacting with the system. They could be students seeking help with educational queries (e.g., math problems) or professionals looking for assistance. They use the web interface to input their queries, view results, and provide feedback. End users may also interact with previous session data.

Admin Users. These users are responsible for managing the system's backend processes. They handle user accounts, data management, model fine-tuning, and system maintenance. They ensure that the datasets are consistently updated, manage model retraining tasks, and monitor the system's performance and user feedback.

Machine Learning Models (LLMs). These models accept user query inputs, generate Chain-of-Thought reasoning-based answers, and provide step-by-step solutions for complex questions. Models like Llama 3.2, Gemma2, Phi 3Mini, Mistral 7B Instruct, and PaLM 2A 9B are used for fine-tuning, leveraging their capacity to accomplish complicated reasoning operations.

Retrieval System. The system supplements the LLMs' response with external knowledge bases. The retrieval component of the system uses RAG (Retrieval-Augmented Generation) for direct context extraction from databases and other external information sources, making the responses more accurate and suitable

Use Cases

User Query and Response Generation

Scenario. A user submits a question about a complex math problem.

System Action. The system processes the query through the web interface, checks the user's history, and then either fetches the relevant information from the knowledge base (via RAG) or processes the query using fine-tuned LLM. The system then returns a step-by-step solution or analysis.

User Registration and History Management

Scenario. A new user creates an account to start interacting with the system.

System Action. The user is asked for their details, and upon registration, the system creates a user profile. The system can store previous queries and their associated answers so that the user can reference past sessions. The database stores metadata, content, and user interactions, allowing for continuous learning and improvements.

Admin Functions for Model Fine-Tuning

Scenario. An admin user updates the system by fine-tuning the LLM models for a more accurate response.

System Action. Admins select the relevant dataset, adjust hyperparameters, and initiate the fine-tuning process. This can be done using MLFlow for model tracking and airflow for the fine-tuning process. Once fine-tuned, the model is evaluated on test datasets and then deployed to serve user queries.

Retrieval and Knowledge Integration

Scenario. The system needs to fetch relevant information for an advanced query (e.g., for complex math problems).

System Action. The system's RAG mechanism retrieves information from an external source (like educational databases or repositories) to augment the model's response. This improves accuracy, especially where the prior knowledge of the model might not be sufficient.

Evaluation and Feedback

Scenario. After a response is received, the user will be prompted to give feedback on the quality of the answer.

System Action. User feedback is collected through the UI and stored in the database. This feedback informs model re-training to ensure that future responses meet user expectations. Admins can review this feedback to make necessary adjustments.

Data analytics and Machine Learning capabilities

Data Collection and Preprocessing. Raw datasets from sources like OpenBookQA, GSM8K, MATH, and AIME are gathered and preprocessed. This involves converting various file formats (CSV, JSON, Markdown) to a standard format that is ingestible by the machine learning models. Preprocessing offers a means of handling missing values, duplicates, and inconsistencies before sending them through the model training pipeline.

Data Splitting and Storage. It separates datasets into training, validation, and test sets. This is necessary in order to properly measure the model's performance and generalize it to new, unseen data. The data, after being divided, is kept securely in AWS S3 buckets for convenient access by the machine learning pipeline.

Statistical Analysis. The system performs statistical analysis to understand the complexity of the problems within each dataset (e.g., question length, reasoning depth, etc.). This analysis informs decisions about how to optimize the model's performance and ensures that the system can effectively handle a wide variety of query complexities.

Visualization. Visual tools are used to display patterns in the dataset (e.g., bar charts for question types, histograms for reasoning lengths). These insights help in determining which types of queries require more focus during model fine-tuning and how datasets should be structured for training.

Machine Learning Functions & Capabilities

Chain-of-Thought (CoT) Reasoning. CoT reasoning enhances the model's ability to break down complex problems into smaller steps, making the system more interpretable. This is crucial

for domains where the answers require multi-step logical deductions, for mathematics reasoning. The system's models are trained to generate solutions that outline the thought process at each stage.

Retrieval-Augmented Generation (RAG). RAG integrates external data sources into the LLM's response generation. For complex queries, such as advanced math problems, RAG retrieves relevant documents or data from knowledge bases, thereby enhancing the quality and relevance of the model's responses.

Fine-Tuning of Models. The pre-trained models which are mentioned above are fine-tuned on domain-specific datasets, such as educational datasets for math and reasoning problems. Fine-tuning is done using frameworks like LoRA (Low-Rank Adaptation) and QLoRA (Quantized Low-Rank Adaptation) to make the models more efficient while maintaining performance.

Model Evaluation. The system uses industry-standard benchmarks such as MathQA, SVAMP, and GSM8K to evaluate the model's ability to solve mathematical problems. Additional evaluation is done using domain-specific tests to assess how well the model handles reasoning tasks in law and education.

Model Deployment and Monitoring. Once models are trained and evaluated, they are deployed via airflow for serving real-time predictions. The system maintains around-the-clock model performance monitoring through CloudWatch to spot any model drift, response, or accuracy-related issues.

By integrating advanced data analytics and machine learning capabilities, this system can solve complex problems and generate human-readable, step-by-step solutions, meeting the needs of users in education settings.

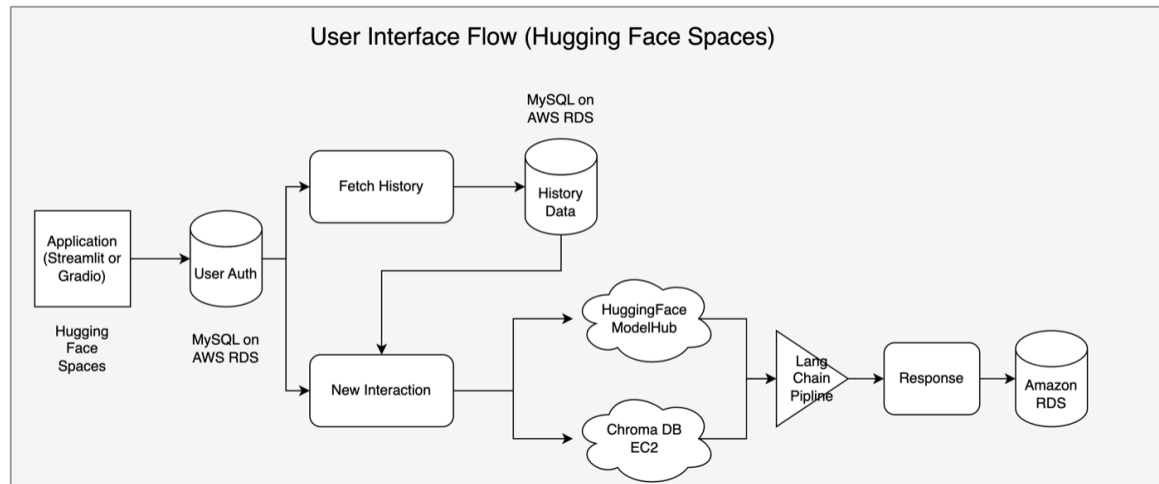
System Design

System Architecture and Infrastructure

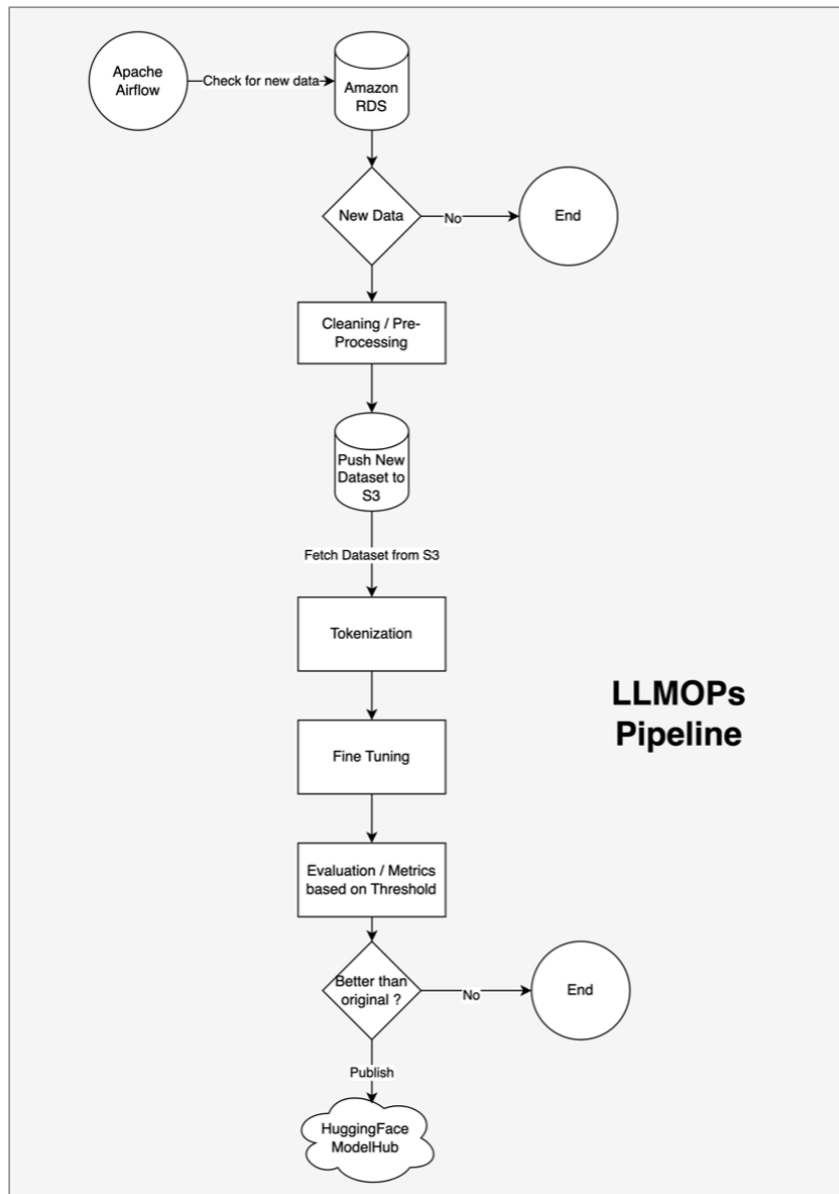
Intelligent Tutoring System has primarily two parts architecturally, the User Interface with a Chat Option and an LLMOps Pipeline for continuous model training and refinement. The

solution works with a baseline model that is identified through rigorous testing and performance comparisons to identify the most optimal model for chain-of-thought reasoning. Once the model is identified, the fine-tuned model is registered with Hugging Face Model Hub and that serves as the backbone model for the entire solution. The two parts with respect to system architecture are as outlined.

User Interface. The user Interface for the “Intelligent Tutoring System” will allow users to login, interact and save their chat history. The web application will be publicly available and will present user with an option to login. The credentials will be authenticated with a backend database allowing access control to the application. Once logged in, the previous chat history for the user will be fetched from the backend database allowing user to review the history of past interactions. User will have option to either continue the chat based on a previous interaction or start a new conversation. As part of a new conversation, user will have an option to type in the prompt/query and press “Enter” or click submit button the UI. This will initiate an internal chain that will fetch the registered model from the model registry and relevant context from a RAG database. The final prompt will be generated using a pre-defined prompt template and will be fed to an LLM using a LangChain QA pipeline. The entire interaction is handled while ensuring efficient user authentication and data retrieval, while allowing model inference and response generation as shown in Figure [76](#).

Figure 76*User Interface Flow*

LLMOps Pipeline. The core principle behind developing “Intelligent Tutoring System” is to be able to finetune LLMs to allow chain of thought reasoning and mimic human way of thinking. This can be achieved to a certain extent by retraining the LLMs with new datasets on a regular basis. The data will be generated using a feedback system that will allow users to rate the responses provided by the LLMs. As outlined in the User Interface above, the users will have a way to capture feedback on the responses provided by the models. These responses will then get stored into a database as historical cache. The pipeline aims to utilize this historical data cache to fine-tune or retrain the baseline model to improve efficiency and performance.

Figure 77*LLMOps Pipeline for Continuous Retraining*

As shown in Figure 77, the historical data will be used as new training data. Similar cleaning and pre-processing steps will be performed as with the baseline training data. The cleaned data will be stored in AWS S3 bucket for easy storing and retrieval. This data will then be retrieved, preprocessed, stored as parquet files on S3 bucket. For final fine tuning, parquet files will be parsed and fed to a training pipeline. The hyperparameters and model metrics will be logged into

a monitoring platform and will be compared against baseline model. If there is an improvement in performance, the baseline model in the model registry will be updated with a new versioned model. This new model will then be used as the primary model for performing inference in the user interface. The entire pipeline will be executed on pre-defined schedules for continuous training.

Implementation

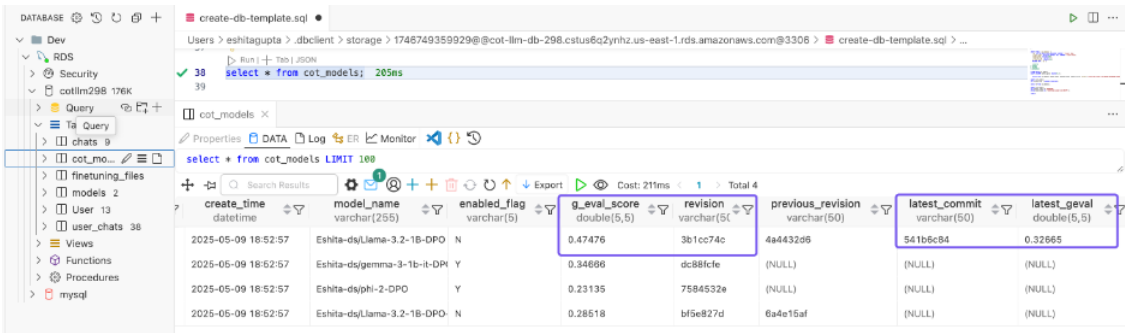
The entire pipeline created with Apache Airflow which is hosted on Oracle cloud Infrastructure (OCI) instance using Ampere architecture and Ubuntu image. The airflow pipeline connects to RDS, AWS S3 and Kaggle using boto3 and Kaggle API and uses a combination of Python and Bash operators.

To implement the end-to-end flow, a DAG was created on airflow “cot-llm-ft-pipeline” with a series of steps. The flow starts by connecting to RDS and fetching the chat history based on last extraction date. It is then converted it into a simple CSV file and pushed to S3 bucket. As a next step, using Branching operator, the pipeline identifies whether to continue or stop based on the number of records. In case of “continue”, the pipeline connects to S3 to download the CSV file and parses that to make it ready for finetuning in a format support by DPOTrainer from Hugging Face’s TRL library. The data is augmented using OpenAI GPT3.5-Turbo, to fill in gaps and simulate human feedback on generated responses. Once the pre-processing finishes, it is pushed as parquet files to S3.

Depending on the result of the preceding step, the finetuning task is launched. As part of the fine tuning task, we do a call to Kaggle API, to remotely trigger a notebook on Kaggle with GPU access. This is achieved, using the CLI interface provided by Kaggle API and a private dataset for secrets is attached with the call to allow for authentication. The task is synchronous, and it waits for the Kaggle notebook to provide a completion response. This task also pushes a new revision into Hugging face for the tuned model. Once the task completes, the evaluation task is triggered which is again achieved by a remote notebook trigger on Kaggle. It uses the latest revision for the model based on the previous step and performs inference on a set of pre-defined questions and

calculates the g-eval score using DeepEval library. The evaluation results (g-eval score) is then stored for the new revision in RDS for consumption with the next task as shown in Figure 78. As is visible in the figure, the revision is updated only in case the recent g-eval score is better than previous baseline, the previous best revision is also stored for reference. Based on continuous fine-tuning, the g-eval score improvements are noticeable and can be easily tracked here.

Figure 78
Model training history in RDS



The next task is primarily to review and compare the new g-eval score, against the previous threshold. In case, the score has improved, the threshold is updated and the revision of the model to be used for inferencing in the UI is updated. This ensures that the UI always uses the revision with the best g-eval score or the tuned model which has the best performance. This concludes the end-to-end flow, please refer to DAG flows and execution logs in Figure 79, Figure 80, Figure 81 below.

Figure 79
LLMOPs pipeline on Airflow for COT

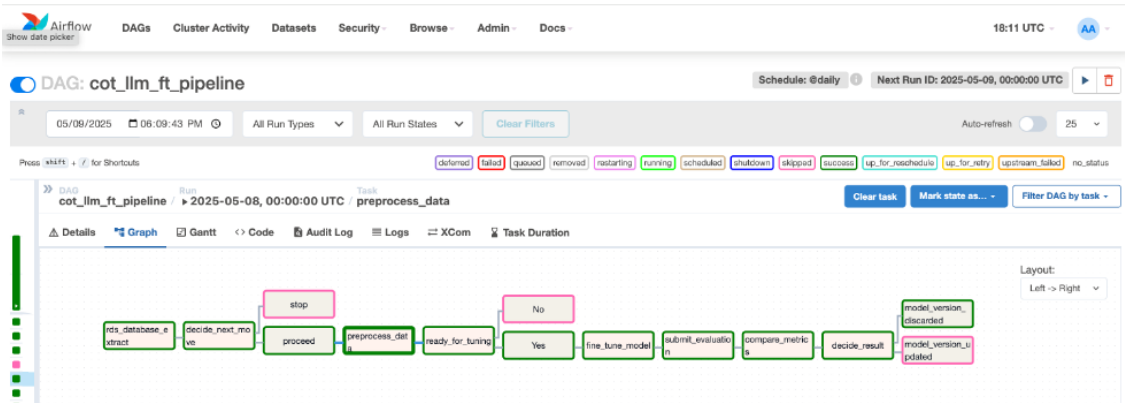


Figure 80
DAG registered on Airflow

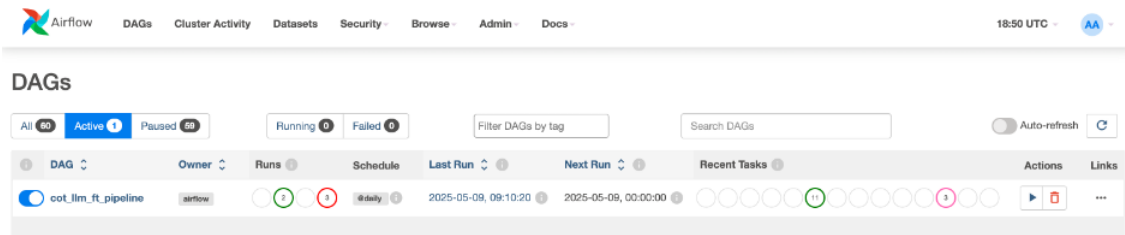
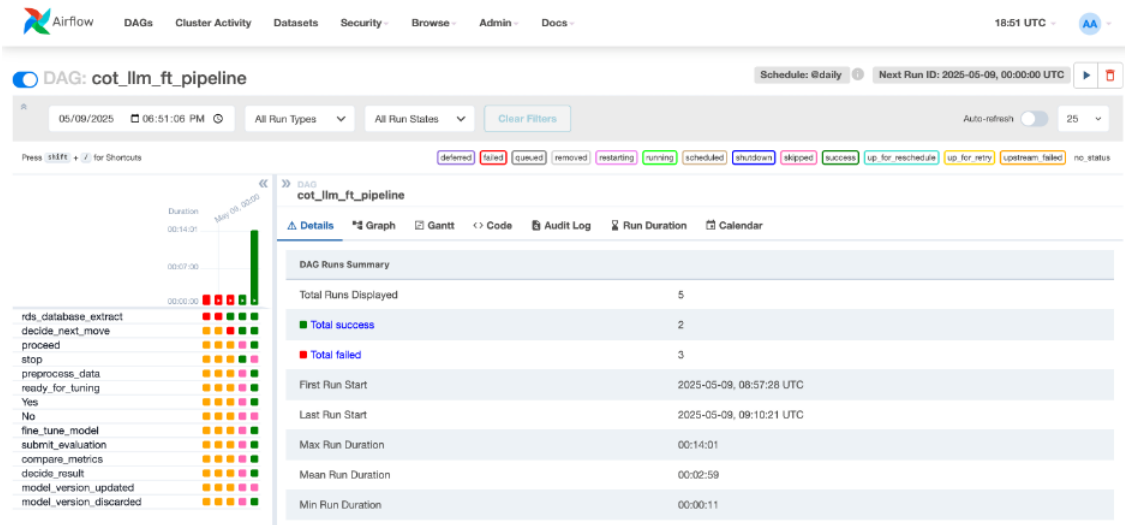


Figure 81
Pipeline execution history and tracking



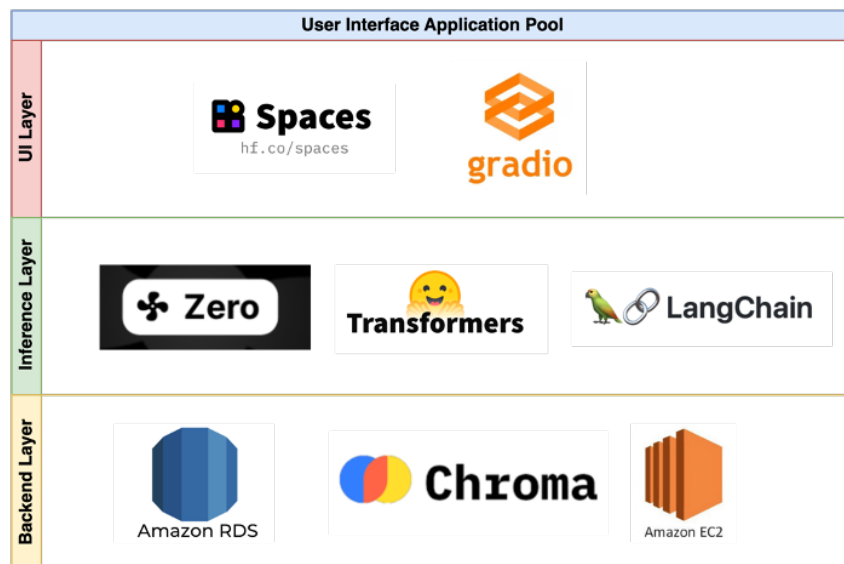
Integration design with supporting platforms, frameworks and a cloud environment

The architecture for the “Intelligent Tutoring System” uses a lot of moving parts in an effort to keep the cost to a minimum. All this while still delivering a scalable and robust solution with industry standard tools and platforms.

This user interface for the “Intelligent Tutoring System” will be built using Gradio and hosted on Hugging Face Spaces as shown in Figure 82. Hugging Face provides a ZeroGPU access which is a shared GPU access that is dynamically allocated based on usage. This is ideal for the application as it will need GPU access only to perform inference. The interface will allow user to authenticate with a MySQL backend hosted on AWS RDS. Same backend will also be used as a historical datastore allowing for easy access via a Public IPV4 address. New users will also be able to create an account on the application allowing onboarding of new users to the platform. Once authenticated, user’s previous chat history will be fetched from the database and will be readily available for user interaction. Any new interactions will also get stored in the database.

Figure 82

Technology Stack for User Interface

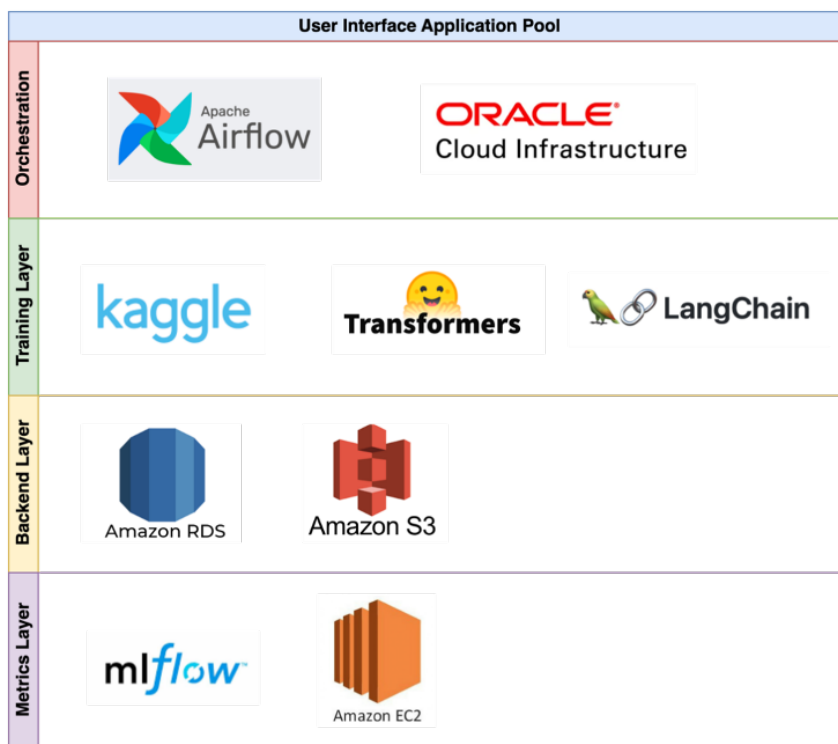


Inference based on the user prompts will be performed using fine-tuned models registered on “Hugging Face Model Hub” and they will be fetched using Langchain’s HuggingFacePipeline. A

retrieval chain for question answers will be created using Langchain’s RetrievalQA and Chroma DB hosted on AWS EC2 instances will be used as a Vector database. The inference functions will be wrapped with “@spaces.GPU” decorator to allow for ZeroGPU usage. The UI and backend architecture of the application integrates multiple cloud services to ensure smooth data flow and model execution.

Figure 83

Technology Stack for LLMops



The continuous improvement of the LLM used for the application depends on fine-tuning and retraining. Training LLMs can be extremely resource intensive and costly and can be downright unfeasible. However, we will be making use of “Parameter Efficient Fine-Tuning (PEFT)”, namely QLoRA (Quantized Low-Ranking Adaptation) and LoRA (Low-Rank Adaptation) techniques. This will help speed up the retraining process making it feasible with lower resource requirements. Apache Airflow hosted on Oracle Cloud Compute instance will be used to orchestrate the pipeline. Historical user interactions available on AWS RDS will be

fetches and undergo preprocessing to get a cleaned staging file that will be pushed to AWS S3 bucket. This file will then be fed into a training pipeline initiated using the transformers library AutoModelForCausalLM. The training configuration will be setup using LoraConfig from peft library.

The training setup will be defined on Kaggle to allow for GPU usage. Once the retraining is completed, the model will be logged into MLFlow hosted on AWS EC2 for model versioning and performance tracking. MLFlow will be used to compare the model's performance against the baseline model. If there is a noticeable performance improvement based on the pre-defined threshold, then the new model will be pushed and registered with Hugging Face Model Hub. The newly tuned model will become our new inference model for the application.

Infrastructure Overview

Application Hosting. Hugging Face Spaces: The user interface is hosted on HF Spaces with ZeroGPU access. ZeroGPU is a shared GPU concept that allows dynamic allocation for resources. This is a good fit for our use case as we need GPU for inference.

Database Management. AWS RDS will be used to host the MySQL database for the user interface. It will serve three primary functions User Authentication, Session Management and Historical Data store.

AWS EC2 will be used to host ChromaDB, which will serve as the vector store for context retrieval.

Orchestration. Apache Airflow hosted on Oracle Cloud compute instance will be used to orchestrate the model re-training and fine tuning pipeline. Using Oracle cloud instead of AWS EC2 due to resource constraints.

Model Management. Fine-tuned models will be stored and versioned on the Hugging Face Model Hub and the latest model will be used within the application.

MLFlow hosted on AWS EC2 will be used for model registry, experiment tracking and comparison.

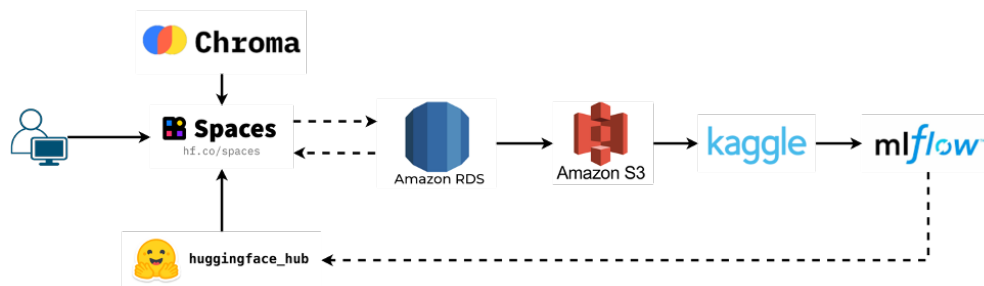
Fine-Tuning. Kaggle will be used for GPU access for fine-training the models.

Data Management Solution and Database

Figure 84 shows the data flow for the application and LLMOps pipeline. As outlined earlier, the user interaction is limited to the gradio application hosted on hugging face spaces. When user makes a query, the application gets the context from Chroma DB while it uses the model published on the hugging face model hub. The authentication details are fetched from the database and the user interaction is stored in the same database. The session control is also managed with the same database ensuring proper connectivity and session isolation. The historical interaction data is used for fine-tuning the LLMs on ongoing basis and the pipelines are designed to run on pre-defined schedules. The data source for the triggering the pipeline remains the same as showing in Figure X. Cleaned and pre-processed data is stored on S3 bucket and used by the notebook hosted on Kaggle that is launched using an API. The metrics for the tuned or retrained models are published on MLFlow and are available for comparison and review. The best models that perform better than baseline models are then pushed to hugging face-hub and become the primary models for the application.

Figure 84

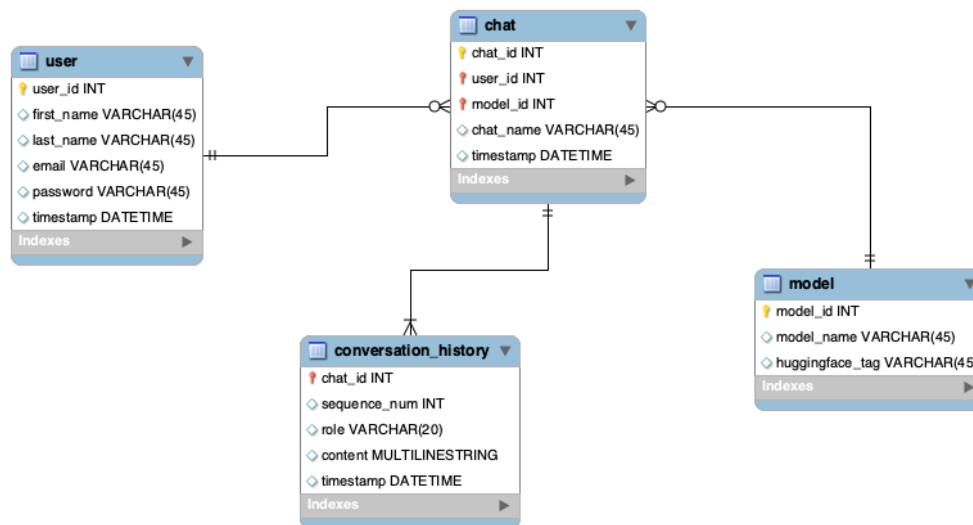
Dataflow for the Application and Training Pipeline



Database Entity Relationships. As outlined earlier, MySQL Database hosted on AWS RDS is used for the user interface. Figure 85 shows the entity relationship diagram for the database with the relevant tables and their attributes.

Figure 85

Entity Relationship Diagram for our Database establishment



The “user” table will be used as the primary authentication table that will store the basic information about the user along with Password in encrypted format. We will have a “chat” table that will act as the historical datastore and will be linked to individual users and will have a title for the original conversation. The “conversation_history” table will have the actual chat content with relevant role. The “chat” table will also link to “model” table which will store the model information available on the hugging face model hub. Using the above database design, the application will be effectively managed and operated seamlessly.

User Interface and Data Visualization

Figure 86 shows the user interface for our “Intelligent Tutoring System”. There will be four screens depending on the action user wants to take. The UI is simple with basic user authentication and onboarding screens. User will also have an option to reset the password. There will be a sign in page for existing users, a create User page for new users and a forgot password page for existing users. Once authenticated, the users will be presented with the chat UI. There will be a sidebar presenting the history of chat interactions for the user. User can select to continue where he left off for an older chat, or he can initiate a new chat. User will also have the

option to clear the history for the existing chats. During conversation, user will be presented with an option for using a “thumbs up” to rate the answer. The responses rated with “Thumbs Up” will be used for the fine-tuning and retraining datasets with the LLMOps pipeline.

Figure 86

User Interface Screenshots

The figure displays two screenshots of the Intelligent Tutoring System (ITS) user interface. The top screenshot shows the login page, which includes a header with the ITS logo and the text "Intelligent Tutoring System AI Powered Learning". Below the header, there are input fields for "Email" (containing "test12@gmail.com") and "Password" (masked with dots). A "Login" button is positioned below the password field, and a "Need an account? Register" link is at the bottom. The bottom screenshot shows the chat interface. It features a sidebar on the left with a "Past Conversations" section containing a dropdown menu for "Chat - 1 - (Session ID: 2iZyLU70)". Below this, there is a "Select Model" dropdown menu set to "Llama-3.2-1B-DPO". At the bottom of the sidebar are buttons for "Load Chat", "Start New Chat", and "Logout". The main chat area on the right has a "Charbot" tab, a "Type your message" input field, and a "Send" button.

Intelligent Solution

AI and Machine Learning Models Development

The project utilizes different advanced pre-trained models to boost chain-of-thought reasoning through specific fine-tuning methods for education domain. The primary goal of our work involves building an efficient framework that promotes step-by-step reasoning throughout multiple operational tasks.

The list combines adapted pre-trained models that add specific functionalities to the reasoning process. Table 3 presents complete details about their designs, core purposes and specific contributions to chain-of-thought reasoning systems. The strategy for model integration seeks to improve both problem-solving accuracy and decision-making sophistication while enhancing the interpretability of the system.

Table 19

Large Language models with their functionality and role in chain of thought reasoning

Model	Functionality	Role in Chain of Thought
Llama 3.2	General-purpose language understanding and generation	This model is good for multi-step logical reasoning which will improve the structured output for chain-of-thought reasoning.
Gemma 3	Fine-tuned for reasoning and logical tasks	Improves logical understating and context understanding. Effective for step-by-step reasoning.
phi 3.5	Effective reasoning for limited resources	Useful for real time reasoning tasks.
Deepseek R1	High-performance comprehension and reasoning Enhances deep reasoning.	This is also crucial for handling complex multi-step logical tasks.

Initial stage Data Collection and Preprocessing. In the initial data collection and preprocessing step, the structured datasets were obtained for fine-tuning cutting-edge Large Language Models (LLMs) to be used for educational domain with enhanced Chain-of-Thought (CoT) reasoning. The datasets are structured in question-answer pairs such that the answers included step-by-step solutions to match the CoT reasoning to solve any problem. Data is

collected from many open-source places for example - HuggingFace datasets such as OpenBookQA and GSM 8K, institutions like Stanford and UC Berkeley (MATH dataset), and wikis on the web providing competitive examination solutions, such as AIME from AoPS. The dataset collected is in different formats, for example - CSV, JSON, Markdown, and HTML formats which are specifically extraction and processing. Some datasets, like OpenBookQA and GSM8K, were pre-split into train, validation, and test splits, while others, e.g., AIME, were manually scraped and split according to an 80-10-10 ratio.

In the data preprocessing stage, the data is cleaned, validated, and normalized. Datasets are loaded as data frames and verified for structure, completeness, and fields. Missing data is taken care of by discarding incomplete records or replacing NaNs with placeholders. Duplicates are identified and removed in MATH dataset. OpenBookQA required choice-based answer parsing by mapping answer identifiers (A, B, C, etc.) to corresponding texts. JSON and Markdown files are also structured, extracting pertinent fields such as difficulty level, problem, and solution for MATH and year, problem, and solution for AIME. Preprocessing is followed by cleaning dataset validation for consistency and storing it in AWS S3 buckets to enable availability for further transformation and fine-tuning.

Intermediate Stage Model Training and Optimization. In the stage of Model Training and Optimization, the Large Language Models (LLMs) are fine-tuned for Chain-of-Thought (CoT) reasoning. Following data preprocessing, models undergo training to enhance their reasoning abilities for the education domain. Different pre-trained models, for example, Llama 3.2, Gemma 2 9B, Phi 3 Mini, Mistral 7B Instruct, and PaLM 2A 9B are selected for fine-tuning. The selected models play a specific role in CoT reasoning. The training procedure is to fine-tune these models on step-by-step explanation embedded structured question-answer datasets, reinforcing logical reasoning. The dataset is split into 80% train, 10% validation, and 10% test to facilitate robust evaluation. Models are trained utilizing the Hugging Face model hub with GPU-based instances for fast computation.

Low-Rank Adaptation (LoRA) and Quantized LoRA (QLoRA) are used for optimal model

training while reducing memory requirements without compromising efficiency. Gradient accumulation is also utilized to allow for large batch sizes without imposing unwarranted computational stress. Iterative training takes place with continuous hyperparameter adjustment such as learning rate, batch size, and weight decay for optimal performance. Hyperparameter tuning is done to make the models generalize. Self-consistency decoding is employed to produce more than one output and choose the most likely output.

For performance measurement different metrics are used. All of these metrics contribute to coherence, correctness, and logically ordered progression of responses. The model performing best is then used for deployment wherein its effectiveness in rendering structured step-by-step explanations is obtained.

Final Stage Model Integration and Deployment. The final model integration and deployment involves a cloud-based data pipeline utilizing various AWS services to ensure seamless data flow and model training automation. The data transformation happens through AWS Glue Jobs and the S3 Buckets serve as storage for raw data alongside intermediate and final staged information. The continuous process from data ingestion to final processing happens with the help of the Amazon RDS (Relational Database Service) while it maintains structured data storage. The LLMOps pipeline operates through an automated model innovation process which conducts efficient data processing while tokenizer operations apply and model fine-tuning and testing thus enabling improved model deployment to HuggingFace Spaces. The systematic workflow enables both persistent learning and ongoing improvement of the system.

The front-end interface bases its user interaction on the Gradio web application. Users need to authenticate before accessing the step-by-step question-answering interface through a chat-based system that provides detailed explanations. The system accesses previous interactions of users which are stored in AWS RDS-backed MySQL databases so users can check their conversation history. With LangChain retrieval and ChromaDB running on AWS EC2, the model improves both accuracy and response quality.

The training process of the Baseline Model consists of multiple stages starting with data

cleaning and continuing through preprocessing and tokenization before reaching the final step of evaluation based on customized metrics. The latest version of the model is released through the HuggingFace spaces following better performance results compared to its predecessor. The deployment system commits to system-wide monitoring as well as system updates through the LLMOps pipeline and Apache Airflow for orchestration with the goal of delivering optimal model performance alongside real-time user efficiency.

Implement Designed System

To allow users to communicate with the system, a web application based on Gradio is created where users can interact with the model using a chat interface. Various fine-tuned LLMs such as Llama 3.2, Gemma 2 9B, Phi 3 Mini, Mistral 7B, and PaLM 2A 9B are used by the system for giving step-by-step explanations for educational questions. Users can sign in to retrieve their past chats saved in Amazon RDS and ask new questions to get live responses. The backend fetches and processes data with the help of LangChain and ChromaDB to provide improved responses. An LLMOps pipeline based in the cloud pre-processes, tokenizes, fine-tunes, and evaluates data automatically so that high-performing models are updated frequently in the Hugging Face Model Hub. Through its modular architecture, the system is then capable of natively incorporating additional models and data sets without affecting high scalability, accuracy, and performance for AI-driven reasoning.

Input and Output Requirements, Supporting Systems and Solution APIs

Input Datasets. The input dataset for Chain of thought reasoning structured question-answer datasets. Datasets utilized are OpenBookQA and GSM 8K from Hugging Face, MATH (UC Berkeley), and AIME (AoPS Online Wiki). The datasets carry step-by-step solutions, which are necessary for CoT reasoning. Data is retrieved in different formats (CSV, JSON, Markdown) and preprocessed, e.g., filling missing values, eliminating duplicates, and restructuring. A few of the datasets, such as OpenBookQA, already provide training, validation, and test splits, but others, such as AIME, must be manually scraped and split (80-10-10 ratio). The datasets are housed in AWS S3, preprocessed using AWS Glue, and joined into Amazon RDS

for convenient access. The datasets form the basis of training models, which allows the LLMs to produce coherent, step-by-step answers to multifaceted questions, enhancing their reasoning and problem-solving capabilities.

Expected Outputs. The outcome is expected to be a fine-tuned LLM-based system that will have the ability to generate step-by-step, structured answers for educational domain based on Chain-of-Thought (CoT) reasoning. Users will interact with the system through a Gradio-based chat interface, and they will be given detailed, logical reasons for their queries. The model fetches previous interactions from Amazon RDS and processes new requests with LangChain and ChromaDB to provide improved accuracy. The answers are coherent, contextually appropriate, and explainable, adopting a structured approach instead of direct solutions.

Supporting Systems and Solution API's. HuggingFace models hub is utilized for storing the trained models along with Apache Airflow for orchestration. With the continuous training and deployment LLMOps is used.

System Support Environment

Google Colab

Google Colab provides free and pay-per-use access to powerful GPUs using Python and Jupyter notebook environment, providing high-performance computing and lowering entry barriers to fine-tune LLMs. Its collaborative feature allows real-time teamwork, enhancing efficiency and fostering innovation. It streamlines project setup since it comes pre-installed with libraries, such as PyTorch and Tensorflow, and up to date tools. Its scalability accommodates projects with large datasets and large models, while its cross-platform compatibility ensures accessibility to all the team members from their individual machines.

Google Drive

Google Drive provides robust cloud storage capabilities. It offers remote accessibility, enabling seamless collaboration among teams regardless of location, and simplifies sharing large datasets without the hassle of multiple email exchanges. Integrating effortlessly with Google's ecosystem, such as Colab, Google Drive enhances workflow efficiency. Its scalability

accommodates growing data needs, from small to large projects, while version control ensures data integrity and prevents accidental overwrites. Additionally, security features like granular permissions help maintain compliance with regulations.

GitHub

GitHub is a large-scale project management, collaboration, and innovation ecosystem. Its Git-based version control provides the ability to track changes to code, and this is vital for developing models iteratively. Collaboration features ensure that the team can work without conflicts and ensure quality via review. Reproducibility is enhanced as others can replicate experiments from shared repositories. Integrations with tools like Jupyter Notebooks streamline workflows, while documentation hosting aids in clear communication of methods.

Hugging Face

Hugging Face provides a readily accessible, open-source platform where developers can easily access, develop and host a vast collection of pre-trained language models and datasets. It offers a vast repository of publicly available datasets like GSM8k and MATH which were used in this project. The transformers library in Python from Hugging Face, provides pre-trained LLMs like Mistral and LLaMa, allowing for supervised fine-tuning without the need to train the LLMs from scratch, as well as utilities that assist in tokenization and data transformation. The platform enables easy sharing and deployment of trained LLMs via APIs or web interfaces using services like Hugging Face Spaces. It also has inbuilt metrics like BLEU and METEOR, which assess model performance while training to guarantee benchmark standards.

Gradio

Gradio is a platform that simplifies the deployment of machine learning models, particularly large language models (LLMs). It simplifies application creation by allowing users to create interactive interfaces without needing extensive technical expertise. Gradio enhances collaboration by having shared workplaces and version control, which makes it simpler for teams with diverse skill sets to collaborate and be productive. Gradio lowers the barrier to entry, making LLMs more accessible to the masses, by providing pre-built components and easy-to-use

dashboards.

Jira

Jira is a powerful project management tool that brings tremendous value to the planning and creation of large engineering projects. It is great for organizing tasks so that the teams can break down projects into actionable epics, stories, and tasks. Its WBS tool helps to decompose projects into smaller tasks so that it is easy to track progress and properly allocate resources, while Gantt charts provide the graphical view of each stage of the project which helps in managing dependencies.

Draw.io

Draw.io is a robust online diagramming tool that aids in the construction of complex data engineering and machine learning projects and enhances project concept communication. Draw.io facilitates the creation of flowcharts, data flow diagrams, and system architecture diagrams. These visual representations clarify processes such as machine learning pipelines and dataflow diagrams. Draw.io simplifies explanations by transforming complex algorithms or data flows into simple, intuitive diagrams that are easily understood by both technical and non-technical individuals.

System Evaluation and Visualization

Analysis of Model Execution and Evaluation Results

In this phase of the project, the focus was on fine-tuning a pre-trained large language model as listed in the earlier sections to generate responses in a step-by-step, chain-of-thought manner. To make the model better understand and mimic how a tutor would explain concepts logically and sequentially different techniques are used which are explained below.

Direct Policy Optimization (DPO)

DPO is a relatively new method in the field of AI training that helps a model learn not just from answers, but from preferences between answers. In other words, instead of only showing the model the “right” response, the model was shown two possible responses and to guide it to prefer the better one. Instead of traditionally fine tuning the model which just mimics the data, this

approach works better for our goal. The aim was for the model to generate thoughtful, step-by-step solutions using Chain of Thought reasoning, just like a good human tutor would. To make this possible, a preference dataset was created as shown in Figure 87, where for each question, industry standards LLM were used to generate multiple responses to the same question. The responses were then manually labelled as chosen or rejected depending on the logical steps and correctness based on human understanding. These preferences help the model understand which types of explanations are better in terms of being clearer, more logical, and educational.

Figure 87

Raw preference dataset for fine tuning

question	option1	option2
1	To find the value of x , we first need to calculate the average of 12, 21, and x . The average of 3 numbers is found by adding the numbers together and then dividing by the total number of values. In this case, the average is given as 18, so we can set up the equation: $\frac{12 + 21 + x}{3} = 18$ Now, we can simplify this equation by first adding 12 and 21: $\frac{33 + x}{3} = 18$ Next, we can multiply both sides of the equation by 3 to eliminate the fraction: $33 + x = 54$ Now, we can solve for x by subtracting 33 from both sides: $x = 54 - 33$ $x = 21$ Therefore, the value of x is 21.	To find the value of x , we can use the formula for the average: $\text{Average} = \frac{\text{Sum of all values}}{\text{Number of values}}$ Given that the average of 12, 21, and x is 18, we can write this as: $\frac{12 + 21 + x}{3} = 18$ Now, we can solve for x by multiplying both sides by 3 to get rid of the denominator: $12 + 21 + x = 54$ Combine like terms: $33 + x = 54$ Subtract 33 from both sides to isolate x : $x = 54 - 33$ $x = 21$ Therefore, the value of x is 21.
2 The average of 12, 21 and x is 18. What is the value of x ?	To find the value of x , we first need to calculate the average of 12, 21, and x . The average of 3 numbers is found by adding the numbers together and then dividing by the total number of values. In this case, the average is given as 18, so we can set up the equation: $\frac{12 + 21 + x}{3} = 18$ Now, we can simplify this equation by first adding 12 and 21: $\frac{33 + x}{3} = 18$ Next, we can multiply both sides of the equation by 3 to eliminate the fraction: $33 + x = 54$ Now, we can solve for x by subtracting 33 from both sides: $x = 54 - 33$ $x = 21$ Therefore, the value of x is 21.	To find the value of x , we first need to calculate the average of 12, 21, and x . The average of three numbers is calculated by adding the numbers together and then dividing by the total number of values. So, the average of 12, 21, and x is: $\frac{12 + 21 + x}{3} = 18$ Now, multiply both sides of the equation by 3 to get rid of the fraction: $33 + x = 54$ Subtract 33 from both sides to solve for x : $x = 54 - 33$ $x = 21$ Therefore, the value of x is 21.
3 The average of 12, 21 and x is 18. What is the value of x ?	To find the value of x , we first need to calculate the average of 12, 21, and x . The average of three numbers is calculated by adding the numbers together and then dividing by the total number of values. So, the average of 12, 21, and x is: $\frac{12 + 21 + x}{3} = 18$ Now, multiply both sides of the equation by 3 to get rid of the fraction: $33 + x = 54$ Subtract 33 from both sides to solve for x : $x = 54 - 33$ $x = 21$ Therefore, the value of x is 21.	To find the value of x , we first need to calculate the average of 12, 21, and x . The average of three numbers is calculated by adding the numbers together and then dividing by the total number of values. So, the average of 12, 21, and x is: $\frac{12 + 21 + x}{3} = 18$ Now, multiply both sides of the equation by 3 to get rid of the fraction: $33 + x = 54$ Subtract 33 from both sides to solve for x : $x = 54 - 33$ $x = 21$ Therefore, the value of x is 21.
4 Compute $\tan 3825^\circ$.	To find the value of $\tan 3825^\circ$, we can use the periodicity of the tangent function. Since the period of the tangent function is 180° , we can first reduce 3825° to an angle within one period of the tangent function. $3825^\circ \div 180^\circ = 21 \text{ remainder } 45$ So, 3825° is equivalent to 45° because the tangent function has a period of 180° . Now, we can find the tangent of 45° . We know that $\tan 45^\circ = 1$. Therefore, $\tan 3825^\circ = \tan 45^\circ = 1$.	To find the value of $\tan 3825^\circ$, we need to find the reference angle. Since the period of the tangent function is 180° , we can subtract multiples of 180° from 3825° until we get an angle between 0° and 180° . $3825^\circ - 3780^\circ = 45^\circ$ So, $\tan 3825^\circ = \tan 45^\circ$. Since $\tan 45^\circ = 1$, the value of $\tan 3825^\circ$ is 1.

DPO helps train the model with this kind of preference learning and makes the model understand how to respond to similar questions. Instead of just mimicking data, it learns to prefer better answers over weaker ones using these labeled preferences. This made it a great fit for building an intelligent tutoring system focused on explainability and learning support.

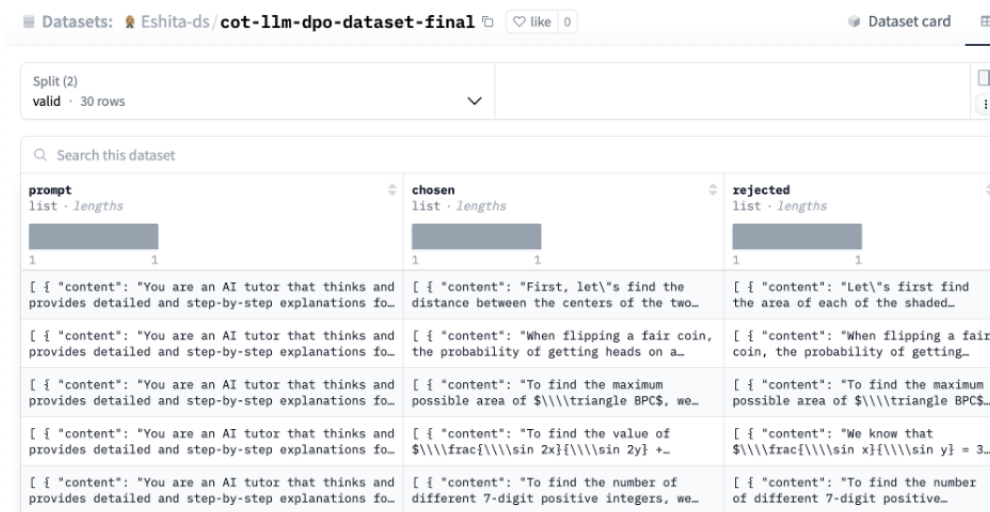
Model Execution. After dataset was curated, it was uploaded as a custom dataset in dictionary format to Hugging Face Hub with the format needed for DPO as shown in Figure 88. Each entry had a question prompt, a preferred (chosen) response, and a less preferred (rejected) response. This made it easy to load and use directly during the fine-tuning process.

This custom dataset was then split into a training set (90%) and a validation set (10%) to ensure the model could learn effectively and be evaluated properly during training. The dataset was shuffled so that both sets had a good mix of examples.

Hugging Face Transformers library was used to load the base “LLaMA 3.2-1B Instruct” model and tokenizer. Gradient checkpoints were enabled to save memory and mixed precision (bfloat16) was used to speed up training on GPU. Padding tokens were set to ensure smooth batch training.

Figure 88

Formatted Dataset for fine tuning on Hugging Face



prompt	chosen	rejected
[{"content": "You are an AI tutor that thinks and provides detailed and step-by-step explanations fo..."}]	[{"content": "First, let's find the distance between the centers of the two..."}]	[{"content": "Let's first find the area of each of the shaded..."}]
[{"content": "You are an AI tutor that thinks and provides detailed and step-by-step explanations fo..."}]	[{"content": "When flipping a fair coin, the probability of getting heads on a..."}]	[{"content": "When flipping a fair coin, the probability of getting..."}]
[{"content": "You are an AI tutor that thinks and provides detailed and step-by-step explanations fo..."}]	[{"content": "To find the maximum possible area of $\triangle BPC$, we..."}]	[{"content": "To find the maximum possible area of $\triangle BPC$..."}]
[{"content": "You are an AI tutor that thinks and provides detailed and step-by-step explanations fo..."}]	[{"content": "To find the value of $\frac{\sin 2x}{\sin 2y} + \dots$ "}]	[{"content": "We know that $\frac{\sin x}{\sin y} = 3$..."}]
[{"content": "You are an AI tutor that thinks and provides detailed and step-by-step explanations fo..."}]	[{"content": "To find the number of different 7-digit positive integers, we..."}]	[{"content": "To find the number of different 7-digit positive..."}]

DPOTrainer from the Hugging Face TRL (Transformers Reinforcement Learning) library was used to perform the fine tuning. The important hyperparameters included - 3 training epochs, small batch sizes (due to model size and GPU constraints) and, regular evaluation and saving checkpoints at each epoch.

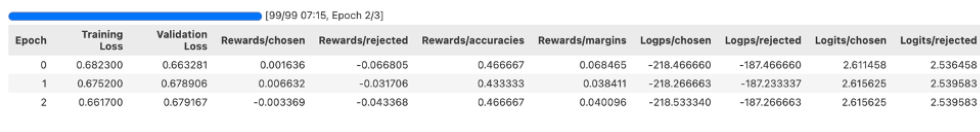
The trainer took the base model and the preference dataset and fine-tuned it to consistently prefer better explanations during response generation. As seen in Figure 89, important metrics like training loss, validation loss, and rewards were tracked at each step to see how the model was learning.

It was noticed during training, the training loss steadily went down from 0.68 to 0.66, which meant the model was getting better at understanding the data. The validation loss stayed consistent around 0.67, which showcase model was learning well without overfitting. Reward values were also reviewed which tell us how much the model prefers the better answers. These

mostly stayed positive, meaning that model favored stronger, clearer responses over weaker ones.

Figure 89

DPO fine-tuning metrics for LLaMA 3.2-1B-Instruct

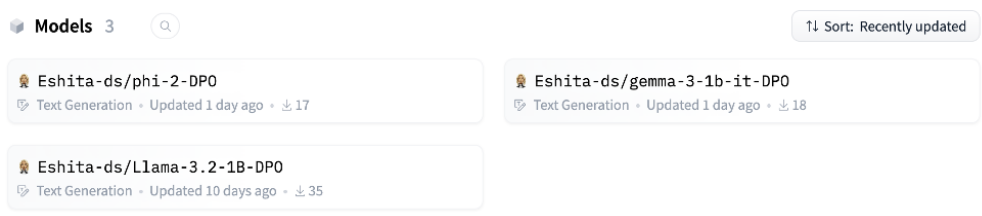


Epoch	Training Loss	Validation Loss	Rewards/chosen	Rewards/rejected	Rewards/accuracies	Rewards/margins	Logps/chosen	Logps/rejected	Logits/chosen	Logits/rejected
0	0.682300	0.663281	0.001636	-0.066805	0.466667	0.068465	-218.466660	-187.466660	2.611458	2.536458
1	0.675200	0.678906	0.006632	-0.031706	0.433333	0.038411	-218.266663	-187.233337	2.615625	2.539583
2	0.661700	0.679167	-0.003369	-0.043368	0.466667	0.040096	-218.533340	-187.266663	2.615625	2.539583

This DPO fine-tuned model was then pushed to the Hugging Face for inferencing and performance evaluation as shown in figure 90. The DPO based finetuning were performed on LLaMA-3.2-1B-Instruct, Phi-2, and Gemma-3-1B-IT to review how it impacts model's performance across key metrics.

Figure 90

List of fine-tuned models published to Hugging Face



Model Name	Category	Updated	Likes
Eshita-ds/phi-2-DPO	Text Generation	Updated 1 day ago	17
Eshita-ds/gemma-3-1b-it-DPO	Text Generation	Updated 1 day ago	18
Eshita-ds/Llama-3.2-1B-DPO	Text Generation	Updated 10 days ago	35

Model Evaluation. After the fine-tuned model was pushed to Hugging Face, a comprehensive evaluation pipeline was created to not only compare raw outputs but also examine the reasoning structure behind them.

Evaluations were performed on the baseline model (pretrained, not fine-tuned) and the fine-tuned model using a custom evaluation framework. While it includes standard NLP metrics like BLEU and ROUGE, the core focus was on evaluating reasoning quality. To do this, an advanced evaluation strategy was used where a large language model (LLM), like ChatGPT (gpt-3.5-turbo), acted as a judge to assess the step-by-step reasoning in the model's responses. This "LLM-as-a-judge" approach, powered by DeepEval's G-Eval framework, allowed the evaluations to go beyond surface-level accuracy and capture the depth, coherence, and correctness

of the chain-of-thought explanation. The metrics considered for DPO fine-tuned model evaluations are as outlined below.

BLEU Score. Measure surface-level similarity and overlaps between model generated and ground truth answers.

ROUGE Score. Evaluates overlap between phrases and sequences that can help to capture content similarity between original and generated responses.

Correctness. ChatGPT (GPT3.5-Turbo) was used to assess if the fine-tuned model's answer were factually correct compared to the ground truth answers.

Logical Coherence. Evaluation for the logical flow from start to end is what these metric tracks, correctness is not a factor during this evaluation. The idea is to ensure that the generated response follow a proper logical flow.

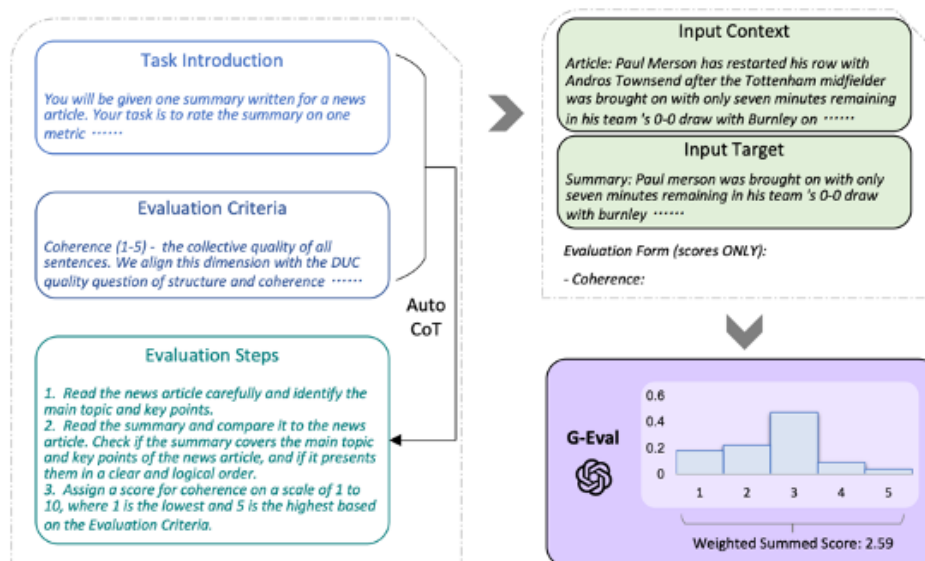
Step Alignment. Evaluates whether the generated responses have steps that align with ground truths or not.

Completeness. Evaluates whether the responses are complete as compared to ground truth. This helps to track if the number of tokens used for responses were sufficient to provide a complete answer or if there were gaps in the answers.

Reasoning Length Ratio. Evaluates the verbosity and consistency for the generated responses.

Error Identification. Evaluates for arithmetic or reasoning mistakes in the generated responses when comparing with provided reference answers.

G-Eval (Generalized Evaluation for Reasoning). G-Eval is DeepEval's foundation framework [\[91\]](#) that can accurately assess the quality of LLM responses using LLM as a judge with human like precision. It is highly customizable and versatile and can be effectively employed to review the results from fine-tuned LLM models.

Figure 91*G-Eval framework*

Sourced from the DeepEval (Confident AI) documentation

This metric was used to perform structured LLM-based evaluation for the reasoning quality of the fine-tuned models using DPO. G-Eval internally uses a combination of different metrics, and it helped to evaluate whether the responses were logically connected, had all the key steps and if the final answer was correct or not along with proper explanations. G-Eval configuration used for evaluating the models is shown in Figure 92.

Figure 92*G-Eval LLM Test Case and Evaluation Steps*

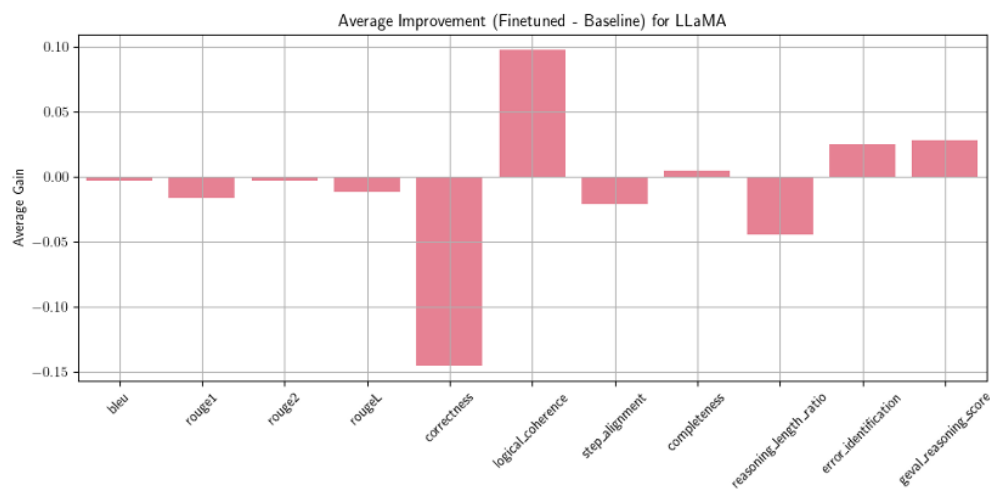
```
def calculate_geval_reasoning(self, question: str, reference: str, hypothesis: str) -> tuple[float, str]:
    reasoning_criteria = (
        "Evaluate the chain-of-thought reasoning in the generated text compared to the reference. ",
        "Assess whether the reasoning steps are logical, complete, and lead to the correct conclusion."
    )

    evaluation_steps = [
        "Check if each reasoning step in the hypothesis is explicitly stated and follows logically from the previous step.",
        "Verify that all critical steps from the reference are present in the hypothesis.",
        "Determine if the final conclusion matches the reference and is supported by the reasoning."
    ]

    test_case = LLMTTestCase(
        input="Solve this problem step-by-step: "+question,
        actual_output=hypothesis,
        expected_output=reference
        # evaluation_params=[LLMTTestCaseParams.ACTUAL_OUTPUT, LLMTTestCaseParams.EXPECTED_OUTPUT]
    )

    reasoning_metric = GEval(
        name="ReasoningQuality",
        criteria=reasoning_criteria,
        evaluation_steps=evaluation_steps,
        evaluation_params=[LLMTTestCaseParams.INPUT, LLMTTestCaseParams.ACTUAL_OUTPUT, LLMTTestCaseParams.EXPECTED_OUTPUT],
        model=self.judge_model,
        threshold=0.7
    )
```

Evaluation Results. Comparison of Baseline vs Fine-Tuned Model MATH dataset's test split was used to pick a sample of 20 questions to validate both baseline and fine-tuned models against the ground truth answers. A basic prompt template was used "f"Question: questionPlease provide a step-by-step solution:" to generate responses using baseline and fine-tuned models. The evaluation results are as outlined in Figures [93](#), [95](#), [97](#).

Figure 93*Evaluation results for LLaMA-3.2-1B (Finetuned vs Baseline)*

With LLaMA-3.2, the metrics looked promising in terms of Logical Coherence, completeness, error identification and the final G-Eval reasoning steps. However, the correctness as compared to ground truths took a hit. This shows that the model even though reasoning well, falls short when it comes to generating accurate responses as shown in Figure 94.

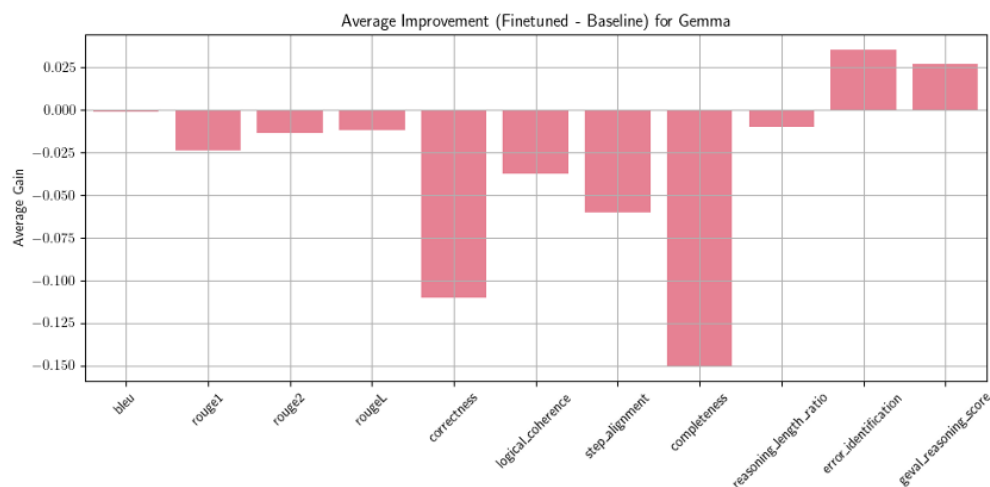
Figure 94

G-eval Reasoning comments for finetuned vs baseline

baseline_bleu	baseline_correctness	baseline_geval_reasoning_score	baseline_geval_reasoning_reason	finetuned_bleu	finetuned_correctness	finetuned_geval_reasoning_score	finetuned_geval_reasoning_reason
0.072138	1.000000	0.318062	The actual output does not explicitly state each reasoning step, critical steps from the reference are not clearly identified, and the final conclusion is not explicitly matched with the reference. Only general steps are provided without specific calculations or conclusions.	0.081153	0.500000	0.097221	The Actual Output does not match the Expected Output as it does not provide the correct step-by-step solution for finding the sum of the $\$(p)_{(a,b)}\$$'s for all possible combinations of a and b.
0.053303	1.000000	0.768678	The actual output precisely follows every step and correctly solves for e as expected from the input.	0.048828	1.000000	0.612788	The steps followed the evaluation criteria, but the final solution is missing the expected output value.
0.096143	1.000000	0.439957	The final conclusion does not match the expected output, as the actual output provides interval notation as $(-\infty, 25/17)$, while the expected output provides the correct answer as $(-\infty, 0)$.	0.118166	1.000000	0.660938	Input, Actual Output, and Expected Output are all consistent and follow the evaluation steps provided.

Figure 95

Evaluation results for Gemma-3-1B-it (Finetuned vs Baseline)

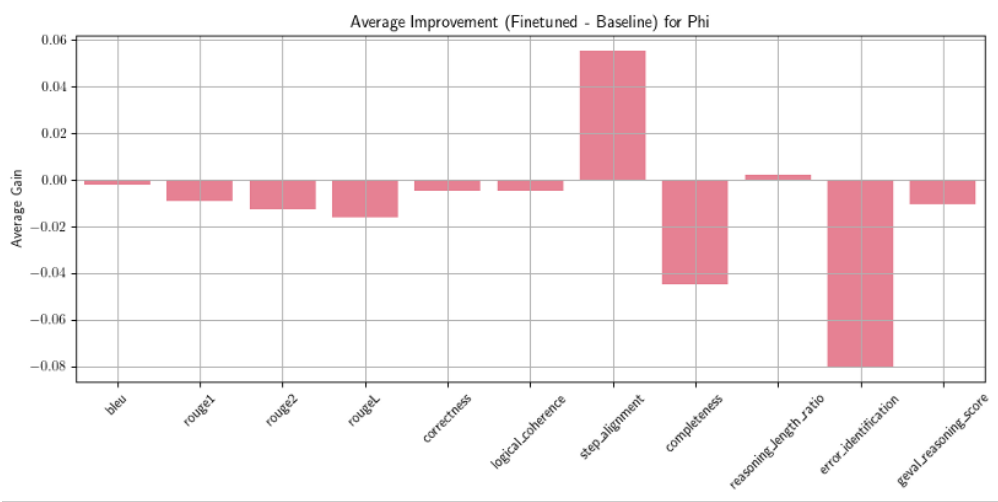


With Gemma-3-1B-it, the results were a mixed-bag, even though the completeness and correctness went down, the error identification improved, improving the final reasoning score as outlined by increase in G-Eval score. It was also noticed, that even with a max token size of 1000, the model was not able to provide a complete answer in some cases as shown in Figure 97.

Figure 96
G-eval Reasoning comments for finetuned vs baseline

baseline_bleu	baseline_correctness	baseline_geval_reasoning_score	baseline_geval_reasoning_reason	finetuned_bleu	finetuned_correctness	finetuned_geval_reasoning_score	finetuned_geval_reasoning_reason
0.239140	1.000000	0.249350	The answer provided does not accurately calculate the ratio of the areas of circle A to circle B, it is missing the step to square the ratio of the radii and only provides the ratio of the radii.	0.253397	0.500000	0.222771	The solution correctly calculates the arc lengths for circles A and B, but does not proceed to find the ratio of the areas as expected. The final conclusion and answer are missing.
0.100207	0.200000	0.339470	The reasoning presented in the actual output does not mention the concept of $f(x)$ being an odd function. It does not provide a step-by-step solution to find $f(-5)$.	0.116710	0.200000	0.347349	The hypothesis evaluation includes reasoning steps and matches the reference for determining $f(-5)$ based on the properties of the function and input-output relationship.

Figure 97
Evaluation results for Phi2 (Finetuned vs Baseline)



With Phi2, the finetuned model did not show significant improvements over the baseline. Even though the step alignment improved, the error decreased quite a bit. Even though the answers generated had the relevant logical steps, they still fell short as compared to the ground truths as is shown in Figure 98.

Figure 98*G-eval Reasoning comments for finetuned vs baseline*

baseline_bleu	baseline_correctness	baseline_geval_reasoning_score	baseline_geval_reasoning_reason	finetuned_bleu	finetuned_correctness	finetuned_geval_reasoning_score	finetuned_geval_reasoning_reason
0.059395	1.000000	0.239846	The input, actual output, and expected output are adequately provided, but the step-by-step solution does not explicitly state each reasoning step and does not logically follow from the previous steps.	0.050825	0.500000	0.149003	The actual output provides the step-by-step solution to find the total area of the two non-shaded crescent-shaped regions, which matches the expected output. The answer given is a numerical value rather than the expected algebraic expression.
0.050959	0.000000	0.220548	Actual Output does not provide the final conclusion matching the reference and does not support the reasoning steps. The answer is not explicitly stated or outlined step-by-step.	0.049217	0.500000	0.314298	The text provides step-by-step simplification of the expression, but does not provide the final conclusion that matches the expected output.

Chain of Draft

Chain of Draft (CoD) is an inspired reasoning and prompting approach following the mental activity of jotting down ideas in few, concise steps—much like the way human beings write ideas before making full explanations. Contrary to Chain of Thought (CoT), which requests extensive, step-by-step reasoning, CoD tells the model to break up difficult questions into a series of short, formal drafts, best each being under five words. This leads to more effective and targeted generation, often with fewer tokens but still logical consistency.

For the project, CoD was utilized to enable math problem-solving through a fine-tuned LLaMA 3.2 model. CoD prompting was aimed at being concise and modular, with the objective of optimizing inference efficiency without sacrificing correctness or semantic quality.

Model Execution. The pipeline for execution was designed to facilitate fine-tuning and reasoning efficiently with LLaMa 3.2 (3.2 billion parameters) using the Unsloth library.

Before fine tuning the model, the dataset is set for training. The dataset was specifically tailored to support both Chain-of-Thought (CoT) and Chain-of-Draft (CoD) reasoning approaches. Each data entry had a question_text, the answer, and an intermediate reasoning field that explained the steps followed to arrive at the answer. In order to satisfy the specific prompting needs of CoT and CoD, a dynamic prompting process was employed in which the question was prefixed with an instruction phrase—either "Let's think step by step." for CoT or "Let's solve using short drafts." for CoD. This ensured that both training and inference always presented the model with clear stylistic instructions in alignment with the method of reasoning under test.

The target output for training was a concatenation of the explanation and the solution, so the model was encouraged to generate both the solution and explanation in a well-formatted style. The dataset was split into training, validation, and test sets, which were each saved as CSVs. These were then converted to Hugging Face Dataset objects to easily integrate into Unsloth’s training pipeline. All the text inputs were pre-trained using a 512-token maximum sequence length by means of padding and truncation to maintain batch consistency prior to training. This structure allowed the model to learn from copious, diverse problem-solving examples without drifting from alignment with the target style of reasoning.

Data was split into train, validation, and test CSVs and converted to Hugging Face Dataset objects. Tokenization with truncation and padding to a maximum length of 512 tokens was performed. Figure 99 shows the dataset prepared for the fine tuning.

Figure 99

Dataset prepared for fine tuning

	dataset_id	question_id	question_text	answer	reasoning
0	2	427	Find the equation whose graph is a parabola with vertex at $(-3, 2)$ and passing through $(-1, 6)$. Hence the answer to this question is $-3x^2+12x-15$.	Since the axis of symmetry is vertical and the vertex is at $(-3, 2)$, the equation of the parabola is $y - 2 = a(x + 3)^2$. Since the parabola passes through $(-1, 6)$, we have $6 - 2 = a(-1 + 3)^2$, so $4 = a(2)^2$, so $a = 1$. Hence the equation of the parabola is $y - 2 = (x + 3)^2$, or $y = x^2 + 6x + 11$. Hence the answer to this question is $-3x^2 + 12x - 15$.	
1	2	1594	If $\$f(x) = \sqrt{x^2 - 4}$, what is the smallest real number x such that $f(x) = 2$? Hence the answer to this question is $\$12$.	Note that $\$f(x)$ is defined if and only if $\$x \geq 2$ or $\$x \leq -2$. Since $f(x) = 2$, we have $\sqrt{x^2 - 4} = 2$, so $x^2 - 4 = 4$, so $x^2 = 8$, so $x = \sqrt{8}$ or $x = -\sqrt{8}$. Since $x \geq 2$ or $x \leq -2$, we have $x = \sqrt{8}$ or $x = -\sqrt{8}$. Hence the smallest real number x such that $f(x) = 2$ is $-\sqrt{8}$. Hence the answer to this question is $\$12$.	
2	2	806	Positive real numbers x and y satisfy the equation $x^2 + y^2 = 1$. Hence the answer to this question is $\$1$.	We have $2x^2y^2 = (x^4 + 2x^2y^2 + y^4) - (x^4 + y^4) = (x^2 + y^2)^2 - (x^4 + y^4) = 1 - (x^4 + y^4)$. Hence the answer to this question is $\$1$.	
3	2	411	If $\$ \log_{(25)}(x-4) = \frac{1}{2}$, find $\$ \frac{1}{x}$. Hence the answer to this question is $\$2$.	First we solve for $\$x$. Converting our logarithm to base 5, we have $\log_5(x-4) = \frac{1}{2}$, so $x-4 = 5^{\frac{1}{2}}$, so $x = 4 + \sqrt{5}$. Hence $\frac{1}{x} = \frac{1}{4 + \sqrt{5}}$. Rationalizing the denominator, we have $\frac{1}{x} = \frac{4 - \sqrt{5}}{(4 + \sqrt{5})(4 - \sqrt{5})} = \frac{4 - \sqrt{5}}{16 - 5} = \frac{4 - \sqrt{5}}{11}$. Hence the answer to this question is $\$2$.	
4	2	6260	How many perfect squares are two-digit and divisible by 3? Hence the answer to this question is $\$25$.	Recall that no perfect squares are negative, but all perfect squares are non-negative. Hence the answer to this question is $\$25$.	
...	...	...	...	...	

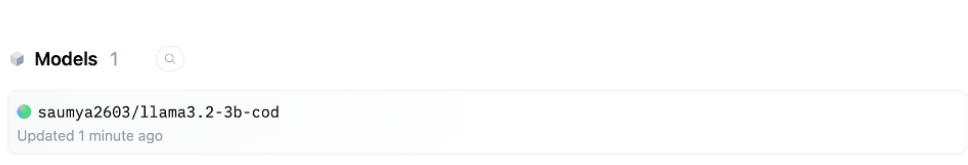
The model was loaded in a quantized 4-bit state to reduce memory usage during training on consumer-grade hardware, such as Google Colab Pro. Unsloth’s FastLanguageModel API was used to load both the model and the tokenizer with support for LoRA (Low-Rank Adaptation) for parameter-efficient fine-tuning. Only 0.15% of model weights were being updated—attention-related layers, specifically (q_proj, k_proj, v_proj, and o_proj)—which kept the computational expense minimal while preserving learning capacity.

Training was conducted using Unsloth’s SFTTrainer, instruction-style tuning-optimized. Each instance of the dataset was tokenized to a sequence length of 512, and the training was conducted for 3 epochs with a batch size 2 per device and 4 steps of gradient accumulation (for an effective batch size of 8). Prompts were dynamically framed to support Chain-of-Thought and

Chain-of-Draft styles with concise templating logic. At inference, generation was performed with the `max_new_tokens` limit set to 256 to capture full reasoning without overloading output length. Model output was stripped of prompt echoes and tested under various metrics, along with capturing runtime and token usage for each output generation. This was conducted to facilitate a full comparison of reasoning quality, efficiency, and compliance across CoT and CoD methods. Below figure [100](#) shows the trained model saved to the hugging face model hub.

Figure 100

Trained LLama 3.2 model saved on hugging face model hub



Model Evaluation. Model evaluation was performed on a held-out test subset of 20 randomly selected questions from the MATH dataset. The primary purpose of the evaluation was to quantify and compare the reasoning performance, efficiency, and alignment of Chain-of-Thought (CoT) and Chain-of-Draft (CoD) approaches. The evaluation employed a diverse set of both lexical and semantic metrics, including BLEU, METEOR, ROUGE, chrF, and BERTScore. In addition, auxiliary information such as token usage, reasoning step counts, token-per-step ratios, and inference time were recorded to estimate the generation efficiency. BERTScore provided a contextual embedding-based semantic similarity score, whereas BLEU and METEOR provided n-gram overlap and linguistic similarity information. These metrics collectively were able to capture both the surface and deep semantic accuracy of generated responses.

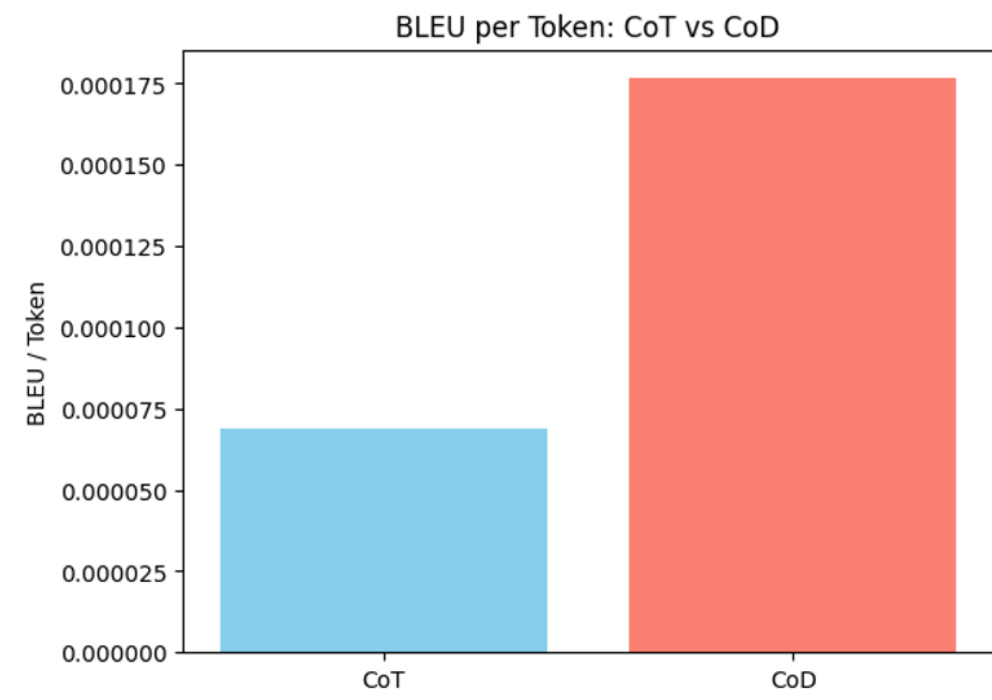
Evaluation Results and Interpretation. The results of the evaluation proved Chain-of-Draft (CoD) to be superior to Chain-of-Thought (CoT) with different metrics. Quantifying lexical alignment, CoD produced higher BLEU and METEOR scores, which indicates that the responses of CoD had more overlap with ground truth responses. This shows that CoD can produce right and well-formed answers. Furthermore, CoD was more efficient in

terms of BLEU-per-token score, i.e., more consistent output with fewer tokens is a positive indication of generation efficiency. While F1 scores for BERTScore were almost equal, additional analysis revealed that CoD obtained higher precision, i.e., more accurate matches to reference responses, while CoT was marginally better in recall as it was more verbose.

Efficiency optimizations also helped CoD. It averaged fewer tokens, fewer tokens per reasoning step, and had faster inference times. Perhaps most significantly of all, CoD produced more reasoning steps than CoT, confirming the hypothesis that CoD's brief, dense steps create modular but efficient sequences of logic. CoD won more frequently on a per-question basis in BLEU as well, confirming its consistency on a broad variety of types of problems. Figure 101 shows that The BLEU per Token evaluation demonstrates that CoD produces fluent answers in fewer tokens than CoT does. The system produces streamlined yet accurate output answers through its efficient solution strategy which proves beneficial in speed-based tasks where token counts matter.

Figure 101

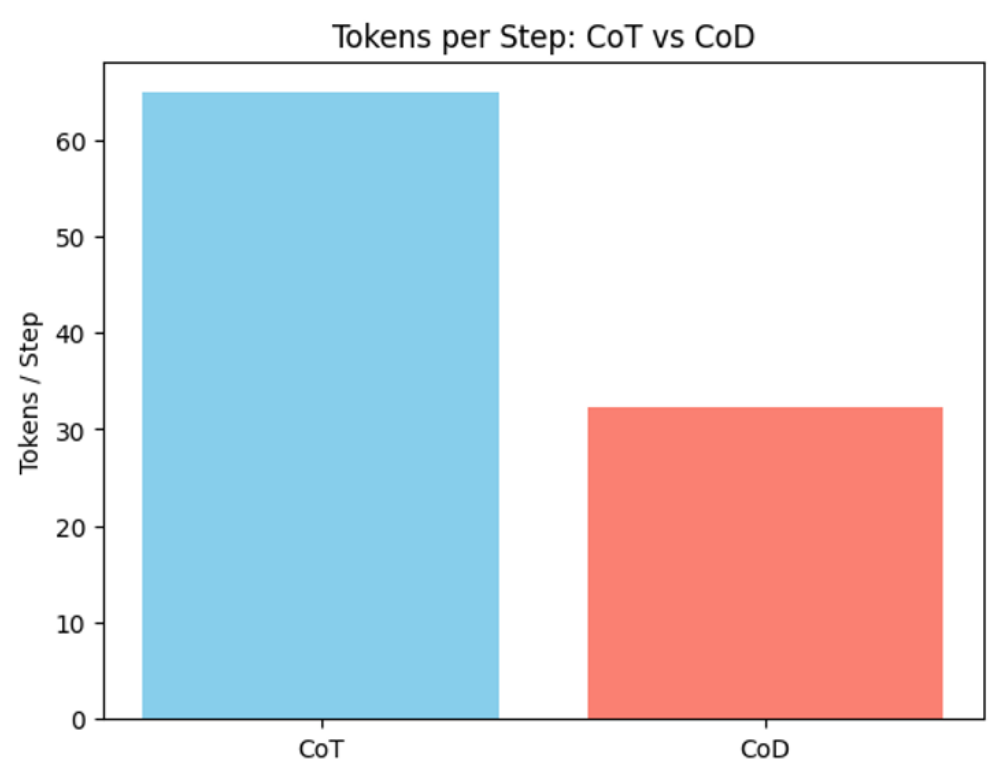
BLEU per token for CoT vs CoD



Each reasoning step in CoD requires less tokens than those used in CoT reasoning steps. Figure 102 demonstrates that CoD generates short precise draft sections which fit easily into modules. CoD results in fewer steps at each point which leads to better clarity while providing better control thus becoming a superior method for structured reasoning tasks especially in mathematical problem-solving.

Figure 102

Tokens per steps CoT vs CoD

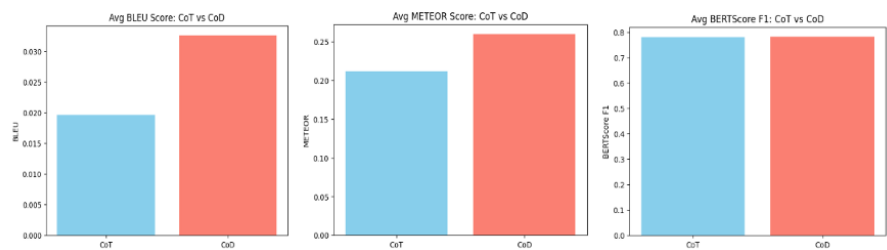


The results in Figure 103 shows Chain of Draft (CoD) as the superior technique for both precision and efficiency over Chain of Thought (CoT). The CoD system performed better than CoT in BLEU and METEOR score measurements, which demonstrates a better lexical and semantic match with reference solutions. The shorter CoD text combination achieved the same BERTScore F1 score as CoT, even though it contained fewer words. The token utilization efficiency, as well as the accuracy of CoD’s structured drafting system, drives its superior performance. The system achieves practical performance in real-world problem-solving tasks by

maintaining proper clarity alongside correct solutions.

Figure 103

Average BLEU, METEOR and BERT Score F1 for CoT vs CoD



As shown in Figure 104, Chain of Draft (CoD) extends Chain of Thought (CoT) but accomplishes these further steps with fewer token sequences combined with less inference time. Micro-step reasoning is a byproduct of CoD as it invokes to transform problems into short pieces of drafts as opposed to arriving at written solutions that are long. CoD is operationally more efficient in terms of shorter steps even though overall steps are greater compared to those of CoT. The tokens that are few result in faster attainment combined with reducing the deployment cost of CoD. The use of CoD as a real-time reasoning solution becomes increasingly popular as it is faster combined with scalability as measured by performance of inference time.

Figure 104

Average reasoning steps, token usage and inference time for CoT vs CoD

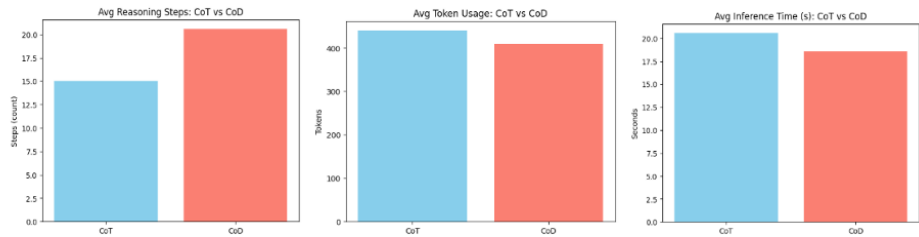


Table 20*Interpretation of CoT vs CoD with different matrices*

Metric	Winner	Interpretation
BLEU / METEOR	CoD	Lexically aligned & semantically rich
BERTScore F1	Tie	Both semantically sound
BLEU per Token	CoD	Better efficiency per token generated
Reasoning Steps	CoD	More granular, modular reasoning
Tokens per Step	CoD	Concise steps as expected
Inference Time	CoD	Slightly faster generation
BLEU Win Count	CoD	Consistently better alignment per question

Group Relative Policy Optimization (GRPO)

Group Relative Policy Optimization (GRPO) is an efficient reinforcement learning algorithm developed to enhance LLM’s reasoning capabilities while reducing computational overhead. Unlike traditional Proximal Policy Optimization (PPO), GRPO eliminates the need for a separate critic model by calculating advantages based on the relative performance within groups of sampled outputs. For each question, it generates multiple responses from the LLM, scores them with simple reward functions, and normalizes these rewards against the group’s statistics. This makes the Reinforcement Learning process significantly more memory-efficient during training. GRPO’s objective function combines an advantage-weighted policy ratio with a KL-divergence term that prevents the policy from deviating too far from a reference model. This approach has proven particularly effective for mathematical reasoning tasks, where the rewards can be implemented with either outcome supervision (rewards at generation completion) or process

supervision (rewards at each reasoning step).

Reward functions help in assigning a quantitative value to an LLM response based on its alignment with the favored response to a question. These rewards are often dependent on the dataset used to fine-tune the LLM. To force an LLM to think about a question before responding, after much trial and error, three reward functions were identified for the MATH dataset.

Format Reward. Checks whether the model response follows the pattern of enclosing the chain-of-thought inside `<think>` `</think>` tags, while giving the concise response inside `<answer>` `</answer>` tags. This forces the model to explicitly think step-by-step about its response before giving the final answer.

Accuracy Reward. Checks whether the answer given by the model inside the `<answer>` tags is correct. Uses a library called math-verify to extract answers from an LLM response and then compares it to the expected answer for the prompt. Rewarding the presence of correct answer in the `<answer>` tags ensures that the models reasoning inside the `<think>` tags is always directed towards obtaining the correct answer.

Similarity Reward. Checks whether the answer given by the model inside the `<answer>` tags is like the reference response. Uses BLEU score to calculate the similarity.

Model Execution. Since the reward functions for GRPO require the exact answers for the accuracy reward, and multiple implementations to solve the problem for the similarity reward, the MATH dataset train and validation partitions (as specified in the previous sections) were further transformed to derive the columns “extracted_answer” and “reasons” from the original reasoning steps provided in the dataset. The “extracted_answer” column was derived using the math-verify library. A sample for the same is shared in Figure [105](#).

Figure 105

Samples of the new derived columns “reasons” and “extracted_answer” for MATH dataset

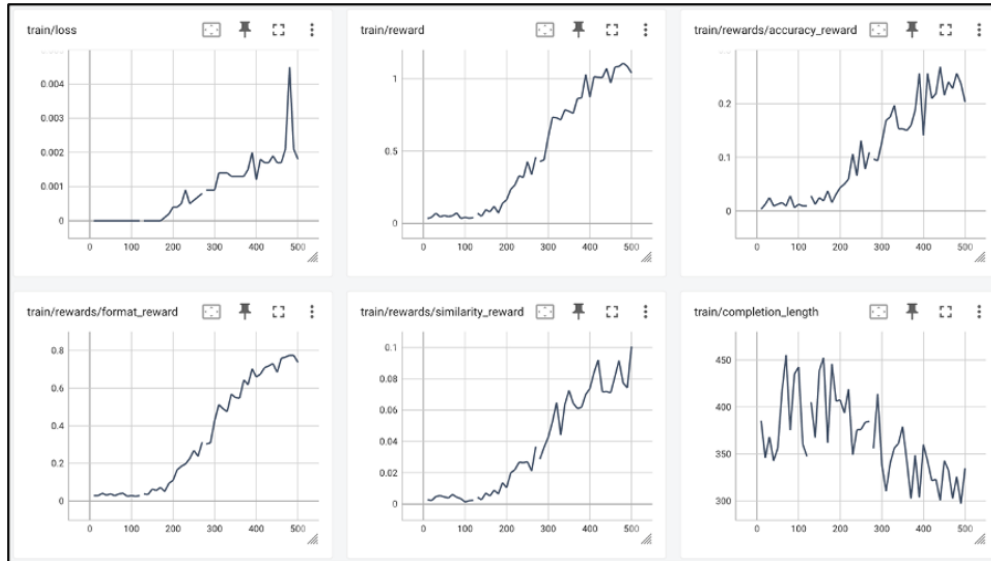
	question_text	reasons	extracted_answer
0	Find the equation whose graph is a parabola wi...	[Since the axis of symmetry is vertical and th...	$-3x^2+12x-8$
1	If $f(x)=\sqrt{x-3}$, what is the smallest rea...	[Note that $f(x)$ is defined if and only if $x...$	12
2	Positive real numbers r,s satisfy the equati...	[We have $2r^2s^2 = (r^4 + 2r^2s^2 + s^4) - (r...$	$\frac{1}{4}$
3	If $\log_{25}(x-4)=\frac{1}{2}$, find $\frac{1}{x}$	[First we solve for x . Converting our logari...	2
4	How many perfect squares are two-digit and div...	[Recall that no perfect squares are negative, ...	2
...	...	...	...
5983	What is the remainder when 2^{19} is divided...	[Since $2^3 \equiv 1 \pmod{7}$ and $a \equiv b...$	2
5984	What is that largest positive integer n for ...	[If $n+10 \mid n^3+100$, $\gcd(n^3+100,n+10)=n...$	890
5985	If a, b , and c are positive integers suc...	[Note that $\gcd(168,693) = 21$. Since $\gcd(...$	21
5986	Evaluate $\lfloor \sqrt{80} \rfloor$.	[Since $\sqrt{64} < \sqrt{80} < \sqrt{81}$, $\lfloor \sqrt{80} \rfloor =$	8
5987	The number $\lfloor e^{7\pi i/60} + e^{17\pi i/60} + ...$	[Let's locate these numbers in the complex pla...	$\frac{9\pi}{20}$

The standard LLM implementation provided by the HuggingFace Transformers library would not fit the Google Colab A100 runtime. Hence, Unsloth library was used instead to efficiently load the base “Llama 3.2 3B Instruct” LLM. Unsloth efficiently switches between a mixed-precision LLM and a vLLM 4-bit quantized version of the same model to efficiently finetune the LLM using LoRA. Unsloth also uses gradient checkpointing to further optimize the GPU usage.

GRPOTrainer from the HuggingFace TRL (Transformers Reinforcement Learning) library was used to guide the GRPO reinforcement learning process. The important hyperparameters for this process were Gradient Accumulation Steps (set to 4 to smoothen learning process), Number of generations (used to generate the group of sampled outputs) and Batch Size (set to 4 due to GPU VRAM constraints). The trainer then uses GRPO to finetune the LLM using LoRA. The training metrics from finetuning the model is shared in Figure [106](#).

Figure 106

Visualizations from Tensorboard depicting the LLM fine-tuning training metrics along with rewards



After the LLM was finetuned, a few samples from the test dataset were selected and prompted to the LLM. The response generated by the LLM (as shown in Figure 107) clearly shows that the LLM enclosed the answers inside <answer> tags and enclosed the reasoning process inside <think> tags. It can also be seen that the model response inside the <think> tags follow a logical, step-by-step workflow, employing a Chain-of-Thought to solve the problem.

Figure 107

Sample responses generated by Llama 3.2 3B Instruct finetuned using GRPO

```
( '<think>First, we need to factorize the denominator to find the values of x '
'for which the function becomes undefined. We can rewrite the denominator as '
'$x^2+x-6 = (x+3)(x-2)$. So, the function becomes $y = '
'\\frac{2}{(x+3)(x-2)}$. Now, we need to find the values of x for which the '
'denominator becomes zero. This happens when $x+3=0$ or $x-2=0$, i.e., when '
'$x=-3$ or $x=2$. These are the potential vertical asymptotes. However, we '
'also need to check if there are any factors in the numerator that cancel out '
'the factors in the denominator. In this case, there are no such factors. '
'Therefore, the function will have vertical asymptotes at $x=-3$ and '
'$x=2$.</think><answer> The graph of $y=\\frac{2}{x^2+x-6}$ has 2 vertical '
'asymptotes at $x=-3$ and $x=2$.</answer>')
```

```
("<think>First, let's recall the formula for compound interest, which is  $A = P(1 + r)^n$ , where A is the amount of money accumulated after n years, '
including interest, P is the principal amount (initial investment), r is the '
annual interest rate (in decimal), and n is the number of years the money is '
invested. In this case, we are given  $A = 1225$ ,  $P = 1000$ , and  $n = 3$ . We need '
to solve for r.</think><answer> To solve for r, we can rearrange the formula '
to isolate r:  $r = (A/P)^{(1/n)} - 1$ . Plugging in the given values, we get  $r = '
(1225/1000)^{(1/3)} - 1 \approx 0.075$  or 7.5%.</answer>')
```

GRPO + COT

To ensure that the reasoning generated by the LLM models was coherent and correct, Chain-of-Thought (COT) prompting technique was combined with the previous setting. Since Chain-of-Thought has proven to promote logical step-by-step problem-solving capabilities, combining this technique with reinforcement learning technique like GRPO proved very successful, resulting in a 2% increase in the g-Eval correctness metric over the simple reasoning model.

DPO + GRPO

We attempted to fine-tune our Llama 3.2 1B model using a combination of DPO and GRPO. GRPO is a reinforcement learning method that efficiently improves reasoning by rewarding grouped responses without needing a separate critic model. GRPO technique alone on the training dataset seemed promising, encouraging step-by-step thinking with rewards for format, accuracy, and similarity, specifically for math problems. However, training our DPO trained model again with GRPO technique underperformed in testing specially in g-eval score. Therefore, it is decided to continue with the DPO-only model for better results.

GRPO + COD

Since GRPO trained reasoning models tend to generate long reasoning chains, these models have been susceptible to losing context in the response, leading to incorrect, unrelated or even unexpected answers. Chain-of-Draft prompting technique promotes LLMs to only think about the problems in short draft, before expanding upon those answers in a step-by-step manner. This prompting technique was combined with LoRA and GRPO, but the results were not impressive. There was a significant drop in performance across all the metrics, which can be attributed to the

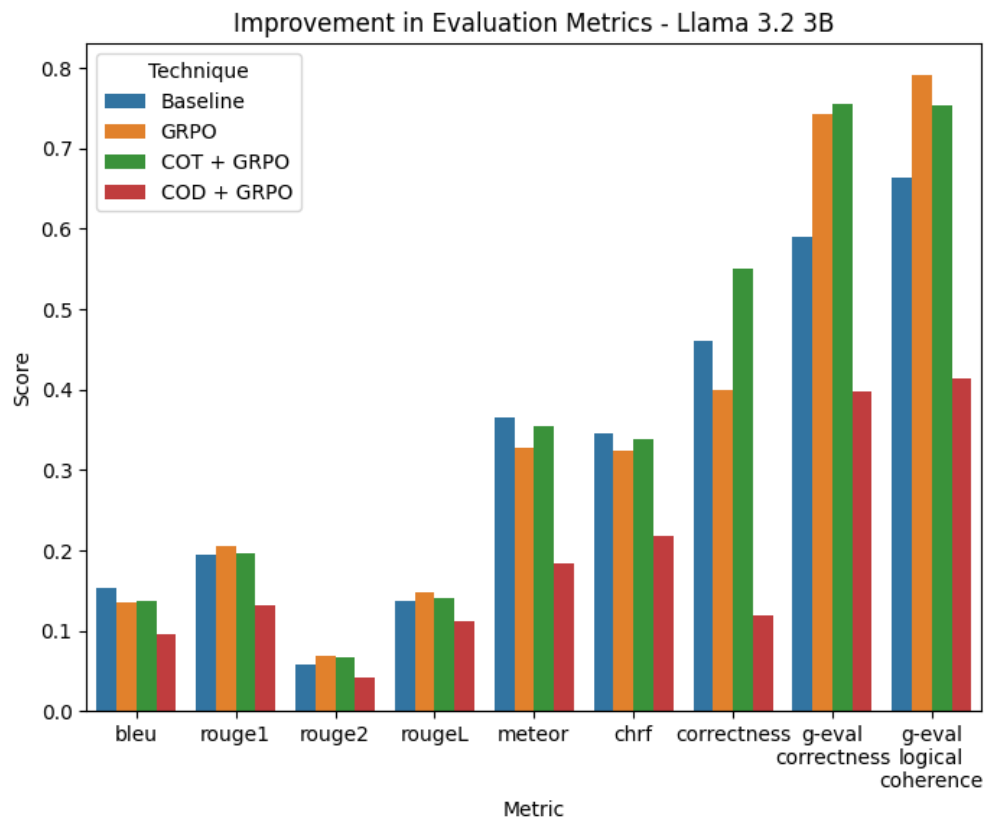
small number of parameters in the baseline model, a constraint that was identified in the original implementation as well.

Model Evaluation. The following metrics were utilized to evaluate the LLM performance after fine-tuning with LoRA and GRPO. BLEU, ROUGE, METEOR, chrF, Correctness (based on math-verify library). These metrics were calculated using the baseline Llama 3.2 3B Instruct LLM as well to provide a thorough comparison of the reasoning model with its baseline counterpart.

Evaluation Results & Interpretation. As shown in Figure [108](#), the reasoning Llama 3.2 model trained with GRPO is performing at par with the baseline Llama 3.2. The biggest difference is in the correctness metric, where the baseline model is performing much better than the reasoning model. This can be attributed to the fact that the correctness metric is dependent on the efficiency of the math-verify library to detect and extract the correct answers from an LLM response. Upon visually inspecting the responses from the reasoning model as shown in Figure [107](#), it can be seen that the LLM responded correctly and logically to the question, but this library is still marking these responses as incorrect. This discrepancy was resolved by using G-Eval framework to implement LLM-as-Judge based evaluation. Correctness from G-Eval framework clearly shows a large improvement from baseline model to the GRPO fine-tuned model. When evaluating logical coherence of the response given by the LLM, the GRPO fine-tuned model had a large improvement over the baseline model, showing that the LLM was successfully fine-tuned to respond in a logical step-by-step manner.

Figure 108

Comparison of baseline and reasoning model across various automated evaluation metrics



Proximal Policy Optimization (PPO)

The reinforcement learning procedure PPO applies explicit reward signals to language models through an adaptation for generation refinement. Instead of supervised learning exclusively PPO gives the model permission to test various output options followed by an improvement update of the model based on chosen superior completions. The approach functions best in problems without absolute solutions but contains distinct answer quality levels.

A PPO optimization system was integrated on top of a DeepSeek-R1 model which had received LoRA modifications. The reward system of the function directed the automation system to select outputs that terminated in boxed solutions while exhibiting logical organization and keeping a minimal number of repetitions. The training method helped models to generate brief logical solutions matching standard math answer formats which became more coherent through

time.

Tree of Thought (ToT)

The structured reasoning framework Tree of Thought (ToT) uses a problem-solving method which functions similarly to searching among different intellectual possibilities just like human thought before reaching an answer. ToT deviates from Chain of Thought by allowing the model to produce multiple reasoning branches before assessing them and selecting the best option.

The proposed project used a fine-tuned DeepSeek-R1 model within a ToT engine which applied ToT to mathematical problems. Each step of the model execution involved multiple intermediate thoughts which were evaluated using reward function to choose the sound logical and complete solution path. The combination of exploration and quality control mechanisms in this method produced more reliable and interpretable answers for multi-step reasoning.

Model Execution. Proximal Policy Optimization (PPO) served as an additional optimization strategy after the supervised learning stage to bring the base model responses closer to pre-defined reasoning methods. There were efforts to improve a LoRA-based DeepSeek-R1-Distill model through reinforcement learning by developing a reward system that directed the model toward creating excellent step-by-step math solutions.

A reward-based format was applied to the original dataset for PPO implementation because the model received scores based on predefined criteria for candidate answers that included boxed answers and logical flow and minimal repetition. The PPOTrainer from TRL required specific configuration with small batch sizes to process math-style CoT answers according to its reward functions.

Although PPO setup demonstrated general success there were intermittent problems during execution between the reward function and LoRA-wrapped model checkpoints. To fix the generation process errors and mismatched configs in the model system I had to conduct manual adjustments for its `.generate()` operation to become functional. The obtained problems showcased a requirement for stronger PPO capabilities when utilizing quantized or adapter-based models. Initially assessed data demonstrated that the model progressively preferred generating

well-structured output.

The researchers studied Tree of Thought (ToT) as a method for creating structured reasoning while bypassing the requirement for additional training. The open-source Tree of Thoughts library enabled implementation of a BFS-style ToT engine but required the fine-tuned DeepSeek model to be wrapped for inference purposes.

The ToT system required the model to produce various intermediate reasoning steps for each mathematical problem while building multiple possible sequence paths to find the solution. The system used basic rewards to evaluate thoughts containing final answers combined with indicative phrases such as “Hence” or “\boxed”. Each promising pathway through the benchmark was selected either through greedy evaluation or sampling evaluation methods.

The ToT framework applied successfully generated various chains of reasoning that humans could easily interpret. End users could check intermediate thoughts generated by the model to uncover its solution exploration methods. The ToT framework offers effective inference-time boosting of accuracy but does not modify the model itself during training to achieve better performance.

During training PPO received beneficial model alignment through the use of PPO while ToT introduced beneficial reasoning guidance during model inference.

Model Evaluation. The evaluation process beyond Chain-of-Thought included testing models using outputs from Proximal Policy Optimization (PPO) and Tree of Thought (ToT) strategies. The evaluation utilized a hundred randomly selected MATH dataset math problems from a separate test-set for this assessment. The evaluation goal measured the effectiveness of sophisticated reasoning methods on answer accuracy together with semantic match and generation speed.

This evaluation used a complete package of semantic and lexical metrics that included BLEU alongside all ROUGE variants together with METEOR and chrF as well as BERTScore evaluation. A complete understanding of n-gram matches and deep semantic consistency in the final answers was possible through the implementation of these multiple evaluation metrics which

used BERT embeddings.

The limitations in training PPO models because of model compatibility issues did not prevent noticeable improvements in BERTScore precision and METEOR scores which indicated that reinforcement learning supported the model to find systematic accurate reasoning steps. The PPO reward system led to outputs that ended in boxed answers while following standard logical templates. The modest results for BLEU and ROUGE appeared because the training process was only partial.

The Tree of Thought (ToT) inference method delivered the greatest BERTScore F1 results because it successfully matched semantic patterns with reference reasoning pathways. ToT achieved its anticipated success because it uses scoring mechanisms that evaluate several chains to pick a winner which unifies beam search and prompt engineering capabilities. Rephrase the ToT method produced responses that were both more consistent and accurate than original CoT responses based on ROUGE-L and chrF assessment results. The BLEU scores experienced lower performance because the diverse responsive approaches of the model deviated from matching reference wording despite being a common limitation of reasoning-based generations.

The efficiency calculation for ToT involved higher computational needs because of its tree expansion and evaluation operations. The inference process of PPO operated efficiently at performing its computations and produced steady token-per-response ratios. The priority of both methods lay in producing concise yet accurate outputs through clear logical structures which led to better results compared to basic CoT methods.

Standard CoT received superior performance from both PPO and ToT in multiple important metrics although each method demonstrated advantages. The PPO system demonstrated superior effectiveness in teaching desirable solution patterns and creating consistent formatting standards, yet the ToT system achieved its best results through creating complex precise multidimensional solutions. The research shows that merging reinforcement-based training and inference-time reasoning exploration yields superior implementation of LLM technology for complex task execution. The evaluation results are shown in the figure 109

Figure 109*Evaluation results*

```

BLEU: {'bleu': 0.09771552956567549, 'precisions': [0.23671115568585094, 0.10752106361598517, 0.06947026022304832, 0.05156359121595406], 'brevity_penalty': 1.0, 'length_ratio': 2.2279}

ROUGE:
rouge1: 0.2781
rouge2: 0.1037
rougeL: 0.1595
rougeLsum: 0.2299

METEOR: {'meteor': 0.27308521629737137}

chrF: {'score': 28.743926888495068, 'char_order': 6, 'word_order': 0, 'beta': 2}

BERTScore:
Precision: 0.8071
Recall:    0.8483
F1:       0.8270

```

Self-Consistency (SC)

Employed Self-Consistency (SC) as a reasoning technique to improve the model's ability to generate more stable and reliable answers for complex tasks. Self-Consistency is based on the idea of generating multiple response candidates for the same question and then selecting the most consistent answer across different generations. This approach aims to reduce errors introduced by randomness during generation and ensures the model's answers are more reliable and coherent.

Model Execution.

To implement SC, fine-tuned the Phi-3.5 Mini-Instruct model using a multi-sampling technique. For each question, the model generated multiple candidate answers. Each response was then evaluated based on its consistency with other responses. The most consistent answer, as judged by similarity and coherence, was selected as the final output. This allowed the model to improve its reasoning ability by using multiple paths to arrive at the correct or most coherent solution.

The dataset was carefully formatted to allow the model to generate multiple response candidates during inference. The Hugging Face Transformers library was used to load the Phi-3.5 Mini-Instruct model to perform the fine-tuning. Similar to other fine-tuning methods, gradient checkpointing and mixed precision were used to optimize memory and computational efficiency.

The simple pipeline is followed for this technique where the model was fine-tuned on the custom dataset consisting of question-answer pairs. For each input question, the model generated n candidate answers (in this case, 5) during inference. The answers were compared for

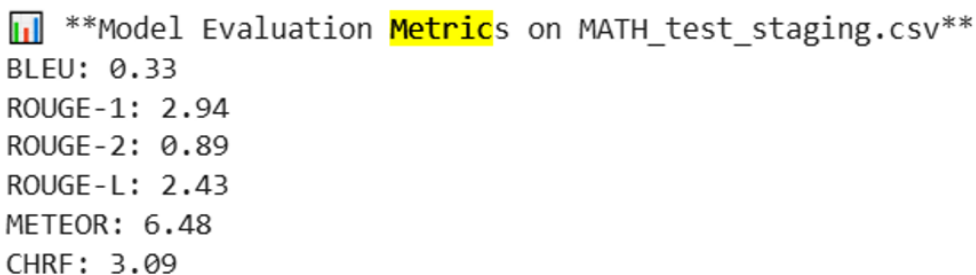
consistency using BLUE, Rogue, and BERTScore. The most consistent answer, measured by these metrics, was chosen as the final output.

Model Evaluation. After fine-tuning the Phi-3.5 Mini-Instruct model using Self-Consistency (SC) techniques, the model was evaluated on its ability to perform reasoning tasks from the MATH dataset. While traditional metrics like BLEU, ROUGE, METEOR, and BERTScore were employed to assess surface-level similarity between the generated responses and the ground truth, SC's real strength lies in its ability to enhance logical consistency. Unlike conventional metrics, SC evaluates the model's logical coherence by generating multiple candidate answers and selecting the one that aligns best with the majority in terms of reasoning quality and correctness.

To quantify the model's performance, a comprehensive evaluation pipeline was used. This involved using BERTScore to capture semantic similarity, BLEU and ROUGE to evaluate surface-level overlaps, and METEOR for synonym and stem alignment. These metrics were essential in ensuring that the model's outputs were both lexically and semantically accurate. By comparing different generated answers, the model could better align with the majority of responses that were deemed logically sound and factually correct. The evaluation metric values are showed in below figure [110](#).

Figure 110

Evaluation Results



```
**Model Evaluation Metrics on MATH_test_staging.csv**  
BLEU: 0.33  
ROUGE-1: 2.94  
ROUGE-2: 0.89  
ROUGE-L: 2.43  
METEOR: 6.48  
CHRF: 3.09
```

Question. 3168
BERTScore F1: 0.7677
BERTScore Precision: 0.7185
BERTScore Recall: 0.8241

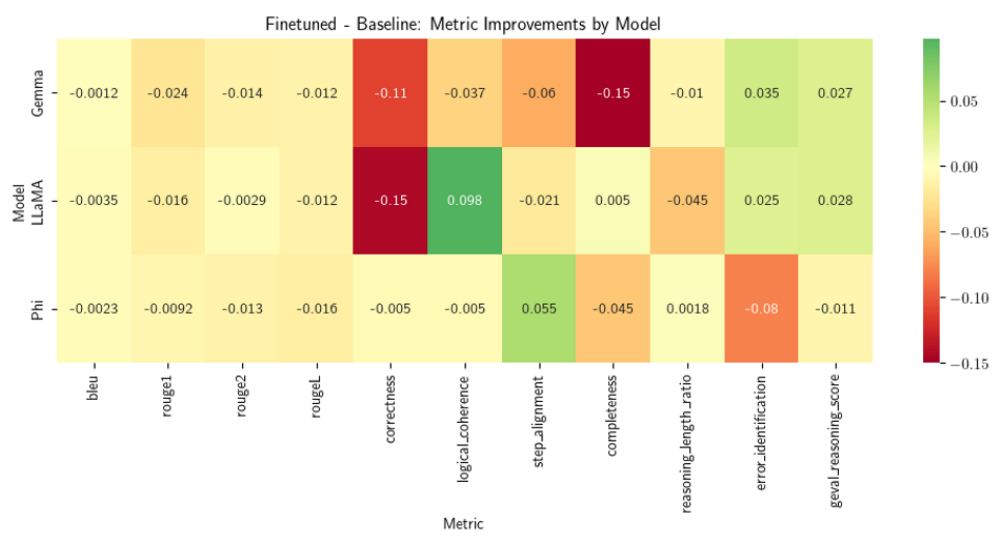
Achievements and Constraints

DPO

Using DPO, we were able to fine tune three models, LLaMA-3.2-1B, Gemma-3-1B, and Phi2 with moderate success. The models were able to reason better with fine-tuned models as compared to baseline models, however, the final answers fell short in terms of accuracy as compared to ground truth answers. However, correctness and logical coherence was improved along with error identification allowing the overall evaluation score to improve across the models as shown in Fig 111 except for Phi2.

Figure 111

Finetuned vs Baseline metrics comparison for all models



Training using DPO was performed on Goggle Colab Pro with access to GPUs like T4 (16GB VRAM), L4 (24 GB VRAM) and A100 (40 GB VRAM). DPO does not support quantization (4-bit or 8-bit using bitsandbytes), so the models must be loaded with full precision (FP16 or BF16) that significantly increases VRAM requirements as compared to techniques like

QLoRA. Without quantization, it's very challenging to tune large size models with limited compute space and VRAM. Even with high end GPUs like A100 the training capability was limited for large sized models. Multiple models were reviewed however, the finally decided models were Phi2, Gemma3-1B and LLaMA-3.2-1B. Following table 21 compares the models considered and why the fine tuning could not be performed given the resource constraints.

Table 21

Comparison of LLM model sizes for training on Colab

Model	Size (Parameters)	Weights (FP16, GB)	Est. Total VRAM (GB)	To-Feasible on Collab	Comments
LLaMA 3.2 1B	1 billion	~2 GB	~6-8 GB	Yes	Safest choice; fits comfortably with small batch sizes.
LLaMA 3.2 3B	3 billion	~6 GB	~15-18 GB	Borderline	May fit with batch size 1 and gradient accumulation, however, with does not work even with L4.
LLaMA 3.2 8B	8 billion	~16 GB	~30-35 GB	No	Exceeds VRAM, requires quantization or higher-end GPU.
Gemma 3 1B	1 billion	~2 GB	~6-8 GB	Yes	Text-only; fits well, ideal for low-resource setups.

Model	Size (Parameters)	Weights (FP16, GB)	Est. total VRAM (GB)	To-Feasible on Collab	Comments
Gemma 4B	3 4 billion	~8 GB	~18-22 GB	No	Multimodal (text+image); exceeds VRAM requirements in FP16.
Gemma 12B	3 12 billion	~24 GB	~50-60 GB	No	Multimodal; far too large without quantization.
Gemma 27B	3 27 billion	~54 GB	~100-120 GB	No	Multimodal; designed for single H100 GPU.
Phi-2 2.7B	2.7 billion	~5.4 GB	~13-15 GB	Yes	Fits with small batch sizes; only Phi-2 version available.

Given the constraints, it was decided to go ahead with smaller sized models for DPO as it was not possible without higher end GPUs, and it was also not cost effective to use A100 for everything.

CoD (Chain of Draft)

The Chain of Draft (CoD) approach delivered various significant outcomes when utilized for mathematical problem-solving. The main success of CoD became its ability to produce shorter more economical outputs than Chain of Thought (CoT) models while maintaining precise semantic conclusions. The modular approach with short-step reasoning enabled the model to create logically sound answers using fewer tokens thus leading to reduced computation time and expenses. The experimental results demonstrated that CoD produced answers which received

better scores in both BLEU and METEOR assessment methods. The assessment indicated CoD delivered enhanced performance together with consistency across different question types through its better BLEU-based evaluation results. CoD proves its worth as an applicable and scalable approach for tasks that require extensive reasoning operations.

However, there are certain limitations and difficulties associated with the CoD approach. One of them is that it is hard to make the "short draft" behavior in a generative model consistent, especially when the model is not particularly trained to adhere to strict token or step limits. This at times can lead to overly verbose or broken drafts. In addition, although CoD is very effective in regimented operations like mathematical calculations, its applicability to open-ended, creative logical reasoning environments has been less explored. Step-by-step formulation may sometimes cause redundancy or explicit fragmentation of the logical process. Yet another drawback is that model coherence to CoD prompting format heavily depends on the quality of fine-tuning as well as on the amount of variety in the training set in terms of the type of reasoning pattern included. Finally, the analysis was conducted on a relatively small sample of 20 cases, which might limit the statistical generalizability of the findings.

Group Relative Policy Optimization (GRPO)

Achievements. Successfully guided the Llama 3.2 3B model to think step by step within the <think> tags, resulting in improved answer correctness. Used LoRA to fine-tune the LLM for Chain-of-Thought reasoning, whereas the original implementation of GRPO suggested a full-finetuning of the LLM.

Constraints. Highest tier Google Colab GPU only offers 40 GB of GPU VRAM with the A100 runtime. This limits the batch size required for LoRA fine-tuning of the model since each question will require 8 responses to be generated from the LLM for reinforcement learning. Llama 3.2 3B requires 40+ GB of GPU VRAM to fine-tune using LoRA and GRPO. Unsloth was used to optimize the resources for finetuning the model. To optimize GPU VRAM, the following techniques had to be used are gradient checkpointing, small batch size and vLLM for fast and efficient inferencing for reward calculations. Each step (of batch size 4) roughly takes 1 minute

on a Colab A100 runtime. The current finetuned model has run only for 500 steps till now and has roughly taken 500 minutes to do so. This is quite slow, since the training dataset contains 6000 samples, and at this pace it would require 6000 minutes, or 100 hours, to complete a single epoch.

Difficulties. The accuracy reward is growing very slowly compared to the format reward and would require further fine-tuning to ensure that the answer enclosed inside the <answer> tags comply with the expected response used by the reward function. A lot of the libraries used in the project keep getting updated daily, and hence continuing the LLM training from checkpoints would fail the next day due to dependencies getting broken. Using the math-verify library by HuggingFace seems to have a lot of False Negatives, as it is counting the answers as incorrect if the answer generated by the LLM does not match its built-in regular expressions. It is suitable for a reward function, but not much useful as an evaluation metric.

Tree of Thought (TOT)

Through the Tree of Thought approach this project obtained distinctive and beneficial contributions to structured problem-solving methods. The key accomplishment of the model consisted of performing parallel reasoning exploration that let it identify better structured solutions than traditional single-path approaches. The ToT approach demonstrated its best performance in complex multi-step problems which required semantic coherence because it improved robustness when singular chains of thought became inadequate.

The implementation of breadth-first search algorithms with reward-based path selection functions within ToT led to better BERTScore F1 and ROUGE-L metrics in generated answers that both preserved semantic meaning and structural correctness of the ground truth. The system recovered from initial errors through its ability to follow different reasoning paths therefore producing better final answers. Each modular component within ToT provided better interpretation capabilities because evaluation and tracing were possible at every tiny step.

The ToT system acted as an effective inference-time reasoning intervention which needed no extra training procedures. The method provided a flexible and scalable application to any pre-existing fine-tuned model, which proved especially beneficial during project work because

reinforcement-based tuning (e.g. PPO) met specific execution constraints. The ToT system improved the reasoning abilities of supervised along with PPO-trained models through its intelligent selection process instead of reinstituting training.

The implementation of ToT was burdened by several important restrictions. The main shortcoming of ToT was its slow inference speed because it involved both spreading multiple reasoning chains and performing multiple scoring tasks which resulted in longer processing times than traditional one-pass methods. The system requires additional optimization to work effectively in real-time use cases and automated large-scale systems.

The outcome quality of Teacher-Foraging resulted from how the designers organized the reward functions. The project employed a fundamental reward system using “Hence” and “□” as keywords that achieved good results for math problems yet proved inadequate for broader types of advanced reasoning questions. Crafting more intelligent reward evaluators would be necessary for broader applicability.

Large model restrictions created memory limits that restricted both the exploration depth of tree branches and the maximum tree depth. The diverse phrasing across multiple paths caused BLEU scores to stay low even though ToT generally improved output quality because surface-level similarity measures are affected by this variation.

The promising reasoning technique named Tree of Thought managed to emerge despite its procedural limitations. The system proved that reasoning control at the inference level serves to boost training-driven improvements for generating high-quality math solutions across multiple problem types.

Self-Consistency (SC)

Achievements. One of the key achievements is the successful fine-tuning of the Phi 3.5 Mini-Instruct model, enabling it to generate reasoning-based responses using techniques like Chain-of-Thought (CoT). The model showed problem-solving, especially in multi-step reasoning, through LoRA fine-tuning and additional methods like RAG. Although smaller than models like Phi 3.5 or Llama, it demonstrated solid reasoning capabilities. The use of Self-Consistency (SC)

further refined its ability to select logically consistent responses. However, its smaller size limited its performance on more complex tasks, though it proved effective for lower-resource applications.

Constraints. Despite the encouraging results, some restrictions were encountered throughout the project. The size of the model was the primary restriction, and while it guaranteed computational effectiveness, it also restricted depth and reasoning complexity. Being a smaller model compared to others in the domain, such as Llama or DeepSeek, Phi 3.5 Mini-Instruct struggled to maintain high performance across all reasoning tasks. The logical coherence and depth of reasoning in the generated responses were often less robust, especially in tasks involving intricate problem-solving. Additionally, memory limitations during training on GPUs with restricted VRAM posed another challenge. Even with optimization techniques like gradient checkpointing and mixed precision, training remained slow, especially for more complex tasks. Another major constraint was the trade-off between the computational efficiency and the richness of reasoning, as the smaller model had to make compromises in its response generation process to fit within hardware limitations.

Difficulties. Several difficulties arose throughout the fine-tuning and evaluation process. While the fine-tuned model demonstrated improvements in logical consistency, particularly through SC techniques, it still showed limitations in terms of reasoning complexity. The smaller model size inherently struggled to match the performance of larger models, particularly when tasked with intricate and multi-step reasoning challenges. The self-consistency approach, though helpful in refining logical reasoning, did not fully address the underlying issue of limited model capacity to generate deeply accurate or comprehensive explanations.

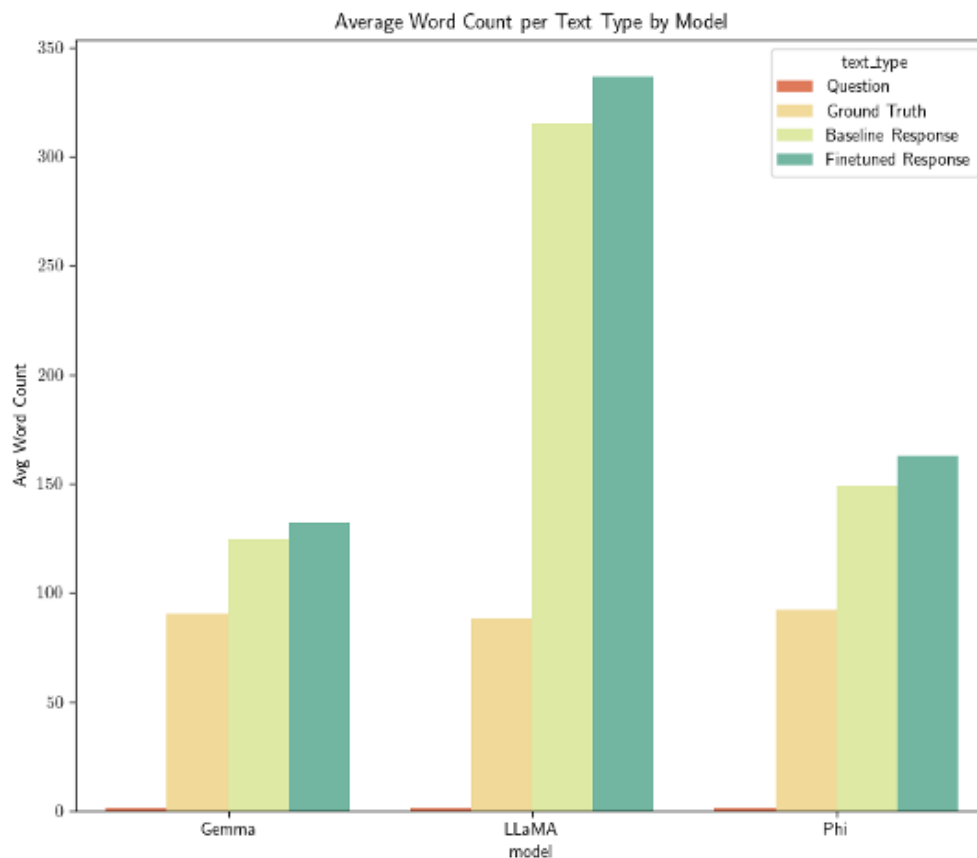
System Quality Evaluation of Model Functions and Performance

Quality of the system was evaluated against baseline and finetuned models for all three models finetuned with DPO. It was noticed that as compared to ground truth answers, the baseline and fine-tuned were longer in terms of word counts. The Figure [112](#) shows, that the fine-tuned models were able to reason better and tried to explain the solution with higher number of words

(or tokens). This shows an improvement in terms of how well the models were able to reason and provide a complete response. Further improvements can be achieved, by training with larger datasets, and higher batch sizes, however, due to the constraints mentioned earlier, its not feasible or cost effective.

Figure 112

Word Count comparisons ground truth vs baseline vs finetuned

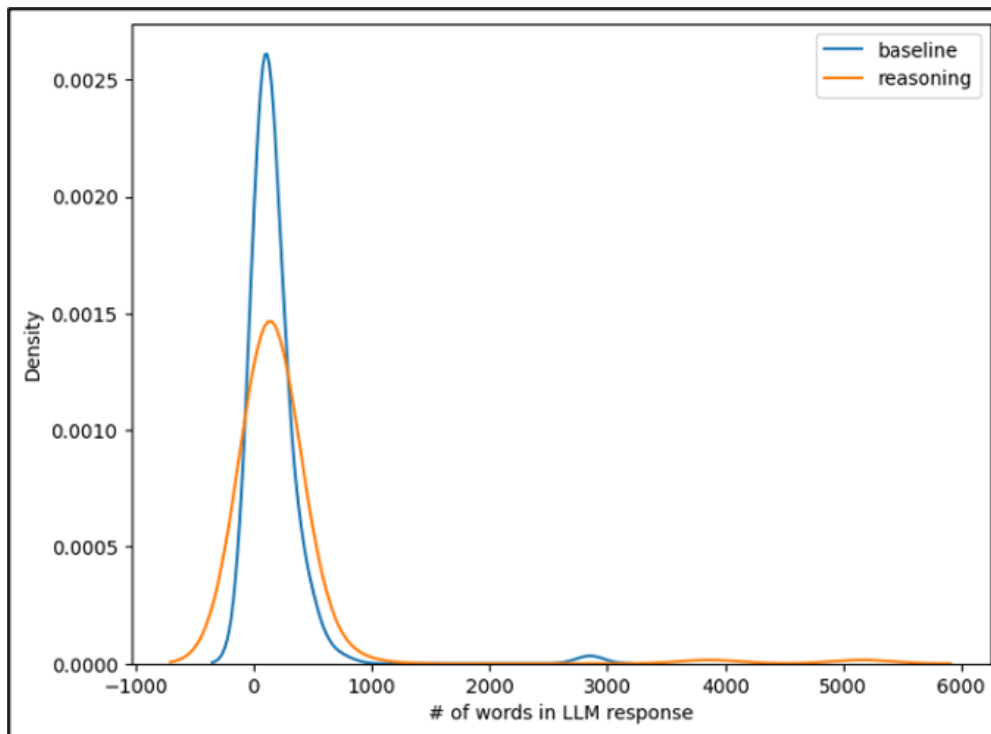


To investigate the quality of responses generated by the Llama 3.2 3B model (both baseline and GRPO fine-tuned version), a variety of metrics were measured. One such metric is measuring the number of words in the LLM response. If a model thinks step-by-step through a problem, its response will typically be longer than a non-finetuned version of the model. From Figure [113](#), the responses generated by the reasoning model were on average longer than the baseline model in terms of number of words. The baseline model had an average response length of 179 words,

while the reasoning model had an average response length of 245 words.

Figure 113

Distribution of words in the Llama 3.2 3B responses on MATH test partition



The Tree of Thought methodology evaluated system quality by determining how it advanced reasoning logic and response organization above both base and refined model outputs. Traditional single-pass generation did not match ToT because this technique enabled the system to search various logical pathways before choosing the best solution combination. Additional tokens provided meaningful value during step-by-step problem resolution that counterbalanced the response length elongation. The ToT system produced outputs containing clear logical organization which strongly matched answers from human experts especially for situations needing step-by-step reasoning or explanatory conclusions. Such findings demonstrate that model enhancements lead to better interpretation of answers which extends to provide reliable explanations of multi-step reasoning tasks.

Improvements in reasoning quality did occur through the application of ToT operations but these advancements included several consequent performance limitations. Through tree-based

thought exploration the system needs additional time for processing each question because it produces multiple thoughts which need evaluation and comparison before selecting a result. ToT requires considerable computational effort together with increased memory usage thus making it incompatible with real-time processing or extensive applications unless specific optimizations are applied. ToT functioned as an effective control system during inference to boost output stability while requiring no extra training and maintaining inference stability. Adding ToT to systems produces positive effects on system quality metrics which include reasoning accuracy and answer completeness as well as interpretability thus making ToT an important enhancement tool for logical multi-step processes.

The quality of the system was evaluated by comparing the baseline model with the fine-tuned models, including the Phi 3.5 Mini-Instruct model fine-tuned with Self-Consistency (SC). It was observed that, similar to other models, the fine-tuned versions exhibited longer responses compared to ground truth answers. This increase in word count or tokens signifies that the fine-tuned models were able to generate more detailed and comprehensive solutions, which indicates improved reasoning capabilities. The SC technique allowed the model to select the most consistent and coherent responses from multiple outputs, enhancing the logical coherence of the generated solutions. This improvement is especially important for tasks requiring step-by-step problem-solving and reasoning, where multiple answers are generated, and the best one is chosen based on logical consistency.

The SC-based approach evaluated system quality by determining how well the model could generate logically consistent answers by selecting the most coherent and accurate response among the candidates. In contrast to traditional methods, where a single output is generated, SC enables the model to evaluate different logical pathways, providing a more reliable and robust solution. As a result, fine-tuned models using SC were able to present solutions with improved accuracy and coherence, making them more effective in solving multi-step reasoning tasks.

However, the application of SC also led to certain limitations. The need for generating multiple responses and selecting the most consistent one introduced additional computational

overhead, which resulted in increased memory usage and longer inference times. This trade-off means that while SC improves the reasoning quality, it also requires more time and computational resources. Moreover, for tasks requiring real-time processing or large-scale applications, this added complexity could be a barrier unless further optimizations are applied. Despite these challenges, SC remains a valuable tool for enhancing logical consistency and generating more reliable multi-step reasoning outputs, significantly improving system quality metrics such as reasoning accuracy and interpretability.

System Visualization

Figure 114, 115, 116 shows the comparison of how the LLaMA, Gemma and Phi models (baseline and finetuned) performed for a sample set of 20 questions from the test dataset. This clearly shows the variations in performance in terms of Bleu and G-Eval reasoning scores.

Figure 114

Comparisons for 20 sample questions for LLaMA baseline vs finetuned

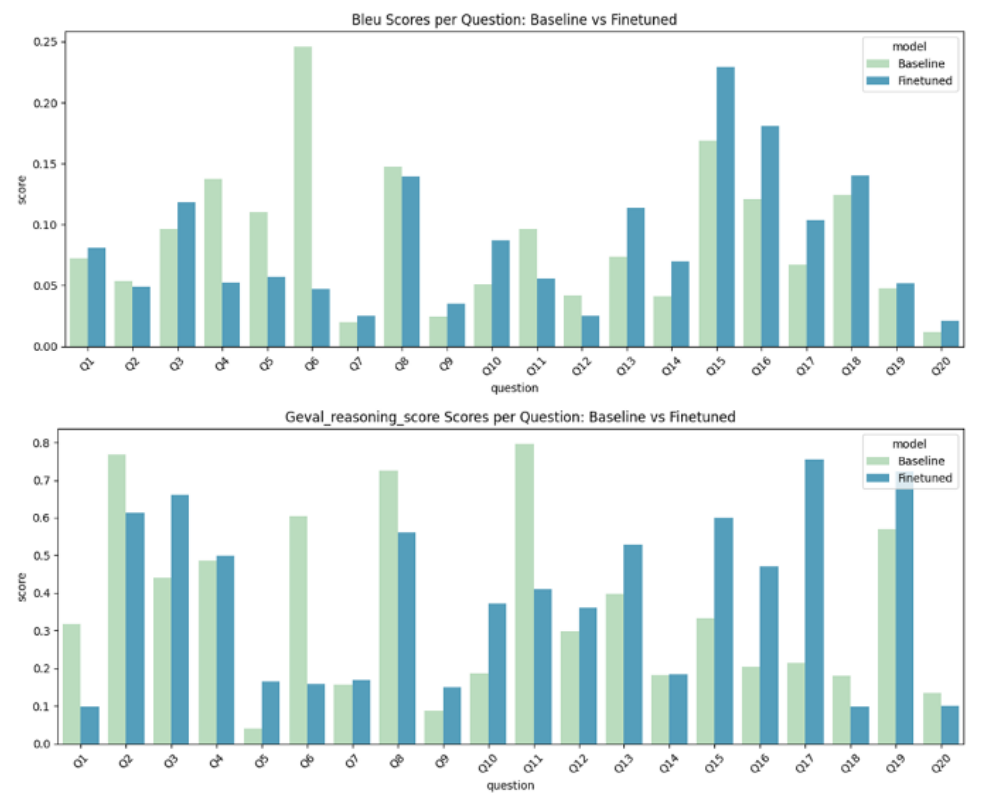


Figure 115

Comparisons for 20 sample questions for Gemma baseline vs finetuned

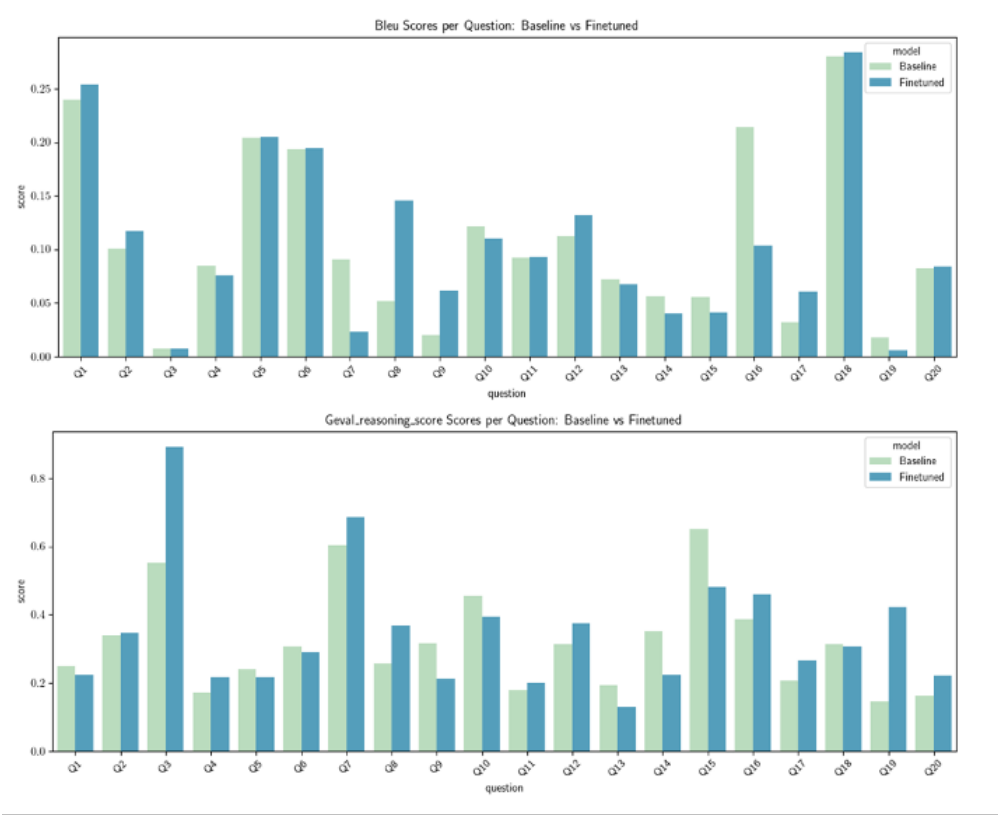
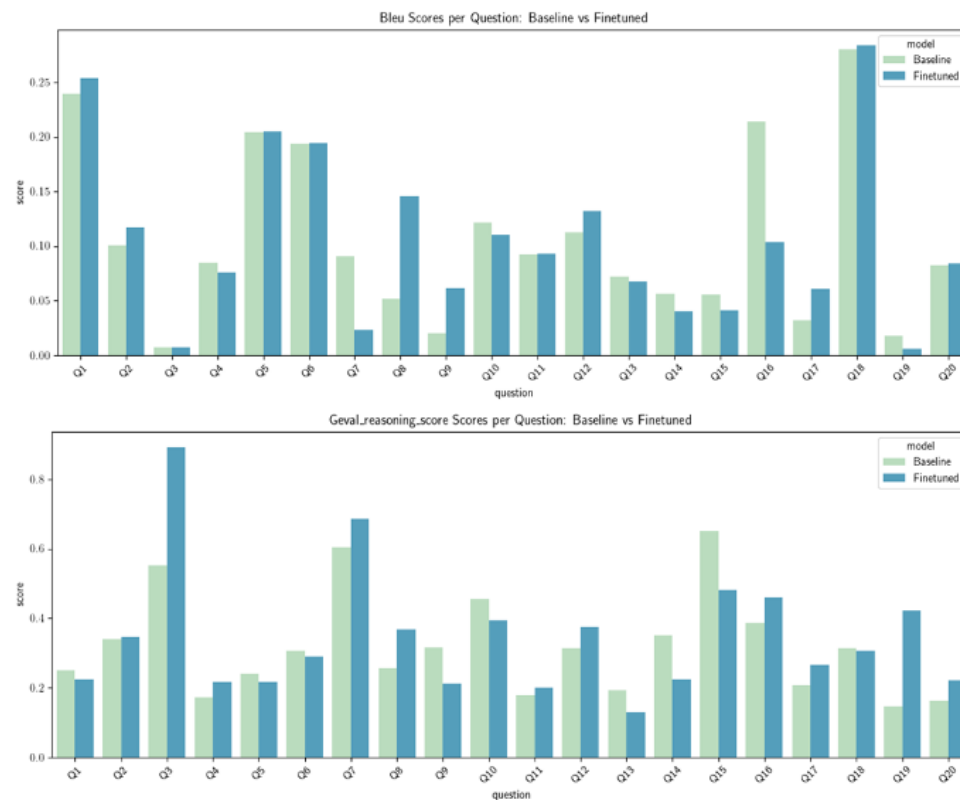


Figure 116

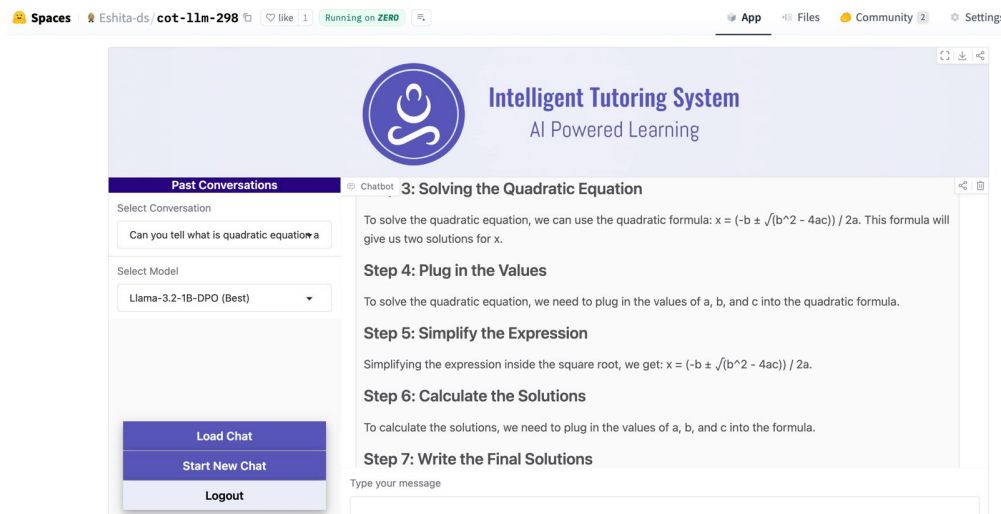
Comparisons for 20 sample questions for Phi baseline vs finetuned



The same models were added to the Web application to allow inferencing for real-time usage. Using the pre-trained / fine-tuned models, the application will be able to perform inferencing on the user questions. Each user will have separate chat sessions and history will be stored in a backend database. The users will be able to reload old chat history at any point in time using the web application. The generated responses will be used for further fine tuning using continuous pipelines and the models will be updated in case improvements are identified based on predefined thresholds. Example of the response generated in user interface is shown in the figure [117](#) below.

Figure 117

Inferencing using custom models in Intelligent Tutoring Application



The figure 118 shows the visual distribution of response lengths in relation to the question lengths. Blue color represents baseline responses while green represents finetuned responses and yellow represents ground truth answers.

LLaMA chart shows uniform spread with responses varying in length from 100 to 300 words which are longer than ground truths. Fine-tuned responses show a higher density with 200+ words which indicates better reasoning and verbosity after fine tuning. Fine-tuned model is showing a tendency of over explaining on the responses which is a desired outcome with chain of thought reasoning. LLaMA shows the most impact of fine tuning using DPO.

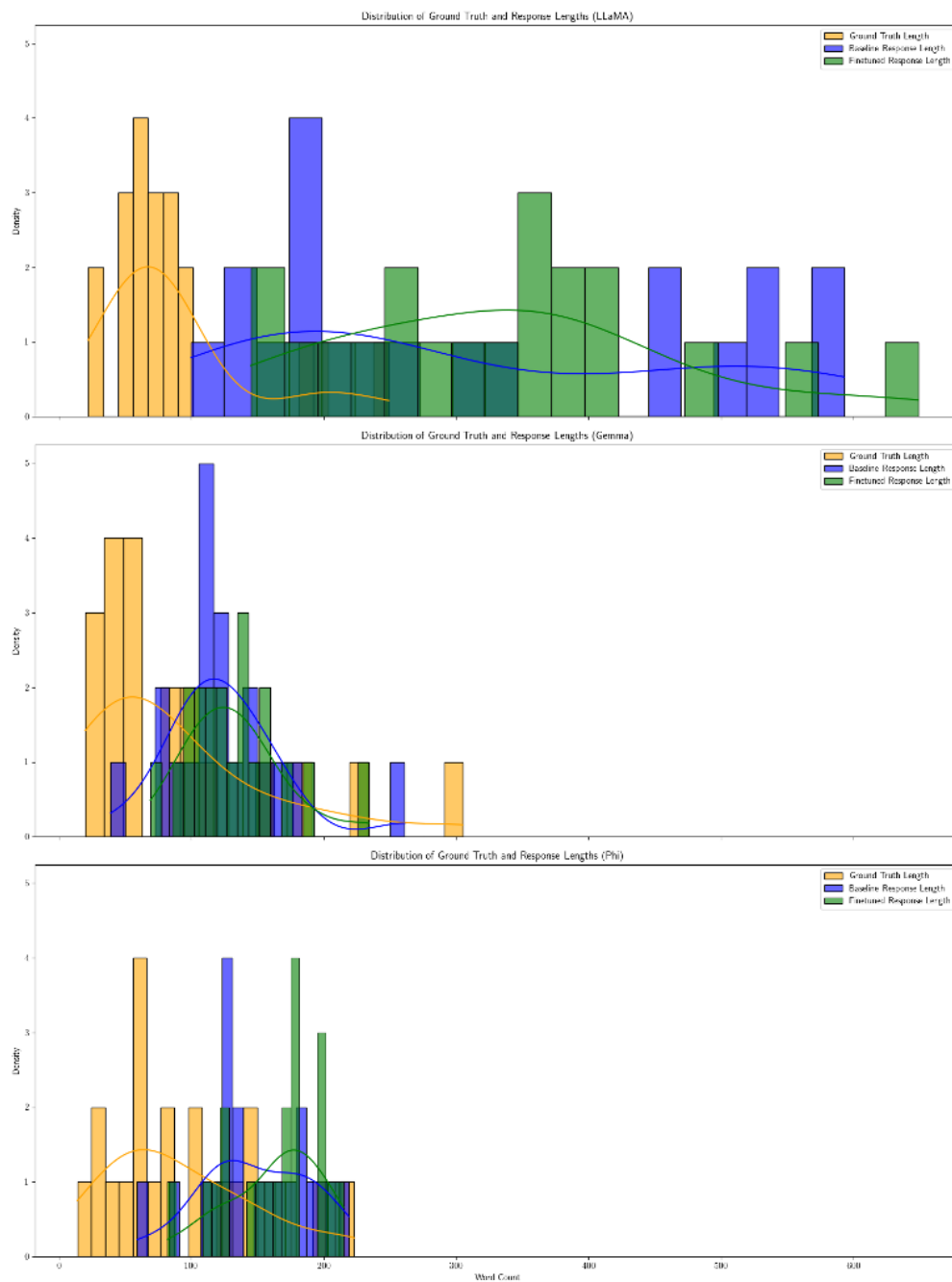
Gemma chart shows that baseline responses average around 80–140-word range while finetuned show a slightly higher average word count around 200+ words. This shows, even though Gemma is adapting to chain of thought reasoning, it is still generating compact answers. Gemma performs closer to real world responses while maintaining coherence and context.

Phi chart shows that all responses are centered with all outputs tracking in the range of 80-180 words. Fine-tuned shows a slight increase in word counts, however, nothing of too much significance. Phi is less prone to over generation and seems to be well aligned, however, the reasoning capabilities are limited as the answers are not as detailed as expected with chain of

thought reasoning models.

Figure 118

Distribution of Response Lengths in absolute word counts



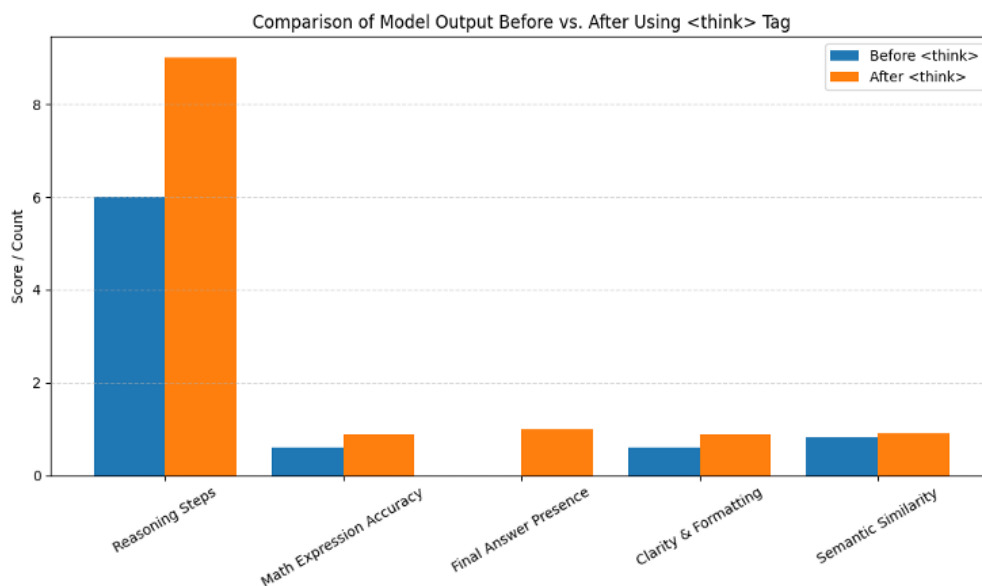
Deepseek executed a test which analyzed the effects of prompting strategies by assessing textual outputs generated with and without implementation of the think tag. The modification of

the input with think tag produced significant enhancements across different dimensions of reasoning in Figure 119. The implementation of the thinking process tag resulted in reasoning outputs that expanded to nine steps which amplified the structured framework of the thoughts. The mathematical solution showed better clarity and fewer missing steps when the students received prompting strategies. The think tag resulted in the model producing a final boxed answer in its output indicating its enhanced capacity to express precise and clear conclusions.

The think tag led to better formatting clarity as well as semantic similarity to ground truth comparisons that performed at slightly higher levels when compared to the base version. The think tag proved effective in both thought organization and the production of more human-sensible explanations. The structured format provided by think served as an intellectual anchor for the model leading it to produce increased coherence in logical outputs along with easier interpretation. The research demonstrates how structured prompt tags including think serve to enhance large language model reasoning capabilities without needing any additional model fine-tuning steps.

Figure 119

Comparison of Model Output before & after using Think Tag



Conclusion

Summary

The research project “Specializing Large Language Models Using Chain of Thought Reasoning” is particularly aimed at enhancing the interpretative faculties alongside the performance level of large language models (LLMs) concerning reasoning in educational domain. The structured datasets GSM8K, MATH, AIME, and OpenbookQA served as the primary building blocks for the system intended to address complicated multi-step reasoning problems. This study aimed to determine the capabilities of advanced LLMs - LLaMA 3.2, Gemma 3, phi 3.5, and Mistral 7B - within the context of Chain-of-Thought (CoT) reasoning by fine-tuning each of the models. Combined techniques of fine-tuning such as LoRA, QLoRA and reasoning techniques like Direct Preference Optimization (DPO), Chain of Draft (CoD), and Tree of Thought (ToT) were applied. The structure of these approaches facilitated the models to answer logically and consistently in a stepwise method similar to human thought processes. Scalable deployment and user interaction were enabled through a custom cloud data pipeline built on AWS (S3, Glue, RDS, CloudWatch) and a Gradio front-end. Evaluation of the models includes (BLEU, ROUGE, METEOR, chrF), semantic (BERTScore), and structure-based evaluation (coherence, correctness, completeness, step alignment). The results showed that CoT-boosted models significantly outperformed default prompting in reasoning depth, whereas metrics like BLEU did not always improve. This research for the education domain lay a solid groundwork for interpretable, scalable systems for applications requiring high logical accuracy and transparency. Ongoing retraining features and modular system design assure flexibility and sustained impact in education based on AI.

Benefits and Shortcoming

Benefits

The system built in this project presents several notable advantages for users engaging with Large Language Models. The deployment architecture leveraging HuggingFace Spaces, Gradio and Amazon RDS offers a cost-effective and straightforward implementation that reduces

technical barriers to entry. This approach incorporates user account functionality, enabling user isolation for privacy and personalization, along with efficient management of previous conversations that helps in maintaining context across interactions. The interface has been deliberately designed to be clean and free from distractions, promoting focused engagement with the LLMs trained in this project.

From a technical perspective, the LLMs have been specifically trained to approach educational and reasoning questions methodically. They solve problems in a step-by-step manner, explicitly outlining the reasoning process that leads to the correct solution. This transparency in problem-solving is particularly valuable in educational contexts where understanding the methodology is as important as reaching the correct answer. For LLaMa 3.2 (3B), fine-tuned using LoRA and GRPO, one of the most significant advantages was its ability to identify errors in its own reasoning and subsequently correct those mistakes within the same response. This self-correction feature serves as an educational tool by highlighting common pitfalls in complex problem-solving and helping users develop better critical thinking skills. This can be seen in Figure 120 below.

Figure 120

LLM fine-tuned using GRPO is able to identify contradictions and solve the problem correctly

First, let's denote the four distinct integers as a , b , c , and d . We are given that when added in pairs, the sums 10, 18, 19, 20, 21, and 29 are obtained. This gives us the following equations: $a + b = 10$, $a + c = 18$, $a + d = 19$, $b + c = 20$, $b + d = 21$, and $c + d = 29$. We can solve these equations to find the values of a , b , c , and d .

From the first equation, we can express b as $b = 10 - a$. Substituting this into the second equation, we get $a + (10 - a) = 18$, which simplifies to $10 = 18$.

This is a contradiction, so we need to reconsider our approach.

Let's re-examine the equations. From the equation $b + d = 21$, we can express d as $d = 21 - b$.

Substituting this into the equation $c + d = 29$, we get $c + (21 - b) = 29$, which simplifies to $c + 21 - b = 29$. Rearranging the terms, we get $c - b = 8$. Now we have two equations: $c - b = 8$ and $b + c = 20$.

We can solve these two equations to find the values of b and c . Adding the two equations, we get $2c = 28$, which simplifies to $c = 14$.

Substituting $c = 14$ into the equation $b + c = 20$, we get $b + 14 = 20$, which simplifies to $b = 6$.

Now that we have the values of b and c , we can find the value of d . Substituting $b = 6$ into the equation $d = 21 - b$, we get $d = 21 - 6 = 15$.

Finally, we can find the value of a . Substituting $d = 15$ into the equation $a + d = 19$, we get $a + 15 = 19$, which simplifies to $a = 4$.

Therefore, the four integers in increasing order are 4, 6, 14, and 15.

Shortcomings

Despite its strengths, this system has some limitations that affect its functionality and scalability. A significant constraint is the LLMs inability to reference previous conversations when solving current problems, which limits its capacity to build upon past interactions or learn from user history. The models selected for this project are limited to 1-7 billion parameters due to training, deployment, and monetary constraints, potentially limiting the quality of responses compared to larger, more sophisticated models. This parameter limitation reflects the practical trade-offs made to balance performance with resource availability. From a scalability perspective, the current implementation supports at most 100 concurrent user conversations, which would require dedicated hardware infrastructure to handle larger user bases or enterprise-level deployments. This constraint may limit the system's applicability in high-traffic educational environments or larger institutional settings where simultaneous access by many users is expected.

Potential System and Model Applications

The developed system and fine tuned large language models have far-reaching and deep impacts in various fields. First and foremost, the system is going to transform the education technology landscape by serving as an intelligent tutoring system (ITS) that can provide personalized, step-by-step explanations in math, logical reasoning, and science subjects. The chain-of-thought (CoT) reasoning capability allows the models to simulate human-like problem-solving behavior, making them extremely well-suited for online learning platforms, homework helpers, and classroom support tools. For non-educational use cases, the models have enormous potential in standardized test prep settings, e.g., SAT, GRE, and olympiad tests like AIME, where multi-step logical responses are the core. Further, decision-support domains like medical diagnosis, engineering design, and scientific inquiry can benefit from these models' capability to produce structured chains of reasoning for elaborate queries.

In business applications, the patterns can be utilized in knowledge management or automated customer service systems in which clear logic and contextual memory need to be applied in order to effectively respond to user inquiries. Due to its implementation with scalable cloud

infrastructure, the solution can be extremely suitable for mobile apps, voice assistants, and edtech platforms. The capacity for multiple retraining of models on actual user feedback also improves personalization and relevance, which makes the system useful in lifelong learning setups and adaptive testing systems. The system in totality closes the gap between raw AI capabilities and actual world usability by providing transparency, accuracy, and human-aligned reasoning, establishing a new standard for LLM applications in education, and high-stakes decision-making fields.

Experience and Lessons Learned

Experience

The project yielded valuable insights into the practical aspects of fine-tuning and deploying LLMs. Fine-tuning LLMs in the 1-7 billion parameter range proved to be achievable in just a few hours on a single GPU using Low-Rank Adaptation (LoRA), demonstrating the accessibility of this technology even with limited computing resources. Additionally, extending training beyond two epochs resulted in significant overfitting, highlighting the importance of careful epoch selection.

During Group Relative Policy Optimization (GRPO) implementation, setting the `lora_alpha` parameter equal to the `lora_rank` parameter, rather than following the conventional approach of making `lora_alpha` twice the `lora_rank`, created a better balance between new reasoning capabilities and base model performance. The GRPO training process yielded significant gains in reasoning performance over the base LLM with only two epochs of training across the entire dataset and required more than 25 hours of training time on a Google Colab A100 GPU.

Attempting to fine-tune models larger than seven billion parameters proved infeasible on services like Google Colab or Kaggle Notebooks, indicating the need for more substantial GPU resources for larger models. The integration of various HuggingFace services to host the trained LLMs required considerably more effort than anticipated, and finalizing the Gradio-based chat interface that properly maintained chat history and rendered responses correctly demanded numerous iterations.

Learnings

The project generated several key technical insights about effective LLM fine-tuning methodologies. LoRA demonstrated its value in helping LLMs avoid catastrophic forgetting, preserving the core capabilities of the base model while adding new reasoning abilities. Direct Preference Optimization (DPO) eliminated the need for a separate reward model entirely, transforming the reinforcement learning problem into a simpler classification problem based on human preference data. This approach used curated samples from the training dataset, further streamlining the fine-tuning process while maintaining or improving performance.

For GRPO implementation, focusing fine-tuning exclusively on non-attention parameters proved sufficient to ensure smooth LLM training, streamlining the process and reducing computational requirements. GRPO demonstrated effectiveness by allowing LLMs to learn from multiple ranked completions per prompt (configured to 8 for this project), which significantly improved response quality, stability, and human alignment across various metrics compared to the base model. A notable advantage of Reinforcement Learning with GRPO is its elimination of two traditionally expensive components: the reward and critic models, making the training process more efficient.

Recommendations for Future Work

For future work, the system can be enhanced by incorporating conversational memory to allow the model to reference and build upon prior user interactions. This would enable more personalized and context-aware responses, particularly valuable in educational settings. Leveraging vector databases or memory-augmented retrieval techniques could significantly improve continuity across sessions. Additionally, scaling to larger model architectures such as 7B or 13B parameter models using multi-GPU or distributed training frameworks like DeepSpeed or FSDP would likely improve reasoning depth and language fluency. Another promising direction is the integration of multi-agent collaboration strategies, such as debate-style prompting or ensemble reasoning, where multiple LLMs critique and refine each other's answers before finalizing a response.

Future iterations could also benefit from implementing real-time feedback loops, allowing users to rate answers and feed this data into active learning cycles via preference-based fine-tuning methods like DPO or GRPO. Expanding the evaluation pipeline beyond math to include domains like science, law, or programming would validate model generalizability. Further improvements could involve supporting multilingual capabilities and accessibility features such as text-to-speech or screen-reader compatibility. Finally, for scalability in production settings, migrating to cloud-native infrastructure with autoscaling and monitoring would make the system viable for institutional deployments, while exploring hybrid symbolic-neural reasoning could bridge gaps in logical rigor and accuracy

Contributions and Impacts on Society

Key contributions

The project delivers an end-to-end, reproducible pipeline that finetunes Llama 3(3B), Gemma 2(1B), DeepSeek-R1(1.5B) and Phi-2(2B) using lightweight adapters (LoRA/QLoRA) so the entire training runs on a single 24 GB GPU. On top of these checkpoints implemented and benchmarked six reasoning strategies (Chain-of-Thought, Chain-of-Draft, Self-Consistency, GRPO, Tree-of-Thought, DPO) against math problems, scoring with open metrics (BLEU, ROUGE-L, chrF, METEOR, BERTScore) plus custom G-Eval (LLM as a judge) for correctness and logical coherence compared to the expected responses from the test set. All code, prompts and metrics are released, giving practitioners a drop-in framework for replicating or extending reasoning research on any model family.

Societal Impact

Because the pipeline works with small–medium open models, universities, non-profits and small businesses can now build transparent AI tutors or domain assistants without expensive cloud access. The step-by-step solutions generated by our best strategy help students understand how to solve problems instead of copying answers, while the open evaluation suite lets educators audit model accuracy. Running the quantized models locally cuts both inference cost and data-privacy risk, promoting wider, responsible adoption of AI-based learning tools.

References

- Abdin, M., Aneja, J., Awadalla, H., Awadallah, A., Awan, A. A., Bach, N., Bahree, A., Bakhtiari, A., Bao, J., Behl, H., Benhaim, A., Bilenko, M., Bjorck, J., Bubeck, S., Cai, M., Cai, Q., Chaudhary, V., Chen, D., Chen, D., . . . Zhou, X. (2024, April 22). *PHI-3 Technical Report: A highly capable language model locally on your phone*. ArXiv.org.
- <https://arxiv.org/abs/2404.14219>
- AIME Problems and Solutions. (n.d.). Retrieved
- https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions
- Amini, A., Gabriel, S., Lin, P., Koncel-Kedziorski, R., Choi, Y., & Hajishirzi, H. (2019). MathQA: Towards Interpretable Math Word Problem Solving with Operation-Based Formalisms. *arXiv (Cornell University)*.
- <https://doi.org/10.48550/arxiv.1905.13319>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., . . . Amodei, D. (2020). Language Models are Few-Shot Learners. *arXiv (Cornell University)*.
- <https://doi.org/10.48550/arxiv.2005.14165>
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). Training verifiers to solve math word problems. *arXiv (Cornell University)*.
- <https://doi.org/10.48550/arxiv.2110.14168>
- Colombo, A. (2024). Leveraging Knowledge GraphsandLLMstoSupport and Monitor

Legislative Systems. In Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24), October 21–25, 2024, Boise, ID, USA. ACM, New York, NY, USA, 4 pages.

<https://doi.org/10.1145/3627673.3680268>

Chalkidis, I., Jana, A., Hartung, D., Bommarito, M., Androutsopoulos, I., Katz, D. M., & Aletras, N. (2021). LexGLUE: a benchmark dataset for legal language understanding in English. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2110.00976>

Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., & Androutsopoulos, I. (2020).

LEGAL-BERT: The Muppets straight out of Law School. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2010.02559>

Chen, W., et al. (2023). QLoRA: Efficient Adaptation Techniques for Large Models.

<https://arxiv.org/abs/2306.04031>

Chowdhery, A., et al. (2023). PaLM 2: Scaling Language Models with Sparse Mixture of Experts.

<https://arxiv.org/abs/2304.14067>

Devlin, J., et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.

<https://aclanthology.org/N19-1423/>

Ding, J., Cen, Y., Wei, X. (2023). Using Large Language Model to Solve and Explain Physics WordProblems Approaching Human Level.

<https://arxiv.org/abs/2309.08182>

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A.,

- Korenev, A., Hinsvark, A., Rao, A., Zhang, A., . . . Zhao, Z. (2024). The Llama 3 herd of models. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.2407.21783>
- Fedus, W., et al. (2021). Scaling Sparse Mixture of Experts.
<https://arxiv.org/abs/2107.03374>
- Geva, M., Khashabi, D., Segal, E., Khot, T., Roth, D., & Berant, J. (2021). Did Aristotle Use a Laptop? A Question Answering Benchmark with Implicit Reasoning Strategies. *Transactions of the Association for Computational Linguistics*, 9, 346-361.
https://doi.org/10.1162/tac1_a_00370
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D.X., & Steinhardt, J. (2021). Measuring Mathematical Problem Solving With the MATH Dataset.
<https://doi.org/10.48550/arXiv.2103.03874>
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., & Chen, W. (2021). LORA: Low-Rank adaptation of Large Language Models. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.2106.09685>
- Hou, A., Jurayj, W., Holgenberger, N., Durme, B. (2024). Gaps or Hallucinations? Gazing into Machine-Generated Legal Analysis for Fine-grained Text Evaluations.
<https://arxiv.org/abs/2409.09947>
- Huang, J., Gu, S. S., Hou, L., Wu, Y., Wang, X., Yu, H., & Han, J. (2022). Large language models can Self-Improve. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.2210.11610>
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., De Las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M., Stock,

P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., & Sayed, W. E. (2023). Mistral 7B. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2310.06825>

Kim, S., Joo, S. J., Kim, D., Jang, J., Ye, S., Shin, J., & Seo, M. (2023). The COT collection: Improving zero-shot and few-shot learning of language models via Chain-of-Thought Fine-Tuning. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2305.14045>

Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). Large Language Models are Zero-Shot Reasoners. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2205.11916>

Lee, K., Firat, O., Agarwal, A., Fannjiang, C., & Sussillo, D. (2019). Hallucinations in neural machine translation.

<https://openreview.net/pdf?id=SkxJ-309FQ>

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP tasks. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2005.11401>

Ling, W., Yogatama, D., Dyer, C., & Blunsom, P. (2017). Program Induction by Rationale Generation : Learning to solve and explain algebraic word problems. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.1705.04146>

Liu, X., He, P., Chen, W., & Gao, J. (2019). Multi-Task deep neural networks for natural

language understanding. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.1901.11504>

Liu, Y., et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach.

<https://arxiv.org/abs/1907.11692>

Lu, P., Mishra, S., Xia, T., Qiu, L., Chang, K., Zhu, S., Tafjord, O., Clark, P., & Kalyan, A.

(2022). Learn to explain: Multimodal reasoning via thought chains for science question answering. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2209.09513>

Mihaylov, T., Clark, P., Khot, T., & Sabharwal, A. (2018). Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering. Conference on Empirical Methods in Natural Language Processing.

<https://aclanthology.org/D18-1260>

Nye, M., Andreassen, A., Uesato, J., Schauble, P., Creswell, A., Shanahan, M., & Kohli, P.

(2021). *Show Your Work: Scratchpads for Intermediate Computation with Language Models*. *arXiv (Cornell University)*.

<https://arxiv.org/abs/2112.00114>.

Papineni, K., Roukos, S., Ward, T., & Zhu, W. (2002). BLEU. *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, 311–318.

<https://doi.org/10.3115/1073083.1073135>

Patel, A., Bhattamishra, S., & Goyal, N. (2021). Are NLP Models really able to Solve Simple Math Word Problems? *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2103.07191>

Rae, J. W., Borgeaud, S., Cai, T., Millican, K., Hoffmann, J., Song, F., Aslanides, J., Henderson, S., Ring, R., Young, S., Rutherford, E., Hennigan, T., Menick, J., Cassirer, A., Powell, R.,

Van Den Driessche, G., Hendricks, L. A., Rauh, M., Huang, P., . . . Irving, G. (2021).

Scaling Language Models: Methods, Analysis & Insights from Training Gopher. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2112.11446>

Reppert, J., Rachbach, B., George, C., Stebbing, L., Byun, J., Appleton, M., & Stuhlmüller, A.

(n.d.). Iterated decomposition: Improving science Q&A by supervising reasoning processes. ArXiv.

<https://doi.org/10.48550/arxiv.2303.08261>

Sap, M., Shwartz, V., Bosselut, A., Choi, Y., & Roth, D. (2020). Commonsense reasoning for natural language processing. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: *Tutorial Abstracts*, 27-33.

<https://doi.org/10.18653/v1/2020.acl-tutorials.7>

Schröer, C., Kruse, F., Jorge, M. G. & Gómez, J. M. (2021). A Systematic Literature Review on Applying CRISP-DM Process Model, Retrieved 4 2, 2024, from

<https://doi.org/10.1016/j.procs.2021.01.199>

Schulhoff, S., Ilie, M., Balepur, N., Kahadze, K., Liu, A., Si, C., Li, Y., Gupta, A., Han, H.,

Schulhoff, S., Dulepet, P. S., Vidyadhara, S., Ki, D., Agrawal, S., Pham, C., Kroiz, G.,

Li, F., Tao, H., Srivastava, A., . . . Resnik, P. (2024). The Prompt Report: A Systematic survey of prompting techniques. arXiv (Cornell University).

<https://doi.org/10.48550/arxiv.2406.06608>

Sprague, Z., Yin, F., Rodriguez, J. D., Jiang, D., Wadhwa, M., Singhal, P., Zhao, X., Ye, X.,

Mahowald, K., & Durrett, G. (2024, September 18). *To CoT or not to CoT? Chain-of-*

thought helps mainly on math and symbolic reasoning. arXiv.org.

<https://arxiv.org/abs/2409.12183>

Talmor, A., Herzig, J., Lourie, N., & Berant, J. (2018). CoMMonSenseQA: A question answering challenge targeting commonsense knowledge. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.1811.00937>

Team, G., Riviere, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard, T., Shahriari, B., Ramé, A., Ferret, J., Liu, P., Tafti, P., Friesen, A., Casbon, M., Ramos, S., Kumar, R., Lan, C. L., Jerome, S., . . . Andreev, A. (2024). Gemma 2: Improving open language models at a practical size. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2408.00118>

Touvron, H., et al. (2023). LLaMA: Open and Efficient Foundation Models.

<https://arxiv.org/abs/2307.09288>

Wang, B., Min, S., Deng, X., Shen, J., Wu, Y., Zettlemoyer, L., & Sun, H. (n.d.). Towards understanding chain-of-thought prompting: An empirical study of what matters. *arXiv*.

<https://arxiv.org/abs/2208.04014>

Wang, J., & Wibisono, A. (2023). Continuized acceleration for Quasar Convex functions in Non-Convex optimization. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2302.07851>

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., & Zhou, D. (2022). Self-Consistency improves chain of thought reasoning in language models. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2203.11171>

Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang,

- Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., Yue, X., & Chen, W. (2024). MMLU-Pro: a more robust and challenging Multi-Task Language Understanding benchmark. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.2406.01574>
- Wang, Z., Liu, A., Lin, H., Li, J., Ma, X., & Liang, Y. (n.d.).
RAT: Retrieval augmented thoughts elicit context-aware reasoning in long-horizon generation. *arXiv*.
<https://arxiv.org/abs/2303.06744>
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q., & Zhou, D. (2022b). Chain-of-Thought prompting elicits reasoning in large language models. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.2201.11903>
- Weston, J., Chopra, S., & Bordes, A. (2014). Memory networks. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.1410.3916>
- Xia, Y., Zhou, J., Shi, Z., Chen, J., & Huang, H. (2024). Improving Retrieval Augmented Language Model with Self-Reasoning. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.2407.19813>
- Xu, X., Tao, C., Shen, T., Xu, C., Xu, H., Long, G., & Lou, J. (2023). Re-Reading improves reasoning in language models. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.2309.06275>
- Ye, X., Iyer, S., Celikyilmaz, A., Stoyanov, V., Durrett, G., & Pasunuru, R. (2022).
Complementary Explanations for Effective In-Context Learning. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.2211.13892>

Zelikman, E., Wu, Y., & Goodman, N. D. (2022). STAR: Bootstrapping reasoning with reasoning. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2203.14465>

Zhang, J., et al. (2023). Self-Consistency in Language Models: A Method for Reliable Responses.

<https://openreview.net/forum?id=UhvN6M6ms6G>

Zhao, S., Yang, Y., Wang, Z., He, Z., Qiu, L. K., & Qiu, L. (n.d.). Retrieval augmented generation (RAG) and beyond: A comprehensive survey on how to make your LLMs use external data more wisely.

<https://doi.org/10.48550/arxiv.2301.00795>

Zhao, Y., Cao, H., Zhao, X., & Ou, Z. (n.d.). An empirical study of retrieval augmented generation with chain-of-thought. *arXiv*.

<https://doi.org/10.48550/arxiv.2304.14248>

Zheng, L., Guha, N., Anderson, B. R., Henderson, P., & Ho, D. E. (2021). When does pretraining help? Assessing Self-Supervised Learning for Law and the CaseHOLD Dataset. *arXiv (Cornell University)*.

<https://doi.org/10.48550/arxiv.2104.08671>

Zhou, Y., Muresanu, A., Han, Z., Paster, K., Pitis, S., Chan, S., Ba, J. (2022). Large Language Models Are Human-Level Prompt Engineers.

<https://arxiv.org/abs/2211.01910>

Appendix

Appendix A - System Testing (UI)

Use Case	Description	Result
User login	User is able to login and proceed to the next step.	Worked as expected, please refer figure 121
New User Registration	User is able to put in his details to register himself on the application and is able to successfully register	User was able to register with his details, refer figure 122
Initiate a new chat	User should be able to click on “start a new chat” option to interact with the models	User was able to start a new chat, refer figure 124
Reload old chat and continue the conversation	User should be able to load the old chat from dropdown option in “Select Conversation” and then select “Load chat” button	User was able to successfully load chat option 9, refer figure 125
Model Selection	User is able to select which model under “Select model” they are willing to use for interaction. Different Model will yield different results	User was able to select model “Llama-3.2-1B-DPO” for the interaction, refer figure 125
User Logout	To successfully logging out from the application, user to select the Logout option.	User was successfully able to logout and returned to the main screen , refer figure 126
Application Link	Link to Hugging face spaces Application	.
Airflow Pipeline Link	LLMOps Airflow Pipeline	.

Figure 121*User Login Screen*

The figure displays two screenshots of the Intelligent Tutoring System (ITS) user interface. The top screenshot shows the login screen, and the bottom screenshot shows the chat interface.

Intelligent Tutoring System
AI Powered Learning

Login Screen:

- Email:
- Password:
- [Login](#)
- [Need an account? Register](#)

Chat Interface:

- Past Conversations**
- Select Conversation: [Chat - 1 - \(Session ID: 2fZyLU70\)](#)
- Select Model: [Llama-3.2-1B-DPO](#)
- [Load Chat](#)
- [Start New Chat](#)
- [Logout](#)
- Charbot
- Type your message:
- [Send](#)

Figure 122*New user registration Screen*

Register New Account!!

First Name
SJSU

Last Name
University

Email
testjsu@gmail.com

Password

Confirm Password

Message
Registration successful

Register

Already have an account? [Login](#)

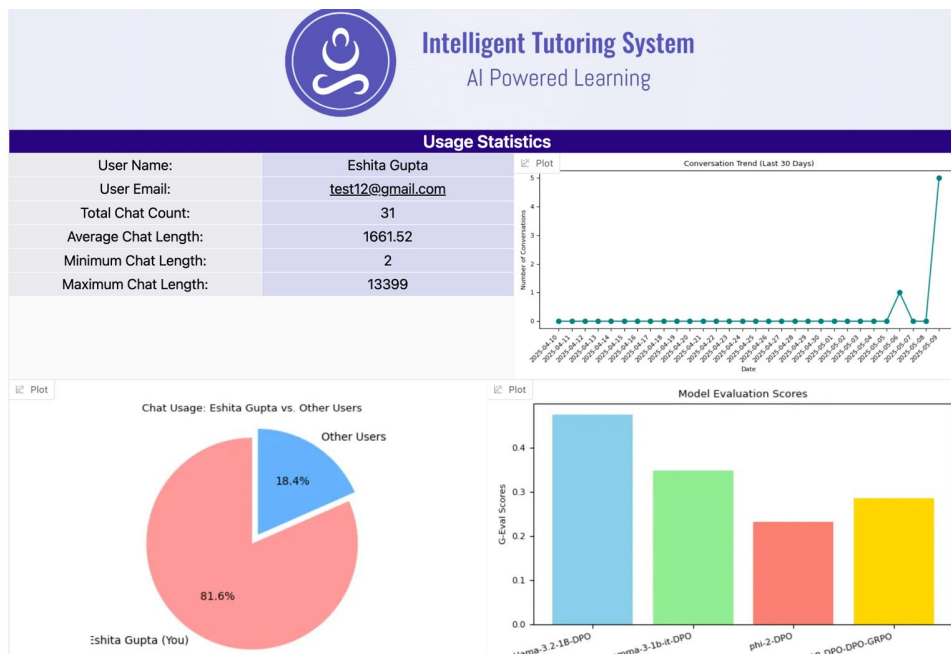
Figure 123*Stats of the User after Logging in*

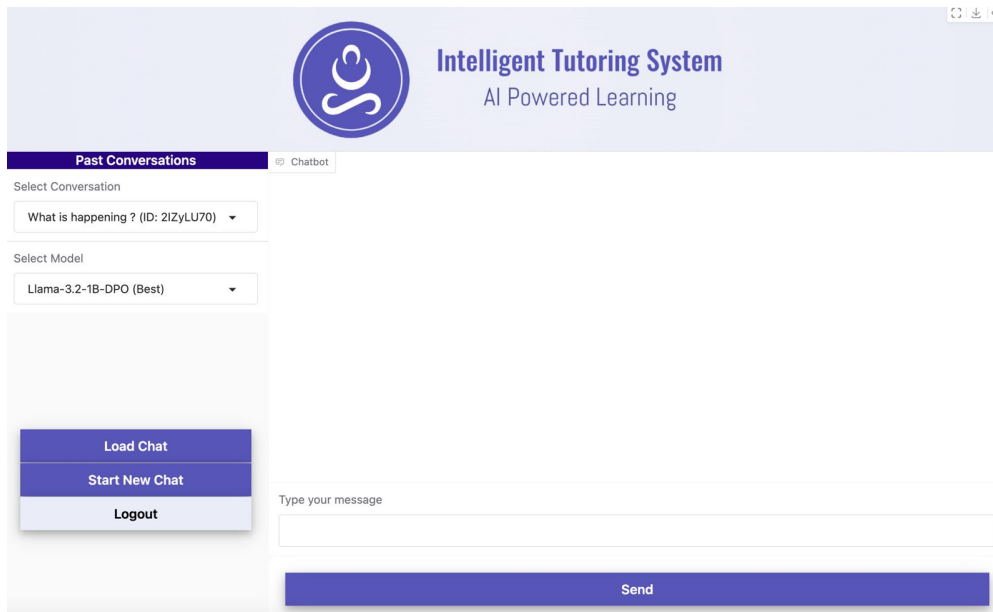
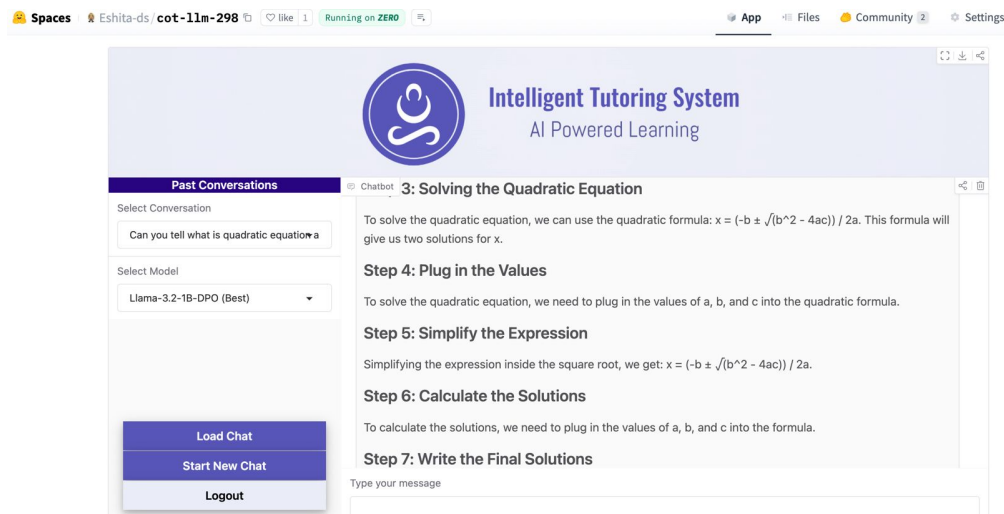
Figure 124*Initiate a new Chat Screen***Figure 125***Load the past chat sessions to continue the chat*

Figure 126

Screen after User successfully logs out



The screenshot shows a web browser window displaying the login page of the 'Intelligent Tutoring System'. The header features a blue circular logo with a stylized 'S' and the text 'Intelligent Tutoring System' and 'AI Powered Learning'. Below the header, there are two input fields: 'Email' with the value 'test12@gmail.com' and 'Password' with masked characters '.....'. A blue 'Login' button is positioned below the password field. At the bottom, a light blue bar contains the text 'Need an account? Register'.

Intelligent Tutoring System
AI Powered Learning

Email
test12@gmail.com

Password
.....

Login

Need an account? Register

Appendix B - Project Data Source and Management Store

Data Source	Links
OpenBookQA	OpenBookQA Dataset from AllenAI
GSM 8K	GSM8K Dataset from OpenAI
MATH	MATH Dataset from UC Berkeley
AIME	AIME Dataset from AoPS
Staging Datasets	Staging Datasets in shared drive

Appendix C - Project Program Source Library, Presentation, and Demonstration

Resource	Links
Program Source Library/ Code Artifacts	Code Artifacts - DATA298 - Team 6 - Google Drive
Presentation	Presentation .ppt file - DATA298 - Team 6 - Google Drive
Submitted documents	Documents/Report - DATA298 - Team 6 - Google Drive
Demo Video	Demo Video - DATA298 - Team 6 - Google Drive